100 MINI PROJECTS

You Can Build With AI

From Beginner to Builder — Practical Projects, AI Prompts & Complete Code

A practical, build-it-today book for complete beginners — students, self-learners and anyone curious about what AI can do when you put it to work on real projects.

Includes 100 projects · 100 AI prompts · complete runnable code · zero prior coding required

Copyright & Disclaimer

© 2026 — All rights reserved. This book is intended for personal, educational use. You are welcome to build every project in it, share what you create, and modify the source code freely.

The code samples in this book are written for educational purposes. They are runnable as published, but as with any software, you should review, test and adapt them before using them in a real product. The author makes no claim about fitness for any particular purpose, and disclaims all liability arising from the use of these examples.

Names of technologies, services or libraries used in this book belong to their respective owners. Web links and API names were correct at the time of writing; specific URLs may change over time.

No part of this book should be construed as a promise of income, employment or any specific outcome from completing the projects. The value of the book is in the practice and the skills you build by doing.

Table of Contents

Copyright & Disclaimer
How to Use This Book
How to Use AI as Your Coding Partner
How to Debug When AI Gives You Broken Code
How to Modify Any Project
Recommended Free Tools for Beginners
Part 1 — Super Easy Projects
- Project 1: Hello Name Greeter
- Project 2: Age in Days Calculator
- Project 3: Days Until Summer
- Project 4: Random Compliment Generator
- Project 5: Colour of the Day
- Project 6: Even or Odd Checker
- Project 7: Temperature Mood Translator
- Project 8: Reverse a Word
- Project 9: Simple Counter with Limits
- Project 10: Tip Calculator
- Project 11: Dice Roller Simulator
- Project 12: BMI Calculator
- Project 13: Coin Toss Counter
- Project 14: Word Length Sorter
- Project 15: Magic 8-Ball
- Project 16: Number Pyramid Printer
- Project 17: Basic Calculator
- Project 18: Time Zone Buddy
- Project 19: Word Counter Tool
- Project 20: Digital Greeting Card

Part 2 — Web Projects
- Project 21: Personal Landing Page
- Project 22: Notes Saver
- Project 23: Responsive Photo Grid
- Project 24: Markdown Previewer
- Project 25: Real-time Clock Display
- Project 26: Sticky Notes Board
- Project 27: Animated CSS Buttons

Project 28: Form Validator
Project 29: Drag-and-Drop Kanban
Project 30: Mini Online Storefront
Project 31: Weather UI with Mock Data
Project 32: Tabbed Interface
Project 33: Image Filter Playground
Project 34: Accordion FAQ
Project 35: Searchable Movie List
Project 36: Toast Notification System
Project 37: Analog Clock
Project 38: Personal Blog Template
Project 39: Recipe Card Component
Project 40: Resume Builder
Part 3 — Games & Interactive Projects
Project 41: Number Guessing Game
Project 42: Rock Paper Scissors
Project 43: Tic Tac Toe
Project 44: Memory Match Game
Project 45: Snake Game
Project 46: Breakout / Brick Breaker
Project 47: Typing Speed Tester
Project 48: Reaction Time Tester
Project 49: Whack-a-Mole
Project 50: Pomodoro Timer
Project 51: Hangman
Project 52: Maze Generator and Solver
Project 53: Quiz Game
Project 54: Tower of Hanoi Visualiser
Project 55: Drawing Pad
Project 56: Space Shooter
Project 57: Bouncing DVD Logo
Project 58: Connect Four AI
Project 59: Simon Says Colour Memory
Project 60: Stop The Clock Race
Part 4 — Productivity & Useful Tools
Project 61: Habit Tracker
Project 62: Expense Tracker
Project 63: Flashcard Study App
Project 64: Markdown Editor

Project 65: QR Code Generator UI
Project 66: Text Analyzer
Project 67: Daily Planner
Project 68: Pomodoro Habit Combo
Project 69: Stock Portfolio Tracker (Mock)
Project 70: Personal CRM
Project 71: Unit Converter
Project 72: Random Quote Generator with Originals
Project 73: Password Strength Tester
Project 74: Workout Timer
Project 75: Book Reading Tracker
Project 76: Daily Affirmation Spinner
Project 77: Movie Watch List
Project 78: Invoice Generator
Project 79: Markdown Notes Wiki
Project 80: Reading Speed Adjuster
Part 5 — Advanced Beginner Projects
Project 81: Interactive Solar System
Project 82: Colour Palette Generator
Project 83: Procedural Terrain Generator
Project 84: Particle Explosion Simulator
Project 85: Drum Machine / Step Sequencer
Project 86: Projectile Motion Simulator
Project 87: Chord Progression Player
Project 88: ASCII Art Image Converter
Project 89: Cipher Encoder & Decoder
Project 90: Random Story Generator
Project 91: Meme Generator
Project 92: Data Dashboard
Project 93: Snakes & Ladders Game
Project 94: Mini Search Engine
Project 95: Recipe Randomizer
Project 96: Decision Spinner (Weighted)
Project 97: Mini Synthesizer
Project 98: Sudoku Solver
Project 99: Tiny Generative Art Wallpaper
Project 100: Capstone: Personal Portfolio Site
30-Day Project Challenge
Project Difficulty Roadmap

Skills Gained by Completing the Book
What to Build Next
Final AI Coding Cheat Sheet
Author Note

How to Use This Book

This book is built so you can read it in any order, but it is most powerful when you read top-to-bottom and build everything.

A simple plan that works for most people:

1. Each project takes between 30 minutes and a few hours to complete. Pick what you can finish in one sitting, finish it, and move on. By the time you reach project 100 you will have built the equivalent of a small portfolio.
2. Read the AI prompt at the top of every project before you read the code. The prompt is your starting point — paste it into ChatGPT, Claude or Gemini if you need help, want different ideas, or want a deeper explanation of any concept.
3. Type the code yourself instead of copy-pasting. Yes, it's slower. Yes, it builds real understanding.
4. Tweak every project. Change the colours. Change the wording. Replace the icons with ones you like. The upgrade challenge is your friend; finishing it is where most learning actually happens.

How to Use AI as Your Coding Partner

An AI coding assistant is not magic. It's a very patient tutor that will read your code, point out mistakes, suggest fixes and rewrite parts on request. It is not a replacement for understanding — but it is an excellent amplifier when you already know what you want to build.

Use this three-step pattern whenever you start a project with AI:

1. Describe the goal as if you were writing for a junior developer. Example: 'I want a webpage where the user types a temperature and clicks a button, and the page shows one of five mood messages based on the temperature range.' The AI prompt section at the top of every project in this book shows you how to do this.
2. Describe the constraints up-front. Tell the AI which language you want to use, which technology is allowed (vanilla JS only, no React), and any files or APIs you already have. Constraints save you time because the AI doesn't propose solutions you can't use.
3. Describe UX details. Colour scheme, button labels, what happens when something goes wrong, what data the user sees. The more specific you are, the better the first answer the AI gives you.
4. When something breaks, paste the actual error message into the AI along with the smallest possible code snippet. 'I'm getting Uncaught TypeError in line 42' is much more useful than 'my code doesn't work'.
5. When the AI gives you code, run it first. If it works, change one small thing at a time to understand what the code does. If it works perfectly but you don't know why, ask the AI to explain each line.

How to Debug When AI Gives You Broken Code

Sometimes the AI's first answer is wrong. That is normal. Code is correct only as long as the human reviewing it is paying attention. Here's a reliable workflow.

1. Read the error message from the bottom up. The last line is usually the actual error name and message. Above it are the lines of your code the error refers to. The very top tells you which function caused it.
2. Common mistakes to look for first: missing semicolons (in JavaScript-using-parameters this is rare), unmatched brackets ({[()]}), mistyped variable names, typos in function names, missing commas in object declarations, and missing closing tags in HTML.
3. Open the browser's developer console (F12 or right-click → Inspect → Console). Almost every JavaScript error shows up there with a line number. The line number tells you exactly where to look.
4. If the AI's code uses a library you don't know, ask it: 'Explain this code line by line as if I am a complete beginner, and tell me which parts would not work in a plain HTML file.' Usually the answer is a single line you need to delete.
5. When in doubt, restart. Save a copy of the broken code, then ask the AI: 'Rewrite this project from scratch using only vanilla HTML, CSS and JavaScript in a single file.' A clean rewrite is faster than fixing a tangled first attempt.

How to Modify Any Project

Every project in this book has a Customization Ideas list with three small tweaks. Try them in order from simplest to hardest; each one teaches a slightly larger idea than the last.

After every project, ask yourself: 'What would I change so this matches my taste?' The answer might be colours, wording or layout. Apply that change, then write down what you changed. This habit is the single fastest way to grow as a builder.

Finally, after the three customisation ideas, do the Upgrade Challenge. The upgrade challenges are deliberately a bit harder than the project itself. They are designed to push you into territory where you have to read and write more on your own, or consult an AI assistant to learn something new.

Recommended Free Tools for Beginners

You don't need to buy anything to build everything in this book. These are the tools you'll use most, and they're all free.

1. **A modern web browser**

Chrome, Edge, Firefox or Safari. All four work. Pick one and learn where its developer tools live (F12 or Cmd+Opt+I on Mac).

1. **A code editor**

Visual Studio Code (free, popular, beginner-friendly). Alternatives: Sublime Text, Notepad++, or even the simplest text app on your computer — many of the early projects work just fine in NotePad.

1. **An AI coding assistant**

ChatGPT, Claude or Gemini (free tiers are enough for this book). The book assumes you have at least one of these open in another tab while you build.

1. **A folder on your computer**

Make a folder called 100-projects. Inside it, create 100 small folders, one per project. Save each project's HTML file inside its own folder. This way nothing gets lost when you start something new.

1. **A smartphone (optional)**

Several of the early projects work fine on a phone using an app like Replit, an online HTML preview tool, or any cloud-based editor that runs in mobile Safari or Chrome.

Part 1 — Super Easy Projects

20 quick projects to start with. Each one is short, runnable in your browser, and teaches a single new idea.

Project 1: Hello Name Greeter

Difficulty: ☆

What you will build

Build a tiny webpage that asks for a name and greets the visitor with a personalized message.

Skills you will learn

- HTML form
- JavaScript basics
- DOM selection

Features

- Text input field
- Greet button
- Live preview as you type

Build it with AI — the prompt to paste

> *You are a patient coding tutor. Create a single-file HTML page called 'Hello Name Greeter'. Requirements: a text input where the user types their name, a button labelled 'Greet me', and a paragraph below that shows 'Hello, <name>!' when clicked. Also update the greeting live as the user types. Use only vanilla HTML, CSS and JavaScript inside one file. Add beginner-friendly comments explaining each line.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Hello Name Greeter</title>  6    <style>  7      body { font-family:
system-ui, sans-serif; max-width: 480px; margin: 40px auto; padding: 20px; }  8
h1 { color: #2563eb; }  9      input { padding: 10px; font-size: 16px; width:
100%; box-sizing: border-box; } 10      p.greeting { margin-top: 20px; font-
size: 20px; color: #0f172a; } 11    </style> 12  </head> 13  <body> 14
<h1>Hello Name Greeter</h1> 15    <p>Type your name below:</p> 16    <input
id="nameInput" placeholder="e.g. Aisha"> 17    <p class="greeting"
id="output">Hello, friend!</p> 18   19    <script> 20      // Grab references to
the input and the output paragraph 21      const input =
document.getElementById("nameInput"); 22      const output =
document.getElementById("output"); 23   24      // Update greeting live whenever
the user types 25      input.addEventListener("input", () => { 26        const
name = input.value.trim(); 27        output.textContent = name ? `Hello, $
{name}!` : "Hello, friend!"; 28      }); 29    </script> 30  </body> 31  </html>
```

File structure

```
single file: greeter.html
```

How to run it

Open the file in any web browser (double-click it).

How it works

The browser reads the HTML for structure, the CSS for styling, and the JavaScript inside the <script> tag for behaviour. addEventListener('input', ...) fires every time the input value changes, then we update the paragraph's text using template strings with backticks.

Customization ideas

- Add a colour picker that tints the greeting
- Greet in multiple languages using a dropdown
- Add a 'clear' button

🕹 Upgrade Challenge

> *Keep a counter showing how many times the user has been greeted this session.*

Project 2: Age in Days Calculator

Difficulty: ☆

What you will build

Take a birth year as input and show how many days old the person is, with a fun twist for upcoming birthdays.

Skills you will learn

- Basic arithmetic
- Date math
- Form handling

Features

- Birth year input
- Days-lived output
- Days until next birthday

Build it with AI — the prompt to paste

> *Create a single-file HTML page called 'Age in Days Calculator'. It takes a birth year, current year, and birth month/day from the user and outputs (a) approximate days lived assuming 365.25 days/year, (b) exact days lived until today, (c) days until the next birthday. Validate that the year is between 1900 and the current year. Show errors in red. Use only vanilla JavaScript, no libraries.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Age in Days</title>  6    <style>  7      body { font-family: system-ui;
max-width: 420px; margin: 40px auto; }  8      label { display:block; margin-
top: 12px; font-weight: 600; }  9      input { width: 100%; padding: 8px; font-
size: 14px; } 10      button { margin-top: 16px; padding: 10px 16px; background:
#2563eb; color: white; border: 0; border-radius: 6px; cursor: pointer; } 11
.output { margin-top: 20px; padding: 12px; background: #f1f5f9; border-radius:
6px; } 12      .err { color: #b91c1c; } 13    </style> 14  </head> 15  <body> 16
<h1>Age in Days</h1> 17    <label>Birth year <input id="year" type="number"
placeholder="2010"></label> 18    <label>Birth month (1-12) <input id="month"
type="number" placeholder="1" min="1" max="12"></label> 19    <label>Birth day
(1-31) <input id="day" type="number" placeholder="15" min="1" max="31"></label>
20    <button id="go">Calculate</button> 21    <div class="output"
id="out"></div> 22   23    <script> 24      const out =
document.getElementById("out"); 25
document.getElementById("go").addEventListener("click", () => { 26        const
y = parseInt(document.getElementById("year").value); 27        const m =
parseInt(document.getElementById("month").value); 28        const d =
parseInt(document.getElementById("day").value); 29        const current = new
Date(); 30   31        if (!y || !m || !d || y < 1900 || y >
current.getFullYear()) { 32          out.innerHTML = '<span class="err">Please
enter valid numbers.</span>'; 33          return; 34        } 35   36
const birth = new Date(y, m - 1, d); 37        const ms = current - birth; 38
const daysLived = Math.floor(ms / 86400000); 39        const daysEstimate =
Math.floor((current.getFullYear() - y) * 365.25); 40   41        // Days until
```

```
next birthday 42        let next = new Date(current.getFullYear(), m - 1, d); 43
if (next < current) next = new Date(current.getFullYear() + 1, m - 1, d); 44
const daysToNext = Math.ceil((next - current) / 86400000); 45   46
out.innerHTML = 47          "Approximate days lived: <b>" + daysEstimate +
"</b><br>" + 48          "Exact days lived: <b>" + daysLived + "</b><br>" + 49
"Days until next birthday: <b>" + daysToNext + "</b>"; 50      });
  1    </script>  2  </body>  3  </html>
```

File structure

```
single file: age-in-days.html
```

How to run it

Open the file in any browser.

How it works

We use the Date object to compare two points in time. Subtracting two Date objects gives milliseconds; dividing by 86,400,000 (the number of milliseconds in a day) gives days.

Customization ideas

- Add hours/minutes lived
- Show seconds as a live counter
- Display the weekday of birth

🕹 Upgrade Challenge

> *Display the number of full moons lived (assume one every 29.53 days).*

Project 3: Days Until Summer

Difficulty: ☆

What you will build

A countdown page showing how many days, hours and minutes remain until the next summer solstice.

Skills you will learn

- Date arithmetic
- Conditional logic

Features

- Live countdown
- Updates every minute
- Encouraging messages

Build it with AI — the prompt to paste

> *Build a single-file 'Days Until Summer' countdown page. Compute days, hours and minutes from now until June 21 of the current year. If June 21 has already passed this year, count down to next year. Add three encouraging messages that change based on how close the date is. Refresh every minute. Pure HTML/CSS/JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Days Until Summer</title>  6    <style>  7      body { font-family:
system-ui; background: linear-gradient(180deg,#fef3c7,#fde68a); text-align:
center; padding: 60px 20px; }  8      h1 { font-size: 28px; }  9      .count
{ font-size: 48px; font-weight: 800; color: #b45309; margin: 20px 0; }
10      .msg { font-size: 18px; color: #78350f; } 11    </style> 12  </head> 13
<body> 14    <h1>☀ Days Until Summer</h1> 15    <div class="count" id="count">--
</div> 16    <div class="msg" id="msg">Counting down...</div> 17   18
<script> 19      function daysUntilSummer() { 20        const now = new Date();
21        let target = new Date(now.getFullYear(), 5, 21); // June = month 5 22
if (target < now) target = new Date(now.getFullYear() + 1, 5, 21); 23
const diffMs = target - now; 24        const days = Math.floor(diffMs /
86400000); 25        const hours = Math.floor((diffMs % 86400000) / 3600000); 26
const mins = Math.floor((diffMs % 3600000) / 60000); 27        return { days,
hours, mins }; 28      } 29   30      function paint() { 31        const t =
daysUntilSummer(); 32        document.getElementById("count").textContent = `$
{t.days}d ${t.hours}h ${t.mins}m`; 33        let msg = "Hang in there!"; 34
if (t.days < 7) msg = "□ Almost summer!"; 35        else if (t.days < 30) msg =
"Soon!"; 36        else if (t.days < 100) msg = "Plan that trip!"; 37
document.getElementById("msg").textContent = msg; 38      } 39   40
paint(); 41      setInterval(paint, 60000); // refresh every minute 42
</script> 43  </body> 44  </html>
```

File structure

```
single file: days-until-summer.html
```

How to run it

Open in any browser.

How it works

We compute the next summer solstice with the Date constructor (months are 0-indexed). setInterval fires the same paint function every minute, so the display stays close to current.

Customization ideas

- Count down to your next birthday instead
- Add seconds too
- Show a progress bar

🕹 Upgrade Challenge

> *Let the user pick any future date and count down to that instead.*

Project 4: Random Compliment Generator

Difficulty: ☆

What you will build

Click a button and receive a friendly, original compliment from a curated list.

Skills you will learn

- Arrays
- Random selection
- Event handling

Features

- Button-triggered compliment
- Animation on click
- Original phrasing only

Build it with AI — the prompt to paste

> *Make a single-file 'Random Compliment Generator'. Show a button labelled 'Cheer me up'. When clicked, display a random compliment chosen from an array of at least 20 positive, original short messages such as 'Your curiosity is your superpower.' No internet, no APIs, only vanilla JS. Add a small fade animation.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Random Compliment</title>  6    <style>  7      body { font-family:
system-ui; max-width: 520px; margin: 60px auto; text-align: center; background:
#fef9c3; min-height: 100vh; padding: 20px; }  8      h1 { color: #a16207; }  9
.card { background: white; padding: 30px; border-radius: 14px; box-shadow: 0 4px
14px rgba(0,0,0,.08); font-size: 22px; min-height: 80px; opacity: 1; transition:
opacity .4s; } 10      .fade { opacity: 0; } 11      button { margin-top: 20px;
padding: 12px 22px; background: #ca8a04; color: white; border: 0; border-radius:
8px; font-size: 16px; cursor: pointer; } 12      button:hover { background:
#a16207; } 13    </style> 14  </head> 15  <body> 16    <h1>Cheer Me Up □</h1> 17
<div class="card" id="card">Click the button for a kind word.</div> 18
<button id="btn">Cheer me up</button> 19   20    <script> 21      const
compliments = [ 22        "Your curiosity is your superpower.", 23        "You
learn things faster than you think.", 24        "Small steps still move you
forward.", 25        "You make problems smaller just by trying.", 26
"Your ideas deserve to be heard.", 27        "You are getting better every day,
quietly.", 28        "Rest is also progress.", 29        "You are allowed to
start over.", 30        "Your effort matters more than your speed.", 31
"Trying again is brave.", 32        "You handle more than you realise.", 33
"Your future self is cheering for you.", 34        "Done is better than
perfect.", 35        "You don't need anyone's permission to start.", 36
"Mistakes are just data with feelings.", 37        "You bring something no one
else can.", 38        "Today is a fine day to figure things out.", 39
"You are not behind.", 40        "Your questions are valuable.", 41
"Showing up counts." 42      ]; 43   44      const card =
document.getElementById("card"); 45
document.getElementById("btn").addEventListener("click", () => { 46
```

```
card.classList.add("fade"); 47        setTimeout(() => { 48          const i =
Math.floor(Math.random() * compliments.length); 49          card.textContent =
compliments[i]; 50          card.classList.remove("fade");
  1        }, 400);  2      });  3    </script>  4  </body>  5  </html>
```

File structure

```
single file: compliment.html
```

How to run it

Open in any browser.

How it works

Math.random() returns a number between 0 and 1. Multiplying by array length and flooring gives a valid random index. We briefly hide the card, pick a new compliment, then show it again — that creates a fade.

Customization ideas

- Add a 'copy to clipboard' button
- Let users add their own compliments
- Pick a compliment every 10 seconds automatically

🕹 Upgrade Challenge

> *Build a 'kindness jar' that saves favourites to localStorage so the reader can revisit them.*

Project 5: Colour of the Day

Difficulty: ☆

What you will build

Each day of the week gets its own colour, and the page paints the screen with it.

Skills you will learn

- Date.getDay()
- Switch-style logic
- CSS backgrounds

Features

- Day-based colour
- Big colour swatch display
- Hex code shown

Build it with AI — the prompt to paste

> *Build 'Colour of the Day'. Determine the day of the week using new Date().getDay() and assign a unique colour to each day. Display the colour as a full-screen block, show the day name, and show the hex code. Auto-detect the user's day on load, no buttons. Single file. Pure HTML/CSS/JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Colour of the Day</title>  6    <style>  7      body { margin: 0; min-
height: 100vh; display:flex; flex-direction:column; align-items:center; justify-
content:center; font-family: system-ui; transition: background .6s; }  8      h1
{ color: white; font-size: 40px; text-shadow: 0 2px 6px rgba(0,0,0,.3); }  9
p.color { color: white; font-size: 28px; font-family: monospace; } 10
</style> 11  </head> 12  <body> 13    <h1 id="day"></h1> 14    <p class="color"
id="hex"></p> 15   16    <script> 17      const days = [ 18        { name:
"Sunday",    hex: "#fb7185" }, 19        { name: "Monday",    hex: "#60a5fa" },
20        { name: "Tuesday",   hex: "#34d399" }, 21        { name: "Wednesday",
hex: "#a78bfa" }, 22        { name: "Thursday",  hex: "#fbbf24" }, 23
{ name: "Friday",    hex: "#f472b6" }, 24        { name: "Saturday",  hex:
"#22d3ee" } 25      ]; 26      const today = days[new Date().getDay()]; 27
document.body.style.background = today.hex; 28
document.getElementById("day").textContent = today.name; 29
document.getElementById("hex").textContent = today.hex; 30    </script> 31
</body> 32  </html>
```

File structure

```
single file: colour-of-day.html
```

How to run it

Open in any browser.

How it works

getDay() returns 0 for Sunday through 6 for Saturday. We use that as an index into the days array. CSS transition fades between any background changes.

Customization ideas

- Add a button to cycle the colour manually
- Show a complementary colour as well
- Animate a subtle pulse

🕹 Upgrade Challenge

> *Show two extra 'lucky colours' that are mathematically matched using hue rotation in HSL.*

Project 6: Even or Odd Checker

Difficulty: ☆

What you will build

Type an integer and instantly see whether it is even or odd with a short explanation.

Skills you will learn

- Modulo operator
- Conditional rendering
- Input validation

Features

- Instant feedback as you type
- Handles negative numbers
- Educational note on mod 2

Build it with AI — the prompt to paste

> *Make a single-file 'Even or Odd Checker'. As the user types an integer, instantly display either 'Even' or 'Odd' beneath the input. Show a small explanation: 'Even numbers are divisible by 2 with no remainder.' If the input is not a valid integer (e.g. 3.5 or 'abc'), show 'Not a valid integer'. Pure vanilla JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Even or Odd</title>  6    <style>  7      body { font-family: system-ui;
max-width: 400px; margin: 60px auto; padding: 20px; }  8      input { width:
100%; padding: 10px; font-size: 18px; box-sizing: border-box; }  9      .result
{ margin-top: 16px; padding: 14px; border-radius: 8px; font-size: 18px; text-
align: center; } 10      .even { background: #dcfce7; color: #166534; }
11      .odd { background: #fef9c3; color: #854d0e; } 12      .err { background:
#fee2e2; color: #991b1b; } 13      .note { font-size: 13px; color: #475569;
margin-top: 14px; } 14    </style> 15  </head> 16  <body> 17    <h1>Even or Odd?
</h1> 18    <input id="num" type="number" placeholder="Type any integer..."> 19
<div class="result" id="result">Enter a number above.</div> 20    <p
class="note">Even numbers leave no remainder when divided by 2. Odd numbers
leave 1.</p> 21   22    <script> 23      const num =
document.getElementById("num"); 24      const result =
document.getElementById("result"); 25   26      num.addEventListener("input", ()
=> { 27        const v = num.value.trim(); 28        if (v === "" || isNaN(v) ||
!Number.isInteger(Number(v))) { 29          result.textContent = "Not a valid
integer."; 30          result.className = "result err"; 31          return; 32
} 33        const n = Number(v); 34        if (n % 2 === 0) { 35
result.textContent = n + " is Even ✓"; 36          result.className = "result
even"; 37        } else { 38          result.textContent = n + " is Odd ✗"; 39
result.className = "result odd"; 40        } 41      }); 42    </script> 43
</body> 44  </html>
```

File structure

```
single file: even-odd.html
```

How to run it

Open in any browser.

How it works

The % (modulo) operator returns the remainder after division. If n % 2 equals 0, the number is divisible by 2 and therefore even. Otherwise it is odd.

Customization ideas

- Show whether the number is divisible by 3, 5, 10 too
- Convert to binary and show the last bit
- Let the user test multiple numbers

🕹 Upgrade Challenge

> *Add a 'prime checker' panel using the same input.*

Project 7: Temperature Mood Translator

Difficulty: ☆

What you will build

Type the current temperature in Celsius and get a friendly, kid-safe 'weather mood' message.

Skills you will learn

- Conditionals
- String concatenation

Features

- Celsius input
- Friendly mood messages
- Emoji indicators

Build it with AI — the prompt to paste

> *Build a single-file "Temperature Mood Translator". User enters a temperature in °C and the page shows a moody one-liner such as 'Sweater weather ☕' for 15°C, 'Time for ice cream 🍦' for 30°C. Buckets: below 0, 0-10, 11-20, 21-30, above 30. Add an emoji. Pure HTML/CSS/JS one file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Temperature Mood</title>  6    <style>  7      body { font-family:
system-ui; max-width: 420px; margin: 60px auto; padding: 20px; text-align:
center; }  8      input { padding: 10px; font-size: 18px; width: 100%; box-
sizing: border-box; }  9      .mood { margin-top: 20px; padding: 20px;
background: #f1f5f9; border-radius: 12px; font-size: 22px; } 10    </style> 11
</head> 12  <body> 13    <h1>Temperature Mood</h1> 14    <p>Enter a temperature
in °C</p> 15    <input id="t" type="number" placeholder="e.g. 22"> 16    <div
class="mood" id="mood">Tell me the temperature.</div> 17   18    <script> 19
document.getElementById("t").addEventListener("input", () => { 20        const t
= parseFloat(document.getElementById("t").value); 21        let msg = "Pick a
number first."; 22        if (isNaN(t)) msg = "Pick a number first."; 23
else if (t < 0) msg = "🥶 Freezing - hot chocolate weather."; 24        else if
(t <= 10) msg = "🧥 Coat weather - layer up."; 25        else if (t <= 20) msg =
"☕ Sweater weather."; 26        else if (t <= 30) msg = "👕 T-shirt weather -
enjoy!"; 27        else msg = "🍦 Ice cream weather - stay hydrated."; 28
document.getElementById("mood").textContent = msg; 29      }); 30    </script>
31  </body> 32  </html>
```

File structure

```
single file: temp-mood.html
```

How to run it

Open in any browser.

How it works

We chain 'if/else if' conditions in order from coldest to hottest — only the first matching one runs, so each bucket gets a unique temperature range.

Customization ideas

- Add a Fahrenheit toggle
- Show the humidity mood too (user enters %)
- Show a colour-coded gradient

🕹 Upgrade Challenge

> *Save the user's last 5 inputs in localStorage.*

Project 8: Reverse a Word

Difficulty: ☆

What you will build

Type a word and instantly see it reversed, with palindrome detection.

Skills you will learn

- String methods
- Loops
- Comparison logic

Features

- Live reverse display
- Palindrome detection
- Length counter

Build it with AI — the prompt to paste

> *Build 'Reverse a Word'. As the user types, show the word reversed character by character, detect whether the original is a palindrome ('madam', 'level'), and show character + word counts. One file. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Reverse a Word</title>  6    <style>  7      body { font-family: system-
ui; max-width: 480px; margin: 50px auto; padding: 20px; }  8      input { width:
100%; padding: 10px; font-size: 18px; box-sizing: border-box; }  9      .panel {
margin-top: 20px; padding: 16px; background: #f8fafc; border-radius: 10px; } 10
.big { font-size: 26px; word-break: break-all; } 11      .info { color: #475569;
margin-top: 10px; font-size: 14px; } 12      .palin { color: #16a34a; font-
weight: 600; } 13    </style> 14  </head> 15  <body> 16    <h1>Reverse a
Word</h1> 17    <input id="w" placeholder="Type a word..."> 18    <div
class="panel"> 19      <div class="big" id="rev"></div> 20      <div
class="info" id="info"></div> 21    </div> 22   23    <script> 24      const w =
document.getElementById("w"); 25      const rev =
document.getElementById("rev"); 26      const info =
document.getElementById("info"); 27   28      function reverseString(s) { 29
let out = ""; 30        for (let i = s.length - 1; i >= 0; i--) out += s[i]; 31
return out; 32      } 33   34      w.addEventListener("input", () => { 35
const original = w.value.trim(); 36        if (!original) { rev.textContent =
""; info.textContent = ""; return; } 37        const reversed =
reverseString(original); 38        const isPalin = original.toLowerCase() ===
reversed.toLowerCase(); 39        rev.textContent = reversed; 40        let html
= `${original.length} characters · ${original.split(/\s+/).length} word(s)`; 41
if (isPalin) html += ` · <span class="palin">Palindrome!</span>`; 42
info.innerHTML = html; 43      }); 44    </script> 45  </body> 46  </html>
```

File structure

```
single file: reverse.html
```

How to run it

Open in any browser.

How it works

We loop from the last index to 0 and build a new string. For palindrome check, we compare the lowercase original to the lowercase reversed so 'Madam' still matches.

Customization ideas

- Reverse whole sentences, not just words
- Add an animation that flips each letter
- Highlight vowel count

🕹 Upgrade Challenge

> *Detect palindromes in any sentence (ignoring spaces and punctuation).*

Project 9: Simple Counter with Limits

Difficulty: ☆

What you will build

A counter with + and - buttons that prevents going below zero or above a chosen limit.

Skills you will learn

- useState-style state management
- Event listeners
- Conditions

Features

- Increment and decrement
- Configurable max
- Reset button
- Disable buttons at limits

Build it with AI — the prompt to paste

> *Build 'Simple Counter with Limits'. UI has two buttons, '+' and '-', a current count display, a number input for the max value, and a reset button. Disable the '-' button when the count is 0 and the '+' button when the count hits the max. Reset returns to zero. Single file, vanilla JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Limited Counter</title>  6    <style>  7      body { font-family: system-
ui; max-width: 360px; margin: 60px auto; text-align: center; }  8      .count
{ font-size: 64px; font-weight: 800; color: #1d4ed8; margin: 20px 0; }  9
button { font-size: 24px; padding: 10px 22px; margin: 0 6px; border-radius: 8px;
border: 0; background: #1d4ed8; color: white; cursor: pointer; } 10
button:disabled { background: #94a3b8; cursor: not-allowed; } 11
input[type=number] { width: 80px; padding: 6px; font-size: 16px; } 12
</style> 13  </head> 14  <body> 15    <h1>Counter</h1> 16    <label>Max: <input
id="max" type="number" value="10" min="1"></label> 17    <div class="count"
id="c">0</div> 18    <button id="minus">−</button> 19    <button
id="plus">+</button> 20    <p><button id="reset">Reset</button></p> 21   22
<script> 23      let count = 0; 24      const c = document.getElementById("c");
25      const plus = document.getElementById("plus"); 26      const minus =
document.getElementById("minus"); 27      const maxInp =
document.getElementById("max"); 28   29      function paint() { 30
c.textContent = count; 31        const max = parseInt(maxInp.value) || 1; 32
plus.disabled = count >= max; 33        minus.disabled = count <= 0; 34      }
35   36      plus.addEventListener("click", () => { count++; paint(); }); 37
minus.addEventListener("click", () => { count--; paint(); }); 38
document.getElementById("reset").addEventListener("click", () => { count = 0;
paint(); }); 39      maxInp.addEventListener("input", paint); 40      paint();
41    </script> 42  </body> 43  </html>
```

File structure

```
single file: counter.html
```

How to run it

Open in any browser.

How it works

We store the count in a variable and re-render the UI inside paint(). Setting disabled = true on a button automatically greys it out and blocks clicks via the browser.

Customization ideas

- Add +5 / -5 jump buttons
- Vibrate the screen (mobile) at the limit
- Persist the count with localStorage

🕹 Upgrade Challenge

> *Make the counter wrap around (max + 1 → 0, 0 − 1 → max).*

Project 10: Tip Calculator

Difficulty: ☆

What you will build

Calculate tip and total bill from a bill amount and a percentage.

Skills you will learn

- Arithmetic
- Input handling
- Number formatting

Features

- Bill amount input
- Tip percentage input
- Per-person split
- Currency formatting

Build it with AI — the prompt to paste

> *Build a single-file 'Tip Calculator'. Inputs: bill amount (positive number), tip percentage (slider 0-50%), number of people (number input, default 1). Outputs: tip amount, total bill, and per-person amount. Use .toFixed(2) for currency. Validate that bill and people are > 0. Vanilla JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Tip Calculator</title>  6    <style>  7      body { font-family: system-
ui; max-width: 420px; margin: 40px auto; padding: 20px; background: #f8fafc; }
8      .row { display: flex; justify-content: space-between; margin: 8px 0; }  9
label { font-weight: 600; } 10      input[type=number] { padding: 6px; width:
100px; } 11      input[type=range] { width: 100%; } 12      .box { margin-top:
20px; padding: 14px; background: white; border-radius: 10px; } 13      .total
{ font-size: 22px; font-weight: 700; color: #0f766e; } 14    </style> 15
</head> 16  <body> 17    <h1>□ Tip Calculator</h1> 18    <div
class="row"><label>Bill ₹</label><input id="bill" type="number" value=""></div>
19    <div class="row"><label>Tip %</label><span id="pctLbl">15%</span></div> 20
<input id="pct" type="range" min="0" max="50" value="15"> 21    <div
class="row"><label>People</label><input id="people" type="number" value="1"
min="1"></div> 22   23    <div class="box"> 24      <div
class="row"><span>Tip</span><span id="tip">₹0.00</span></div> 25      <div
class="row"><span>Total</span><span id="total">₹0.00</span></div> 26      <hr>
27      <div class="row total"><span>Per person</span><span
id="per">₹0.00</span></div> 28    </div> 29   30    <script> 31      const bill
= document.getElementById("bill"); 32      const pct =
document.getElementById("pct"); 33      const pctLbl =
document.getElementById("pctLbl"); 34      const people =
document.getElementById("people"); 35   36      function fmt(n) { return "₹" +
n.toFixed(2); } 37   38      function update() { 39        pctLbl.textContent =
pct.value + "%"; 40        const b = parseFloat(bill.value) || 0; 41
const p = people.value <= 0 ? 1 : parseInt(people.value); 42        const tip =
b * (pct.value / 100); 43        const total = b + tip; 44
tip.textContent = fmt(tip);  // (repurposed - intentional bug fix below) 45   46
```

```
document.getElementById("tip").textContent = fmt(tip); 47
document.getElementById("total").textContent = fmt(total); 48
document.getElementById("per").textContent = fmt(total / p); 49      } 50
  1      [bill, pct, people].forEach(el => el.addEventListener("input",
update));  2      update();  3    </script>  4  </body>  5  </html>
```

File structure

```
single file: tip.html
```

How to run it

Open in any browser.

How it works

Tip = bill × percentage / 100. Total = bill + tip. Per-person = total / people. toFixed(2) keeps two decimal places, perfect for currency display.

Customization ideas

- Add quick buttons (10%, 15%, 18%, 20%)
- Round up option for per-person
- Remember the last tip % chosen

🕹 Upgrade Challenge

> *Add a 'split quality check' that warns if anyone has to pay more than 60% of the bill.*

Project 11: Dice Roller Simulator

Difficulty: ☆

What you will build

Click to roll any number of dice. Each die shows dots and the total is summed.

Skills you will learn

- Loops
- Random numbers
- DOM building

Features

- Roll 1-10 dice
- SVG dot faces
- Roll history
- Total sum

Build it with AI — the prompt to paste

> *Make 'Dice Roller Simulator'. User picks how many dice to roll (1-10 using a slider). Each click rolls all dice, drawing pips on a 3x3 grid for each die. Show the sum. Use SVG circles for the dots. Vanilla JS only.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Dice Roller</title>  6    <style>  7      body { font-family: system-ui;
max-width: 700px; margin: 30px auto; text-align: center; }  8      .dice
{ display: inline-block; width: 60px; height: 60px; background: white; border:
2px solid #0f172a; border-radius: 10px; margin: 6px; position: relative; }  9
.pip { width: 10px; height: 10px; background: #0f172a; border-radius: 50%;
position: absolute; } 10      .p1 { top: 25px; left: 25px; } 11      .p2 { top:
10px; left: 10px; } .p2b { top: 40px; left: 40px; } 12      .p3 { top: 10px;
left: 10px; } .p3b { top: 25px; left: 25px; } .p3c { top: 40px; left: 40px; } 13
.p4 { top: 10px; left: 10px; } .p4b { top: 10px; left: 40px; } .p4c { top: 40px;
left: 10px; } .p4d { top: 40px; left: 40px; } 14      .p5 { top: 40px; left:
40px; } 15      .p6 { top: 10px; left: 10px; } .p6b { top: 10px; left:
40px; } .p6c { top: 30px; left: 10px; } .p6d { top: 30px; left: 40px; } .p6e
{ top: 50px; left: 10px; } .p6f { top: 50px; left: 40px; } 16   17      button {
margin-top: 20px; padding: 12px 28px; font-size: 18px; background: #16a34a;
color: white; border: 0; border-radius: 8px; cursor: pointer; } 18      .sum
{ font-size: 24px; font-weight: 700; margin: 14px 0; } 19    </style> 20
</head> 21  <body> 22    <h1>🎲 Dice Roller</h1> 23    <label>Number of dice:
<input id="n" type="range" min="1" max="10" value="2"> 24      <span
id="nLbl">2</span> 25    </label> 26    <div id="board"></div> 27    <div
class="sum" id="sum">Total: 0</div> 28    <button id="roll">Roll!</button> 29
30    <script> 31      const layouts = { 32        1: ["p1"], 33        2:
["p2","p2b"], 34        3: ["p3","p3b","p3c"], 35        4:
["p4","p4b","p4c","p4d"], 36        5: ["p4","p4b","p4c","p4d","p1"], 37
6: ["p6","p6b","p6c","p6d","p6e","p6f"] 38      }; 39      const board =
document.getElementById("board"); 40      const sumEl =
document.getElementById("sum"); 41      const n = document.getElementById("n");
42      const nLbl = document.getElementById("nLbl"); 43   44      function
makeDie(value) { 45        const d = document.createElement("div"); 46
```

```
d.className = "dice"; 47        layouts[value].forEach(cls => { 48
const p = document.createElement("div"); 49          p.className = "pip " + cls;
50          d.appendChild(p);
  1        });  2        return d;  3      }  4    5      function roll() {  6
board.innerHTML = "";  7        const count = parseInt(n.value);  8        let
total = 0;  9        for (let i = 0; i < count; i++) { 10          const v = 1 +
Math.floor(Math.random() * 6); 11          total += v; 12
board.appendChild(makeDie(v)); 13        } 14        sumEl.textContent = "Total:
" + total; 15      } 16   17      n.addEventListener("input", () =>
nLbl.textContent = n.value); 18
document.getElementById("roll").addEventListener("click", roll); 19      roll();
20    </script> 21  </body> 22  </html>
```

File structure

```
single file: dice.html
```

How to run it

Open in any browser.

How it works

1 + Math.floor(Math.random() * 6) gives a random integer between 1 and 6, inclusive. We pick one of the predefined layouts for each value and place dot divs inside a styled square.

Customization ideas

- Add a 'roll until doubles' mode
- Animate dice while rolling
- Allow user-defined sides (4, 8, 20)

🕹 Upgrade Challenge

> *Add a probability chart: roll 100 dice in a background loop and display each face's frequency.*

Project 12: BMI Calculator

Difficulty: ☆

What you will build

Compute Body Mass Index from height and weight with a clear category label.

Skills you will learn

- Forms
- Math
- Conditional categories

Features

- Metric units
- Category explanation
- Healthy-range visual

Build it with AI — the prompt to paste

> *Build 'BMI Calculator'. Inputs: weight in kilograms and height in centimetres. Output: BMI value (weight / height²), a category label (Underweight / Normal / Overweight / Obese), and a short explanation. Show a colour-coded bar. No data storage, no internet. Single file, vanilla JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>BMI Calculator</title>  6    <style>  7      body { font-family: system-
ui; max-width: 420px; margin: 50px auto; padding: 20px; }  8      label
{ display: block; margin: 10px 0 4px; font-weight: 600; }  9      input { width:
100%; padding: 8px; box-sizing: border-box; } 10      .box { margin-top: 20px;
padding: 16px; background: #f1f5f9; border-radius: 10px; } 11      .val { font-
size: 32px; font-weight: 700; } 12      .cat { font-weight: 700; padding: 6px
10px; display: inline-block; border-radius: 6px; } 13      .bar { height: 12px;
background: linear-gradient(90deg,#0ea5e9 25%,#16a34a 25% 50%,#f59e0b 50%
75%,#dc2626 75%); border-radius: 6px; position: relative; } 14      .marker
{ position: absolute; top: -4px; width: 4px; height: 20px; background:
#0f172a; } 15    </style> 16  </head> 17  <body> 18    <h1>BMI Calculator</h1>
19    <label>Weight (kg) <input id="w" type="number" placeholder="60"></label>
20    <label>Height (cm) <input id="h" type="number" placeholder="170"></label>
21    <div class="box"> 22      <div class="val" id="val">-</div> 23      <div
class="cat" id="cat">--</div> 24      <p id="note">Enter values to see your
BMI.</p> 25      <div class="bar"><div class="marker" id="marker"
style="left:0"></div></div> 26    </div> 27   28    <script> 29      const w =
document.getElementById("w"), h = document.getElementById("h"); 30      const
val = document.getElementById("val"), cat = document.getElementById("cat"); 31
const note = document.getElementById("note"); 32      const marker =
document.getElementById("marker"); 33   34      function update() { 35
const wv = parseFloat(w.value), hv = parseFloat(h.value); 36        if (!wv || !
hv || hv < 30) { 37          val.textContent = "-"; cat.textContent = "--";
cat.style.background = ""; 38          note.textContent = "Enter your height and
weight."; marker.style.left = "0"; 39          return; 40        } 41
const m = hv / 100; 42        const bmi = wv / (m * m); 43
val.textContent = bmi.toFixed(1); 44        // Position marker on bar (BMI 15 ->
0%, 35 -> 100%) 45        let pos = ((bmi - 15) / 20) * 100; 46        pos =
```

```
Math.max(0, Math.min(100, pos)); 47        marker.style.left = pos + "%"; 48
49        let label, color, msg; 50        if (bmi < 18.5) { label =
"Underweight"; color = "#bae6fd"; msg = "Consider talking to a doctor or
nutritionist."; }
  1        else if (bmi < 25) { label = "Normal"; color = "#bbf7d0"; msg = "Keep
up your healthy routine!"; }  2        else if (bmi < 30) { label =
"Overweight"; color = "#fde68a"; msg = "Small daily habits can make a big
difference."; }  3        else { label = "Obese"; color = "#fecaca"; msg =
"Please consult a healthcare professional."; }  4    5        cat.textContent =
label; cat.style.background = color;  6        note.textContent = msg;  7      }
8    9      [w, h].forEach(el => el.addEventListener("input", update)); 10
</script> 11  </body> 12  </html>
```

File structure

```
single file: bmi.html
```

How to run it

Open in any browser.

How it works

BMI = weight (kg) / (height (m))². The square of a number in JS is just x * x. We divide the result into categories using the World Health Organization's standard thresholds.

Customization ideas

- Add imperial units (lb, in)
- Show ideal weight range for the height
- Track BMI over time in localStorage

🕹 Upgrade Challenge

> *Add a daily calorie estimator using the Mifflin-St Jeor equation.*

Project 13: Coin Toss Counter

Difficulty: ☆

What you will build

Flip coins and watch a live tally of Heads vs Tails as a small bar chart.

Skills you will learn

- Loops
- Arrays
- Updating counters

Features

- Animated counter
- Live bar chart
- Reset and auto-flip

Build it with AI — the prompt to paste

> *Build 'Coin Toss Counter'. A button 'Flip 100 times' runs an animation that flips several coins over a few seconds, updating Heads and Tails totals. A small bar chart visualises the ratio. Auto-flip option runs indefinitely until stopped. Vanilla JS, no libraries.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Coin Toss</title>  6    <style>  7      body { font-family: system-ui;
max-width: 480px; margin: 40px auto; padding: 20px; text-align: center; }  8
.tally { font-size: 28px; font-weight: 700; margin: 20px 0; }  9      .bar
{ display: flex; height: 30px; border-radius: 6px; overflow: hidden; background:
#e2e8f0; } 10      .heads { background: #facc15; text-align: center; color:
#422006; padding: 4px; } 11      .tails { background: #94a3b8; text-align:
center; color: white; padding: 4px; } 12      button { padding: 10px 20px;
margin: 6px; background: #2563eb; color: white; border: 0; border-radius: 6px;
cursor: pointer; } 13      button.danger { background: #dc2626; } 14    </style>
15  </head> 16  <body> 17    <h1>🪙 Coin Toss</h1> 18    <div
class="tally">Heads: <span id="h">0</span> · Tails: <span id="t">0</span></div>
19    <div class="bar"><div class="heads" id="hb"></div><div class="tails"
id="tb"></div></div> 20    <p><button id="flip1">Flip 1</button> 21
<button id="flip100">Flip 100</button> 22       <button
id="auto">Auto-flip</button> 23       <button id="reset"
class="danger">Reset</button></p> 24   25    <script> 26      let h = 0, t = 0;
27      const hEl = document.getElementById("h"), tEl =
document.getElementById("t"); 28      const hb = document.getElementById("hb"),
tb = document.getElementById("tb"); 29      let autoId = null; 30   31
function paint() { 32        hEl.textContent = h; tEl.textContent = t; 33
const total = h + t; 34        hb.style.width = (total ? h / total * 100 : 50) +
"%"; 35        tb.style.width = (total ? t / total * 100 : 50) + "%"; 36
hb.textContent = total ? Math.round(h / total * 100) + "%" : ""; 37
tb.textContent = total ? Math.round(t / total * 100) + "%" : ""; 38      } 39
40      function flip() { 41        if (Math.random() < 0.5) h++; else t++; 42
paint(); 43      } 44   45
document.getElementById("flip1").addEventListener("click", flip); 46
document.getElementById("flip100").addEventListener("click", () => { for (let i
```

```
= 0; i < 100; i++) flip(); }); 47
document.getElementById("reset").addEventListener("click", () => { h = 0; t = 0;
if (autoId) clearInterval(autoId); paint(); }); 48
document.getElementById("auto").addEventListener("click", () => { 49        if
(autoId) { clearInterval(autoId); autoId = null; } 50        else autoId =
setInterval(flip, 80);
  1      });  2    3      paint();  4    </script>  5  </body>  6  </html>
```

File structure

```
single file: coin.html
```

How to run it

Open in any browser.

How it works

Math.random() < 0.5 gives a 50/50 split. setInterval runs flip every 80 milliseconds while automatic mode is on, giving a live feel. Stopping clears the interval.

Customization ideas

- Add a 'flip until skew' button
- Save totals to localStorage
- Add streak detection (longest HH or TT run)

🕹 Upgrade Challenge

> *Simulate a biased coin: let the user set P(Heads) anywhere from 0 to 1.*

Project 14: Word Length Sorter

Difficulty: ☆

What you will build

Paste a sentence. Get back each word sorted by length.

Skills you will learn

- String split
- Array sort with comparator
- Mapping

Features

- Paste textarea
- Sort by length
- Show count

Build it with AI — the prompt to paste

> *Build 'Word Length Sorter'. Pastebox takes a sentence or paragraph. Button 'Sort by length' displays words from longest to shortest in a vertical list with the length beside each. Sort is case-insensitive. Pure JS. One file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Word Length Sorter</title>  6    <style>  7      body { font-family:
system-ui; max-width: 600px; margin: 40px auto; padding: 20px; }  8
textarea { width: 100%; min-height: 100px; padding: 10px; font-size: 15px; box-
sizing: border-box; }  9      button { padding: 10px 20px; margin-top: 10px;
background: #2563eb; color: white; border: 0; border-radius: 6px; cursor:
pointer; } 10      ol { margin-top: 20px; } 11      li { padding: 4px 0; } 12
.len { color: #64748b; font-size: 13px; margin-left: 6px; } 13    </style> 14
</head> 15  <body> 16    <h1>Word Length Sorter</h1> 17    <textarea id="text"
placeholder="Paste anything here..."></textarea> 18    <br> 19    <button
id="go">Sort by length</button> 20    <p id="info"></p> 21    <ol
id="list"></ol> 22   23    <script> 24      const text =
document.getElementById("text"); 25      const list =
document.getElementById("list"); 26      const info =
document.getElementById("info"); 27   28
document.getElementById("go").addEventListener("click", () => { 29        const
words = text.value.trim().split(/\s+/).filter(w => w.length); 30        if (!
words.length) { list.innerHTML = ""; info.textContent = ""; return; } 31
const sorted = words.slice().sort((a, b) => b.length - a.length); 32
info.textContent = `${words.length} words, longest ${sorted[0].length} chars.`;
33        list.innerHTML = sorted.map(w => `<li>${w}<span class="len">($
{w.length})</span></li>`).join(""); 34      }); 35    </script> 36  </body> 37
</html>
```

File structure

```
single file: word-sorter.html
```

How to run it

Open in any browser.

How it works

split(/\s+/) breaks the text on any whitespace. The sort callback returns b.length - a.length so longer words bubble to the top. map() builds the final HTML list.

Customization ideas

- Allow sorting alphabetically
- Highlight palindromes
- Group by first letter

🕹 Upgrade Challenge

> *Build a sentence generator that picks the longest word from each sentence and stitches them together.*

Project 15: Magic 8-Ball

Difficulty: ☆

What you will build

Ask a yes/no question out loud, shake the 8-ball, and get a randomised answer from a finite set.

Skills you will learn

- Random selection
- CSS animation
- Tap-friendly UI

Features

- Tap to reveal
- 20+ answers
- Shake animation

Build it with AI — the prompt to paste

> *Build a 'Magic 8-Ball'. A black circle with a small blue window. The user types a question, taps the ball, the ball 'shakes' (CSS animation), then displays one of 20 pre-written answers (10 positive, 5 neutral, 5 negative, all clearly placeholder humour). One file vanilla JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Magic 8 Ball</title>  6    <style>  7      body { font-family: system-ui;
background: #0f172a; color: white; min-height: 100vh; margin: 0; display: flex;
flex-direction: column; align-items: center; justify-content: center; }  8
input { padding: 10px; font-size: 16px; width: 280px; border-radius: 6px;
border: 0; }  9      .ball { margin-top: 36px; width: 220px; height: 220px;
background: radial-gradient(circle at 30% 30%, #334155, #000); border-radius:
50%; display: flex; align-items: center; justify-content: center; cursor:
pointer; user-select: none; box-shadow: 0 0 40px rgba(255,255,255,.1);
transition: transform .05s; } 10      .ball.shake { animation: shake .5s; } 11
.window { width: 120px; height: 80px; background: radial-gradient(circle at 30%
30%, #60a5fa, #1e40af); border-radius: 50%; display: flex; align-items: center;
justify-content: center; padding: 10px; text-align: center; font-weight: 600;
font-size: 14px; color: white; } 12      @keyframes shake { 13        0%
{ transform: rotate(0); } 14        25% { transform: rotate(15deg); } 15
50% { transform: rotate(-15deg); } 16        75% { transform: rotate(10deg); }
17        100% { transform: rotate(0); } 18      } 19    </style> 20  </head> 21
<body> 22    <h1>🎱 Magic 8 Ball</h1> 23    <input id="q" placeholder="Ask a
yes/no question..."> 24    <div class="ball" id="ball"><div class="window"
id="win">Tap me</div></div> 25   26    <script> 27      const answers = [ 28
"Yes, definitely.", "Most likely.", "Outlook good.", "Signs say yes.", 29
"You may rely on it.", "As I see it, yes.", "It is certain.", 30        "Yes -
in good time.", "Without a doubt.", "Absolutely.", 31        "Ask again later.",
"Better not tell you now.", "Cannot predict now.", 32        "Concentrate and
ask again.", "Reply hazy, try again.", 33        "Don't count on it.", "My reply
is no.", "Outlook not so good.", 34        "Very doubtful.", "My sources say
no." 35      ]; 36      const ball = document.getElementById("ball"); 37
```

```
const win = document.getElementById("win"); 38   39
ball.addEventListener("click", () => { 40        ball.classList.remove("shake");
41        void ball.offsetWidth; // force reflow to restart animation 42
ball.classList.add("shake"); 43        setTimeout(() => { 44
win.textContent = answers[Math.floor(Math.random() * answers.length)]; 45
}, 500); 46      }); 47    </script> 48  </body> 49  </html>
```

File structure

```
single file: 8ball.html
```

How to run it

Open in any browser.

How it works

We force a 'reflow' by reading offsetWidth after removing a class. This lets the same animation play again immediately. setTimeout waits until the shake animation finishes before swapping the text.

Customization ideas

- Add sound effect on shake
- Track how many times each answer was given
- Add a typing animation

🕹 Upgrade Challenge

> *Make the answers adapt to the question's tone: detect positive/negative words and bias the answer set.*

Project 16: Number Pyramid Printer

Difficulty: ☆

What you will build

Generate a centred numeric pyramid in the browser up to a chosen height.

Skills you will learn

- Loops
- String building
- DOM rendering

Features

- Configurable height
- Multi-number pyramids
- Live preview

Build it with AI — the prompt to paste

> *Build 'Number Pyramid Printer'. User picks a height (1-12). Display builds the classic pyramid, e.g. for height 4 shows: ' 1 ', ' 1 2 1 ', '1 2 3 2 1', '1 2 3 4 3 2 1'. Use printf-style spacing with   entities. Show a 'monospace' <pre> block. Vanilla JS.*

Complete code

— HTML —

```
 1 <!DOCTYPE html> 2 <html> 3 <head> 4   <meta charset="utf-8"> 5
<title>Number Pyramid</title> 6   <style> 7     body { font-family: system-
ui; max-width: 600px; margin: 40px auto; padding: 20px; text-align: center; } 8
pre { background: #0f172a; color: #7dd3fc; padding: 20px; border-radius: 10px;
font-size: 22px; overflow: auto; } 9     input { padding: 8px; width: 80px;
font-size: 16px; } 10   </style> 11 </head> 12 <body> 13   <h1>Number
Pyramid</h1> 14   <label>Height: <input id="h" type="number" min="1" max="12"
value="5"></label> 15   <pre id="out"></pre> 16  17   <script> 18     const
h = document.getElementById("h"); 19     const out =
document.getElementById("out"); 20  21     function build() { 22       const
height = Math.max(1, Math.min(12, parseInt(h.value) || 1)); 23       let s =
""; 24       for (let row = 1; row <= height; row++) { 25         const pad =
" ".repeat(height - row); 26         const part = Array.from({length: row}, (_,
i) => i + 1).join(" ") + 27                     Array.from({length: row - 1},
(_, i) => row - 1 - i).join(" "); 28         s += pad + part + "\n";
29       } 30       out.textContent = s; 31     } 32  33
h.addEventListener("input", build); 34     build(); 35   </script> 36 </body>
37 </html>
```

File structure

```
single file: pyramid.html
```

How to run it

Open in any browser.

How it works

Outer loop runs once per row. For each row we build the ascending sequence in one Array.from pass and the descending sequence in a second; both joined by spaces, then padded with leading spaces.

Customization ideas

- Try an alphabet pyramid (A, B, C…)
- Multiplication tables pyramid
- Random digit pyramid

🕹 Upgrade Challenge

> *Build a Pascal's triangle printer up to row 20 — each cell is the sum of the two above.*

Project 17: Basic Calculator

Difficulty: ☆

What you will build

A four-function calculator with a clean button grid.

Skills you will learn

- Event delegation
- Arithmetic
- String parsing

Features

- + − × ÷
- Decimal support
- Sign toggle
- Clear
- Backspace

Build it with AI — the prompt to paste

> *Build 'Basic Calculator'. Buttons for digits 0-9, operators +, -, *, /, decimal point, equals, clear (AC), backspace (⌫) and sign toggle (±). Display shows the live expression. Pure vanilla JS, one file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Calculator</title>  6    <style>  7      body { font-family: system-ui;
max-width: 320px; margin: 30px auto; }  8      .display { background: #0f172a;
color: white; padding: 20px; text-align: right; font-size: 28px; border-radius:
10px 10px 0 0; min-height: 60px; word-wrap: break-word; }  9      .second
{ font-size: 14px; color: #94a3b8; min-height: 18px; } 10      .pad { display:
grid; grid-template-columns: repeat(4, 1fr); background: #e2e8f0; padding: 10px;
gap: 6px; border-radius: 0 0 10px 10px; } 11      button { padding: 18px; font-
size: 20px; border: 0; background: white; border-radius: 6px; cursor: pointer; }
12      button.op { background: #fde68a; } 13      button.eq { background:
#16a34a; color: white; grid-column: span 2; } 14      button.clear { background:
#fca5a5; } 15    </style> 16  </head> 17  <body> 18    <div class="display"><div
class="second" id="prev"></div><div id="cur">0</div></div> 19    <div
class="pad"> 20      <button class="clear" data-k="C">AC</button> 21
<button data-k="±">±</button> 22      <button data-k="B">⌫</button> 23
<button class="op" data-k="/">÷</button> 24      <button
data-k="7">7</button><button data-k="8">8</button><button data-k="9">9</button>
25      <button class="op" data-k="*">×</button> 26      <button
data-k="4">4</button><button data-k="5">5</button><button data-k="6">6</button>
27      <button class="op" data-k="-">−</button> 28      <button
data-k="1">1</button><button data-k="2">2</button><button data-k="3">3</button>
29      <button class="op" data-k="+">+</button> 30      <button data-k="0"
style="grid-column: span 2;">0</button> 31      <button data-k=".">.</button> 32
<button class="eq" data-k="=">=</button> 33    </div> 34   35    <script> 36
const cur = document.getElementById("cur"); 37      const prev =
document.getElementById("prev"); 38      let expr = ""; 39   40      function
paint() { 41        cur.textContent = expr || "0"; 42      } 43   44
```

```
document.querySelectorAll("button").forEach(b => { 45
b.addEventListener("click", () => { 46          const k = b.dataset.k; 47
if (k === "C") { expr = ""; prev.textContent = ""; } 48          else if (k ===
"B") { expr = expr.slice(0, -1); } 49          else if (k === "±") { 50
// Wrap the last number in parentheses with a minus sign.
  1            const m = expr.match(/(.*?)(-?\d*\.?\d*)$/);  2            if (m
&& m[2]) {  3              const head = m[1];  4              const num = m[2];
5              expr = head + (num.startsWith("-") ? num.slice(1) : "-" + num);
6            }  7          }  8          else if (k === "=") {  9            try
{ 10              const safe = expr.replace(/×/g, "*").replace(/÷/g, "/"); 11
// Don't allow anything outside digits and operators 12              if (!/^[\d.
+\-*\/() ]+$/.test(safe)) throw 0; 13              const r = Function('"use
strict"; return (' + safe + ')')(); 14              prev.textContent = expr + "
="; 15              expr = String(r); 16            } catch { prev.textContent =
"Error"; expr = ""; } 17          } 18          else { expr += k; } 19
paint(); 20        }); 21      }); 22   23      paint(); 24    </script> 25
</body> 26  </html>
```

File structure

```
single file: calculator.html
```

How to run it

Open in any browser.

How it works

We keep the live expression in a string and only evaluate when '=' is pressed. Function('return ...') gives a tiny sandboxed arithmetic evaluator for valid expressions.

Customization ideas

- Add percent key (%)
- Add memory keys (M+, MR)
- Support keyboard input

🕹 Upgrade Challenge

> *Add a history tape showing the last 20 calculations with timestamps.*

Project 18: Time Zone Buddy

Difficulty: ☆

What you will build

Compare current times across a list of major world cities using built-in browser time zones.

Skills you will learn

- Intl.DateTimeFormat
- Time zone APIs
- Dynamic lists

Features

- Auto-refresh every second
- 10 cities
- Difference from local time

Build it with AI — the prompt to paste

> *Build 'Time Zone Buddy'. Display current time for 10 cities (London, New York, Tokyo, Dubai, Sydney, Mumbai, Berlin, Singapore, Lagos, San Francisco) using Intl.DateTimeFormat with each city's IANA time zone. Update every second. Highlight any city currently in business hours. Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Time Zone Buddy</title>  6    <style>  7      body { font-family: system-
ui; max-width: 600px; margin: 40px auto; }  8      table { width: 100%; border-
collapse: collapse; }  9      th, td { padding: 10px; text-align: left; border-
bottom: 1px solid #e2e8f0; } 10      th { background: #f1f5f9; } 11      .biz
{ background: #dcfce7; } 12      .hour { font-family: monospace; font-size:
20px; font-weight: 600; } 13    </style> 14  </head> 15  <body> 16    <h1>🌍
Time Zone Buddy</h1> 17
<table><thead><tr><th>City</th><th>Time</th><th>Date</th></tr></thead><tbody
id="t"></tbody></table> 18   19    <script> 20      const cities = [ 21
["Mumbai", "Asia/Kolkata"], 22        ["London", "Europe/London"], 23
["New York", "America/New_York"], 24        ["Tokyo", "Asia/Tokyo"], 25
["Dubai", "Asia/Dubai"], 26        ["Sydney", "Australia/Sydney"], 27
["Berlin", "Europe/Berlin"], 28        ["Singapore", "Asia/Singapore"], 29
["Lagos", "Africa/Lagos"], 30        ["San Francisco", "America/Los_Angeles"] 31
]; 32      const body = document.getElementById("t"); 33   34      function
update() { 35        const now = new Date(); 36        body.innerHTML =
cities.map(([c, tz]) => { 37          const t = new Intl.DateTimeFormat("en-GB",
{ timeZone: tz, hour: "2-digit", minute: "2-digit", second: "2-digit", hour12:
false }).format(now); 38          const d = new Intl.DateTimeFormat("en-GB",
{ timeZone: tz, weekday: "short", day: "2-digit", month: "short" }).format(now);
39          const hour = parseInt(t.split(":")[0]); 40          const biz = hour
>= 9 && hour < 18 ? "biz" : ""; 41          return `<tr class="${biz}"><td>$
{c}</td><td class="hour">${t}</td><td>${d}</td></tr>`; 42        }).join(""); 43
} 44   45      setInterval(update, 1000); 46      update(); 47    </script> 48
</body> 49  </html>
```

File structure

```
single file: timezones.html
```

How to run it

Open in any browser.

How it works

Intl.DateTimeFormat is a built-in browser API. We pass a timeZone option and the same Date.now moment is formatted against each city's IANA time zone, letting time-zone math happen inside the engine itself.

Customization ideas

- Highlight cities where it is currently night-time
- Add user-defined city input
- Show 24-hour vs 12-hour toggle

🕹 Upgrade Challenge

> *Show a world map with a glowing dot at the centre of each city, animated to local time of day.*

Project 19: Word Counter Tool

Difficulty: ☆

What you will build

Paste any text and get word, character and sentence counts, plus average word length.

Skills you will learn

- Text analysis
- Regex
- Statistics

Features

- Word count
- Character count
- Sentence count
- Reading time

Build it with AI — the prompt to paste

> *Build 'Word Counter Tool'. Pastebox accepts any text. Live output shows: word count, character count (with and without spaces), sentence count, paragraph count, average word length, and estimated reading time at 200 wpm. Vanilla JS, no libraries, single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Word Counter</title>  6    <style>  7      body { font-family: system-ui;
max-width: 720px; margin: 40px auto; padding: 20px; }  8      textarea { width:
100%; min-height: 160px; padding: 12px; font-size: 15px; box-sizing: border-box;
}  9      .stats { display: grid; grid-template-columns: repeat(auto-fit,
minmax(120px, 1fr)); gap: 10px; margin-top: 20px; } 10      .stat { background:
#f1f5f9; padding: 14px; border-radius: 10px; text-align: center; } 11      .num
{ font-size: 26px; font-weight: 700; color: #1d4ed8; } 12      .lbl { font-size:
13px; color: #475569; } 13    </style> 14  </head> 15  <body> 16    <h1>Word
Counter</h1> 17    <textarea id="t" placeholder="Paste or type
anything..."></textarea> 18    <div class="stats"> 19      <div
class="stat"><div class="num" id="w">0</div><div class="lbl">Words</div></div>
20      <div class="stat"><div class="num" id="c">0</div><div
class="lbl">Characters</div></div> 21      <div class="stat"><div class="num"
id="cws">0</div><div class="lbl">Char (no spaces)</div></div> 22      <div
class="stat"><div class="num" id="s">0</div><div
class="lbl">Sentences</div></div> 23      <div class="stat"><div class="num"
id="p">0</div><div class="lbl">Paragraphs</div></div> 24      <div
class="stat"><div class="num" id="r">0m</div><div class="lbl">Reading
time</div></div> 25    </div> 26   27    <script> 28      const t =
document.getElementById("t"); 29      function stat(id, v)
{ document.getElementById(id).textContent = v; } 30   31
t.addEventListener("input", () => { 32        const text = t.value; 33
const words = text.trim().match(/[\w'’-]+/g) || []; 34        const sentences =
text.split(/[.!?]+/).filter(s => s.trim().length); 35        const paragraphs =
text.split(/\n+/).filter(p => p.trim().length); 36        const avgLen =
```

```
words.length ? (words.join("").length / words.length).toFixed(1) : 0; 37
const minutes = (words.length / 200).toFixed(1); 38   39        stat("w",
words.length); 40        stat("c", text.length); 41        stat("cws",
text.replace(/\s/g, "").length); 42        stat("s", sentences.length); 43
stat("p", paragraphs.length); 44        stat("avg", avgLen); 45        stat("r",
minutes + "m"); 46      }); 47    </script> 48  </body> 49  </html>
```

File structure

```
single file: word-counter.html
```

How to run it

Open in any browser.

How it works

We use a regex /[\w’'-]+/g that handles contractions and possessives as single words. Paragraphs are split on newlines, sentences on . ! ? followed by anything.

Customization ideas

- Show word frequency table
- Detect language of the pasted text
- Highlight the longest sentence

🕹 Upgrade Challenge

> *Add per-paragraph stats with collapsible rows.*

Project 20: Digital Greeting Card

Difficulty: ☆

What you will build

Build an animated greeting card with confetti and personalised text — perfect for sharing.

Skills you will learn

- CSS animations
- Event listeners
- DOM templating

Features

- Custom name input
- Animated message
- Confetti on click
- Downloadable as standalone HTML

Build it with AI — the prompt to paste

> *Build a 'Digital Greeting Card'. A lovely styled card with the user's chosen name, a short message, and click-anywhere confetti animation (a few dozen coloured squares falling). Use CSS keyframes and JS DOM creation. Single HTML file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Greeting Card</title>  6    <style>  7      body { margin: 0; min-height:
100vh; background: linear-gradient(135deg,#fbcfe8,#a5b4fc); display: flex;
align-items: center; justify-content: center; font-family: 'Georgia', serif;
overflow: hidden; }  8      .card { background: white; padding: 40px 60px;
border-radius: 18px; box-shadow: 0 20px 50px rgba(0,0,0,.15); text-align:
center; max-width: 480px; }  9      .to { color: #db2777; font-size: 24px; } 10
.msg { color: #1e3a8a; font-size: 20px; margin-top: 10px; min-height: 50px;
font-style: italic; } 11      input { padding: 8px 12px; font-size: 16px;
border-radius: 6px; border: 1px solid #cbd5e1; } 12      button { padding: 8px
16px; font-size: 16px; background: #db2777; color: white; border: 0; border-
radius: 6px; cursor: pointer; margin-left: 6px; } 13      .confetti { position:
fixed; top: -10px; width: 8px; height: 14px; animation: fall linear forwards; }
14      @keyframes fall { 15        to { transform: translateY(120vh)
rotate(720deg); } 16      } 17      p.controls { margin-top: 20px; } 18
</style> 19  </head> 20  <body> 21    <div class="card"> 22      <div
class="to">For <span id="who">Friend</span>,</div> 23      <div class="msg"
id="msg">a wonderful day is just beginning. ✳</div> 24      <p class="controls">
25        <input id="name" placeholder="Recipient's name"> 26        <button
id="ok">Update</button> 27      </p> 28      <p>Tap anywhere for confetti 🎉</p>
29    </div> 30   31    <script> 32
document.getElementById("ok").addEventListener("click", () => { 33        const
v = document.getElementById("name").value.trim(); 34        if (v)
document.getElementById("who").textContent = v; 35      }); 36   37      const
colors = ["#f87171","#fbbf24","#34d399","#60a5fa","#a78bfa","#f472b6"]; 38
document.body.addEventListener("click", e => { 39        for (let i = 0; i < 30;
```

```
i++) { 40          const c = document.createElement("div"); 41
c.className = "confetti"; 42          c.style.left = (e.clientX + (Math.random()
* 200 - 100)) + "px"; 43          c.style.background =
colors[Math.floor(Math.random() * colors.length)]; 44
c.style.animationDuration = (Math.random() * 2 + 2) + "s"; 45
document.body.appendChild(c); 46          setTimeout(() => c.remove(), 4000); 47
} 48      }); 49    </script> 50  </body>
  1  </html>
```

File structure

```
single file: greeting.html
```

How to run it

Open in any browser.

How it works

We listen for click events on the body, then spawn 30 elements at the click position. Each is given a random colour and a CSS animation that translates them off-screen. setTimeout cleans them up.

Customization ideas

- Add a message customiser with 5 templates
- Add a 'share via link' option
- Show a heart-shaped photo frame

🕹 Upgrade Challenge

> *Add a real 'share image' button: render the card to canvas and let the user download it as a PNG.*

Part 2 — Web Projects

Real interactive websites. Each project adds a new layer — DOM manipulation, layouts, components, persistence.

Project 21: Personal Landing Page

Difficulty: ☆☆

What you will build

Your first real website: name, headline, three project cards, and a contact form (no real backend).

Skills you will learn

- HTML semantics
- CSS layout
- Responsive design

Features

- Hero section
- Project cards
- Contact form
- Mobile-first responsive

Build it with AI — the prompt to paste

> *Create a single-file personal landing page. Include: a hero with the user's name and a one-line tagline, three featured 'project cards' with placeholder descriptions, an about-me paragraph, and a contact form. Mobile-first responsive. Use vanilla CSS (no frameworks) and a single HTML file. Add comments explaining the structure.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<meta name="viewport" content="width=device-width,initial-scale=1">  6
<title>Your Name · Landing</title>  7    <style>  8      * { box-sizing: border-
box; }  9      body { margin: 0; font-family: system-ui, sans-serif; color:
#0f172a; line-height: 1.5; } 10      header { padding: 80px 20px 60px; text-
align: center; background: linear-gradient(135deg,#0ea5e9,#6366f1); color:
white; } 11      header h1 { font-size: clamp(28px, 6vw, 48px); margin: 0; } 12
header p { font-size: clamp(16px, 3vw, 20px); opacity: .95; } 13      main
{ max-width: 880px; margin: 0 auto; padding: 40px 20px; } 14      h2 { border-
left: 4px solid #6366f1; padding-left: 12px; } 15      .cards { display: grid;
grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 16px; } 16
.card { background: #f8fafc; padding: 18px; border-radius: 10px; border: 1px
solid #e2e8f0; } 17      .card h3 { margin: 0 0 6px; color: #4f46e5; } 18
form { display: grid; gap: 10px; max-width: 480px; } 19      input, textarea
{ padding: 10px; border: 1px solid #cbd5e1; border-radius: 6px; font-size: 15px;
font-family: inherit; } 20      button { padding: 10px 18px; background:
#4f46e5; color: white; border: 0; border-radius: 6px; cursor: pointer; font-
size: 16px; } 21    </style> 22  </head> 23  <body> 24    <header> 25
<h1>Hello, I'm Aisha</h1> 26      <p>A beginner builder learning to ship small
projects every week.</p> 27    </header> 28   29    <main> 30      <section> 31
<h2>About</h2> 32        <p>I love turning ideas into small, useful tools. This
page is my very first website.</p> 33      </section> 34   35      <section> 36
<h2>Featured Projects</h2> 37        <div class="cards"> 38          <div
class="card"><h3>Tip Calculator</h3><p>A friendly utility for splitting bills
fairly.</p></div> 39          <div class="card"><h3>Magic 8-Ball</h3><p>A
```

```
playful decision helper with shake animation.</p></div> 40          <div
class="card"><h3>Habit Tracker</h3><p>A minimal weekly habit tracker with
streaks.</p></div> 41        </div> 42      </section> 43   44      <section> 45
<h2>Contact</h2> 46        <form onsubmit="event.preventDefault(); alert('Demo
only — not sent.');"> 47          <input type="text" placeholder="Your name"
required> 48          <input type="email" placeholder="Your email" required> 49
<textarea placeholder="Message" rows="4" required></textarea> 50
<button type="submit">Send</button>
  1        </form>  2      </section>  3    </main>  4  </body>  5  </html>
```

File structure

```
single file: landing.html
```

How to run it

Open in any browser.

How it works

We use semantic tags (header, main, section) for structure. The grid uses auto-fit with a minimum column size, so it collapses to one column on mobile and grows on desktops. clamp() makes font-sizes responsive too.

Customization ideas

- Add a dark mode toggle
- Replace cards with your real projects
- Add subtle scroll-reveal animation

🕹 Upgrade Challenge

> *Add a small portfolio gallery that reads project thumbnails from a local JSON file using fetch().*

Project 22: Notes Saver

Difficulty: ☆☆

What you will build

A notepad where notes are saved to localStorage and restored when you reload.

Skills you will learn

- localStorage
- CRUD operations
- JSON serialisation

Features

- Add, edit, delete notes
- Search across all notes
- Auto-save on every keystroke

Build it with AI — the prompt to paste

> *Build a single-file 'Notes Saver'. Users can add new notes with a title and content, edit existing ones, delete them and search across all notes. Notes are stored in localStorage as JSON. Single file vanilla JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Notes</title>  6    <style>  7      body { font-family: system-ui; max-
width: 800px; margin: 30px auto; padding: 20px; background: #fafafa; }
8      .row { display: flex; gap: 10px; margin-bottom: 12px; }  9      input,
textarea { width: 100%; padding: 8px; border: 1px solid #cbd5e1; border-radius:
6px; font-family: inherit; font-size: 14px; } 10      button { padding: 8px
14px; background: #2563eb; color: white; border: 0; border-radius: 6px; cursor:
pointer; } 11      .note { background: white; padding: 14px; border: 1px solid
#e2e8f0; border-radius: 8px; margin-top: 10px; } 12      .note h3 { margin: 0;
color: #1e40af; } 13      .note small { color: #64748b; } 14      .actions
{ margin-top: 8px; } 15      .actions button { background: #94a3b8; margin-
right: 6px; } 16      .actions .del { background: #dc2626; } 17    </style> 18
</head> 19  <body> 20    <h1>□ Notes Saver</h1> 21    <div class="row"> 22
<input id="title" placeholder="Title"> 23      <textarea id="body"
placeholder="Write a note..." rows="2"></textarea> 24      <button
id="add">Add</button> 25    </div> 26    <input id="search" placeholder="Search
notes..."> 27    <div id="list"></div> 28   29    <script> 30      const KEY =
"notes-saver-v1"; 31      const list = document.getElementById("list"); 32
let notes = JSON.parse(localStorage.getItem(KEY) || "[]"); 33   34      function
save() { localStorage.setItem(KEY, JSON.stringify(notes)); } 35      function
render(filter = "") { 36        list.innerHTML = ""; 37        notes 38
.filter(n => n.title.toLowerCase().includes(filter.toLowerCase()) ||
n.body.toLowerCase().includes(filter.toLowerCase())) 39          .forEach(n => {
40            const div = document.createElement("div"); 41
div.className = "note"; 42            div.innerHTML =
`<h3>${escape(n.title)}</h3><p>${escape(n.body)}</p><small>${new
Date(n.t).toLocaleString()}</small> 43              <div class="actions"><button
onclick="editNote('${n.id}')">Edit</button> 44              <button class="del"
onclick="deleteNote('${n.id}')">Delete</button></div>`; 45
list.appendChild(div); 46          }); 47      } 48      function escape(s)
```

```
{ return s.replace(/[&<>"]/g, c =>
({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;'}[c])); } 49   50
document.getElementById("add").addEventListener("click", () => {
  1        const t = document.getElementById("title").value.trim();  2
const b = document.getElementById("body").value.trim();  3        if (!t)
return;  4        notes.unshift({ id: Math.random().toString(36).slice(2),
title: t, body: b, t: Date.now() });  5
document.getElementById("title").value = "";  6
document.getElementById("body").value = "";  7        save(); render();
8      });  9   10
document.getElementById("search").addEventListener("input", e =>
render(e.target.value)); 11   12      window.editNote = id => { 13        const
n = notes.find(x => x.id === id); 14        if (!n) return; 15        const t =
prompt("New title", n.title); if (t === null) return; 16        const b =
prompt("New content", n.body); if (b === null) return; 17        n.title = t;
n.body = b; save(); render(); 18      }; 19      window.deleteNote = id =>
{ notes = notes.filter(x => x.id !== id); save(); render(); }; 20   21
render(); 22    </script> 23  </body> 24  </html>
```

File structure

```
single file: notes.html
```

How to run it

Open in any browser.

How it works

Notes are stored as a JSON array inside localStorage. JSON.stringify turns it into a string for storage; JSON.parse turns it back. The escape function prevents simple XSS in the rendered HTML.

Customization ideas

- Add 'pinned' notes
- Add colour tags per note
- Encrypt notes with a user-chosen password

🕹 Upgrade Challenge

> *Add Markdown rendering using a small client-side parser (marked.js mini) so notes support headings and lists.*

Project 23: Responsive Photo Grid

Difficulty: ☆☆

What you will build

A clean, auto-arranging grid of photos with a lightbox preview on click.

Skills you will learn

- CSS Grid
- Modal handling
- Keyboard accessibility

Features

- Auto-fit columns
- Click to enlarge
- Keyboard navigation
- Captions

Build it with AI — the prompt to paste

> *Build a 'Responsive Photo Grid'. Use CSS Grid with auto-fit to display 12 placeholder images (use https://picsum.photos/seed/x/200 or any local images you have). Clicking opens a full-screen lightbox with arrow-key navigation and Escape to close. Single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Photo Grid</title>  6    <style>  7      body { margin: 0; background:
#0f172a; color: white; font-family: system-ui; }  8      h1 { text-align:
center; padding: 30px; }  9      .grid { display: grid; grid-template-columns:
repeat(auto-fit, minmax(180px, 1fr)); gap: 8px; padding: 8px; } 10      .grid
img { width: 100%; aspect-ratio: 1/1; object-fit: cover; border-radius: 4px;
cursor: pointer; transition: transform .2s; } 11      .grid img:hover
{ transform: scale(1.03); } 12      .lightbox { display: none; position: fixed;
inset: 0; background: rgba(0,0,0,.9); align-items: center; justify-content:
center; z-index: 10; } 13      .lightbox.open { display: flex; }
14      .lightbox img { max-width: 90vw; max-height: 90vh; box-shadow: 0 0 30px
rgba(255,255,255,.2); } 15      .nav { position: absolute; top: 50%; transform:
translateY(-50%); background: rgba(255,255,255,.12); border: 0; color: white;
font-size: 30px; width: 60px; height: 60px; border-radius: 50%; cursor: pointer;
} 16      .nav.prev { left: 20px; } .nav.next { right: 20px; } 17      .cap
{ position: absolute; bottom: 20px; left: 0; right: 0; text-align: center;
color: white; } 18    </style> 19  </head> 20  <body> 21    <h1>Photo
Gallery</h1> 22    <div class="grid" id="grid"></div> 23    <div
class="lightbox" id="lb"> 24      <button class="nav prev" id="prev">‹</button>
25      <img id="big" src="" alt=""> 26      <button class="nav next"
id="next">›</button> 27      <div class="cap" id="cap"></div> 28    </div> 29
30    <script> 31      const items = Array.from({length: 12}, (_, i) => ({ 32
src: `https://picsum.photos/seed/p${i}/800/800`, 33        cap: `Photo ${i+1}`
34      })); 35      let idx = 0; 36   37      const grid =
document.getElementById("grid"); 38      items.forEach((it, i) => { 39
const img = document.createElement("img"); 40        img.src =
it.src.replace("/800/800", "/200/200"); 41        img.alt = it.cap; 42
```

```
img.addEventListener("click", () => open(i)); 43        grid.appendChild(img);
44      }); 45   46      const lb = document.getElementById("lb"); 47      const
big = document.getElementById("big"); 48      const cap =
document.getElementById("cap"); 49      function open(i) { idx = i; show();
lb.classList.add("open"); } 50      function show() { big.src = items[idx].src;
cap.textContent = items[idx].cap; }
  1      function close() { lb.classList.remove("open"); }  2      function
delta(d) { idx = (idx + d + items.length) % items.length; show(); }  3    4
document.getElementById("prev").addEventListener("click", () => delta(-1));  5
document.getElementById("next").addEventListener("click", () => delta(1));  6
lb.addEventListener("click", e => { if (e.target === lb) close(); });  7
document.addEventListener("keydown", e => {  8        if (!
lb.classList.contains("open")) return;  9        if (e.key === "Escape")
close(); 10        else if (e.key === "ArrowLeft") delta(-1); 11        else if
(e.key === "ArrowRight") delta(1); 12      }); 13    </script> 14  </body> 15
</html>
```

File structure

```
single file: gallery.html
```

How to run it

Open in any browser. Needs an internet connection for picsum.photos.

How it works

CSS Grid with repeat(auto-fit, minmax(180px, 1fr)) automatically adds or removes columns to fit the screen width. The lightbox is a fixed-position overlay controlled by adding/removing the open class.

Customization ideas

- Replace picsum with your own image folder
- Add swipe support on touch
- Add fade transitions between photos

🕹 Upgrade Challenge

> *Add image filters: black-and-white, sepia and blur, applied via CSS variables and a small controls row.*

Project 24: Markdown Previewer

Difficulty: ☆☆

What you will build

Type markdown on the left, see formatted HTML on the right, live as you type.

Skills you will learn

- Two-pane layout
- Regex-based mini parser
- Live binding

Features

- Real-time preview
- Supports headings, lists, bold, italics, links, code
- Reset button

Build it with AI — the prompt to paste

> *Build a 'Markdown Previewer'. The page has two columns: a textarea (left) and a styled preview (right). Write a tiny markdown-to-HTML converter in vanilla JS that handles headings (#, ##, ###), bold (**x**), italic (*x*), links [text](url), lists (- item) and code (`x`). No external libraries. Single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Markdown Previewer</title>  6    <style>  7      body { margin: 0; font-
family: system-ui; height: 100vh; display: flex; }  8      .pane { flex: 1;
padding: 18px; box-sizing: border-box; overflow: auto; }  9      textarea
{ width: 100%; height: 100%; border: 0; outline: none; font-size: 15px; font-
family: ui-monospace, monospace; resize: none; } 10      .preview { background:
#f8fafc; } 11      .preview h1, .preview h2, .preview h3 { border-bottom: 1px
solid #e2e8f0; padding-bottom: 4px; } 12      .preview code { background:
#e2e8f0; padding: 2px 5px; border-radius: 3px; } 13      .preview pre
{ background: #0f172a; color: #e2e8f0; padding: 12px; border-radius: 6px;
overflow: auto; } 14      .preview a { color: #4f46e5; } 15      .preview ul
{ padding-left: 22px; } 16    </style> 17  </head> 18  <body> 19    <div
class="pane"> 20      <h2>Editor</h2> 21      <textarea id="md"
spellcheck="false"># Hello, markdown! 22  Try **bold**, *italic*, or [a link]
(https://example.com). 23   24  ## Lists 25  - First 26  - Second 27  - Third 28
29  ## Code 30  `inline code`</textarea> 31    </div> 32    <div class="pane
preview"> 33      <h2>Preview</h2> 34      <div id="pv"></div> 35    </div> 36
37    <script> 38      function md(input) { 39        // Escape HTML first so
user-supplied content can't inject tags. 40        const esc = s =>
s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;"); 41
let s = esc(input); 42   43        s = s.replace(/^### (.+)$/gm, "<h3>$1</h3>");
44        s = s.replace(/^## (.+)$/gm, "<h2>$1</h2>"); 45        s =
s.replace(/^# (.+)$/gm, "<h1>$1</h1>"); 46        s =
s.replace(/\*\*(.+?)\*\*/g, "<b>$1</b>"); 47        s = s.replace(/\*(.+?)\*/g,
"<i>$1</i>"); 48        s = s.replace(/`([^`]+)`/g, "<code>$1</code>"); 49
s = s.replace(/\[([^\]]+)\]\(([^)]+)\)/g, '<a href="$2" target="_blank"
rel="noopener">$1</a>'); 50        s = s.replace(/(^- .+(?:\n- .+)*)/gm, block
=> {
```

```
  1          const items = block.split("\n").map(l => "<li>" + l.replace(/^- /,
"") + "</li>").join("");  2          return "<ul>" + items + "</ul>";
3        });  4        s = s.replace(/\n{2,}/g, "</p><p>");  5        return
"<p>" + s + "</p>";  6      }  7    8      const mdEl =
document.getElementById("md");  9      const pv = document.getElementById("pv");
10      mdEl.addEventListener("input", () => pv.innerHTML = md(mdEl.value)); 11
pv.innerHTML = md(mdEl.value); 12    </script> 13  </body> 14  </html>
```

File structure

```
single file: previewer.html
```

How to run it

Open in any browser.

How it works

We escape HTML first so any literal < and > typed by the user can't break the page. Then apply a series of replacements that mimic markdown syntax. Order matters — escape before transforming, headings before bold.

Customization ideas

- Add blockquotes
- Add tables
- Add image embedding via ![alt](url)

🕹 Upgrade Challenge

> *Add an export-to-HTML button so the user can save the preview as a standalone .html file.*

Project 25: Real-time Clock Display

Difficulty: ☆☆

What you will build

A beautiful large clock showing time, day progress, and date with smooth colour transitions.

Skills you will learn

- CSS animations
- RequestAnimationFrame
- Time progression

Features

- Hours, minutes, seconds displays
- Day progress bar
- Smooth colour transitions between moments

Build it with AI — the prompt to paste

> *Build a 'Real-time Clock Display'. A large clock with hours, minutes, seconds. Underneath, a horizontal bar shows how far through the day you are. Background colour smoothly transitions based on the time of day (night blue → morning yellow → day blue → sunset orange → night). Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Real-time Clock</title>  6    <style>  7      body { margin: 0; min-
height: 100vh; display: flex; flex-direction: column; align-items: center;
justify-content: center; font-family: system-ui; color: white; transition:
background-color 1s; }  8      .time { font-size: clamp(60px, 15vw, 140px);
font-weight: 200; font-variant-numeric: tabular-nums; letter-spacing: 4px; }  9
.date { font-size: 18px; margin-top: 8px; opacity: .9; } 10      .progress
{ width: min(80vw, 600px); height: 8px; background: rgba(255,255,255,.25);
border-radius: 4px; margin-top: 30px; } 11      .progress > div { height: 100%;
background: white; border-radius: 4px; } 12    </style> 13  </head> 14  <body>
15    <div class="time" id="t">00:00:00</div> 16    <div class="date" id="d">-
</div> 17    <div class="progress"><div id="bar" style="width:0%"></div></div>
18   19    <script> 20      function colour(h) { 21        // smoothly
interpolate between colours for the time of day 22        if (h < 6)  return
"#0f172a";    // night 23        if (h < 11) return "#fb923c";    // morning 24
if (h < 16) return "#0ea5e9";    // day 25        if (h < 19) return "#f97316";
// sunset 26        return "#1e293b";               // evening 27      } 28
function pad(n) { return n.toString().padStart(2, "0"); } 29      function
tick() { 30        const now = new Date(); 31        const ms = now.getTime();
32        const start = new Date(now); start.setHours(0,0,0,0); 33        const
pct = ((ms - start.getTime()) / 86400000) * 100; 34
document.getElementById("t").textContent = pad(now.getHours()) + ":" +
pad(now.getMinutes()) + ":" + pad(now.getSeconds()); 35
document.getElementById("d").textContent = now.toLocaleDateString(undefined,
{ weekday: "long", year: "numeric", month: "long", day: "numeric" }); 36
document.getElementById("bar").style.width = pct.toFixed(4) + "%"; 37
document.body.style.backgroundColor = colour(now.getHours()); 38
```

```
requestAnimationFrame(tick); 39      } 40      tick(); 41    </script> 42
</body> 43  </html>
```

File structure

```
single file: clock.html
```

How to run it

Open in any browser.

How it works

% of the day elapsed = current time / 86,400,000 ms. requestAnimationFrame redraws every frame which is more efficient than setInterval for smooth visual updates like this.

Customization ideas

- Add moon/sun icon that rotates through phases
- Show timezone offset
- Add a 12/24 hour toggle

🕹 Upgrade Challenge

> *Add a companion 'week progress' toast that appears once a week with a friendly message.*

Project 26: Sticky Notes Board

Difficulty: ☆☆

What you will build

A draggable, savable board of sticky notes that reappears each time you open the page.

Skills you will learn

- Drag and drop
- localStorage
- Coordinate maths

Features

- Create notes with one click
- Drag to reposition
- Persistent positions
- Edit and delete

Build it with AI — the prompt to paste

> *Build a 'Sticky Notes Board'. Click anywhere to drop a new sticky note with a default message. Each note is draggable. Notes can be edited by double-click. Positions and content persist in localStorage. Pure JS, no libraries.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Sticky Board</title>  6    <style>  7      body { margin: 0; min-height:
100vh; background: #f59e0b; font-family: system-ui; overflow: hidden; }
8      .hint { position: fixed; top: 12px; left: 12px; background: white;
padding: 8px 12px; border-radius: 6px; font-size: 14px; }  9      .note
{ position: absolute; width: 180px; min-height: 180px; padding: 14px;
background: #fef9c3; border-radius: 4px; box-shadow: 0 6px 18px rgba(0,0,0,.18);
cursor: move; user-select: none; } 10      .note textarea { width: 100%; height:
140px; background: transparent; border: 0; outline: 0; resize: none; font:
inherit; color: #1f2937; } 11      .note .del { position: absolute; top: 4px;
right: 4px; width: 22px; height: 22px; border-radius: 50%; background: #ef4444;
color: white; border: 0; cursor: pointer; opacity: 0; } 12      .note:hover .del
{ opacity: 1; } 13    </style> 14  </head> 15  <body> 16    <div
class="hint">Click anywhere to add a note · double-click to edit</div> 17   18
<script> 19      const KEY = "sticky-board-v1"; 20      let notes =
JSON.parse(localStorage.getItem(KEY) || "[]"); 21      const colors =
["#fef9c3","#bae6fd","#bbf7d0","#fbcfe8","#fed7aa"]; 22   23      function
save() { localStorage.setItem(KEY, JSON.stringify(notes)); } 24      function
make(x, y, text = "Double-click to edit") { 25        const div =
document.createElement("div"); 26        div.className = "note"; 27
div.style.left = x + "px"; 28        div.style.top = y + "px"; 29
div.style.background = colors[Math.floor(Math.random() * colors.length)]; 30
31        const ta = document.createElement("textarea"); 32        ta.value =
text; 33        ta.readOnly = true; 34        ta.addEventListener("dblclick", ()
=> { ta.readOnly = false; ta.focus(); }); 35        ta.addEventListener("blur",
() => { 36          ta.readOnly = true; 37          const id = div.dataset.id;
38          const note = notes.find(n => n.id === id); 39          if (note)
```

```
{ note.text = ta.value; save(); } 40        }); 41        div.appendChild(ta);
42   43        const del = document.createElement("button"); 44
del.className = "del"; del.textContent = "×"; 45
del.addEventListener("click", () => { 46          const id = div.dataset.id; 47
notes = notes.filter(n => n.id !== id); 48          save(); div.remove(); 49
}); 50        div.appendChild(del);
  1    2        div.dataset.id = "n" + Math.random().toString(36).slice(2);  3
notes.push({ id: div.dataset.id, x, y, text });  4        save();
document.body.appendChild(div);  5        enableDrag(div);  6      }  7    8
function enableDrag(el) {  9        let ox, oy, dragging = false; 10
el.addEventListener("mousedown", e => { 11          if (e.target.tagName ===
"TEXTAREA" || e.target.classList.contains("del")) return; 12          dragging =
true; 13          ox = e.clientX - el.offsetLeft; 14          oy = e.clientY -
el.offsetTop; 15        }); 16        document.addEventListener("mousemove", e
=> { 17          if (!dragging) return; 18          el.style.left = (e.clientX -
ox) + "px"; 19          el.style.top = (e.clientY - oy) + "px"; 20        }); 21
document.addEventListener("mouseup", () => { 22          if (!dragging) return;
dragging = false; 23          const id = el.dataset.id; 24          const note =
notes.find(n => n.id === id); 25          if (note) { note.x = el.offsetLeft;
note.y = el.offsetTop; save(); } 26        }); 27      } 28   29
document.body.addEventListener("click", e => { 30        if (e.target.tagName
=== "TEXTAREA" || e.target.classList.contains("del") ||
e.target.classList.contains("hint")) return; 31        make(e.clientX - 40,
e.clientY - 40); 32      }); 33   34      // Restore 35      notes.forEach(n =>
{ 36        const dummy = document.createElement("div"); 37
document.body.appendChild(dummy); 38        make(n.x, n.y, n.text); 39
dummy.remove(); 40      }); 41    </script> 42  </body> 43  </html>
```

File structure

```
single file: sticky.html
```

How to run it

Open in any browser.

How it works

We track mouse position during drag and update the element's CSS left/top. localStorage stores the positions so the board looks identical when the user returns.

Customization ideas

- Add note colours from a palette
- Add a 'shuffle' button
- Add a 'reset board' confirm

🕹 Upgrade Challenge

> *Sync notes between two browser tabs in real time using BroadcastChannel API.*

Project 27: Animated CSS Buttons

Difficulty: ☆☆

What you will build

A playground of seven creative CSS button animations you can copy into your own projects.

Skills you will learn

- CSS transitions
- Keyframes
- Pseudo-elements

Features

- Pulse
- Slide
- Glow
- 3D press
- Neon flicker
- Liquid fill
- Icon spin

Build it with AI — the prompt to paste

> *Build a playground page that displays seven distinct CSS button animations. Add comments before each explaining the technique. The buttons must be reusable in any other website. Single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>CSS Buttons Lab</title>  6    <style>  7      body { font-family: system-
ui; background: #f1f5f9; min-height: 100vh; margin: 0; padding: 30px; }  8
h1 { text-align: center; }  9      .grid { display: grid; grid-template-columns:
repeat(auto-fit, minmax(220px, 1fr)); gap: 16px; max-width: 900px; margin: 0
auto; } 10      .cell { background: white; padding: 30px; border-radius: 10px;
text-align: center; box-shadow: 0 2px 8px rgba(0,0,0,.06); } 11      .cell small
{ display: block; margin-top: 10px; color: #64748b; } 12   13      /* 1. Pulse
*/ 14      .b1 { padding: 12px 24px; border: 0; background: #2563eb; color:
white; border-radius: 6px; cursor: pointer; animation: pulse 2s infinite; } 15
@keyframes pulse { 0%,100% { box-shadow: 0 0 0 0 rgba(37,99,235,.5); } 50%
{ box-shadow: 0 0 0 10px rgba(37,99,235,0); } } 16   17      /* 2. Slide */ 18
.b2 { padding: 12px 24px; border: 2px solid #16a34a; background: transparent;
color: #16a34a; border-radius: 6px; cursor: pointer; position: relative;
overflow: hidden; transition: color .3s; z-index: 1; } 19      .b2::before
{ content: ""; position: absolute; inset: 0; background: #16a34a; transform:
translateX(-100%); transition: transform .3s; z-index: -1; } 20      .b2:hover {
color: white; } 21      .b2:hover::before { transform: translateX(0); } 22   23
/* 3. Glow */ 24      .b3 { padding: 12px 24px; border: 0; background: #7c3aed;
color: white; border-radius: 6px; cursor: pointer; transition: box-shadow .3s; }
25      .b3:hover { box-shadow: 0 0 25px #7c3aed, 0 0 50px #7c3aed; } 26   27
/* 4. 3D Press */ 28      .b4 { padding: 12px 24px; border: 0; background:
#f59e0b; color: white; border-radius: 6px; cursor: pointer; box-shadow: 0 5px 0
#b45309; transition: transform .1s, box-shadow .1s; } 29      .b4:active
```

```
{ transform: translateY(5px); box-shadow: 0 0 0 #b45309; } 30   31      /* 5.
Neon Flicker */ 32      .b5 { padding: 12px 24px; background: #0f172a; color:
#67e8f9; border: 2px solid #67e8f9; border-radius: 4px; cursor: pointer; text-
shadow: 0 0 4px #67e8f9; box-shadow: 0 0 8px #67e8f9; animation: flicker 4s
infinite; } 33      @keyframes flicker { 0%,19%,21%,50%,100% { opacity: 1; }
20%,49% { opacity: .6; } } 34   35      /* 6. Liquid Fill */ 36      .b6
{ padding: 12px 24px; border: 0; background: #ef4444; color: white; border-
radius: 6px; cursor: pointer; position: relative; overflow: hidden; }
37      .b6::after { content: ""; position: absolute; inset: 0; background:
rgba(255,255,255,.3); transform: translateY(100%); transition: transform .3s; }
38      .b6:hover::after { transform: translateY(0); } 39   40      /* 7. Icon
Spin */ 41      .b7 { padding: 12px 18px 12px 24px; border: 0; background:
#0891b2; color: white; border-radius: 6px; cursor: pointer; } 42      .b7::after
{ content: " →"; display: inline-block; transition: transform .3s; }
43      .b7:hover::after { transform: rotate(360deg); } 44    </style> 45
</head> 46  <body> 47    <h1>CSS Button Lab</h1> 48    <p style="text-
align:center;color:#475569">Hover or click to see each animation. The CSS below
each button is yours to copy.</p> 49    <div class="grid"> 50      <div
class="cell"><button class="b1">Pulse</button><small>Pulsing box-shadow
ring</small></div>
  1      <div class="cell"><button class="b2">Slide</button><small>Background
slides in from the left</small></div>  2      <div class="cell"><button
class="b3">Glow</button><small>Outer glow on hover</small></div>  3      <div
class="cell"><button class="b4">Press</button><small>3D press effect on
click</small></div>  4      <div class="cell"><button
class="b5">Neon</button><small>Flickering neon outline</small></div>  5
<div class="cell"><button class="b6">Liquid</button><small>Liquid floods up on
hover</small></div>  6      <div class="cell"><button
class="b7">Spin</button><small>Arrow icon rotates fully</small></div>  7
</div>  8  </body>  9  </html>
```

File structure

```
single file: buttons.html
```

How to run it

Open in any browser.

How it works

Each button uses a different CSS technique. Pseudo-elements (::before / ::after) are key for many effects because they let us draw extra visual layers without adding more HTML elements.

Customization ideas

- Combine effects for your own buttons
- Re-skin with brand colours
- Add sound effects on hover

🕹 Upgrade Challenge

> *Build a full 'Design System' page that exports all your favourite buttons as a ready-to-paste CSS theme.*

Project 28: Form Validator

Difficulty: ☆☆

What you will build

Reusable client-side form validation with email, password strength and live error messages.

Skills you will learn

- Form validation
- Regex
- CSS feedback states

Features

- Email pattern check
- Password strength
- Required fields
- Live feedback
- Custom messages

Build it with AI — the prompt to paste

> *Build a 'Form Validator' page with a single form: name, email, password and confirm password. Each field shows inline error or success as the user types. Password strength is rated Weak/Medium/Strong based on length and character variety. On submit, only valid forms show a success message. Vanilla JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Form Validator</title>  6    <style>  7      body { font-family: system-
ui; max-width: 460px; margin: 40px auto; padding: 20px; }  8      .field
{ margin-bottom: 14px; }  9      label { display: block; margin-bottom: 4px;
font-weight: 600; } 10      input { width: 100%; padding: 10px; border: 1px
solid #cbd5e1; border-radius: 6px; font-size: 15px; box-sizing: border-box; } 11
input.invalid { border-color: #dc2626; } 12      input.valid { border-color:
#16a34a; } 13      .msg { font-size: 13px; margin-top: 4px; min-height: 16px; }
14      .msg.bad { color: #b91c1c; } 15      .msg.good { color: #15803d; } 16
.strength { height: 6px; background: #e2e8f0; border-radius: 3px; margin-top:
6px; } 17      .strength > div { height: 100%; border-radius: 3px; transition:
width .3s, background .3s; } 18      button { padding: 10px 18px; background:
#2563eb; color: white; border: 0; border-radius: 6px; font-size: 16px; cursor:
pointer; width: 100%; } 19      .success { margin-top: 14px; padding: 12px;
background: #dcfce7; color: #166534; border-radius: 6px; display: none; } 20
</style> 21  </head> 22  <body> 23    <h1>Sign Up</h1> 24    <form id="f"> 25
<div class="field"> 26        <label>Name</label> 27        <input id="name"
required> 28        <div class="msg" id="nameMsg"></div> 29      </div> 30
<div class="field"> 31        <label>Email</label> 32        <input id="email"
type="email" required> 33        <div class="msg" id="emailMsg"></div> 34
</div> 35      <div class="field"> 36        <label>Password</label> 37
<input id="pw" type="password" required minlength="8"> 38        <div
class="strength"><div id="bar" style="width:0%; background:#dc2626"></div></div>
39        <div class="msg" id="pwMsg"></div> 40      </div> 41      <div
```

```
class="field"> 42        <label>Confirm password</label> 43        <input
id="pw2" type="password" required> 44        <div class="msg" id="pw2Msg"></div>
45      </div> 46      <button type="submit">Create account</button> 47
<div class="success" id="ok">Account created (demo)!</div> 48    </form> 49   50
<script>
  1      function setMsg(id, txt, kind, valid) {  2        const m =
document.getElementById(id);  3        m.textContent = txt;  4
m.className = "msg " + kind;  5        const inp =
m.previousElementSibling.tagName === "INPUT" ? m.previousElementSibling :
m.previousElementSibling.querySelector("input");  6        if (inp)
inp.className = valid === true ? "valid" : (valid === false ? "invalid" : "");
7      }  8      function strength(p) {  9        let s = 0; 10        if
(p.length >= 8) s++; 11        if (/[A-Z]/.test(p)) s++; 12        if (/[a-
z]/.test(p)) s++; 13        if (/[0-9]/.test(p)) s++; 14        if (/[^A-Za-z0-
9]/.test(p)) s++; 15        return s; // 0..5 16      } 17   18
document.getElementById("name").addEventListener("input", e => { 19        const
v = e.target.value.trim(); 20        if (!v) { setMsg("nameMsg", "Name is
required.", "bad", false); return; } 21        if (v.length < 2)
{ setMsg("nameMsg", "Name is too short.", "bad", false); return; } 22
setMsg("nameMsg", "Looks good.", "good", true); 23      }); 24   25
document.getElementById("email").addEventListener("input", e => { 26
const v = e.target.value.trim(); 27        const ok = /^[^\s@]+@[^\s@]+\.[^\s@]+
$/.test(v); 28        if (!v) { setMsg("emailMsg", "Email is required.", "bad",
false); return; } 29        if (!ok) { setMsg("emailMsg", "That doesn't look
like an email.", "bad", false); return; } 30        setMsg("emailMsg", "Valid
email.", "good", true); 31      }); 32   33
document.getElementById("pw").addEventListener("input", e => { 34        const v
= e.target.value; 35        const s = strength(v); 36        const bar =
document.getElementById("bar"); 37        const labels = ["", "Very weak",
"Weak", "Decent", "Good", "Strong"]; 38        const colors = ["", "#dc2626",
"#f59e0b", "#facc15", "#16a34a", "#0ea5e9"]; 39        bar.style.width = (s *
20) + "%"; 40        bar.style.background = colors[s]; 41        if (!v)
{ setMsg("pwMsg", "Password is required.", "bad", false); return; } 42        if
(v.length < 8) { setMsg("pwMsg", "Use at least 8 characters.", "bad", false);
return; } 43        setMsg("pwMsg", labels[s] + ".", "good", true); 44      });
45   46      document.getElementById("pw2").addEventListener("input", e => { 47
const v = e.target.value; 48        const p =
document.getElementById("pw").value; 49        if (v && v === p)
{ setMsg("pw2Msg", "Passwords match.", "good", true); return; } 50        if (v)
{ setMsg("pw2Msg", "Passwords don't match.", "bad", false); return; }
  1        setMsg("pw2Msg", "", "", null);  2      });  3    4
document.getElementById("f").addEventListener("submit", e => {  5
e.preventDefault();  6        const ok = ["name","email","pw","pw2"].every(id =>
document.getElementById(id).classList.contains("valid"));  7        if (ok)
document.getElementById("ok").style.display = "block";  8      });  9
</script> 10  </body> 11  </html>
```

File structure

```
single file: form.html
```

How to run it

Open in any browser.

How it works

Each field has its own input listener. We use the change in class on the input (valid / invalid) to drive border colour, then read those classes at submit time to know whether the form is currently valid.

Customization ideas

- Add a phone number field with country code
- Show 'show password' toggle
- Save state on blur across reloads

🕹 Upgrade Challenge

> *Add a real 'username availability' check by simulating an async lookup with setTimeout.*

Project 29: Drag-and-Drop Kanban

Difficulty: ☆☆

What you will build

A three-column Kanban board (To Do / Doing / Done) where tasks can be dragged between columns.

Skills you will learn

- HTML5 drag-and-drop API
- localStorage
- Dynamic rendering

Features

- Three columns
- Add and delete tasks
- Drag to move
- Persist to localStorage

Build it with AI — the prompt to paste

> *Build a 'Kanban' page with three columns: To Do, Doing, Done. The user can add tasks to To Do, drag any task to another column, and delete a task. State persists across reload via localStorage. Pure HTML5 drag API.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Kanban Lite</title>  6    <style>  7      body { font-family: system-ui;
background: #f1f5f9; margin: 0; padding: 20px; }  8      h1 { text-align:
center; }  9      .board { display: grid; grid-template-columns: repeat(3, 1fr);
gap: 14px; max-width: 900px; margin: 0 auto; } 10      .col { background:
#e2e8f0; border-radius: 10px; padding: 12px; min-height: 60vh; } 11      .col h2
{ margin: 0 0 10px; color: #475569; font-size: 16px; } 12      .card { padding:
10px 12px; background: white; border-radius: 6px; margin-bottom: 8px; display:
flex; justify-content: space-between; align-items: center; cursor: grab; box-
shadow: 0 1px 3px rgba(0,0,0,.08); } 13      .card.dragging { opacity: .4; } 14
.col.dragover { outline: 2px dashed #6366f1; } 15      .del { background:
#fee2e2; border: 0; color: #b91c1c; border-radius: 4px; padding: 2px 8px;
cursor: pointer; } 16      form { max-width: 900px; margin: 0 auto 14px;
display: flex; gap: 8px; } 17      form input { flex: 1; padding: 10px; border:
1px solid #cbd5e1; border-radius: 6px; } 18      form button { padding: 10px
16px; background: #2563eb; color: white; border: 0; border-radius: 6px; cursor:
pointer; } 19    </style> 20  </head> 21  <body> 22    <h1>Kanban</h1> 23
<form id="newForm"> 24      <input id="newText" placeholder="New task..."
required> 25      <button>Add to To Do</button> 26    </form> 27    <div
class="board"> 28      <div class="col" data-col="todo"><h2>To Do</h2></div> 29
<div class="col" data-col="doing"><h2>Doing</h2></div> 30      <div class="col"
data-col="done"><h2>Done</h2></div> 31    </div> 32   33    <script> 34
const KEY = "kanban-v1"; 35      let state =
JSON.parse(localStorage.getItem(KEY)) || { todo: [], doing: [], done: [] }; 36
37      function save() { localStorage.setItem(KEY, JSON.stringify(state)); } 38
39      function render() { 40
document.querySelectorAll(".col").forEach(col => { 41          const key =
```

```
col.dataset.col; 42          col.querySelectorAll(".card").forEach(c =>
c.remove()); 43          state[key].forEach(text => { 44            const card =
document.createElement("div"); 45            card.className = "card"; 46
card.draggable = true; 47            card.dataset.text = text; 48
card.innerHTML = `<span>${text}</span><button class="del">×</button>`; 49
card.querySelector(".del").addEventListener("click", () => { 50
state[key] = state[key].filter(t => t !== text); save(); render();
  1           });  2           card.addEventListener("dragstart", e =>
{ card.classList.add("dragging"); e.dataTransfer.setData("text", text); });  3
card.addEventListener("dragend", () => card.classList.remove("dragging"));  4
col.appendChild(card);  5          });  6        });  7      }  8    9
document.querySelectorAll(".col").forEach(col => { 10
col.addEventListener("dragover", e => { e.preventDefault();
col.classList.add("dragover"); }); 11        col.addEventListener("dragleave",
() => col.classList.remove("dragover")); 12        col.addEventListener("drop",
e => { 13          e.preventDefault(); 14
col.classList.remove("dragover"); 15          const text =
e.dataTransfer.getData("text"); 16          if (!text) return; 17          for
(const k of ["todo","doing","done"]) state[k] = state[k].filter(t => t !==
text); 18          state[col.dataset.col].push(text); 19          save();
render(); 20        }); 21      }); 22   23
document.getElementById("newForm").addEventListener("submit", e => { 24
e.preventDefault(); 25        const v =
document.getElementById("newText").value.trim(); 26        if (!v) return; 27
state.todo.push(v); 28        document.getElementById("newText").value = ""; 29
save(); render(); 30      }); 31   32      render(); 33    </script> 34  </body>
35  </html>
```

File structure

```
single file: kanban.html
```

How to run it

Open in any browser.

How it works

The HTML5 drag-and-drop API uses dragstart (sets the data) → dragover (must call preventDefault to allow drop) → drop (reads the data and updates state). We save state to localStorage after every change.

Customization ideas

- Add multi-select and bulk move
- Add a 'pinned' lane at the top
- Add due dates with colour coding

🕹 Upgrade Challenge

> *Add a 'next-up' queue inspired by real agile flows, with priority labels and WIP limits.*

Project 30: Mini Online Storefront

Difficulty: ☆☆

What you will build

A beautifully styled product grid with cart, total, and a fake checkout page.

Skills you will learn

- Component-thinking
- Cart state
- Routing via URL hash

Features

- Product grid
- Add to cart
- Cart with quantity controls
- Fake checkout
- localStorage cart

Build it with AI — the prompt to paste

> *Build a 'Mini Online Storefront'. Display at least six mock products in a grid with name, price and image (use placeholder.com via https://picsum.photos/seed/pN/300/200). Click 'Add to cart', then a separate view via URL hash (#/cart) shows the cart with qty + / - and a 'Checkout' button that shows a fake success screen. Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Mini Store</title>  6    <style>  7      body { font-family: system-ui;
margin: 0; min-height: 100vh; background: #f8fafc; }  8      header
{ background: #1e293b; color: white; padding: 16px 20px; display:flex; justify-
content:space-between; align-items:center; }  9      header a { color: white;
text-decoration: none; } 10      .cart-link { background: #facc15; color:
#422006; padding: 6px 14px; border-radius: 6px; font-weight: 600; }
11      .grid { display: grid; grid-template-columns: repeat(auto-fit,
minmax(220px, 1fr)); gap: 16px; padding: 20px; } 12      .product { background:
white; border-radius: 10px; padding: 14px; text-align: center; box-shadow: 0 1px
4px rgba(0,0,0,.08); } 13      .product img { width: 100%; border-radius: 6px;
aspect-ratio: 4/3; object-fit: cover; } 14      .product button { padding: 8px
14px; background: #2563eb; color: white; border: 0; border-radius: 6px; cursor:
pointer; } 15      main { max-width: 1000px; margin: 0 auto; } 16      .cart-
item { background: white; padding: 12px; border-radius: 8px; margin-bottom: 8px;
display: flex; justify-content: space-between; align-items: center; }
17      .qty button { padding: 4px 10px; } 18      .total { font-size: 22px;
font-weight: 700; margin-top: 16px; } 19      .checkout { padding: 12px 20px;
background: #16a34a; color: white; border: 0; border-radius: 6px; font-size:
16px; cursor: pointer; } 20    </style> 21  </head> 22  <body> 23    <header><a
href="#/"><strong>Mini Store</strong></a><a href="#/cart" class="cart-link">🛒
<span id="cnt">0</span></a></header> 24    <main id="app"></main> 25   26
<script> 27      const products = [ 28        {id:1, name:"Wireless Headphones",
price:2499, img:"https://picsum.photos/seed/p1/300/200"}, 29        {id:2,
name:"Cotton T-Shirt",      price:499,
```

```
img:"https://picsum.photos/seed/p2/300/200"}, 30        {id:3, name:"Travel
Mug",          price:349,  img:"https://picsum.photos/seed/p3/300/200"}, 31
{id:4, name:"Notebook",            price:199,
img:"https://picsum.photos/seed/p4/300/200"}, 32        {id:5, name:"USB-C
Cable",         price:299,  img:"https://picsum.photos/seed/p5/300/200"}, 33
{id:6, name:"Bluetooth Speaker",   price:1599,
img:"https://picsum.photos/seed/p6/300/200"} 34      ]; 35      let cart =
JSON.parse(localStorage.getItem("cart") || "{}"); 36   37      const save = ()
=> localStorage.setItem("cart", JSON.stringify(cart)); 38      const count = ()
=> Object.values(cart).reduce((a,b) => a + b, 0); 39      const total = () =>
products.filter(p=>cart[p.id]).reduce((s,p) => s + p.price * cart[p.id], 0); 40
41      function paintCount() { document.getElementById("cnt").textContent =
count(); } 42   43      function viewShop() { 44        const app =
document.getElementById("app"); 45        app.innerHTML = '<div class="grid">' +
products.map(p => ` 46          <div class="product"> 47            <img src="$
{p.img}" alt="${p.name}"> 48            <h3>${p.name}</h3> 49            <p>₹$
{p.price}</p> 50            <button onclick="add(${p.id})">Add to cart</button>
  1          </div>`).join("") + '</div>';  2      }  3    4      function
viewCart() {  5        const app = document.getElementById("app");  6
const items = products.filter(p => cart[p.id]);  7        if (!items.length)
{ app.innerHTML = '<p style="padding:20px">Cart is empty. <a href="#/">Shop
now</a>.</p>'; return; }  8        app.innerHTML = items.map(p => `  9
<div class="cart-item"> 10            <strong>${p.name}</strong> 11
<span class="qty"> 12              <button onclick="chg(${p.id}, -1)">−</button>
13              ${cart[p.id]} 14              <button onclick="chg(${p.id},
1)">+</button> 15            </span> 16            <span>₹${p.price *
cart[p.id]}</span> 17          </div> 18        `).join("") + `<p
class="total">Total: ₹${total()}</p> 19          <button class="checkout"
onclick="checkout()">Checkout</button> 20          <p style="margin-top:10px"><a
href="#/">← back to shop</a></p>`; 21      } 22   23      window.add = id =>
{ cart[id] = (cart[id] || 0) + 1; save(); paintCount(); }; 24      window.chg =
(id, d) => { cart[id] = (cart[id] || 0) + d; if (cart[id] <= 0) delete cart[id];
save(); paintCount(); viewCart(); }; 25      window.checkout = () => { 26
cart = {}; save(); paintCount(); 27
document.getElementById("app").innerHTML = '<div style="padding:60px;text-
align:center"><h2>Order placed □</h2><p>This is a demo. Nothing was
charged.</p><a href="#/">Continue shopping</a></div>'; 28      }; 29   30
window.addEventListener("hashchange", route); 31      function route() { 32
paintCount(); 33        if (location.hash === "#/cart") viewCart(); else
viewShop(); 34      } 35      route(); 36    </script> 37  </body> 38  </html>
```

File structure

```
single file: shop.html
```

How to run it

Open in any browser. Needs internet for product images.

How it works

Routing is just a hashchange listener. The cart is a plain object keyed by product id with quantities as values. Re-rendering the cart costs almost nothing because the items are simple HTML strings.

Customization ideas

- Add product search
- Show subtotals with coupon code field
- Add wishlist feature

🕹 Upgrade Challenge

> *Add a real 'place order' workflow with form fields for shipping and a summary page.*

Project 31: Weather UI with Mock Data

Difficulty: ☆☆

What you will build

A clean, mobile-friendly weather card UI driven by a curated, locally-stored JSON weather dataset.

Skills you will learn

- Mock data
- Component UI
- Day/time logic

Features

- 7-day forecast cards
- Current weather card
- Hourly bar
- Day/night icons

Build it with AI — the prompt to paste

> *Build a 'Weather UI' page using ONLY mock data defined in a JSON object within the JavaScript (no API needed). Display the current weather as a large card with city, temperature, condition and an icon, plus a 7-day forecast row and a 12-hour hourly forecast bar. Use CSS for icons. Pure JS, single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Weather UI</title>  6    <style>  7      body { font-family: system-ui;
background: linear-gradient(180deg,#60a5fa,#93c5fd); min-height: 100vh; margin:
0; padding: 20px; color: #0f172a; }  8      .now { background:
rgba(255,255,255,.7); backdrop-filter: blur(4px); padding: 28px; border-radius:
16px; max-width: 720px; margin: 0 auto 20px; text-align: center; }
9      .now .city { font-size: 18px; color: #475569; } 10      .now .temp
{ font-size: 80px; font-weight: 200; } 11      .now .cond { font-size: 20px; }
12      .now .icon { font-size: 60px; } 13      .row { display: flex; gap: 10px;
overflow-x: auto; max-width: 720px; margin: 0 auto; padding-bottom: 10px; } 14
.row::-webkit-scrollbar { display: none; } 15      .day { flex: 0 0 100px;
background: rgba(255,255,255,.6); padding: 14px 8px; border-radius: 10px; text-
align: center; } 16      .day .icon { font-size: 32px; } 17      .day .t { font-
weight: 700; font-size: 16px; } 18      .hour { flex: 0 0 80px; background:
rgba(255,255,255,.5); padding: 10px; border-radius: 8px; text-align: center;
font-size: 13px; } 19      .hour .t { font-weight: 700; font-size: 15px; } 20
</style> 21  </head> 22  <body> 23    <div class="now" id="now"></div> 24    <h3
style="text-align:center">Next hours</h3> 25    <div class="row"
id="hours"></div> 26    <h3 style="text-align:center">Next 7 days</h3> 27
<div class="row" id="days"></div> 28   29    <script> 30      // Mock dataset —
designed for the demo to be visually rich. 31      const data = { 32
city: "Mumbai", 33        current: { temp: 31, cond: "Partly Cloudy", icon: "⛅"
}, 34        hours: [ 35          {h:14,t:31,c:31},{h:15,t:32,c:32},
{h:16,t:31,c:31},{h:17,t:30,c:30}, 36          {h:18,t:29,c:29},
{h:19,t:28,c:28},{h:20,t:27,c:27},{h:21,t:27,c:27}, 37
{h:22,t:26,c:26},{h:23,t:26,c:26},{h:0,t:25,c:25},{h:1,t:25,c:25} 38        ],
```

```
39        days: [ 40          {d:"Today", icon:"☀", hi:33, lo:25}, 41
{d:"Tue",   icon:"⛅", hi:32, lo:25}, 42          {d:"Wed",   icon:"🌦", hi:30,
lo:24}, 43          {d:"Thu",   icon:"🌧", hi:28, lo:24}, 44          {d:"Fri",
icon:"⛈", hi:27, lo:23}, 45          {d:"Sat",   icon:"🌤", hi:31, lo:25}, 46
{d:"Sun",   icon:"☀", hi:33, lo:26} 47        ] 48      }; 49   50      const
now = document.getElementById("now");
  1      now.innerHTML = `  2        <div class="city">${data.city}</div>  3
<div class="icon">${data.current.icon}</div>  4        <div class="temp">$
{data.current.temp}°</div>  5        <div
class="cond">${data.current.cond}</div>`;  6    7
document.getElementById("hours").innerHTML = data.hours.map(h =>  8        `<div
class="hour"><div>${h.h}:00</div><div class="t">${h.t}°</div></div>`).join("");
9   10      document.getElementById("days").innerHTML = data.days.map(d => 11
`<div class="day"><div>${d.d}</div><div class="icon">${d.icon}</div> 12
<div class="t">${d.hi}°</div><div
style="color:#64748b">${d.lo}°</div></div>`).join(""); 13    </script> 14
</body> 15  </html>
```

File structure

```
single file: weather-mock.html
```

How to run it

Open in any browser.

How it works

All forecast data lives in a single JS object. Because no real API is involved, the page works offline and is privacy-friendly — both great for learning.

Customization ideas

- Let the user switch between sample cities
- Add simulated day/night themes
- Add wind/humidity indicator

🕹 Upgrade Challenge

> *Replace the mock object with a real fetch() to Open-Meteo (free, no key), keeping the same data shape.*

Project 32: Tabbed Interface

Difficulty: ☆☆

What you will build

A reusable tabbed interface with deep-linkable URLs (#hash) and keyboard arrow navigation.

Skills you will learn

- ARIA roles
- URL hash sync
- Keyboard events

Features

- Three or more tabs
- URL hash sync (#about, #skills, etc.)
- Arrow-key navigation
- Fade transition

Build it with AI — the prompt to paste

> *Build a 'Tabbed Interface'. Three or more tabs, each showing different content. The active tab is reflected in the URL hash so users can bookmark or share direct links. Arrow keys cycle tabs. Vanilla JS only.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Tabs Demo</title>  6    <style>  7      body { font-family: system-ui;
max-width: 800px; margin: 30px auto; padding: 20px; }  8      .tabs { display:
flex; gap: 6px; border-bottom: 2px solid #e2e8f0; }  9      .tab { padding: 10px
16px; cursor: pointer; background: #f1f5f9; border-radius: 6px 6px 0 0; border:
1px solid #e2e8f0; border-bottom: 0; } 10      .tab[aria-selected="true"]
{ background: white; color: #1d4ed8; border-color: #c7d2fe; font-weight: 600; }
11      .panel { padding: 20px; background: white; border: 1px solid #e2e8f0;
border-top: 0; border-radius: 0 0 6px 6px; } 12      .panel[hidden] { display:
none; } 13    </style> 14  </head> 15  <body> 16    <h1>Tabs Demo</h1> 17
<div class="tabs" role="tablist"> 18      <button class="tab" role="tab" data-
tab="about"  aria-selected="true">About</button> 19      <button class="tab"
role="tab" data-tab="skills" aria-selected="false">Skills</button> 20
<button class="tab" role="tab" data-tab="projects" aria-
selected="false">Projects</button> 21      <button class="tab" role="tab" data-
tab="contact" aria-selected="false">Contact</button> 22    </div> 23    <div
class="panel" id="about"    role="tabpanel">Tab content: about.</div> 24    <div
class="panel" id="skills"   role="tabpanel" hidden>Tab content: skills.</div> 25
<div class="panel" id="projects" role="tabpanel" hidden>Tab content:
projects.</div> 26    <div class="panel" id="contact"  role="tabpanel"
hidden>Tab content: contact.</div> 27   28    <script> 29      const tabs =
document.querySelectorAll(".tab"); 30      function activate(name) { 31
tabs.forEach(t => t.setAttribute("aria-selected", t.dataset.tab === name)); 32
document.querySelectorAll(".panel").forEach(p => p.hidden = p.id !== name); 33
if (location.hash !== "#" + name) history.replaceState(null, "", "#" + name); 34
} 35      tabs.forEach(t => t.addEventListener("click", () =>
activate(t.dataset.tab))); 36      document.addEventListener("keydown", e =>
```

```
{ 37        if (!["ArrowLeft","ArrowRight"].includes(e.key)) return; 38
const order = ["about","skills","projects","contact"]; 39        const cur =
order.indexOf((location.hash || "#about").slice(1)); 40        const next =
e.key === "ArrowRight" ? (cur + 1) % order.length : (cur - 1 + order.length) %
order.length; 41        activate(order[next]); 42      }); 43
activate((location.hash || "#about").slice(1)); 44    </script> 45  </body> 46
</html>
```

File structure

```
single file: tabs.html
```

How to run it

Open in any browser.

How it works

Only one panel is visible at a time. We use ARIA roles for accessibility, then sync the active tab to location.hash so deep links work. Arrow keys compute the next index in a circular list.

Customization ideas

- Animate panels on activation
- Lazy-load panel content
- Save last active tab in localStorage

🕹 Upgrade Challenge

> *Build a 'multi-pane dashboard' using nested tab sets as a tiny SPA pattern.*

Project 33: Image Filter Playground

Difficulty: ☆☆

What you will build

Sliders that apply various CSS filters to an image in real time.

Skills you will learn

- CSS filters
- Live UI binding
- Image loading

Features

- Brightness, contrast, blur, hue, saturation, sepia, grayscale
- Reset to default
- Save preset

Build it with AI — the prompt to paste

> *Build an 'Image Filter Playground'. Loads a sample image (use one from picsum) and shows six sliders for brightness, contrast, blur, hue-rotate, saturate and sepia. Sliding updates CSS variables on the image instantly. Add 'reset' and 'randomise' buttons.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Image Filters</title>  6    <style>  7      body { font-family: system-
ui; max-width: 800px; margin: 20px auto; padding: 20px; background: #0f172a;
color: white; min-height: 100vh; }  8      .preview { text-align: center; }  9
.preview img { max-width: 100%; border-radius: 10px; } 10      .controls
{ display: grid; grid-template-columns: repeat(auto-fit, minmax(200px,1fr));
gap: 10px; margin-top: 20px; } 11      .sl { background: rgba(255,255,255,.08);
padding: 10px; border-radius: 8px; } 12      .sl label { display: block; font-
size: 13px; } 13      .sl input { width: 100%; } 14      .actions { margin-top:
16px; display: flex; gap: 8px; } 15      button { padding: 10px 16px;
background: #facc15; color: #422006; border: 0; border-radius: 6px; cursor:
pointer; font-weight: 700; } 16    </style> 17  </head> 18  <body> 19
<h1>Filter Playground</h1> 20    <div class="preview"> 21      <img id="img"
src="https://picsum.photos/seed/filter/800/500" alt="Sample image"> 22    </div>
23    <div class="controls" id="ctrls"></div> 24    <div class="actions"> 25
<button id="reset">Reset</button> 26      <button id="random">Randomize</button>
27    </div> 28   29    <script> 30      const defs = [ 31        { name:
"Brightness", key: "brightness", unit: "", min: 0, max: 200, d: 100 }, 32
{ name: "Contrast",   key: "contrast",   unit: "%", min: 0, max: 200, d: 100 },
33        { name: "Blur",       key: "blur",       unit: "px", min: 0, max: 20,
d: 0 }, 34        { name: "Hue",        key: "hue-rotate", unit: "deg", min: 0,
max: 360, d: 0 }, 35        { name: "Saturate",   key: "saturate",   unit: "%",
min: 0, max: 200, d: 100 }, 36        { name: "Sepia",      key: "sepia",
unit: "%", min: 0, max: 100, d: 0 } 37      ]; 38      const state =
Object.fromEntries(defs.map(d => [d.key, d.d])); 39   40      const img =
document.getElementById("img"); 41      const ctrls =
document.getElementById("ctrls"); 42      ctrls.innerHTML = defs.map(d => ` 43
<div class="sl"> 44          <label>${d.name}: <span id="v-${d.key}">${d.d}$
{d.unit}</span></label> 45          <input type="range" min="${d.min}" max="$
```

```
{d.max}" value="${d.d}" data-key="${d.key}" data-unit="${d.unit}"> 46
</div>`).join(""); 47      ctrls.querySelectorAll("input").forEach(inp => { 48
inp.addEventListener("input", () => { 49          state[inp.dataset.key] =
inp.value; 50          document.getElementById("v-" +
inp.dataset.key).textContent = inp.value + inp.dataset.unit;
  1          paint();  2        });  3      });  4    5      function paint() {
6        img.style.filter = defs.map(d =>  7          `${d.key}(${state[d.key]}$
{d.unit})`).join(" ");  8      }  9   10
document.getElementById("reset").addEventListener("click", () => { 11
defs.forEach(d => state[d.key] = d.d); 12
ctrls.querySelectorAll("input").forEach(inp => { 13          inp.value =
state[inp.dataset.key]; 14          document.getElementById("v-" +
inp.dataset.key).textContent = inp.value + inp.dataset.unit; 15        }); 16
paint(); 17      }); 18   19
document.getElementById("random").addEventListener("click", () => { 20
defs.forEach(d => state[d.key] = Math.floor(Math.random() * (d.max - d.min) +
d.min)); 21        ctrls.querySelectorAll("input").forEach(inp => { 22
inp.value = state[inp.dataset.key]; 23          document.getElementById("v-" +
inp.dataset.key).textContent = inp.value + inp.dataset.unit; 24        }); 25
paint(); 26      }); 27      paint(); 28    </script> 29  </body> 30  </html>
```

File structure

```
single file: filters.html
```

How to run it

Open in any browser. Needs internet for picsum image.

How it works

CSS filter property accepts a list of functions. We set each in a shared template string and the browser applies them all in one pass — far cheaper than re-rendering any image data.

Customization ideas

- Add presizable vignette
- Add background colour overlay
- Add download button to save the styled image

🕹 Upgrade Challenge

> *Use canvas to convert filtered image to a downloadable PNG.*

Project 34: Accordion FAQ

Difficulty: ☆☆

What you will build

An accessible FAQ component where users can expand and collapse questions.

Skills you will learn

- ARIA roles
- Height transitions
- Accordion behaviour

Features

- Multiple sections
- One open at a time
- Smooth open animation
- Keyboard accessible

Build it with AI — the prompt to paste

> *Build a reusable 'Accordion FAQ'. Show at least six questions. Only one section open at a time. Clicking expands smoothly via CSS height transitions. The component must be accessible: arrow keys move focus, Enter / Space toggle, ARIA states correct.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>FAQ</title>  6    <style>  7      body { font-family: system-ui; max-
width: 720px; margin: 30px auto; padding: 20px; }  8      h1 { color: #1e40af; }
9      details { background: white; border: 1px solid #e2e8f0; border-radius:
8px; margin-bottom: 10px; } 10      summary { padding: 14px 18px; cursor:
pointer; font-weight: 600; list-style: none; position: relative; } 11
summary::after { content: "+"; position: absolute; right: 18px; transition:
transform .3s; } 12      details[open] summary::after { content: "−"; }
13      .body { padding: 0 18px 16px; color: #475569; line-height: 1.5; } 14
summary:focus { outline: 2px solid #6366f1; } 15    </style> 16  </head> 17
<body> 18    <h1>Frequently Asked Questions</h1> 19   20    <details> 21
<summary>What is this FAQ about?</summary> 22      <div class="body">It explains
how to build accessible web components. The answer is intentionally short.</div>
23    </details> 24    <details> 25      <summary>Do I need a
framework?</summary> 26      <div class="body">No. Everything here uses native
HTML and JavaScript only.</div> 27    </details> 28    <details> 29
<summary>Can I copy and reuse this pattern?</summary> 30      <div
class="body">Yes — the &lt;details&gt; element is the simplest accessible
accordion.</div> 31    </details> 32    <details> 33      <summary>How do I make
only one open at a time?</summary> 34      <div class="body">Listen for the
'toggle' event on each details, then close the rest.</div> 35    </details> 36
<details> 37      <summary>Is it keyboard-accessible?</summary> 38      <div
class="body">Yes — the &lt;summary&gt; element is naturally focusable and
toggles on Enter or Space.</div> 39    </details> 40    <details> 41
<summary>Can I add icons?</summary> 42      <div class="body">Yes — using CSS
pseudo-elements like ::before or ::after.</div> 43    </details> 44   45
<script> 46      // Close other details when one opens 47
```

```
document.querySelectorAll("details").forEach(d => { 48
d.addEventListener("toggle", () => { 49          if (d.open)
document.querySelectorAll("details").forEach(o => { if (o !== d) o.open = false;
}); 50        });
  1      });  2    </script>  3  </body>  4  </html>
```

File structure

```
single file: faq.html
```

How to run it

Open in any browser.

How it works

<details> and <summary> are built-in HTML elements that handle all open/close state for free. We only need a small JS shim to enforce 'one at a time' and CSS to style the +/− icon.

Customization ideas

- Add a search box that filters questions
- Persist the last-open item across reload
- Add an animation that slides content

🕹 Upgrade Challenge

> *Build a multi-section FAQ where each section is itself an accordion (nested accordions).*

Project 35: Searchable Movie List

Difficulty: ☆☆

What you will build

A searchable, filterable, paginated list of mock movies with a clean detail view.

Skills you will learn

- Live search
- Filtering
- Pagination

Features

- Live text search
- Genre filter
- Sort by year or rating
- Pagination (10 per page)

Build it with AI — the prompt to paste

> *Build a 'Searchable Movie List'. Use a hard-coded array of at least 30 fictional movies with title, year, genre and rating. The user can search by title, filter by genre (dropdown), sort by year or rating, and page through results. Pure JS, single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Movie List</title>  6    <style>  7      body { font-family: system-ui;
max-width: 800px; margin: 30px auto; padding: 20px; }  8      .controls
{ display: flex; flex-wrap: wrap; gap: 10px; margin-bottom: 14px; }  9
input, select, button { padding: 8px 12px; font-size: 14px; border-radius: 6px;
border: 1px solid #cbd5e1; } 10      .card { background: white; padding: 14px;
border: 1px solid #e2e8f0; border-radius: 8px; margin-bottom: 8px; display:
flex; justify-content: space-between; } 11      .meta { color: #64748b; font-
size: 13px; } 12      .rating { background: #facc15; color: #422006; padding:
4px 10px; border-radius: 4px; font-weight: 600; } 13      .pages { display:
flex; gap: 6px; margin-top: 14px; } 14      .pages button { padding: 6px 10px;
cursor: pointer; } 15      .pages button.active { background: #2563eb; color:
white; } 16    </style> 17  </head> 18  <body> 19    <h1>Movies</h1> 20    <div
class="controls"> 21      <input id="q" placeholder="Search title..."> 22
<select id="g"><option value="">All genres</option> 23
<option>Drama</option><option>Comedy</option><option>Action</option><option>Sci-
Fi</option><option>Animation</option> 24      </select> 25      <select
id="s"><option value="rating">Sort: rating</option> 26        <option
value="year">Sort: year</option></select> 27    </div> 28    <div
id="list"></div> 29    <div class="pages" id="pages"></div> 30   31    <script>
32      const movies = Array.from({length: 32}, (_, i) => ({ 33        title:
["Quiet Rain","Velvet Sky","Long Path","Hidden Light","The Letter","Empty
Room","Crimson Day","Twin Stars", 34                  "Letters Home","Far
Echo","Autumn Room","Last Mile"][i % 12] + " " + (i + 1), 35        year: 2005 +
(i * 3 % 18), 36        genre: ["Drama","Comedy","Action","Sci-Fi","Animation"]
[i % 5], 37        rating: Math.round((5 + (i * 7 % 50) / 10) * 10) / 10 38
}));  39      let page = 1;  40      const PER = 8;  41   42      function render()
```

```
{ 43        const q = document.getElementById("q").value.toLowerCase(); 44
const g = document.getElementById("g").value; 45        const s =
document.getElementById("s").value; 46        let view = movies.filter(m => 47
(!q || m.title.toLowerCase().includes(q)) && 48          (!g || m.genre === g));
49        view.sort((a,b) => s === "year" ? b.year - a.year : b.rating -
a.rating); 50        const total = Math.max(1, Math.ceil(view.length / PER));
  1        page = Math.min(page, total);  2        const slice =
view.slice((page - 1) * PER, page * PER);  3    4
document.getElementById("list").innerHTML = slice.length ? slice.map(m =>  5
`<div class="card"><div><strong>${m.title}</strong><div class="meta">${m.year} ·
${m.genre}</div></div>  6           <div class="rating">★
${m.rating}</div></div>`).join("") :  7          '<p>No movies match.</p>';  8
9        const p = document.getElementById("pages"); 10        p.innerHTML = "";
11        for (let i = 1; i <= total; i++) { 12          const b =
document.createElement("button"); 13          b.textContent = i; 14          if
(i === page) b.className = "active"; 15          b.addEventListener("click", ()
=> { page = i; render(); }); 16          p.appendChild(b); 17        } 18      }
19   20      ["q","g","s"].forEach(id =>
document.getElementById(id).addEventListener("input", () => { page = 1;
render(); })); 21      render(); 22    </script> 23  </body> 24  </html>
```

File structure

```
single file: movies.html
```

How to run it

Open in any browser.

How it works

We use array.filter to combine two predicates and array.sort with a custom comparator to choose the sort. Pagination maths is just math: total pages = ceil(length / perPage).

Customization ideas

- Add a 'favourites' toggle
- Add a 'random pick' button
- Show genre histogram

🕹 Upgrade Challenge

> *Add a detail page that opens when you click a movie, with a synopsis and a 'go back' flow.*

Project 36: Toast Notification System

Difficulty: ☆☆

What you will build

A reusable toast (popup) component on a page that demonstrates four types of notifications.

Skills you will learn

- Component pattern
- Timers
- Queue management

Features

- Success, error, info, warning
- Auto-dismiss
- Click to dismiss
- Stack at the top

Build it with AI — the prompt to paste

> *Build a 'Toast Notification System'. Provide four buttons: Show Success, Show Error, Show Info, Show Warning. Each spawns a toast message at the top-right. Toasts auto-dismiss after 3 seconds and can be clicked to dismiss earlier. Stack vertically if multiple are open.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Toasts</title>  6    <style>  7      body { font-family: system-ui;
padding: 30px; background: #f8fafc; min-height: 100vh; }  8      button
{ padding: 10px 14px; margin: 4px; border: 0; border-radius: 6px; cursor:
pointer; color: white; font-weight: 600; }  9      .ok { background:
#16a34a; } .err { background: #dc2626; } .info { background: #2563eb; } .warn
{ background: #f59e0b; } 10      .stack { position: fixed; top: 20px; right:
20px; display: flex; flex-direction: column; gap: 10px; max-width: 320px; z-
index: 50; } 11      .toast { padding: 12px 16px; border-radius: 8px; color:
white; display: flex; justify-content: space-between; align-items: center; box-
shadow: 0 4px 12px rgba(0,0,0,.18); animation: slidein .25s; cursor: pointer; }
12      .ok-t { background: #16a34a; } .err-t { background: #dc2626; } .info-t {
background: #2563eb; } .warn-t { background: #f59e0b; } 13      @keyframes
slidein { from { transform: translateX(120%); } to { transform: translateX(0); }
} 14    </style> 15  </head> 16  <body> 17    <h1>Toast Demo</h1> 18    <button
class="ok"   onclick="toast('Saved!', 'ok-t')">Show Success</button> 19
<button class="err"  onclick="toast('Something failed.', 'err-t')">Show
Error</button> 20    <button class="info" onclick="toast('Heads up: new update
available.', 'info-t')">Show Info</button> 21    <button class="warn"
onclick="toast('Check your input.', 'warn-t')">Show Warning</button> 22    <div
class="stack" id="stack"></div> 23   24    <script> 25      function toast(text,
kind) { 26        const el = document.createElement("div"); 27
el.className = "toast " + kind; 28        el.innerHTML =
`<span>${text}</span><span style="opacity:.7;margin-left:10px">×</span>`; 29
const remove = () => { el.style.opacity = "0"; setTimeout(() => el.remove(),
250); }; 30        el.addEventListener("click", remove); 31
```

```
document.getElementById("stack").appendChild(el); 32        setTimeout(remove,
3000); 33      } 34    </script> 35  </body> 36  </html>
```

File structure

```
single file: toasts.html
```

How to run it

Open in any browser.

How it works

Each toast is plain DOM. Clicking immediately removes it; otherwise setTimeout dismisses it after 3s. The stack is just a vertical flex container on the top-right.

Customization ideas

- Add a 'progress bar' on each toast
- Add a 'pause on hover' option
- Limit the visible count to 5

🕹 Upgrade Challenge

> *Build a confirm dialog using the same pattern but blocking — modal with overlay.*

Project 37: Analog Clock

Difficulty: ☆☆

What you will build

A real-time analog wall clock face with smooth rotating hands and tick marks.

Skills you will learn

- CSS transforms
- requestAnimationFrame
- Geometric time-math

Features

- Smooth hour, minute, second hands
- 12 tick marks
- Digital time underneath

Build it with AI — the prompt to paste

> *Build an 'Analog Clock'. A circular face with 12 tick marks and three hands (hour, minute, seconds) that rotate smoothly. Hands are absolutely-positioned divs rotated by transform: rotate(). Pure JS one file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Analog Clock</title>  6    <style>  7      body { display:flex; flex-
direction: column; align-items: center; justify-content: center; min-height:
100vh; margin: 0; background: #0f172a; color: white; font-family: system-ui; }
8      .face { width: 320px; height: 320px; border-radius: 50%; background:
radial-gradient(circle at top, #f8fafc, #cbd5e1); border: 6px solid #1e293b;
position: relative; box-shadow: 0 0 30px rgba(0,0,0,.5); }  9      .tick
{ position: absolute; left: 50%; top: 6px; width: 4px; height: 16px; background:
#1e293b; transform-origin: 50% 154px; margin-left: -2px; } 10      .hand
{ position: absolute; left: 50%; bottom: 50%; transform-origin: 50% 100%;
border-radius: 4px; } 11      .h { width: 6px; height: 80px; background:
#1e293b; margin-left: -3px; } 12      .m { width: 4px; height: 110px;
background: #334155; margin-left: -2px; } 13      .s { width: 2px; height:
130px; background: #dc2626; margin-left: -1px; } 14      .center { position:
absolute; left: 50%; top: 50%; width: 12px; height: 12px; background: #1e293b;
border-radius: 50%; margin: -6px; } 15      .digital { margin-top: 18px; font-
size: 28px; font-variant-numeric: tabular-nums; } 16    </style> 17  </head> 18
<body> 19    <div class="face" id="face"> 20      <div class="center"></div> 21
<div class="hand h" id="h"></div> 22      <div class="hand m" id="m"></div> 23
<div class="hand s" id="s"></div> 24    </div> 25    <div class="digital"
id="d"></div> 26   27    <script> 28      const face =
document.getElementById("face"); 29      // Add 12 tick marks at 30° each 30
for (let i = 0; i < 12; i++) { 31        const t =
document.createElement("div"); 32        t.className = "tick"; 33
t.style.transform = `rotate(${i * 30}deg)`; 34        face.appendChild(t); 35
} 36   37      function tick() { 38        const now = new Date(); 39
const ms = now.getMilliseconds(); 40        const s = now.getSeconds() + ms /
1000; 41        const m = now.getMinutes() + s / 60; 42        const h =
(now.getHours() % 12) + m / 60; 43
document.getElementById("s").style.transform = `rotate(${s * 6}deg)`; 44
```

```
document.getElementById("m").style.transform = `rotate(${m * 6}deg)`; 45
document.getElementById("h").style.transform = `rotate(${h * 30}deg)`; 46
document.getElementById("d").textContent = 47
now.toLocaleTimeString(undefined, { hour: "2-digit", minute: "2-digit", second:
"2-digit", hour12: false }); 48        requestAnimationFrame(tick); 49      } 50
tick();
  1    </script>  2  </body>  3  </html>
```

File structure

```
single file: analog.html
```

How to run it

Open in any browser.

How it works

Each hand is a div whose transform-origin is its base. Seconds spin 360° in 60 seconds → 6° per second. The hour hand also moves as minutes pass, giving smooth motion.

Customization ideas

- Add Roman numerals
- Add an alarm that glows when triggered
- Add chime on the hour

🕹 Upgrade Challenge

> *Add a world clock face with multiple smaller dials, one per city.*

Project 38: Personal Blog Template

Difficulty: ☆☆

What you will build

A clean, readable blog layout with article cards and a sample article reader.

Skills you will learn

- Article layout
- Typography
- Content modelling

Features

- Article list cards
- Article reader view
- Reading time
- Tag list
- Share links

Build it with AI — the prompt to paste

> *Build a 'Personal Blog Template'. Two views: a list of articles with a thumbnail, title, excerpt and tag, and a single-article view that opens when a card is clicked. Include at least three mock articles. Pure JS and HTML, single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>My Blog</title>  6    <style>  7      body { font-family: Georgia, serif;
max-width: 760px; margin: 30px auto; padding: 0 20px; color: #1f2937; line-
height: 1.7; }  8      header h1 { border-bottom: 2px solid #1e3a8a; padding-
bottom: 10px; }  9      .card { border-bottom: 1px solid #e2e8f0; padding: 20px
0; cursor: pointer; } 10      .card h2 { margin: 0; color: #1e3a8a; }
11      .card small { color: #64748b; } 12      .card p { margin: 6px 0; } 13
.tag { background: #e0e7ff; color: #3730a3; padding: 2px 8px; border-radius:
4px; font-size: 13px; } 14      article h1 { color: #1e3a8a; } 15      .meta
{ color: #64748b; font-size: 14px; } 16      a.back { color: #2563eb; text-
decoration: none; } 17    </style> 18  </head> 19  <body> 20    <header><h1>📝 My
Blog</h1></header> 21    <main id="app"></main> 22   23    <script> 24
const posts = [ 25        { id:1, title:"How I Built My First Website",
date:"2026-07-12", tag:"Coding", body:"Building my first website was the most
fun I have ever had with a computer. I started with a blank file, wrote one line
at a time, and reloaded the page to see what changed. Slowly, words turned into
boxes, then boxes turned into a layout, and finally the layout turned into
something I could call mine. In this post I'll walk you through that journey,
the mistakes I made, and what I'd do differently next time." }, 26
{ id:2, title:"Why I Stopped Watching Tutorials", date:"2026-08-01",
tag:"Learning", body:"Tutorials are great. But after a while, I noticed that I
could follow along with anything yet still couldn't build anything on my own. So
I switched. I started small: instead of watching, I tried to recreate things I'd
already seen. Some days I succeeded. Most days I failed. But I learned ten times
faster." }, 27        { id:3, title:"Lessons from 100 Days of Code", date:"2026-
08-20", tag:"Habits", body:"I just finished 100 days of code. The biggest
lesson: small daily practice beats big weekly sprints. On tired days, even
```

```
twenty minutes counted. On busy days, I read other people's code instead of
writing my own. The streak was the trick — it kept me coming back." } 28      ];
29   30      function list() { 31
document.getElementById("app").innerHTML = posts.map(p => ` 32          <div
class="card" onclick="location.hash = '#/post/${p.id}'"> 33            <h2>$
{p.title}</h2> 34            <small>${p.date} · <span
class="tag">${p.tag}</span></small> 35            <p>${p.body.slice(0,
140)}...</p> 36          </div>`).join(""); 37      } 38      function post(id)
{ 39        const p = posts.find(x => x.id === +id); 40        if (!p) return
list(); 41        document.getElementById("app").innerHTML = `<a class="back"
href="#/">← all posts</a> 42          <article> 43
<h1>${p.title}</h1> 44            <p class="meta">${p.date} · ${p.tag} · $
{Math.max(1, Math.round(p.body.length / 1200))} min read</p> 45            <p>$
{p.body}</p> 46          </article>`; 47      } 48      function route() { 49
const m = location.hash.match(/^#\/post\/(\d+)/); 50        m ? post(m[1]) :
list();
  1      }  2      addEventListener("hashchange", route);  3      route();  4
</script>  5  </body>  6  </html>
```

File structure

```
single file: blog.html
```

How to run it

Open in any browser.

How it works

Routing is a tiny regex match on location.hash. The list posts show excerpts (first 140 chars). The full post is a typographic-friendly article using Georgia for readability.

Customization ideas

- Add comments using localStorage
- Add a tag-based filter on the list
- Add a 'related posts' section

🕹 Upgrade Challenge

> *Add a simple admin mode: type 'admin' in the URL hash to compose a new post via a textarea, saved to localStorage.*

Project 39: Recipe Card Component

Difficulty: ☆☆

What you will build

A reusable recipe card component showing ingredients, steps, and a serving-size adjuster.

Skills you will learn

- Component pattern
- Scaling logic
- Localised formatting

Features

- Hero image
- Ingredients with checkable boxes
- Numbered steps
- Servings adjuster
- Cooking time

Build it with AI — the prompt to paste

> *Build a 'Recipe Card Component'. Display one recipe with a hero image, description, list of ingredients, numbered steps, and a 'servings' adjuster that scales ingredient amounts. Pure JS one file. Use a real placeholder image from picsum.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Recipe Card</title>  6    <style>  7      body { font-family: system-ui;
max-width: 720px; margin: 30px auto; padding: 20px; background: #fffbeb; }  8
.hero { width: 100%; aspect-ratio: 16/9; object-fit: cover; border-radius: 10px;
}  9      h1 { color: #b45309; margin-bottom: 4px; } 10      .meta { color:
#78350f; font-size: 14px; } 11      .grid { display: grid; grid-template-
columns: 1fr 1fr; gap: 16px; margin: 20px 0; } 12      .panel { background:
white; padding: 14px; border-radius: 10px; } 13      .panel h2 { margin-top: 0;
color: #b45309; } 14      ul { padding-left: 20px; } 15      li { margin: 4px 0;
} 16      input[type=checkbox] { transform: scale(1.2); margin-right: 6px; } 17
.step { background: white; padding: 12px; border-radius: 8px; margin-bottom:
8px; } 18      .step b { color: #b45309; margin-right: 8px; } 19      .servings
{ display: flex; align-items: center; gap: 10px; margin: 14px 0; }
20      .servings button { padding: 6px 12px; font-size: 16px; background:
#b45309; color: white; border: 0; border-radius: 6px; cursor: pointer; } 21
</style> 22  </head> 23  <body> 24    <img class="hero"
src="https://picsum.photos/seed/recipe/720/405" alt="Recipe"> 25    <h1>One-Pan
Tomato Pasta</h1> 26    <p class="meta">⏱ 25 min · 🍽 Servings: <span
id="servings">4</span></p> 27   28    <div class="servings"> 29      <button
onclick="adjust(-1)">−</button> 30      <button onclick="adjust(1)">+</button>
31      <span>Adjust servings</span> 32    </div> 33   34    <div class="grid">
35      <div class="panel"> 36        <h2>Ingredients</h2> 37        <ul
id="ings"></ul> 38      </div> 39      <div class="panel"> 40
<h2>Steps</h2> 41        <div id="steps"></div> 42      </div> 43    </div> 44
45    <script> 46      const base = 4; 47      const ingr = [ 48        { name:
```

```
"Pasta",         amount: 400, unit: "g" }, 49        { name: "Cherry tomatoes",
amount: 300, unit: "g" }, 50        { name: "Garlic cloves",   amount: 4,
unit: "" },
  1        { name: "Olive oil",       amount: 3,   unit: "tbsp" },  2
{ name: "Salt",            amount: 1,   unit: "tsp" },  3        { name: "Basil
leaves",    amount: 10,  unit: "" }  4      ];  5    6      function paint()
{  7        const s = parseInt(document.getElementById("servings").textContent);
8        const factor = s / base;  9
document.getElementById("ings").innerHTML = ingr.map(i => 10
`<li><label><input type="checkbox"> ${Math.round(i.amount * factor * 10) / 10} $
{i.unit} ${i.name}</label></li>` 11        ).join(""); 12
document.getElementById("steps").innerHTML = `        <div
class="step"><b>1</b>Boil water. Add pasta and cook 9 minutes, drain and reserve
a cup of water.</div> 13          <div class="step"><b>2</b>In a wide pan, heat
olive oil over medium. Add sliced garlic for 30 seconds.</div> 14          <div
class="step"><b>3</b>Add tomatoes. Cook until they collapse, about 6
minutes.</div> 15          <div class="step"><b>4</b>Add pasta and a splash of
reserved water. Toss, season, tear basil on top.</div>`; 16      } 17
function adjust(d) { 18        const el = document.getElementById("servings");
19        const v = Math.max(1, Math.min(12, parseInt(el.textContent) + d)); 20
el.textContent = v; 21        paint(); 22      } 23      paint(); 24
</script> 25  </body> 26  </html>
```

File structure

```
single file: recipe.html
```

How to run it

Open in any browser. Needs internet for the picsum image.

How it works

We base ingredient amounts on a known serving count and multiply by a factor. The same approach scales any measurement-based content (like ingredient lists in any recipe).

Customization ideas

- Add nutritional estimate (calories, protein)
- Add unit converter cup/g/oz
- Add 'shopping list total' button

🕹 Upgrade Challenge

> *Allow the user to swap ingredient amounts to a 'metric' or 'imperial' variant using unit conversions.*

Project 40: Resume Builder

Difficulty: ☆☆☆

What you will build

Fill in a form and instantly see a downloadable, printable styled resume preview.

Skills you will learn

- Form bindings
- Print CSS
- DOM templating

Features

- Real-time preview
- Multiple sections
- Print-friendly @media print
- Download as HTML

Build it with AI — the prompt to paste

> *Build a 'Resume Builder'. Fill-in form on the left, live-rendered resume on the right. Sections: Personal, Summary, Experience, Education, Skills, Projects. Print button uses window.print() with @media print rules. Download HTML button saves the preview to a downloadable .html file via a Blob URL.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Resume Builder</title>  6    <style>  7      body { font-family: system-
ui; margin: 0; display: grid; grid-template-columns: 380px 1fr; min-height:
100vh; }  8      form { background: #f1f5f9; padding: 20px; overflow: auto; }  9
form h2 { font-size: 16px; margin: 18px 0 6px; color: #1e40af; } 10      form
input, form textarea { width: 100%; padding: 8px; margin-top: 4px; border: 1px
solid #cbd5e1; border-radius: 6px; font: inherit; box-sizing: border-box; } 11
form textarea { min-height: 60px; } 12      .actions { display: flex; gap: 8px;
margin-top: 14px; } 13      .actions button { padding: 10px 14px; border: 0;
border-radius: 6px; cursor: pointer; } 14      .actions .print { background:
#2563eb; color: white; } 15      .actions .dl { background: #16a34a; color:
white; } 16      .preview { padding: 40px; background: white; overflow: auto; }
17      .preview h1 { margin: 0; color: #0f172a; } 18      .preview .contact
{ color: #64748b; font-size: 13px; margin-bottom: 16px; } 19      .preview h3
{ color: #1e40af; border-bottom: 1px solid #e2e8f0; padding-bottom: 4px; } 20
.preview .item { margin-bottom: 8px; } 21      .item b { display: block; } 22
.item small { color: #64748b; } 23      @media print { 24        form, .actions
{ display: none; } 25        body { display: block; } 26        .preview
{ padding: 10mm; } 27      } 28    </style> 29  </head> 30  <body> 31    <form
id="f"> 32      <h2>Personal</h2> 33      <input data-id="name"
placeholder="Full name"> 34      <input data-id="role"   placeholder="Title
(e.g. Junior Developer)"> 35      <input data-id="email"  placeholder="Email">
36      <input data-id="phone"  placeholder="Phone"> 37      <input data-
id="loc"    placeholder="City, Country"> 38   39      <h2>Summary</h2> 40
<textarea data-id="summary"></textarea> 41   42      <h2>Experience</h2> 43
<textarea data-id="exp" rows="5" placeholder="One item per line: Role | Company
| Start–End | Description"></textarea> 44   45      <h2>Education</h2> 46
```

```
<textarea data-id="edu" rows="3" placeholder="One item per line: Degree |
Institute | Year"></textarea> 47   48      <h2>Skills</h2> 49      <textarea
data-id="skills" placeholder="Comma separated"></textarea> 50
  1      <h2>Projects</h2>  2      <textarea data-id="projects" rows="3"
placeholder="One item per line: Name | Link | Description"></textarea>  3    4
<div class="actions">  5        <button type="button" class="print"
onclick="window.print()">Print / PDF</button>  6        <button type="button"
class="dl" onclick="download()">Download HTML</button>  7      </div>  8
</form>  9   10    <div class="preview" id="pv"></div> 11   12    <script> 13
const inp = document.querySelectorAll("[data-id]"); 14      function escape(s) {
return s.replace(/[&<>]/g, c => ({'&':'&amp;','<':'&lt;','>':'&gt;'}[c])); } 15
function get(id) { return document.querySelector(`[data-id="${id}"]`).value; }
16   17      function paint() { 18        const lines = (s, sep) =>
get(s).split('\n').filter(l => l.trim()); 19        const exp =
lines('exp').map(l => l.split('|').map(x => x.trim())); 20        const edu =
lines('edu').map(l => l.split('|').map(x => x.trim())); 21        const proj =
lines('projects').map(l => l.split('|').map(x => x.trim())); 22   23
document.getElementById("pv").innerHTML = ` 24          <h1>${escape(get('name')
|| 'Your Name')}</h1> 25          <div class="contact">${escape(get('role') ||
'Title')} · ${escape(get('email'))} · ${escape(get('phone'))} · $
{escape(get('loc'))}</div> 26          ${get('summary') ? `<h3>Summary</h3><p>$
{escape(get('summary'))}</p>` : ''} 27          ${exp.length ?
`<h3>Experience</h3>` + exp.map(x => `<div class="item"><b>${escape(x[0] ||
'')}</b> <small>${escape(x[1] || '')}</small><br>${escape(x[2] ||
'')}</div>`).join("") : ''} 28          ${edu.length ? `<h3>Education</h3>` +
edu.map(x => `<div class="item"><b>${escape(x[0] || '')}</b> <small>$
{escape(x[1] || '')} ${escape(x[2] || '')}</small></div>`).join("") : ''} 29
${get('skills') ? `<h3>Skills</h3><p>$
{get('skills').split(',').map(escape).join(' · ')}</p>` : ''} 30          $
{proj.length ? `<h3>Projects</h3>` + proj.map(x => `<div class="item"><b>$
{escape(x[0] || '')}</b> <small>${escape(x[1] || '')}</small><br>${escape(x[2]
|| '')}</div>`).join("") : ''} 31        `; 32      } 33   34
inp.forEach(el => el.addEventListener("input", paint)); 35   36      function
download() { 37        const html = `<!doctype html><html><head><meta
charset="utf-8"><title>${escape(get('name'))} - Resume</title> 38
<style>${document.querySelector("style").textContent}</style></head> 39
<body><div class="preview">${document.getElementById("pv").innerHTML}</div></
body></html>`; 40        const blob = new Blob([html], { type: "text/html" });
41        const a = document.createElement("a"); 42        a.href =
URL.createObjectURL(blob); 43        a.download = (get('name') ||
'resume').replace(/\s+/g, '-') + ".html"; 44        a.click(); 45      } 46
</script> 47  </body> 48  </html>
```

File structure

```
single file: resume.html
```

How to run it

Open in any browser.

How it works

Each input has a data-id attribute. We attach one input listener per field and re-render on every change — cheap because the preview is just a few dozen nodes. @media print hides the form when printing.

Customization ideas

- Add a profile photo upload

- Allow section reordering
- Save to localStorage and reload

🕹 Upgrade Challenge

> *Add multiple resume templates and let the user select one.*

Part 3 — Games & Interactive Projects

Games teach timing, state, animation and user feedback — the parts of code that feel alive.

Project 41: Number Guessing Game

Difficulty: ☆

What you will build

The classic: the game picks a number, you guess, it tells you higher or lower.

Skills you will learn

- Loops
- Conditions
- Random numbers

Features

- Three difficulty levels
- Guess counter
- Higher/lower hints
- Win celebration

Build it with AI — the prompt to paste

> *Build a 'Number Guessing Game'. The computer picks a random number. The user guesses. The page says 'Higher' or 'Lower' until the guess is right. Tracks attempts and best score. Three difficulty levels (Easy 1-50, Medium 1-100, Hard 1-500). Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Number Guess</title>  6    <style>  7      body { font-family: system-ui;
max-width: 420px; margin: 50px auto; text-align: center; }  8      h1 { color:
#1d4ed8; }  9      .diff { display: flex; gap: 6px; justify-content: center;
margin-bottom: 20px; } 10      .diff button { padding: 8px 14px; border: 0;
border-radius: 6px; cursor: pointer; background: #e2e8f0; } 11      .diff
button.active { background: #1d4ed8; color: white; } 12      input { padding:
10px; font-size: 18px; width: 120px; } 13      button.go { padding: 10px 18px;
background: #16a34a; color: white; border: 0; border-radius: 6px; cursor:
pointer; font-size: 16px; } 14      .msg { margin-top: 20px; font-size: 20px;
min-height: 30px; } 15      .win { color: #16a34a; font-weight: 700; }
16      .hi { color: #f59e0b; } .lo { color: #2563eb; } 17      .stats { margin-
top: 16px; color: #64748b; font-size: 14px; } 18    </style> 19  </head> 20
<body> 21    <h1>🔢 Number Guessing Game</h1> 22    <div class="diff" id="diff">
23      <button data-max="50"  class="active">Easy (1-50)</button> 24
<button data-max="100">Medium (1-100)</button> 25      <button data-
max="500">Hard (1-500)</button> 26    </div> 27    <input id="g" type="number"
placeholder="Your guess"> 28    <button class="go" id="go">Guess</button> 29
<div class="msg" id="msg">Pick a difficulty and guess!</div> 30    <div
class="stats" id="stats"></div> 31   32    <script> 33      let target, max =
50, attempts = 0, best = JSON.parse(localStorage.getItem("best") || "{}"); 34
function newGame() { 35        target = Math.floor(Math.random() * max) + 1; 36
attempts = 0; 37        paint(); 38
document.getElementById("msg").textContent = `Guess between 1 and ${max}`; 39
} 40      function paint() { 41
document.getElementById("stats").textContent = "Attempts: " + attempts +
(best[max] ? " · Best: " + best[max] : ""); 42      } 43
```

```
document.querySelectorAll("#diff button").forEach(b => { 44
b.addEventListener("click", () => { 45          document.querySelectorAll("#diff
button").forEach(x => x.classList.remove("active")); 46
b.classList.add("active"); 47          max = +b.dataset.max; 48
newGame(); 49        }); 50      });
  1      document.getElementById("go").addEventListener("click", () => {  2
const v = +document.getElementById("g").value;  3        attempts++;  4
paint();  5        if (v === target) {  6
document.getElementById("msg").innerHTML = "<span class='win'>Correct!
</span>";  7          best[max] = best[max] ? Math.min(best[max], attempts) :
attempts;  8          localStorage.setItem("best", JSON.stringify(best));  9
setTimeout(newGame, 1500); 10        } else if (v < target) { 11
document.getElementById("msg").innerHTML = "<span class='lo'>Higher ⬆</span>";
12        } else { 13          document.getElementById("msg").innerHTML = "<span
class='hi'>Lower ⬇</span>"; 14        } 15
document.getElementById("g").value = ""; 16
document.getElementById("g").focus(); 17      }); 18      newGame(); 19
</script> 20  </body> 21  </html>
```

File structure

```
single file: guess.html
```

How to run it

Open in any browser.

How it works

Math.floor(Math.random() * max) + 1 gives an integer in [1, max]. We compare the guess, branch on higher or lower, and use localStorage to remember the player's best score per difficulty.

Customization ideas

- Add a 'binary search hint' button
- Animate a thermometer of closeness
- Add multiplayer hot-seat

🕹 Upgrade Challenge

> *Build a 'reverse mode': the player thinks of a number, and the program guesses it in 7 questions.*

Project 42: Rock Paper Scissors

Difficulty: ☆

What you will build

Click rock, paper or scissors. The computer picks its move and the page declares the winner.

Skills you will learn

- Random selection
- Win/lose logic
- Score tracking

Features

- Three animated hand icons
- Score tracker
- Streak counter
- Reset

Build it with AI — the prompt to paste

> *Build 'Rock Paper Scissors'. Three buttons for player moves. Computer picks randomly. Show both choices with emoji animations, the result, and update the score. Streak counter shows consecutive wins. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>RPS</title>  6    <style>  7      body { font-family: system-ui; max-
width: 480px; margin: 40px auto; text-align: center; }  8      h1 { color:
#4338ca; }  9      .moves { display: flex; justify-content: center; gap: 12px;
margin: 18px 0; } 10      .moves button { font-size: 36px; padding: 12px 18px;
border: 0; border-radius: 10px; cursor: pointer; background: white; box-shadow:
0 2px 6px rgba(0,0,0,.1); } 11      .vs { font-size: 60px; margin: 16px 0; min-
height: 70px; } 12      .res { font-size: 22px; font-weight: 700; min-height:
28px; } 13      .win { color: #16a34a; } .lose { color: #dc2626; } .draw
{ color: #64748b; } 14      .score { margin-top: 14px; font-size: 16px; color:
#475569; } 15      .streak { font-weight: 700; color: #ea580c; } 16    </style>
17  </head> 18  <body> 19    <h1>✊✋✌ Rock · Paper · Scissors</h1> 20    <div
class="moves"> 21      <button onclick="play('rock')">✊</button> 22      <button
onclick="play('paper')">✋</button> 23      <button onclick="play('scissors')">✌
</button> 24    </div> 25    <div class="vs" id="vs">– ? –</div> 26    <div
class="res" id="res"></div> 27    <div class="score"> 28      Wins: <span
id="w">0</span> · Losses: <span id="l">0</span> · Draws: <span
id="d">0</span><br> 29      <span class="streak" id="streak"></span> 30
<button onclick="reset()" style="margin-left:10px;padding:4px
10px;">Reset</button> 31    </div> 32   33    <script> 34      const beats =
{ rock: "scissors", paper: "rock", scissors: "paper" }; 35      const emoji =
{ rock: "✊", paper: "✋", scissors: "✌" }; 36      const KEY = "rps-stats"; 37
let stats = JSON.parse(localStorage.getItem(KEY) ||
'{"w":0,"l":0,"d":0,"streak":0,"best":0}'); 38      function paint() { 39
document.getElementById("w").textContent = stats.w; 40
document.getElementById("l").textContent = stats.l; 41
document.getElementById("d").textContent = stats.d; 42
```

```
document.getElementById("streak").textContent = 43          stats.streak > 1 ?
`🔥 Streak: ${stats.streak} (best ${stats.best})` : ""; 44      } 45
function play(p) { 46        const c = ["rock","paper","scissors"]
[Math.floor(Math.random()*3)]; 47        const out =
document.getElementById("vs"); 48        const r =
document.getElementById("res"); 49        out.textContent = `${emoji[p]}  vs  $
{emoji[c]}`; 50        if (p === c) { r.textContent = "Draw"; r.className = "res
draw"; stats.d++; stats.streak = 0; }
  1        else if (beats[p] === c) {  2          r.textContent = "You win!";
r.className = "res win";  3          stats.w++; stats.streak++;  4          if
(stats.streak > stats.best) stats.best = stats.streak;  5        } else {  6
r.textContent = "You lose!"; r.className = "res lose";  7          stats.l++;
stats.streak = 0;  8        }  9        localStorage.setItem(KEY,
JSON.stringify(stats)); 10        paint(); 11      } 12      window.play = play;
13      window.reset = () => { stats = {w:0,l:0,d:0,streak:0,best:0};
localStorage.setItem(KEY, JSON.stringify(stats)); paint(); }; 14      paint();
15    </script> 16  </body> 17  </html>
```

File structure

```
single file: rps.html
```

How to run it

Open in any browser.

How it works

We encode who beats whom in a single dictionary: beats.rock = 'scissors' means rock beats scissors. With that one map we can derive both win and loss logic symmetrically.

Customization ideas

- Add 'lizard' and 'spock' (5-move RPS)
- Show a 10-round history
- Best-of-N tournament mode

🕹 Upgrade Challenge

> *Build a small AI opponent that learns the player's most-used move (uses a frequency table).*

Project 43: Tic Tac Toe

Difficulty: ☆☆

What you will build

Two-player Tic Tac Toe on a 3x3 grid with win detection and a reset button.

Skills you will learn

- 2D array logic
- Win-line detection
- Game state

Features

- 3x3 grid
- Two-player mode
- Win line highlight
- Draw detection
- Reset

Build it with AI — the prompt to paste

> *Build 'Tic Tac Toe'. Two players click on a 3x3 grid. X always starts. After each move, check for a winning line (row, column or diagonal). Draw if 9 moves without a winner. Highlight the winning line. Buttons to reset and start over. Vanilla JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Tic Tac Toe</title>  6    <style>  7      body { font-family: system-ui;
max-width: 360px; margin: 40px auto; text-align: center; }  8      h1 { color:
#be185d; }  9      .grid { display: grid; grid-template-columns: repeat(3, 1fr);
gap: 6px; margin: 18px auto; width: 320px; } 10      .cell { aspect-ratio: 1;
font-size: 56px; background: white; border: 3px solid #db2777; border-radius:
8px; cursor: pointer; display:flex; align-items: center; justify-content:
center; font-weight: 700; color: #be185d; transition: background .15s; } 11
.win { background: #86efac !important; } 12      .draw { color: #64748b; } 13
.status { font-size: 18px; margin-top: 12px; min-height: 24px; } 14      button
{ padding: 10px 18px; background: #be185d; color: white; border: 0; border-
radius: 6px; cursor: pointer; } 15    </style> 16  </head> 17  <body> 18
<h1>Tic Tac Toe</h1> 19    <div class="grid" id="g"></div> 20    <p
class="status" id="s">X's turn</p> 21    <button
onclick="reset()">Reset</button> 22   23    <script> 24      const WINS =
[[0,1,2],[3,4,5],[6,7,8],[0,3,6],[1,4,7],[2,5,8],[0,4,8],[2,4,6]]; 25      let
board = Array(9).fill(""); 26      let turn = "X"; 27      let over = false; 28
const g = document.getElementById("g"); 29   30      function setup() { 31
g.innerHTML = ""; 32        board.forEach((_, i) => { 33          const c =
document.createElement("button"); 34          c.className = "cell"; c.onclick =
() => move(i); 35          g.appendChild(c); 36        }); 37      } 38
function move(i) { 39        if (board[i] || over) return; 40        board[i] =
turn; 41        const cells = document.querySelectorAll(".cell"); 42
cells[i].textContent = turn; 43        const w = winner(); 44        if (w)
{ over = true; w.forEach(j => cells[j].classList.add("win")); 45
document.getElementById("s").textContent = (turn === "X" ? "X" : "O") + "
```

```
wins!"; 46        } else if (board.every(Boolean)) { 47          over = true;
document.getElementById("s").textContent = "Draw!"; 48        } else { 49
turn = turn === "X" ? "O" : "X"; 50
document.getElementById("s").textContent = turn + "'s turn";
  1        }  2      }  3      function winner() {  4        for (const [a,b,c]
of WINS) {  5          if (board[a] && board[a] === board[b] && board[a] ===
board[c]) return [a,b,c];  6        }  7        return null;  8      }  9
window.reset = () => { board = Array(9).fill(""); turn = "X"; over = false; 10
document.getElementById("s").textContent = "X's turn"; setup(); }; 11
setup(); 12    </script> 13  </body> 14  </html>
```

File structure

```
single file: ttt.html
```

How to run it

Open in any browser.

How it works

The board is a flat array of 9 cells. We check all 8 winning combinations against the current board state. If any triple is identical and non-empty, that's the winning line.

Customization ideas

- Add a single-player mode vs a simple AI
- Track wins/losses/draws
- Add an undo button

🕹 Upgrade Challenge

> *Build a stronger AI for single-player mode using the minimax algorithm.*

Project 44: Memory Match Game

Difficulty: ☆☆

What you will build

Flip pairs of cards and find matches. Score is tracked by moves and time.

Skills you will learn

- Card state machine
- Shuffling
- Timer

Features

- 12 cards (6 pairs)
- Move counter
- Timer
- Match animation
- Win screen

Build it with AI — the prompt to paste

> *Build 'Memory Match'. Show 12 cards face down. Click to flip. Two flipped cards either match (stay open) or don't match (flip back after 1s). Track moves and time. Show a win screen with best score. Vanilla JS, single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Memory Game</title>  6    <style>  7      body { font-family: system-ui;
max-width: 520px; margin: 30px auto; text-align: center; }  8      h1 { color:
#7c3aed; }  9      .stats { display: flex; justify-content: space-around;
margin: 12px 0; color: #475569; } 10      .grid { display: grid; grid-template-
columns: repeat(4, 1fr); gap: 10px; margin: 14px 0; } 11      .card { aspect-
ratio: 1; font-size: 36px; background: #7c3aed; color: white; display:flex;
align-items:center; justify-content:center; border-radius: 10px; cursor:
pointer; user-select: none; font-weight: 700; perspective: 600px; }
12      .card.open { background: white; color: #1f2937; } 13      .card.matched
{ background: #86efac; color: #166534; } 14      .card.flip { background: white;
color: #1f2937; } 15      button { padding: 10px 18px; background: #7c3aed;
color: white; border: 0; border-radius: 6px; cursor: pointer; } 16    </style>
17  </head> 18  <body> 19    <h1>🧠 Memory Match</h1> 20    <div
class="stats"><span>Moves: <b id="moves">0</b></span><span>Time: <b
id="time">0s</b></span><span>Best: <b id="best">—</b></span></div> 21    <div
class="grid" id="g"></div> 22    <p id="msg"></p> 23    <button
onclick="reset()">New Game</button> 24   25    <script> 26      const symbols =
["🍎","🍌","🍇","🍓","🍒","🥝"]; 27      let cards, opened, moves, time, timer,
locked; 28      function shuffle(a) { 29        for (let i = a.length - 1; i >
0; i--) { 30          const j = Math.floor(Math.random() * (i + 1)); 31
[a[i], a[j]] = [a[j], a[i]]; 32        } 33        return a; 34      } 35
function reset() { 36        cards = shuffle([...symbols, ...symbols]); 37
opened = []; moves = 0; time = 0; locked = false; 38
clearInterval(timer); 39        timer = setInterval(() =>
{ document.getElementById("time").textContent = (++time) + "s"; }, 1000); 40
const g = document.getElementById("g"); 41        g.innerHTML = ""; 42
```

```
cards.forEach((s, i) => { 43          const c = document.createElement("div");
44          c.className = "card"; c.textContent = "?"; 45          c.onclick =
() => flip(i, c); 46          g.appendChild(c); 47        }); 48
document.getElementById("moves").textContent = 0; 49
document.getElementById("msg").textContent = ""; 50      }
  1      function flip(i, c) {  2        if (locked ||
c.classList.contains("matched") || c.classList.contains("open")) return;  3
c.classList.add("open"); c.textContent = cards[i]; opened.push({ i, c });  4
if (opened.length === 2) {  5          moves++;  6
document.getElementById("moves").textContent = moves;  7          const [a, b] =
opened;  8          if (cards[a.i] === cards[b.i]) {  9
a.c.classList.add("matched"); b.c.classList.add("matched"); opened = []; 10
if (document.querySelectorAll(".matched").length === cards.length) { 11
clearInterval(timer); 12              const best =
JSON.parse(localStorage.getItem("mem-best") || "null"); 13              if (!
best || moves < best) { localStorage.setItem("mem-best", JSON.stringify(moves));
document.getElementById("best").textContent = moves; } 14
document.getElementById("msg").textContent = "🎉 You won in " + moves + " moves
and " + time + "s."; 15            } 16          } else { 17            locked =
true; 18            setTimeout(() => { a.c.classList.remove("open");
b.c.classList.remove("open"); a.c.textContent = "?"; b.c.textContent = "?";
opened = []; locked = false; }, 800); 19          } 20        } 21      } 22
reset(); 23    </script> 24  </body> 25  </html>
```

File structure

```
single file: memory.html
```

How to run it

Open in any browser.

How it works

We use Fisher-Yates shuffle for truly random ordering. The locked flag prevents rapid clicks from breaking the comparison logic when the second card has just flipped.

Customization ideas

- Add a difficulty toggle (8, 16, 24 cards)
- Add a 'preview' before the game starts
- Add sound effects

🕹 Upgrade Challenge

> *Add a 'leaderboard' that stores the top 5 best scores per difficulty in localStorage.*

Project 45: Snake Game

Difficulty: ☆☆☆

What you will build

The classic Snake: a growing snake moves around the board and eats fruit. Don't hit the walls or yourself.

Skills you will learn

- Game loop
- Direction queue
- Collision detection

Features

- Arrow-key controlled snake
- Score and high score
- Self-collision and wall-collision
- Restart

Build it with AI — the prompt to paste

> *Build 'Snake'. The snake moves on a fixed grid, eats food to grow, and the game ends if it hits a wall or itself. Arrow keys change direction (no instant 180° turns). Spacebar to pause. Local high score. Canvas or grid divs — your choice. Vanilla JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Snake</title>  6    <style>  7      body { font-family: system-ui;
display: flex; flex-direction: column; align-items: center; background: #0f172a;
min-height: 100vh; margin: 0; color: white; }  8      canvas { background:
#1e293b; border: 2px solid #22d3ee; border-radius: 6px; margin: 20px; }  9
h1 { color: #22d3ee; } 10      .status { font-size: 16px; } 11      .gameover
{ color: #f87171; font-weight: 700; } 12    </style> 13  </head> 14  <body> 15
<h1>🐍 Snake</h1> 16    <p class="status">Score: <span id="sc">0</span> · High:
<span id="hi">0</span></p> 17    <canvas id="cv" width="400"
height="400"></canvas> 18    <p>Arrow keys to move · Space to pause</p> 19   20
<script> 21      const cv = document.getElementById("cv"); 22      const ctx =
cv.getContext("2d"); 23      const SIZE = 20, COLS = 20, ROWS = 20; 24      let
snake, dir, nextDir, food, score, high = +localStorage.getItem("snake-hi") || 0,
paused = false, over = false, speed = 120; 25   26      function reset() { 27
snake = [{x:10,y:10},{x:9,y:10},{x:8,y:10}]; 28        dir = {x:1,y:0}; nextDir
= dir; 29        food = placeFood(); score = 0; 30        paused = false; over =
false; 31        document.getElementById("sc").textContent = 0; 32
document.getElementById("hi").textContent = high; 33      } 34      function
placeFood() { 35        while (true) { 36          const x =
Math.floor(Math.random()*COLS), y = Math.floor(Math.random()*ROWS); 37
if (!snake.some(s => s.x === x && s.y === y)) return {x,y}; 38        }
39      } 40      function step() { 41        if (paused || over) return; 42
dir = nextDir; 43        const head = { x: snake[0].x + dir.x, y: snake[0].y +
dir.y }; 44        if (head.x < 0 || head.x >= COLS || head.y < 0 || head.y >=
ROWS || 45            snake.some(s => s.x === head.x && s.y === head.y)) { 46
```

```
over = true; return; 47        } 48        snake.unshift(head); 49        if
(head.x === food.x && head.y === food.y) { 50          score += 10;
  1          document.getElementById("sc").textContent = score;  2          if
(score > high) { high = score; localStorage.setItem("snake-hi", high);
document.getElementById("hi").textContent = high; }  3          food =
placeFood();  4        } else {  5          snake.pop();  6        }  7      }
8      function draw() {  9        ctx.fillStyle = "#1e293b";
ctx.fillRect(0,0,400,400); 10        ctx.fillStyle = "#22d3ee";
snake.forEach((s,i) => { ctx.globalAlpha = i ? 0.7 : 1; ctx.fillRect(s.x*SIZE+1,
s.y*SIZE+1, SIZE-2, SIZE-2); }); 11        ctx.globalAlpha = 1; 12
ctx.fillStyle = "#f87171"; ctx.fillRect(food.x*SIZE+2, food.y*SIZE+2, SIZE-4,
SIZE-4); 13        if (over) { ctx.fillStyle = "white"; ctx.font = "26px sans-
serif"; ctx.textAlign = "center"; ctx.fillText("Game Over — refresh to retry",
200, 200); } 14      } 15      addEventListener("keydown", e => { 16
const m = {ArrowUp:{x:0,y:-1},ArrowDown:{x:0,y:1},ArrowLeft:{x:-
1,y:0},ArrowRight:{x:1,y:0}}[e.key]; 17        if (m && !(m.x === -dir.x && m.y
=== -dir.y)) { nextDir = m; e.preventDefault(); } 18        if (e.key === " ") {
paused = !paused; e.preventDefault(); } 19      }); 20      reset(); 21
setInterval(() => { step(); draw(); }, speed); 22    </script> 23  </body> 24
</html>
```

File structure

```
single file: snake.html
```

How to run it

Open in any browser.

How it works

The snake is a list of grid cells. Each tick we move the head in 'dir' and either grow (if we hit food) or drop the tail. The line '!(m.x === -dir.x && m.y === -dir.y)' guarantees the snake never U-turns into itself on the very next move.

Customization ideas

- Add a wrap-around mode (no walls)
- Add a timer / speed-up
- Add bonus fruits

🕹 Upgrade Challenge

> *Make a two-player snake on one board that shares the playfield without colliding intentionally.*

Project 46: Breakout / Brick Breaker

Difficulty: ☆☆☆

What you will build

Paddle, ball, and bricks. Bounce the ball, clear the level, lose if it falls off.

Skills you will learn

- Physics
- Game loop
- Collision math

Features

- Paddle moved by arrow keys or mouse
- Multiple rows of bricks
- Score and lives
- Win/game over

Build it with AI — the prompt to paste

> *Build 'Breakout'. Paddle at the bottom, ball bouncing with collision against walls, paddle and bricks. Three rows of 8 bricks. Score increases per brick, lose a life when the ball falls. Win when all bricks are gone. Vanilla JS canvas.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Breakout</title>  6    <style>  7      body { font-family: system-ui;
background: #0f172a; color: white; display:flex; flex-direction: column; align-
items: center; min-height: 100vh; margin: 0; }  8      canvas { background:
#1e293b; border: 2px solid #fb923c; border-radius: 6px; margin: 16px; }  9
h1 { color: #fb923c; } 10    </style> 11  </head> 12  <body> 13    <h1>
Breakout</h1> 14    <p>Score: <span id="sc">0</span> · Lives: <span
id="lives">3</span> · Use ← → or mouse</p> 15    <canvas id="cv" width="480"
height="320"></canvas> 16   17    <script> 18      const cv =
document.getElementById("cv"); 19      const ctx = cv.getContext("2d"); 20
const W = cv.width, H = cv.height; 21      const paddle = { x: W/2 - 50, y: H -
20, w: 100, h: 12, dx: 0 }; 22      const ball = { x: W/2, y: H - 50, r: 8, vx:
3, vy: -3 }; 23      let bricks = [], score = 0, lives = 3, over = false, win =
false; 24   25      function buildBricks() { 26        bricks = []; 27
const cols = 8, rows = 4; 28        const bw = 50, bh = 18, pad = 4; 29
for (let r = 0; r < rows; r++) 30          for (let c = 0; c < cols; c++) 31
bricks.push({ x: 30 + c * (bw + pad), y: 30 + r * (bh + pad), w: bw, h: bh,
alive: true }); 32      } 33      function reset() { buildBricks(); ball.x =
W/2; ball.y = H - 50; ball.vx = 3; ball.vy = -3; 34        paddle.x = W/2 - 50;
score = 0; lives = 3; over = false; win = false; paint(); } 35   36
function step() { 37        if (over || win) return; 38        paddle.x +=
paddle.dx; 39        paddle.x = Math.max(0, Math.min(W - paddle.w, paddle.x));
40        ball.x += ball.vx; ball.y += ball.vy; 41        if (ball.x < ball.r ||
ball.x > W - ball.r) ball.vx *= -1; 42        if (ball.y < ball.r) ball.vy *= -
1; 43        if (ball.y > H) { 44          lives--;
document.getElementById("lives").textContent = lives; 45          if (lives <=
0) { over = true; return; } 46          ball.x = W/2; ball.y = H - 50; ball.vx =
3; ball.vy = -3; 47          return; 48        } 49        // paddle 50
```

```
if (ball.y > paddle.y - ball.r && ball.x > paddle.x && ball.x < paddle.x +
paddle.w && ball.vy > 0)
  1          ball.vy = -Math.abs(ball.vy);  2        // bricks  3
bricks.forEach(b => {  4          if (!b.alive) return;  5          if (ball.x >
b.x && ball.x < b.x + b.w && ball.y > b.y && ball.y < b.y + b.h) {  6
b.alive = false; ball.vy *= -1; score += 10;  7
document.getElementById("sc").textContent = score;  8          }  9        });
10        if (bricks.every(b => !b.alive)) win = true; 11      } 12   13
function paint() { 14        ctx.fillStyle = "#1e293b"; ctx.fillRect(0,0,W,H);
15        ctx.fillStyle = "#fb923c"; ctx.fillRect(paddle.x, paddle.y, paddle.w,
paddle.h); 16        ctx.fillStyle = "white"; ctx.beginPath(); ctx.arc(ball.x,
ball.y, ball.r, 0, 7); ctx.fill(); 17        bricks.forEach((b, i) => { 18
if (!b.alive) return; 19          ctx.fillStyle =
["#fbbf24","#f97316","#f87171","#ef4444"][Math.floor(i/8)]; 20
ctx.fillRect(b.x, b.y, b.w, b.h); 21        }); 22        if (over)
{ ctx.fillStyle = "white"; ctx.font = "24px sans-serif"; ctx.fillText("Game
Over", W/2 - 60, H/2); } 23        if (win) { ctx.fillStyle = "white"; ctx.font
= "24px sans-serif"; ctx.fillText("You win!", W/2 - 50, H/2); } 24      } 25
26      addEventListener("keydown", e => { 27        if (e.key === "ArrowLeft")
paddle.dx = -6; 28        if (e.key === "ArrowRight") paddle.dx = 6; 29      });
30      addEventListener("keyup", () => paddle.dx = 0); 31
cv.addEventListener("mousemove", e => { 32        const r =
cv.getBoundingClientRect(); 33        paddle.x = e.clientX - r.left - paddle.w /
2; 34      }); 35   36      reset(); 37      setInterval(() => { step();
paint(); }, 20); 38    </script> 39  </body> 40  </html>
```

File structure

```
single file: breakout.html
```

How to run it

Open in any browser.

How it works

Each tick we move ball + paddle, then check four collisions (left/right wall, top wall, paddle, bricks). Ball.vy *= -1 flips the vertical direction. The paddle collision checks happen only when vy > 0 so the ball doesn't get stuck bouncing on the paddle's top.

Customization ideas

- Add a launch mechanic (Space to start)
- Add power-ups
- Add multiple levels

🕹 Upgrade Challenge

> *Build a level editor that stores custom brick layouts in localStorage.*

Project 47: Typing Speed Tester

Difficulty: ☆☆

What you will build

A live WPM-and-accuracy tester with sample prompts and per-character marking.

Skills you will learn

- Timers
- String comparison
- Statistics

Features

- Random prompts from a list
- WPM and accuracy display
- Per-character colour marking
- Best score

Build it with AI — the prompt to paste

> *Build a 'Typing Speed Tester'. Display a prompt sentence. As the user types, each character is coloured green (correct) or red (incorrect). Show elapsed time, words per minute, accuracy and error count. Use a list of 12 short prompts and pick one randomly. Vanilla JS, single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Typing Test</title>  6    <style>  7      body { font-family: system-ui,
sans-serif; max-width: 760px; margin: 30px auto; padding: 20px; }  8      h1
{ color: #0ea5e9; }  9      #prompt { font-size: 20px; line-height: 1.6;
background: #f1f5f9; padding: 16px; border-radius: 8px; margin-bottom: 12px; }
10      .ok { color: #16a34a; } 11      .bad { color: #dc2626; background:
#fee2e2; border-radius: 3px; } 12      .pending { color: #94a3b8; } 13
textarea { width: 100%; min-height: 100px; font-size: 18px; padding: 12px;
border-radius: 8px; border: 1px solid #cbd5e1; } 14      .stats { display: flex;
gap: 18px; margin-top: 16px; font-size: 16px; } 15      .stats b { font-size:
22px; color: #0e7490; } 16    </style> 17  </head> 18  <body> 19    <h1>⌨ Typing
Speed Tester</h1> 20    <div id="prompt">-</div> 21    <textarea id="input"
placeholder="Start typing here..."></textarea> 22    <div class="stats"> 23
<div>Time: <b id="t">0s</b></div> 24      <div>WPM: <b id="wpm">0</b></div> 25
<div>Accuracy: <b id="acc">100%</b></div> 26      <div>Errors: <b
id="err">0</b></div> 27    </div> 28    <p><button onclick="reset()">New
prompt</button></p> 29   30    <script> 31      const prompts = [ 32        "A
small daily habit beats a big weekly plan.", 33        "Code is read more often
than it is written.", 34        "Slow is smooth, smooth is fast.", 35
"Curiosity is the most underrated programming skill.", 36        "Most bugs are
misunderstandings in disguise.", 37        "Bugs found in writing are bugs you
don't have to fix.", 38        "A working prototype answers more questions than
a perfect spec.", 39        "You will spend more time naming things than writing
them.", 40        "Reading other people's code is how you level up fastest.", 41
"Comments explain why, not what.", 42        "Sleeping on a hard problem is a
known debugging technique.", 43        "The simplest fix is usually the right
one." 44      ]; 45      let startTime = null, ended = false, current = ""; 46
```

```
function pick() { current = prompts[Math.floor(Math.random() * prompts.length)];
47        const p = document.getElementById("prompt"); p.innerHTML = ""; 48
current.split("").forEach(ch =>
p.appendChild(Object.assign(document.createElement("span"), {textContent: ch,
className: "pending"}))); 49      } 50      function reset() { pick();
  1        document.getElementById("input").value = "";  2
document.getElementById("t").textContent = "0s";  3
document.getElementById("wpm").textContent = 0;  4
document.getElementById("acc").textContent = "100%";  5
document.getElementById("err").textContent = 0;  6
document.getElementById("input").focus();  7        startTime = null; ended =
false;  8      }  9      function tick() { 10        const typed =
document.getElementById("input").value; 11        const spans =
document.getElementById("prompt").querySelectorAll("span"); 12        let err =
0; 13        spans.forEach((s, i) => { 14          const c = typed[i]; 15
if (c == null) { s.className = "pending"; } 16          else if (c ===
s.textContent) { s.className = "ok"; } 17          else { s.className = "bad";
err++; } 18        }); 19        document.getElementById("err").textContent =
err; 20        if (typed.length && !startTime) startTime = Date.now(); 21
const seconds = startTime ? Math.max(1, Math.round((Date.now() - startTime) /
1000)) : 0; 22        const minutes = seconds / 60; 23        const words =
typed.trim().split(/\s+/).length; 24
document.getElementById("t").textContent = seconds + "s"; 25
document.getElementById("wpm").textContent = Math.round(words / Math.max(0.1,
minutes)); 26        const total = typed.length || 1; 27
document.getElementById("acc").textContent = Math.round(((total - err) / total)
* 100) + "%"; 28      } 29
document.getElementById("input").addEventListener("input", tick); 30
reset(); 31    </script> 32  </body> 33  </html>
```

File structure

```
single file: typing.html
```

How to run it

Open in any browser.

How it works

We split the prompt into spans and compare each character against the user's typed text. By ticking with every input event we get live WPM/accuracy without a setInterval.

Customization ideas

- Highlight the next character
- Show per-word speed heatmap
- Add a 60-second timed mode

🕹 Upgrade Challenge

> *Build a custom prompt generator that uses a tiny Markov chain over a paragraph.*

Project 48: Reaction Time Tester

Difficulty: ☆☆

What you will build

Wait for green, click fast. Five rounds with your reaction time chart.

Skills you will learn

- Timers
- State machine
- Statistics

Features

- Three-colour state machine
- Five rounds per session
- Personal best
- Bar chart of all attempts

Build it with AI — the prompt to paste

> *Build a 'Reaction Time Tester'. Screen is red; click to start. Random delay, then it goes green. Click again as fast as you can. Record the time. Five rounds per session; show best, average and a small bar chart. Click too early and the round is voided. Vanilla JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Reaction Time</title>  6    <style>  7      body { font-family: system-
ui; margin: 0; min-height: 100vh; display:flex; flex-direction: column; align-
items: center; justify-content: center; transition: background .3s; }
8      .red { background: #dc2626; color: white; }  9      .wait { background:
#f59e0b; color: #1f2937; } 10      .green { background: #16a34a; color: white;
cursor: pointer; } 11      .done { background: #1e293b; color: white; } 12
#tap { width: 100vw; height: 70vh; display:flex; flex-direction: column; align-
items: center; justify-content: center; font-size: 36px; font-weight: 700; } 13
.bar { display: flex; gap: 4px; align-items: end; height: 80px; margin-top:
10px; } 14      .bar div { width: 36px; background: #fb923c; color: #0f172a;
font-size: 12px; text-align: center; padding-top: 4px; border-radius: 4px 4px 0
0; } 15    </style> 16  </head> 17  <body> 18    <div id="tap"> 19      <div
id="msg">Click to start</div> 20      <div class="bar" id="bars"
style="display:none"></div> 21    </div> 22   23    <script> 24      let phase =
"red", greenAt = 0, attempts = []; 25      const tap =
document.getElementById("tap"); 26      const msg =
document.getElementById("msg"); 27      const bars =
document.getElementById("bars"); 28      tap.className = "red"; 29   30
function setState(name, txt) { 31        phase = name; 32        tap.className =
name; 33        msg.textContent = txt; 34      } 35   36      function start() {
37        attempts = []; 38        bars.style.display = "none"; bars.innerHTML =
""; 39        setState("red", "Wait for green..."); 40        const delay = 1500
+ Math.random() * 3000; 41        setTimeout(() => { 42          if (phase !==
"red") return;  // user canceled 43          greenAt = performance.now(); 44
setState("green", "CLICK!"); 45        }, delay); 46      } 47   48
```

```
function tapNow() { 49        if (phase === "red") start(); 50        else if
(phase === "wait") { /* no wait phase here */ }
  1        else if (phase === "green") {  2          const t =
Math.round(performance.now() - greenAt);  3          attempts.push(t);  4
setState("red", `${t} ms — Click for next round (${attempts.length}/5)`);  5
if (attempts.length >= 5) showResults();  6        }  7      }  8    9
function showResults() { 10        const avg =
Math.round(attempts.reduce((a,b)=>a+b,0) / attempts.length); 11        const
best = Math.min(...attempts); 12        setState("done", `Best ${best} ms · Avg
${avg} ms · Click to retry`); 13        localStorage.setItem("rxn-best",
Math.min(best, +localStorage.getItem("rxn-best") || 9999)); 14
bars.style.display = "flex"; 15        bars.innerHTML = attempts.map(t => 16
`<div style="height:${Math.min(80, t/5)}px">${t}</div>`).join(""); 17      } 18
19      tap.addEventListener("click", () => { 20        if (phase === "done")
start(); 21        else tapNow(); 22      }); 23    </script> 24  </body> 25
</html>
```

File structure

```
single file: reaction.html
```

How to run it

Open in any browser.

How it works

performance.now() gives sub-millisecond timing. Each phase is a different background colour, and we gate transitions through a single 'phase' variable so the screen never gets confused.

Customization ideas

- Add a 1v2 mode
- Show ms vs global average (sample only)
- Add haptic feedback on mobile

🕹 Upgrade Challenge

> *Visualise 100 random points to compare user performance to a normal distribution bell curve.*

Project 49: Whack-a-Mole

Difficulty: ☆☆☆

What you will build

Moles pop up from random holes. Click them to score before they hide.

Skills you will learn

- SetInterval queues
- State per hole
- Score tracking

Features

- 9 holes grid
- Random popup
- Speed ramps up
- Countdown timer

Build it with AI — the prompt to paste

> *Build 'Whack-a-Mole'. A 3x3 grid of holes; a mole appears in a random hole briefly, then hides. Click moles to score. Game lasts 30 seconds. As time progresses the moles appear for shorter durations. Click a hole even when empty is fine (no penalty). Best score stored.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Whack a Mole</title>  6    <style>  7      body { font-family: system-ui;
text-align: center; padding: 20px; background: #fef3c7; min-height: 100vh; }  8
h1 { color: #92400e; }  9      .stats { font-size: 18px; margin: 12px 0; } 10
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 12px; max-
width: 360px; margin: 0 auto; } 11      .hole { aspect-ratio: 1; background:
#92400e; border-radius: 50%; display: flex; align-items: center; justify-
content: center; cursor: pointer; overflow: hidden; } 12      .mole { font-size:
50px; user-select: none; transform: translateY(100%); transition:
transform .15s; } 13      .up { transform: translateY(0); } 14      button
{ padding: 12px 24px; background: #92400e; color: white; border: 0; border-
radius: 6px; cursor: pointer; font-size: 16px; margin-top: 14px; } 15
</style> 16  </head> 17  <body> 18    <h1>🔨 Whack-a-Mole</h1> 19    <p
class="stats">Score: <span id="sc">0</span> · Time: <span id="t">30</span>s ·
Best: <span id="hi">0</span></p> 20    <div class="grid" id="g"></div> 21
<button id="start">Start Game</button> 22  23    <script> 24      let score =
0, time = 30, running = false, timer = null, popup = null; 25      function
build() { 26        const g = document.getElementById("g"); 27
g.innerHTML = ""; 28        for (let i = 0; i < 9; i++) { 29          const hole
= document.createElement("div"); 30          hole.className = "hole"; 31
const mole = document.createElement("div"); 32          mole.className = "mole";
mole.textContent = "🐹"; 33          hole.appendChild(mole); 34
hole.addEventListener("click", () => { 35            if (!running) return; 36
if (mole.classList.contains("up")) { score++;
document.getElementById("sc").textContent = score;
mole.classList.remove("up"); } 37          }); 38          g.appendChild(hole);
39        } 40      } 41      build(); 42      function loop() { 43        if (!
```

```
running) return; 44        const moles = document.querySelectorAll(".mole"); 45
moles.forEach(m => m.classList.remove("up")); 46        const idx =
Math.floor(Math.random() * 9); 47        const maxUp = Math.max(300, 1500 - (30
- time) * 30); 48        moles[idx].classList.add("up"); 49        setTimeout(()
=> moles[idx].classList.remove("up"), maxUp); 50        popup = setTimeout(loop,
Math.max(200, 1000 - (30 - time) * 20));
  1      }  2      document.getElementById("start").addEventListener("click", ()
=> {  3        if (running) return;  4        running = true; score = 0; time =
30;  5        document.getElementById("sc").textContent = 0;  6
document.getElementById("start").textContent = "Stop";  7        timer =
setInterval(() => {  8          time--;  9
document.getElementById("t").textContent = time; 10          if (time <= 0) { 11
running = false; 12            clearInterval(timer); clearTimeout(popup); 13
document.querySelectorAll(".mole").forEach(m => m.classList.remove("up")); 14
document.getElementById("start").textContent = "Start Game"; 15            const
hi = +localStorage.getItem("whack-hi") || 0; 16            if (score > hi)
{ localStorage.setItem("whack-hi", score);
document.getElementById("hi").textContent = score; } 17            alert("Time's
up! You whacked " + score + " moles."); 18          } 19        }, 1000); 20
loop(); 21      }); 22      document.getElementById("hi").textContent =
+localStorage.getItem("whack-hi") || 0; 23    </script> 24  </body 25  </html>
```

File structure

```
single file: whack.html
```

How to run it

Open in any browser.

How it works

Each round picks a random hole to 'raise' and sets a timer to lower it. The recursion is via setTimeout; each round's delay shortens over time. Two timers coexist: a 1-second countdown and a separate popup scheduler.

Customization ideas

- Add a 'golden mole' worth bonus points
- Add sound effects
- Add a 60-second marathon mode

🕹 Upgrade Challenge

> *Build a 'ghost' variant where holes grey out and only one stays active.*

Project 50: Pomodoro Timer

Difficulty: ☆☆

What you will build

A 25/5 Pomodoro cycle timer with start, pause, reset and a daily task counter.

Skills you will learn

- State machine
- Audio
- localStorage streaks

Features

- 25-minute focus
- 5-minute break
- Auto-cycle
- Today's count
- Sound when changing

Build it with AI — the prompt to paste

> *Build a 'Pomodoro Timer'. Large countdown 25:00 → 'Focus' state. Auto-switches 5:00 → 'Break'. Pause, skip, and reset buttons. Today's completed Pomodoros counter stored in localStorage. Play a beep when the phase changes using a generated audio tone.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Pomodoro</title>  6    <style>  7      body { font-family: system-ui;
max-width: 480px; margin: 30px auto; text-align: center; }  8      .phase
{ font-size: 14px; letter-spacing: 4px; color: #db2777; }  9      .timer { font-
size: 96px; font-weight: 200; font-variant-numeric: tabular-nums; color:
#0f172a; margin: 20px 0; } 10      .focus-bg { background: #fee2e2; min-height:
100vh; padding-top: 40px; } 11      .break-bg { background: #dcfce7; min-height:
100vh; padding-top: 40px; } 12      .ctrls button { padding: 12px 22px; font-
size: 16px; border: 0; border-radius: 6px; cursor: pointer; margin: 0 6px; } 13
.start { background: #16a34a; color: white; } 14      .stop { background:
#f59e0b; color: white; } 15      .reset { background: #94a3b8; color: white; }
16      .count { margin-top: 26px; font-size: 16px; color: #475569; } 17
</style> 18  </head> 19  <body id="b" class="focus-bg"> 20    <div class="phase"
id="phase">FOCUS</div> 21    <div class="timer" id="t">25:00</div> 22    <div
class="ctrls"> 23      <button id="start" class="start">Start</button> 24
<button id="pause" class="stop">Pause</button> 25      <button id="reset"
class="reset">Reset</button> 26    </div> 27    <div class="count">Today: <span
id="c">0</span> 🍅</div> 28   29    <script> 30      const STATE = { focus: 25 *
60, break: 5 * 60 }; 31      let phase = "focus", left = STATE.focus, interval =
null; 32      const fmt = s => String(Math.floor(s/60)).padStart(2,"0") + ":" +
String(s%60).padStart(2,"0"); 33      function today() { return new
Date().toISOString().slice(0,10); } 34      function getCount() { 35
const o = JSON.parse(localStorage.getItem("pomodoros") || "{}"); 36        const
k = today(); 37        return { today: o[k] || 0, record: o }; 38      } 39
function bump() { 40        const o =
JSON.parse(localStorage.getItem("pomodoros") || "{}"); 41        const k =
```

```
today(); 42        o[k] = (o[k] || 0) + 1; 43
localStorage.setItem("pomodoros", JSON.stringify(o)); 44      } 45      function
beep(freq = 660, dur = 0.3) { 46        const ctx = new (window.AudioContext ||
window.webkitAudioContext)(); 47        const o = ctx.createOscillator(); const
g = ctx.createGain(); 48        o.connect(g); g.connect(ctx.destination); 49
o.frequency.value = freq; g.gain.setValueAtTime(0.1, ctx.currentTime); 50
o.start(); o.stop(ctx.currentTime + dur);
  1      }  2      function paint() {  3
document.getElementById("t").textContent = fmt(left);  4
document.getElementById("phase").textContent = phase.toUpperCase();  5
document.body.className = phase === "focus" ? "focus-bg" : "break-bg";  6
document.getElementById("c").textContent = getCount().today;  7      }  8
function tick() {  9        left--; 10        paint(); 11        if (left <= 0)
{ 12          if (phase === "focus") { bump(); beep(880, 0.5); phase = "break";
left = STATE.break; } 13          else { beep(440, 0.5); phase = "focus"; left =
STATE.focus; } 14        } 15      } 16
document.getElementById("start").onclick = () => { 17        if (interval)
return; 18        interval = setInterval(tick, 1000); 19      }; 20
document.getElementById("pause").onclick = () => { clearInterval(interval);
interval = null; }; 21      document.getElementById("reset").onclick = () =>
{ 22        clearInterval(interval); interval = null; 23        phase = "focus";
left = STATE.focus; paint(); 24      }; 25      paint(); 26    </script> 27
</body> 28  </html>
```

File structure

```
single file: pomodoro.html
```

How to run it

Open in any browser.

How it works

setInterval(tick, 1000) decrements 'left' once a second. localStorage keeps a per-day counter so it resets at midnight without any extra code. The beep uses the WebAudio API to generate a tone.

Customization ideas

- Add a 'long break' every 4 Pomodoros
- Add a task list per session
- Add export of the day's log as text

🕹 Upgrade Challenge

> *Build a 'focus stats' view: 7-day Pomodoro history rendered as a simple heatmap.*

Project 51: Hangman

Difficulty: ☆☆

What you will build

The classic word guessing game with stick-figure drawing and lives counter.

Skills you will learn

- String and set operations
- Drawing
- Game state

Features

- At least 50-word dictionary
- Drawing that grows each wrong guess
- Keyboard on-screen

Build it with AI — the prompt to paste

> *Build 'Hangman'. Display blanks for a hidden word. Click letters to guess; correct letters fill in. Wrong letters draw the next part of a hangman figure. Six wrong guesses ends the game. New game picks a random word from a list of 50+. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Hangman</title>  6    <style>  7      body { font-family: system-ui; max-
width: 600px; margin: 30px auto; text-align: center; }  8      .word { font-
size: 36px; letter-spacing: 8px; margin: 20px 0; font-family: monospace; }  9
.wrong { color: #dc2626; min-height: 24px; } 10      svg { background: white;
border-radius: 8px; } 11      button.k { width: 32px; height: 32px; margin: 2px;
padding: 0; border: 1px solid #cbd5e1; border-radius: 4px; cursor: pointer;
font-weight: 600; } 12      button.k:disabled { background: #94a3b8; color:
white; cursor: not-allowed; } 13    </style> 14  </head> 15  <body> 16
<h1>Hangman</h1> 17    <svg width="180" height="220" id="g"> 18      <line
x1="20" y1="200" x2="160" y2="200" stroke="#0f172a" stroke-width="3"/> 19
<line x1="60" y1="200" x2="60" y2="30" stroke="#0f172a" stroke-width="3"/> 20
<line x1="60" y1="30" x2="130" y2="30" stroke="#0f172a" stroke-width="3"/> 21
<line x1="130" y1="30" x2="130" y2="60" stroke="#0f172a" stroke-width="3"/> 22
</svg> 23    <div class="word" id="w"></div> 24    <div class="wrong"
id="wrong"></div> 25    <div id="keys"></div> 26    <button onclick="reset()"
style="margin-top:14px; padding:8px 16px; background:#0f172a; color:white;
border:0; border-radius:6px; cursor:pointer">New Word</button> 27   28
<script> 29      const WORDS =
["apple","bridge","candle","dawn","engine","flower","garden","harvest","island",
"jungle","kettle","lemon","mountain","network","ocean","pyramid","quiet","ribbon
","silent","tunnel","umbrella","village","whisper","yellow","zenith","crystal","
diamond","ember","forest","glacier","horizon","lantern","meadow","noble","oracle
","pebble","quartz","river","sapphire","tide","aurora","cipher","ember","fjord",
"lattice","mantle","nebula","paradox","summit"]; 30      let word, guessed,
wrong, over; 31      function reset() { 32        word =
WORDS[Math.floor(Math.random() * WORDS.length)]; 33        guessed = new Set();
wrong = 0; over = false; 34        drawSvg(0); paint(); 35      } 36
function paint() { 37        const w = document.getElementById("w"); 38
```

```
w.textContent = word.split("").map(c => guessed.has(c) ? c : "_").join(" "); 39
const wrongLetters = [...guessed].filter(c => !word.includes(c)); 40
document.getElementById("wrong").textContent = "Wrong: " + (wrongLetters.join("
") || "none"); 41        const k = document.getElementById("keys"); 42
k.innerHTML = ""; 43        "abcdefghijklmnopqrstuvwxyz".split("").forEach(ch =>
{ 44          const b = document.createElement("button"); 45
b.className = "k"; b.textContent = ch; 46          if (guessed.has(ch))
b.disabled = true; 47          b.onclick = () => guess(ch); 48
k.appendChild(b); 49        }); 50        if (word.split("").every(c =>
guessed.has(c))) { over = true; w.textContent = "You win! " + word; }
  1        if (wrong >= 6) { over = true; w.textContent = "Game over. Word: " +
word; }  2      }  3      function guess(ch) {  4        if (over ||
guessed.has(ch)) return;  5        guessed.add(ch);  6        if (!
word.includes(ch)) wrong++;  7        drawSvg(wrong);  8        paint();  9
} 10      function drawSvg(stage) { 11        const g =
document.getElementById("g"); 12        const parts = [ 13          () =>
{ g.innerHTML += '<circle cx="130" cy="75" r="15" stroke="#0f172a" stroke-
width="3" fill="none"/>'; }, 14          () => { g.innerHTML += '<line x1="130"
y1="90" x2="130" y2="150" stroke="#0f172a" stroke-width="3"/>'; }, 15
() => { g.innerHTML += '<line x1="130" y1="105" x2="105" y2="125"
stroke="#0f172a" stroke-width="3"/>'; }, 16          () => { g.innerHTML +=
'<line x1="130" y1="105" x2="155" y2="125" stroke="#0f172a" stroke-width="3"/>';
}, 17          () => { g.innerHTML += '<line x1="130" y1="150" x2="105" y2="180"
stroke="#0f172a" stroke-width="3"/>'; }, 18          () => { g.innerHTML +=
'<line x1="130" y1="150" x2="155" y2="180" stroke="#0f172a" stroke-width="3"/>';
} 19        ]; 20        if (stage > 0 && stage <= parts.length) parts[stage -
1](); 21      } 22      reset(); 23    </script> 24  </body> 25  </html>
```

File structure

```
single file: hangman.html
```

How to run it

Open in any browser.

How it works

We use Sets for guessed letters — they make lookup O(1). The hangman figure is drawn by appending SVG elements. Wrong count is just an integer that triggers a new SVG piece each time it increments.

Customization ideas

- Add category-based word lists (animals, food, science)
- Add a hint button
- Add a multi-game score

🕹 Upgrade Challenge

> *Allow a custom word list loaded from a textarea for class use.*

Project 52: Maze Generator and Solver

Difficulty: ☆☆☆

What you will build

Generate a random maze and watch a tiny robot solve it using BFS pathfinding.

Skills you will learn

- Recursive backtracking
- BFS algorithm
- Animation

Features

- Random 10x10 maze
- Animated BFS solver
- Start and goal markers
- Speed slider

Build it with AI — the prompt to paste

> *Build 'Maze Generator and Solver'. Generate a 10x10 grid maze using recursive backtracking. Then animate a Breadth-First Search solver from start to goal. Cells light up as visited. Pure JS, single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Maze Game</title>  6    <style>  7      body { font-family: system-ui;
display: flex; flex-direction: column; align-items: center; background: #0f172a;
color: white; min-height: 100vh; margin: 0; padding: 20px; }  8      canvas
{ background: white; border-radius: 6px; }  9      .controls { margin-bottom:
12px; } 10      button { padding: 8px 14px; background: #0ea5e9; color: white;
border: 0; border-radius: 6px; cursor: pointer; } 11      .slow { background:
#f59e0b; } .fast { background: #16a34a; } 12    </style> 13  </head> 14  <body>
15    <h1>🧩 Maze Generator + Solver</h1> 16    <div class="controls"> 17
<button onclick="newMaze()">New Maze</button> 18      <button class="slow"
onclick="solve(60)">Solve (Slow)</button> 19      <button
onclick="solve(20)">Solve (Normal)</button> 20      <button class="fast"
onclick="solve(5)">Solve (Fast)</button> 21    </div> 22    <canvas id="cv"
width="440" height="440"></canvas> 23   24    <script> 25      const cv =
document.getElementById("cv"); 26      const ctx = cv.getContext("2d"); 27
const N = 11, S = 40; 28   29      let walls = [];  // walls[r][c] = { N, E, S,
W } 30   31      function newMaze() { 32        walls = Array.from({length: N},
() => Array.from({length: N}, () => ({N:1,E:1,S:1,W:1}))); 33        const
visited = Array.from({length: N}, () => Array(N).fill(false)); 34
function carve(r, c) { 35          visited[r][c] = true; 36          const dirs
= [["N",-1,0,"S"], ["S",1,0,"N"], ["W",0,-1,"E"], ["E",0,1,"W"]]
37            .sort(() => Math.random() - 0.5); 38          for (const [me, dr,
dc, opp] of dirs) { 39            const nr = r + dr, nc = c + dc; 40
if (nr >= 0 && nr < N && nc >= 0 && nc < N && !visited[nr][nc]) { 41
walls[r][c][me] = 0; walls[nr][nc][opp] = 0; 42              carve(nr, nc); 43
} 44          } 45        } 46        carve(0, 0); 47        draw(); 48      }
49   50      function draw(path = new Set(), frontier = new Set()) {
  1        ctx.fillStyle = "white"; ctx.fillRect(0, 0, N * S, N * S);  2
ctx.strokeStyle = "#0f172a"; ctx.lineWidth = 2;  3        walls.flatMap((row, r)
```

```
=> row.map((w, c) => ({r,c,w}))).forEach(({r,c,w}) => {  4          const x = c
* S, y = r * S;  5          if (w.N) ctx.beginPath(), ctx.moveTo(x, y),
ctx.lineTo(x + S, y), ctx.stroke();  6          if (w.W) ctx.beginPath(),
ctx.moveTo(x, y), ctx.lineTo(x, y + S), ctx.stroke();  7        });  8        //
start and goal cells  9        ctx.fillStyle = "#16a34a"; ctx.fillRect(0, 0, S -
4, S - 4); 10        ctx.fillStyle = "#dc2626"; ctx.fillRect((N - 1) * S + 2, (N
- 1) * S + 2, S - 4, S - 4); 11        // visited frontier 12
frontier.forEach(f => { ctx.fillStyle = "#fde68a"; ctx.fillRect(f.c * S + 6, f.r
* S + 6, S - 12, S - 12); }); 13        path.forEach(p => { ctx.fillStyle =
"#60a5fa"; ctx.fillRect(p.c * S + 6, p.r * S + 6, S - 12, S - 12); }); 14      }
15   16      async function solve(speed) { 17        // BFS from (0,0) to (N-1,
N-1) 18        const start = { r: 0, c: 0 }, goal = { r: N - 1, c: N - 1 }; 19
const came = { "0,0": null }; 20        const queue = [start]; 21        const
seen = new Set(["0,0"]); 22        while (queue.length) { 23          const cur
= queue.shift(); 24          const key = cur.r + "," + cur.c; 25          if
(key === goal.r + "," + goal.c) break; 26          const dirs = [["N",-1,0],
["S",1,0],["W",0,-1],["E",0,1]]; 27          for (const [name, dr, dc] of dirs)
{ 28            if (walls[cur.r][cur.c][name]) continue; 29            const nr
= cur.r + dr, nc = cur.c + dc; 30            const nk = nr + "," + nc; 31
if (seen.has(nk)) continue; 32            seen.add(nk); 33            came[nk] =
cur; 34            queue.push({ r: nr, c: nc }); 35            draw(new Set(),
new Set([cur, {r:nr,c:nc}])); 36            await new Promise(r => setTimeout(r,
speed)); 37          } 38        } 39        // reconstruct path 40        const
path = []; 41        let cur = goal; 42        while (cur) { path.push(cur); cur
= came[cur.r + "," + cur.c]; } 43        for (let i = path.length - 1; i >= 0;
i--) { 44          const s = new Set(path.slice(i)); 45          draw(s, new
Set()); 46          await new Promise(r => setTimeout(r, speed * 2));
47        } 48      } 49   50      newMaze();
  1    </script>  2  </body>  3  </html>
```

File structure

```
single file: maze.html
```

How to run it

Open in any browser.

How it works

Recursive backtracking carves corridors by destroying wall pairs between unvisited cells. BFS explores neighbours level-by-level, always finding the shortest path. await + setTimeout drives the animation.

Customization ideas

- Add diagonal moves
- Add weights to certain cells
- Animate both exploration and path together

🕹 Upgrade Challenge

> *Try A* search using Manhattan distance as heuristic and compare visually against BFS.*

Project 53: Quiz Game

Difficulty: ☆☆

What you will build

A 10-question multiple-choice quiz with score, restart and a finishing screen.

Skills you will learn

- Question shuffling
- Score tally
- Timer per question

Features

- 10 questions across 3 categories
- Per-question 20s timer
- Result screen
- Restart

Build it with AI — the prompt to paste

> *Build a 'Quiz Game'. Display 10 multiple-choice questions from a built-in list of 20+. Each question has a 20-second timer. Score on first attempt. Show progress bar and a result screen with restart. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Quiz</title>  6    <style>  7      body { font-family: system-ui; max-
width: 560px; margin: 30px auto; padding: 20px; }  8      .progress { height:
8px; background: #e2e8f0; border-radius: 4px; margin-bottom: 16px; }
9      .bar { height: 100%; background: #2563eb; border-radius: 4px; transition:
width .3s; } 10      .q { font-size: 18px; margin: 16px 0; } 11      .options
button { display: block; width: 100%; margin: 6px 0; padding: 12px; background:
white; border: 1px solid #cbd5e1; border-radius: 6px; cursor: pointer; text-
align: left; } 12      .options button:hover { background: #f1f5f9; }
13      .options button.ok { background: #bbf7d0; border-color: #16a34a; } 14
.options button.bad { background: #fecaca; border-color: #dc2626; }
15      .timer { font-size: 14px; color: #f59e0b; font-weight: 600; }
16      .result { text-align: center; padding: 20px; } 17    </style> 18
</head> 19  <body> 20    <h1>□ Quiz Game</h1> 21    <div class="progress"><div
class="bar" id="bar" style="width:0%"></div></div> 22    <p>Question <span
id="idx">1</span> / <span id="total">10</span> · <span class="timer"
id="t">20s</span> left</p> 23    <div class="q" id="q">-</div> 24    <div
class="options" id="opts"></div> 25   26    <script> 27      const BANK = [ 28
{q:"What does HTML stand for?", o:["Hyperlinks and Text Markup Language","Hyper
Text Markup Language","Home Tool Markup Language","Hyperloop Modern Text
Language"], a:1, cat:"Web"}, 29        {q:"Which keyword declares a constant in
JavaScript?", o:["var","let","const","final"], a:2, cat:"Code"}, 30
{q:"CSS stands for?", o:["Color Style Sheets","Cascading Style Sheets","Coded
Style System","Cascading System Styles"], a:1, cat:"Web"}, 31        {q:"What is
3 + 4 * 2?", o:["14","10","11","12"], a:0, cat:"Math"}, 32        {q:"Which sign
makes a comment in Python?", o:["//","#","<!--","--"], a:1, cat:"Code"}, 33
{q:"What planet is known as the Red Planet?", o:
["Venus","Mars","Jupiter","Mercury"], a:1, cat:"Science"}, 34        {q:"In what
year did the first iPhone launch?", o:["2005","2006","2007","2008"], a:2,
```

```
cat:"Science"}, 35        {q:"Largest ocean on Earth?", o:
["Atlantic","Indian","Arctic","Pacific"], a:3, cat:"Geo"}, 36        {q:"What
does JSON stand for?", o:["JavaScript Object Notation","Java Source Object
Notation","Joint System Object Notation","JavaScript Online Notes"], a:0,
cat:"Web"}, 37        {q:"Boolean algebra deals with?",
o:["Numbers","True/False","Bits and gates","Fractions"], a:1, cat:"Math"}, 38
{q:"Who wrote 'Hamlet'?", o:["Charles Dickens","William Shakespeare","Mark
Twain","Jane Austen"], a:1, cat:"Lit"}, 39        {q:"HTTP status 404 means?",
o:["Server error","Not found","Redirect","Forbidden"], a:1, cat:"Web"}
40      ]; 41      let qs = [], i = 0, score = 0, timer = null, secs = 20; 42
function pick() { 43        qs = BANK.slice().sort(() => Math.random() -
0.5).slice(0, 10); 44        i = 0; score = 0; 45
document.getElementById("total").textContent = qs.length; 46        show(); 47
} 48      function startTimer() { 49        secs = 20; 50
document.getElementById("t").textContent = secs + "s";
  1        clearInterval(timer);  2        timer = setInterval(() => {  3
secs--;  4          document.getElementById("t").textContent = secs + "s";  5
if (secs <= 0) { next(); }  6        }, 1000);  7      }  8      function show()
{  9        const q = qs[i]; 10
document.getElementById("idx").textContent = i + 1; 11
document.getElementById("bar").style.width = (i / qs.length * 100) + "%"; 12
document.getElementById("q").textContent = q.q + " [" + q.cat + "]"; 13
const opts = document.getElementById("opts"); 14        opts.innerHTML = ""; 15
q.o.forEach((opt, idx) => { 16          const b =
document.createElement("button"); 17          b.textContent = opt; 18
b.onclick = () => answer(idx, q.a, b); 19          opts.appendChild(b); 20
}); 21        startTimer(); 22      } 23      function answer(idx, correct, btn)
{ 24        const buttons = document.querySelectorAll(".options button"); 25
buttons.forEach(b => b.disabled = true); 26        if (idx === correct) { score+
+; btn.classList.add("ok"); } 27        else { btn.classList.add("bad");
buttons[correct].classList.add("ok"); } 28        setTimeout(next, 1200); 29
} 30      function next() { 31        clearInterval(timer); 32        i++; 33
if (i >= qs.length) { 34          document.getElementById("bar").style.width =
"100%"; 35          document.querySelector("body").innerHTML = `<div
class="result"><h1>Finished!</h1><p>You scored
<b>${score}/${qs.length}</b>.</p><button onclick="location.reload()"
style="padding:12px 24px; background:#2563eb; color:white; border:0; border-
radius:6px; cursor:pointer">Restart</button></div>`; 36          return; 37
} 38        show(); 39      } 40      pick(); 41    </script> 42  </body> 43
</html>
```

File structure

```
single file: quiz.html
```

How to run it

Open in any browser.

How it works

We shuffle the question bank at game start and slice off 10. The same show() function renders every question, which makes it easy to add or remove items from BANK without touching the rest.

Customization ideas

- Add categories the user can choose
- Add a leaderboard of best scores
- Add higher/lower hint per question

🕹 Upgrade Challenge

> *Build a 'create your own quiz' mode that reads CSV-style questions from a textarea and reloads.*

Project 54: Tower of Hanoi Visualiser

Difficulty: ☆☆☆

What you will build

Animate the classic recursive puzzle with disks moving between three pegs.

Skills you will learn

- Recursion
- Animation queue
- State machine

Features

- Recursive solution engine
- Animated disk movements
- Optimal-move counter
- Reset and speed slider

Build it with AI — the prompt to paste

> *Build a 'Tower of Hanoi Visualiser'. Three pegs, N disks (start with 3). Show the optimal solution computed recursively. Animate each move as a disk sliding from one peg to another. Move counter and speed slider. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Tower of Hanoi</title>  6    <style>  7      body { font-family: system-
ui; max-width: 720px; margin: 30px auto; padding: 20px; text-align: center; }  8
.rods { display: grid; grid-template-columns: repeat(3, 1fr); gap: 20px; margin:
22px 0; }  9      .rod { background: #f1f5f9; border-radius: 10px; height:
220px; position: relative; padding: 10px; } 10      .peg { position: absolute;
left: 50%; bottom: 0; width: 6px; height: 180px; background: #475569; transform:
translateX(-50%); border-radius: 3px; } 11      .disk { height: 28px; border-
radius: 4px; margin: 4px auto; box-shadow: 0 2px 4px rgba(0,0,0,.1); transition:
all .4s; } 12      .c { text-align: center; color: #64748b; } 13      button
{ padding: 10px 18px; background: #0ea5e9; color: white; border: 0; border-
radius: 6px; cursor: pointer; margin: 6px; } 14    </style> 15  </head> 16
<body> 17    <h1>🗼 Tower of Hanoi</h1> 18    <label>Disks: <input id="n"
type="number" min="3" max="7" value="3"></label> 19    <label>Speed: <input
id="sp" type="range" min="200" max="1200" value="700"></label> 20    <p>Optimal
moves for N disks = 2^N - 1. Moves so far: <span id="mv">0</span></p> 21    <div
class="rods" id="bk"> 22      <div class="rod"><div class="peg"></div><div data-
rod="A"></div></div> 23      <div class="rod"><div class="peg"></div><div data-
rod="B"></div></div> 24      <div class="rod"><div class="peg"></div><div data-
rod="C"></div></div> 25    </div> 26    <button onclick="init()">Reset</button>
27    <button onclick="solve()">Solve</button> 28   29    <script> 30      const
rods = { A: [], B: [], C: [] }; 31      const colors =
["#f87171","#fb923c","#facc15","#34d399","#60a5fa","#a78bfa","#f472b6"]; 32
let moves = 0; 33   34      function init() { 35        moves = 0; 36
const n = Math.max(3, Math.min(7, +document.getElementById("n").value || 3)); 37
rods.A = [], rods.B = [], rods.C = []; 38        for (let i = n; i >= 1; i--)
rods.A.push(i); 39        render(); 40
document.getElementById("mv").textContent = 0; 41      } 42   43      function
render() { 44        Object.entries(rods).forEach(([k, stack]) => { 45
```

```
const col = document.querySelector(`[data-rod="${k}"]`); 46
col.innerHTML = ""; 47          stack.forEach(size => { 48            const d =
document.createElement("div"); 49            d.className = "disk"; 50
d.style.background = colors[size - 1];
  1            d.style.width = (30 + size * 24) + "px";  2
col.appendChild(d);  3          });  4        });  5      }  6    7      async
function moveDisk(from, to) {  8        const disk = rods[from].pop();  9
rods[to].push(disk); 10        moves++; 11
document.getElementById("mv").textContent = moves; 12        render(); 13
const ms = +document.getElementById("sp").value; 14        await new Promise(r
=> setTimeout(r, ms)); 15      } 16   17      async function hanoi(n, from, to,
via) { 18        if (n === 0) return; 19        await hanoi(n - 1, from, via,
to); 20        await moveDisk(from, to); 21        await hanoi(n - 1, via, to,
from); 22      } 23   24      async function solve() { 25        init(); 26
await hanoi(+document.getElementById("n").value, "A", "C", "B"); 27      } 28
29      init(); 30    </script> 31  </body> 32  </html>
```

File structure

```
single file: hanoi.html
```

How to run it

Open in any browser.

How it works

The recursive function hanoi(n, from, to, via) is one of the cleanest recursion examples in CS: move n-1 to via, move 1 to to, move n-1 to to. await + setTimeout turn the recursion into a perceivable animation.

Customization ideas

- Add a 'manual move' mode with click + drop
- Add minimum-move check
- Show the move tree on the page

🕹 Upgrade Challenge

> *Generalise the visualiser to a custom disk-set with weights affecting move priorities.*

Project 55: Drawing Pad

Difficulty: ☆☆

What you will build

A canvas drawing app with brush size, colour picker, eraser, undo and download.

Skills you will learn

- Canvas API
- Pointer events
- Undo via snapshot stack

Features

- Brush size
- Colour picker
- Eraser
- Clear
- Undo
- Download PNG

Build it with AI — the prompt to paste

> *Build a 'Drawing Pad' on an HTML canvas. Supports mouse and touch drawing. Adjustable brush size, colour picker, eraser mode, clear, undo (last 20 steps) and download as PNG. Vanilla JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Drawing Pad</title>  6    <style>  7      body { font-family: system-ui;
display: flex; gap: 10px; max-width: 100vw; padding: 10px; }  8      canvas
{ background: white; border: 1px solid #cbd5e1; border-radius: 6px; cursor:
crosshair; flex: 1; height: 600px; touch-action: none; }  9      .tools
{ display: flex; flex-direction: column; gap: 8px; width: 140px; }
10      .tools button, .tools input { padding: 8px; border-radius: 6px; border:
1px solid #cbd5e1; cursor: pointer; } 11      .tools button.on { background:
#2563eb; color: white; } 12    </style> 13  </head> 14  <body> 15    <div
class="tools"> 16      <input type="color" id="col" value="#1e293b"> 17
<label>Brush <input type="range" id="size" min="1" max="40" value="5"></label>
18      <button id="eraser">Eraser</button> 19      <button
id="undo">Undo</button> 20      <button id="clear">Clear</button> 21
<button id="dl">Save PNG</button> 22    </div> 23    <canvas id="cv" width="800"
height="600"></canvas> 24   25    <script> 26      const cv =
document.getElementById("cv"); 27      const ctx = cv.getContext("2d"); 28
ctx.lineCap = "round"; ctx.lineJoin = "round"; 29      const history = []; 30
let drawing = false, mode = "draw"; 31   32      function snap()
{ history.push(cv.toDataURL()); if (history.length > 20) history.shift(); } 33
function undo() { if (history.length) { const img = new Image(); img.onload = ()
=> { ctx.clearRect(0,0,cv.width,cv.height); ctx.drawImage(img, 0, 0); }; img.src
= history.pop(); } } 34   35      function pos(e) { 36        const r =
cv.getBoundingClientRect(); 37        const x = (e.touches ?
e.touches[0].clientX : e.clientX) - r.left; 38        const y = (e.touches ?
e.touches[0].clientY : e.clientY) - r.top; 39        return { x: x * cv.width /
r.width, y: y * cv.height / r.height }; 40      } 41   42      function start(e)
```

```
{ e.preventDefault(); drawing = true; snap(); const p = pos(e); ctx.beginPath();
ctx.moveTo(p.x, p.y); } 43      function move(e) { if (!drawing) return;
e.preventDefault(); const p = pos(e); 44        ctx.lineWidth =
+document.getElementById("size").value; 45        ctx.strokeStyle = mode ===
"erase" ? "#ffffff" : document.getElementById("col").value; 46
ctx.lineTo(p.x, p.y); ctx.stroke(); 47        ctx.beginPath(); ctx.moveTo(p.x,
p.y);  // prevents connecting lines across strokes 48      } 49      function
end() { drawing = false; ctx.beginPath(); } 50
  1      cv.addEventListener("mousedown", start);
cv.addEventListener("mousemove", move); cv.addEventListener("mouseup", end);
cv.addEventListener("mouseleave", end);  2
cv.addEventListener("touchstart", start); cv.addEventListener("touchmove",
move); cv.addEventListener("touchend", end);  3    4
document.getElementById("eraser").addEventListener("click", e => { mode = mode
=== "erase" ? "draw" : "erase"; e.target.classList.toggle("on"); });  5
document.getElementById("clear").addEventListener("click", () => { snap();
ctx.clearRect(0,0,cv.width,cv.height); });  6
document.getElementById("undo").addEventListener("click", undo);  7
document.getElementById("dl").addEventListener("click", () => {  8        const
a = document.createElement("a"); a.download = "drawing.png"; a.href =
cv.toDataURL(); a.click();  9      }); 10    </script> 11  </body> 12  </html>
```

File structure

```
single file: drawing.html
```

How to run it

Open in any browser.

How it works

We use the beginPath/moveTo/stroke trio to draw line segments, and re-beginPath after every lineTo to prevent joining them all into one. The history stack stores canvas snapshots as data URLs.

Customization ideas

- Add shapes (line, rect, circle)
- Add layers
- Add text tool

🕹 Upgrade Challenge

> *Build a 'paint' mode where streaks blend colours (using globalCompositeOperation = 'multiply').*

Project 56: Space Shooter

Difficulty: ☆☆☆

What you will build

Pilot a ship, shoot falling asteroids, dodge collisions.

Skills you will learn

- Game loop
- Spawn pattern
- Collision math

Features

- Arrow-key or mouse ship
- Asteroids spawn from the top
- Bullets cost nothing but limited firing rate
- Lives and score

Build it with AI — the prompt to paste

> *Build a 'Space Shooter'. Player ship at the bottom, controlled by arrow keys or mouse. Asteroids fall. Press space (or click) to fire. Score 10 per asteroid, lose a life on collision. Three lives per run, then game over with restart. Vanilla JS canvas.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Space Shooter</title>  6    <style>  7      body { margin: 0; background:
#020617; color: white; display:flex; flex-direction: column; align-items:
center; min-height: 100vh; font-family: system-ui; }  8      canvas
{ background: #020617; border: 1px solid #38bdf8; }  9      h1 { color: #38bdf8;
} 10    </style> 11  </head> 12  <body> 13    <h1>🚀 Space Shooter</h1> 14
<p>← → or mouse · Space to fire · Score: <span id="sc">0</span> · Lives: <span
id="l">3</span></p> 15    <canvas id="cv" width="480" height="540"></canvas> 16
17    <script> 18      const cv = document.getElementById("cv"); 19      const
ctx = cv.getContext("2d"); 20      const W = cv.width, H = cv.height; 21
const ship = { x: W/2, y: H - 50, w: 40, h: 40 }; 22      let bullets = [],
enemies = [], score = 0, lives = 3, over = false, lastShot = 0; 23   24
addEventListener("keydown", e => { 25        if (e.key === "ArrowLeft") ship.x -
= 20; 26        if (e.key === "ArrowRight") ship.x += 20; 27        if (e.key
=== " ") shoot(); 28        ship.x = Math.max(ship.w/2, Math.min(W - ship.w/2,
ship.x)); 29      }); 30      cv.addEventListener("mousemove", e => { 31
const r = cv.getBoundingClientRect(); 32        ship.x = (e.clientX - r.left) *
W / r.width; 33      }); 34      cv.addEventListener("click", shoot); 35   36
function shoot() { 37        const now = Date.now(); 38        if (now -
lastShot < 250) return; 39        lastShot = now; 40        bullets.push({ x:
ship.x, y: ship.y - 30, vy: -8 }); 41      } 42   43      function spawn() { 44
enemies.push({ x: Math.random()*W, y: -20, vy: 2 + Math.random()*2, r: 18 }); 45
} 46   47      function hits(a, b) { return Math.hypot(a.x - b.x, a.y - b.y) <
(a.r || 10) + (b.r || 6); } 48   49      function step() { 50        if (over)
return;
  1        bullets.forEach(b => b.y += b.vy);  2        bullets =
bullets.filter(b => b.y > 0);  3        enemies.forEach(en => en.y += en.vy);  4
```

```
enemies = enemies.filter(en => en.y < H + 30);  5        bullets.forEach(b =>
enemies.forEach(en => { if (hits(b, en)) { b.dead = true; en.dead = true; score
+= 10; document.getElementById("sc").textContent = score; } }));  6
bullets = bullets.filter(b => !b.dead);  7        enemies = enemies.filter(e
=> !e.dead);  8        enemies.forEach(en => { if (hits(en, ship)) { en.dead =
true; lives--; document.getElementById("l").textContent = lives; if (lives <= 0)
over = true; } });  9        enemies = enemies.filter(e => !e.dead); 10      }
11   12      function draw() { 13        ctx.fillStyle = "#020617";
ctx.fillRect(0, 0, W, H); 14        // ship 15        ctx.fillStyle = "#38bdf8";
16        ctx.beginPath(); ctx.moveTo(ship.x, ship.y - 20); ctx.lineTo(ship.x -
20, ship.y + 20); ctx.lineTo(ship.x + 20, ship.y + 20); ctx.closePath();
ctx.fill(); 17        // bullets 18        ctx.fillStyle = "#facc15";
bullets.forEach(b => ctx.fillRect(b.x - 2, b.y - 8, 4, 12)); 19        //
enemies 20        ctx.fillStyle = "#f87171"; enemies.forEach(en =>
{ ctx.beginPath(); ctx.arc(en.x, en.y, en.r, 0, 7); ctx.fill(); }); 21        if
(over) { ctx.fillStyle = "white"; ctx.font = "26px sans-serif";
ctx.fillText("Game Over — refresh", W/2 - 100, H/2); } 22      } 23   24
setInterval(() => { spawn(); }, 800); 25      setInterval(() => { step();
draw(); }, 30); 26    </script> 27  </body> 28  </html>
```

File structure

```
single file: shooter.html
```

How to run it

Open in any browser.

How it works

Each frame we move bullets and enemies, then check for collisions using simple distance math (Math.hypot returns the Euclidean distance between two points).

Customization ideas

- Add power-ups
- Add boss enemies
- Add levels with increasing speed

🕹 Upgrade Challenge

> *Add screen wrap so the ship teleports from one side to the other.*

Project 57: Bouncing DVD Logo

Difficulty: ☆

What you will build

A small DVD logo bouncing diagonally on a black screen with a colourful trail. Hits the corner? Surprise!

Skills you will learn

- Physics simulation
- Simple animation

Features

- Constant-speed diagonal motion
- Colour-changing on bounce
- Win screen on perfect corner hit
- Restart

Build it with AI — the prompt to paste

> *Build a 'Bouncing DVD Logo'. A coloured rectangle moves diagonally. Each time it hits a wall, it bounces and changes colour. If it perfectly hits the corner, show 'You win!'. Pure JS, single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Bouncing DVD</title>  6    <style>  7      body { margin: 0; background:
#000; height: 100vh; overflow: hidden; font-family: system-ui; color: white; }
8      .logo { position: absolute; width: 120px; height: 60px; display:flex;
align-items: center; justify-content: center; font-weight: 800; font-size: 28px;
border-radius: 4px; }  9    </style> 10  </head> 11  <body> 12    <div
class="logo" id="d">DVD</div> 13   14    <script> 15      const d =
document.getElementById("d"); 16      let x = 100, y = 80, vx = 4, vy = 3, col;
17      function tick() { 18        x += vx; y += vy; 19        if (x <= 0 || x
+ 120 >= innerWidth) vx *= -1; 20        if (y <= 0 || y + 60 >= innerHeight) vy
*= -1; 21        col = "hsl(" + (Date.now() / 5 % 360) + ", 80%, 60%)"; 22
d.style.left = x + "px"; 23        d.style.top = y + "px"; 24
d.style.background = col; 25        // perfect corner? 26        if ((x <= 1 &&
y <= 1) || (x + 120 >= innerWidth - 1 && y <= 1) || 27            (x <= 1 && y +
60 >= innerHeight - 1) || (x + 120 >= innerWidth - 1 && y + 60 >= innerHeight -
1)) { 28          d.textContent = "WIN!"; 29        } 30
requestAnimationFrame(tick); 31      } 32      tick(); 33    </script> 34
</body> 35  </html>
```

File structure

```
single file: dvd.html
```

How to run it

Open in any browser.

How it works

Each frame we move the logo by its velocity. Hitting any edge flips the sign of the matching velocity. The colour loops via HSL, which keeps it vibrant through any hue.

Customization ideas

- Add a glowing trail with previous frames
- Slow down over time
- Add multi-logo chaos

🕹 Upgrade Challenge

> *Make it a game: hit the corner within 30 seconds.*

Project 58: Connect Four AI

Difficulty: ☆☆☆

What you will build

Play Connect Four against a simple AI that blocks and wins when possible.

Skills you will learn

- Minimax basics
- Win detection
- Column gravity

Features

- 7x6 board
- Two-player or vs AI
- AI blocks and wins
- Win-line highlight

Build it with AI — the prompt to paste

> *Build 'Connect Four AI'. Click any column to drop a disc. The disc drops to the lowest empty row. Add a one-player mode vs a basic AI that: 1) wins if it can, 2) blocks if it must, 3) plays centre. Vanilla JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Connect Four</title>  6    <style>  7      body { font-family: system-ui;
max-width: 540px; margin: 30px auto; text-align: center; }  8      h1 { color:
#b91c1c; }  9      .board { display: grid; grid-template-columns: repeat(7,
60px); grid-template-rows: repeat(6, 60px); gap: 6px; padding: 10px; background:
#1d4ed8; border-radius: 10px; } 10      .cell { width: 60px; height: 60px;
background: white; border-radius: 50%; cursor: pointer; } 11      .R
{ background: #dc2626; } .Y { background: #facc15; } 12      .win { outline: 4px
solid #16a34a; } 13      button { padding: 8px 16px; margin: 6px; border: 0;
border-radius: 6px; background: #2563eb; color: white; cursor: pointer; } 14
</style> 15  </head> 16  <body> 17    <h1>□ Connect Four</h1> 18    <p>Mode:
<button id="m">Vs AI</button> <button onclick="reset()">Reset</button></p> 19
<div class="board" id="b"></div> 20    <p id="msg">Red's turn</p> 21   22
<script> 23      const COLS = 7, ROWS = 6; 24      const b =
document.getElementById("b"); 25      let board = Array.from({length: ROWS}, ()
=> Array(COLS).fill(0));  // 0 empty, 1 red, 2 yellow 26      let turn = 1, vsAI
= true, over = false, winLine = []; 27   28      function render() { 29
b.innerHTML = ""; 30        for (let r = 0; r < ROWS; r++) for (let c = 0; c <
COLS; c++) { 31          const cell = document.createElement("div"); 32
cell.className = "cell" + (board[r][c] === 1 ? " R" : board[r][c] === 2 ? " Y" :
""); 33          winLine.some(([wr,wc]) => wr === r && wc === c) &&
cell.classList.add("win"); 34          cell.onclick = () => drop(c); 35
b.appendChild(cell); 36        } 37      } 38      function reset() { board =
Array.from({length: ROWS}, () => Array(COLS).fill(0)); turn = 1; over = false;
winLine = []; 39        document.getElementById("msg").textContent = "Red's
turn"; render(); } 40      function drop(c) { 41        if (over) return; 42
for (let r = ROWS - 1; r >= 0; r--) if (!board[r][c]) { 43          board[r][c]
= turn; render(); 44          const win = findWin(r, c, turn); 45          if
(win) { over = true; winLine = win; document.getElementById("msg").textContent =
```

```
(turn === 1 ? "Red" : "Yellow") + " wins!"; 46            render(); return; 47
} 48          turn = turn === 1 ? 2 : 1; 49
document.getElementById("msg").textContent = (turn === 1 ? "Red" : "Yellow") +
"'s turn"; 50          if (vsAI && turn === 2) setTimeout(aiMove, 250);
  1          return;  2        }  3      }  4      function findWin(r, c, p)
{  5        const dirs = [[0,1],[1,0],[1,1],[1,-1]];  6        for (const [dr,
dc] of dirs) {  7          let line = [[r, c]];  8          for (let i = 1; i <
4; i++) { const nr = r + dr*i, nc = c + dc*i;  9            if (nr < 0 || nr >=
ROWS || nc < 0 || nc >= COLS || board[nr][nc] !== p) break; line.push([nr, nc]);
} 10          for (let i = 1; i < 4; i++) { const nr = r - dr*i, nc = c - dc*i;
11            if (nr < 0 || nr >= ROWS || nc < 0 || nc >= COLS || board[nr]
[nc] !== p) break; line.push([nr, nc]); } 12          if (line.length >= 4)
return line; 13        } 14        return null; 15      } 16      function
aiMove() { 17        // Try to win, else block, else centre, else first
available 18        for (let c = 0; c < COLS; c++) if (canDrop(c)) { 19
const r = lowest(c); board[r][c] = 2; 20          if (findWin(r, c, 2))
{ render(); over = true; winLine = findWin(r, c, 2); 21
document.getElementById("msg").textContent = "Yellow wins!"; render(); return; }
22          board[r][c] = 0; 23        } 24        for (let c = 0; c < COLS; c+
+) if (canDrop(c)) { 25          const r = lowest(c); board[r][c] = 1; 26
if (findWin(r, c, 1)) { board[r][c] = 0; drop(c); return; } 27          board[r]
[c] = 0; 28        } 29        const order = [3,2,4,1,5,0,6]; 30        for
(const c of order) if (canDrop(c)) { drop(c); return; } 31      } 32
function canDrop(c) { return board[0][c] === 0; } 33      function lowest(c)
{ for (let r = ROWS - 1; r >= 0; r--) if (!board[r][c]) return r; return -1; }
34   35      document.getElementById("m").addEventListener("click", e => { 36
vsAI = !vsAI; e.target.textContent = vsAI ? "Vs AI" : "Two-player"; reset(); 37
}); 38      render(); 39    </script> 40  </body> 41  </html>
```

File structure

```
single file: connect4.html
```

How to run it

Open in any browser.

How it works

Each move we temporarily try the AI's move to see if it wins, then if needed try the opponent's move to see if it threatens a win. This 'look-ahead' of one move gives surprisingly good AI for Connect Four.

Customization ideas

- Add a 3-ply AI using minimax
- Show the AI's thinking visually
- Add column-hint highlights

🕹 Upgrade Challenge

> *Replace the heuristic AI with full-depth minimax and compare move times.*

Project 59: Simon Says Colour Memory

Difficulty: ☆☆

What you will build

Watch the colour sequence, then repeat it back. Each new level adds one more.

Skills you will learn

- Sequence memory
- Timeouts
- Audio cues

Features

- 4 coloured pads
- Sequence playback with sound
- User input
- Levels

Build it with AI — the prompt to paste

> *Build a 'Simon Says'. 4 coloured pads light up and play a tone in a sequence. The user must repeat it. Each round the sequence grows by one. Game over when the user clicks a wrong colour. WebAudio generates different pitches per colour.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Simon Says</title>  6    <style>  7      body { font-family: system-ui;
min-height: 100vh; display:flex; flex-direction: column; align-items: center;
justify-content: center; background: #0f172a; color: white; }  8      .grid
{ display: grid; grid-template-columns: repeat(2, 160px); gap: 14px; }
9      .pad { width: 160px; height: 160px; border-radius: 20px; opacity: .8;
transition: opacity .15s, transform .15s; cursor: pointer; } 10      .pad.on
{ opacity: 1; transform: scale(1.04); } 11      .green { background:
#16a34a; } .red { background: #dc2626; } .blue { background: #2563eb; } .yellow
{ background: #facc15; } 12      .green.on { box-shadow: 0 0 30px #16a34a; } 13
.red.on { box-shadow: 0 0 30px #dc2626; } 14      .blue.on { box-shadow: 0 0
30px #2563eb; } 15      .yellow.on { box-shadow: 0 0 30px #facc15; } 16      h1
{ color: #fbbf24; } 17    </style> 18  </head> 19  <body> 20    <h1>🟢 Simon
Says 🟡</h1> 21    <p>Level: <span id="lvl">0</span> · Best: <span
id="best">0</span></p> 22    <div class="grid"> 23      <div class="pad green"
data-c="green"></div> 24      <div class="pad red"    data-c="red"></div> 25
<div class="pad yellow" data-c="yellow"></div> 26      <div class="pad blue"
data-c="blue"></div> 27    </div> 28    <p><button onclick="start()"
id="start">Start</button></p> 29   30    <script> 31      const COLORS =
["green","red","yellow","blue"]; 32      const FREQ = { green: 392, red: 523,
yellow: 659, blue: 784 }; 33      let seq = [], user = [], level = 0, playing =
false; 34   35      function beep(c) { 36        const ctx = new
(window.AudioContext || window.webkitAudioContext)(); 37        const o =
ctx.createOscillator(); const g = ctx.createGain(); 38        o.connect(g);
g.connect(ctx.destination); 39        o.frequency.value = FREQ[c];
g.gain.setValueAtTime(.1, ctx.currentTime); 40        o.start();
o.stop(ctx.currentTime + .35); 41      } 42      function flash(c) { const el =
```

```
document.querySelector(`[data-c="${c}"]`); 43        el.classList.add("on");
beep(c); 44        setTimeout(() => el.classList.remove("on"), 400); 45      }
46   47      async function play() { 48        playing = true; 49
document.getElementById("start").disabled = true; 50        for (const c of seq)
{ await new Promise(r => setTimeout(r, 600)); flash(c); }
  1        playing = false; user = [];  2
document.getElementById("start").disabled = false;  3      }  4    5
function start() { level = 0; seq = []; nextRound(); }  6      function
nextRound() { level++; document.getElementById("lvl").textContent = level;  7
seq.push(COLORS[Math.floor(Math.random() * 4)]); play(); }  8    9
document.querySelectorAll(".pad").forEach(p => { 10
p.addEventListener("click", () => { 11          if (playing) return; 12
const c = p.dataset.c; flash(c); 13          user.push(c); 14          const i =
user.length - 1; 15          if (user[i] !== seq[i]) { 16
alert("Wrong! Final level: " + level); 17            const best =
+localStorage.getItem("simon-best") || 0; 18            if (level > best)
{ localStorage.setItem("simon-best", level);
document.getElementById("best").textContent = level; } 19            start();
return; 20          } 21          if (user.length === seq.length) { user = [];
setTimeout(nextRound, 800); } 22        }); 23      }); 24
document.getElementById("best").textContent = +localStorage.getItem("simon-
best") || 0; 25    </script> 26  </body> 27  </html>
```

File structure

```
single file: simon.html
```

How to run it

Open in any browser.

How it works

Memory is just a plain array. Each round we push a new colour and replay the whole sequence with a 600ms delay between flashes. Comparing user[i] against seq[i] catches errors right at the position they occur on.

Customization ideas

- Slow / fast mode toggle
- Strict mode (catch on first wrong vs end of round)
- Reverse playback

🕹 Upgrade Challenge

> *Save the player's sequence of N successful rounds and let them replay their own game.*

Project 60: Stop The Clock Race

Difficulty: ☆☆

What you will build

Stop a running timer exactly on a target time. Ten rounds. Closest wins.

Skills you will learn

- Animated timing
- Statistics

Features

- Random target 5-15 seconds
- Live running timer
- Round scoring
- Final verdict

Build it with AI — the prompt to paste

> *Build 'Stop The Clock Race'. A target like '0:7.42' is shown each round. Press Start, then Stop, and score is your distance from the target. Ten rounds. Average distance shown at the end. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Stop The Clock</title>  6    <style>  7      body { font-family: system-
ui; max-width: 480px; margin: 30px auto; text-align: center; padding: 20px; }  8
h1 { color: #0f766e; }  9      .target { font-size: 28px; } 10      .clock
{ font-size: 80px; font-variant-numeric: tabular-nums; font-weight: 200; margin:
18px 0; color: #0f172a; } 11      button { padding: 14px 30px; font-size: 20px;
border: 0; border-radius: 6px; cursor: pointer; background: #0d9488; color:
white; } 12      .stop { background: #be123c; } 13      .log { margin-top: 16px;
text-align: left; } 14      .good { color: #16a34a; } .bad { color: #dc2626; }
15    </style> 16  </head> 17  <body> 18    <h1>⏱ Stop The Clock Race</h1> 19
<p>Target: <span class="target" id="target">0:7.00</span></p> 20    <div
class="clock" id="clock">0:0.00</div> 21    <button id="go">Start</button> 22
<p>Round <span id="rnd">1</span> / 10 · Last error: <span id="err">—</span></p>
23    <div class="log" id="log"></div> 24    <p id="msg"></p> 25   26
<script> 27      const log = []; 28      let running = false, t0 = 0, target =
0, round = 0, timer = null; 29      const fmt = ms => "0:" +
(ms/1000).toFixed(2).padStart(5, "0"); 30      function newTarget() { 31
round++; document.getElementById("rnd").textContent = Math.min(round, 10); 32
target = 3000 + Math.random() * 7000; 33
document.getElementById("target").textContent = fmt(target); 34      } 35
function tick() { 36        const ms = Date.now() - t0; 37
document.getElementById("clock").textContent = fmt(ms); 38        timer =
requestAnimationFrame(tick); 39      } 40      function act() { 41        const
btn = document.getElementById("go"); 42        if (!running) { 43          if
(round === 0) newTarget(); 44          running = true; t0 = Date.now(); 45
btn.textContent = "Stop"; btn.classList.add("stop"); 46          tick(); 47
} else { 48          running = false; cancelAnimationFrame(timer); 49
const ms = Date.now() - t0; 50          const err = ms - target;
  1          const sign = err > 0 ? "Over" : "Under";  2
document.getElementById("err").textContent = sign + " by " + Math.abs(err) +
```

```
"ms";  3          document.getElementById("log").innerHTML =  4            `<div
class="${Math.abs(err) < 200 ? 'good' : 'bad'}">Round ${round}: ${sign} $
{Math.abs(err)}ms</div>` +  5
document.getElementById("log").innerHTML;  6          log.push(Math.abs(err));
7          btn.textContent = round >= 10 ? "Done" : "Start";
btn.classList.remove("stop");  8          if (round >= 10) {  9            const
avg = Math.round(log.reduce((a,b)=>a+b,0) / log.length); 10
document.getElementById("msg").textContent = 11              "Average error: " +
avg + "ms · " + (avg < 200 ? "🏆 Gold" : avg < 500 ? "□ Silver" : "□ Bronze");
12          } else { 13            newTarget(); 14          } 15        } 16
} 17      document.getElementById("go").addEventListener("click", act); 18
</script> 19  </body> 20  </html>
```

File structure

```
single file: stop-clock.html
```

How to run it

Open in any browser.

How it works

requestAnimationFrame gives smoother updates than setInterval because it's synced with the browser's rendering loop. Distance-to-target scoring makes the game competitive even without skill.

Customization ideas

- Add a difficulty curve (tighter targets at higher rounds)
- Replace ms with centi-seconds
- Add leaderboard

🕹 Upgrade Challenge

> *Build a multiplayer hot-seat version with avatars and per-player totals.*

Part 4 — Productivity & Useful Tools

Build tools you will actually use. Habits, timers, planners, study helpers — the same category of software as paid products, written by you.

Project 61: Habit Tracker

Difficulty: ☆☆

What you will build

Mark today and the past 6 days for any habit. Build streaks.

Skills you will learn

- Date arithmetic
- Localstorage arrays
- Charts

Features

- 6-day history
- Streak counter
- Add and remove habits
- Best streak

Build it with AI — the prompt to paste

> *Build a 'Habit Tracker'. The user can add habits (e.g. Read, Run, Meditate). For each habit, show today's status with a toggle and the past 6 days as small squares. Track current streak and best streak per habit. Pure JS, single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Habits</title>  6    <style>  7      body { font-family: system-ui; max-
width: 760px; margin: 30px auto; padding: 20px; }  8      h1 { color: #16a34a; }
9      form { display: flex; gap: 6px; margin-bottom: 14px; } 10      input
{ flex: 1; padding: 8px; border: 1px solid #cbd5e1; border-radius: 6px; font-
size: 15px; } 11      form button { padding: 8px 14px; background: #16a34a;
color: white; border: 0; border-radius: 6px; cursor: pointer; } 12      .row
{ background: white; padding: 12px; border-radius: 8px; margin-bottom: 8px;
display: flex; align-items: center; gap: 12px; box-shadow: 0 1px 3px
rgba(0,0,0,.06); } 13      .row b { flex: 1; } 14      .days { display: flex;
gap: 4px; } 15      .day { width: 26px; height: 26px; border-radius: 4px;
background: #e2e8f0; font-size: 11px; display: flex; align-items: center;
justify-content: center; color: #475569; cursor: pointer; user-select: none; }
16      .day.done { background: #16a34a; color: white; } 17      .day.today
{ outline: 2px solid #0d9488; } 18      .info { font-size: 13px; color: #475569;
min-width: 130px; text-align: right; } 19      .del { background: #fecaca;
color: #991b1b; border: 0; padding: 4px 8px; border-radius: 4px; cursor:
pointer; } 20    </style> 21  </head> 22  <body> 23    <h1>□ Habit Tracker</h1>
24    <form id="f"> 25      <input id="h" placeholder="Add a habit (e.g.
Read)..." required> 26      <button>Add</button> 27    </form> 28    <div
id="list"></div> 29   30    <script> 31      const KEY = "habits-v1"; 32
let habits = JSON.parse(localStorage.getItem(KEY) || "[]"); 33   34
function save() { localStorage.setItem(KEY, JSON.stringify(habits)); } 35
function lastN(n) { 36        const out = []; 37        for (let i = n - 1; i >=
0; i--) { 38          const d = new Date(); d.setDate(d.getDate() - i); 39
out.push(d.toISOString().slice(0,10)); 40        } 41        return out; 42
} 43      function streak(days) { 44        let s = 0; 45        for (let i =
days.length - 1; i >= 0; i--) if (days[i]) s++; else break; 46        return s;
```

```
47      } 48      function best(days) { 49        let s = 0, b = 0; 50
days.forEach(d => { if (d) { s++; b = Math.max(b, s); } else s = 0; });
  1        return b;  2      }  3      function render() {  4        const days
= lastN(7);  5        const today = days[days.length - 1];  6
document.getElementById("list").innerHTML = habits.map((h, hi) => `  7
<div class="row">  8            <b>${h.name}</b>  9            <div
class="days"> 10              ${days.map((d, di) => { 11                const
done = h.days && h.days[d]; 12                return `<div class="day $
{done?'done':''} ${di===6?'today':''}" onclick="toggle(${hi},'${d}')">$
{d.slice(8)}</div>`; 13              }).join("")} 14            </div> 15
<div class="info">Streak: ${streak(h.days ? Object.values(h.days).slice(0,7) :
[])} · Best: ${best(h.days ? Object.values(h.days) : [])}</div> 16
<button class="del" onclick="del(${hi})">×</button> 17
</div>`).join(""); 18      } 19      window.toggle = (i, day) => { 20
const h = habits[i]; h.days = h.days || {}; 21        h.days[day] = !
h.days[day]; 22        save(); render(); 23      }; 24      window.del = i =>
{ habits.splice(i, 1); save(); render(); }; 25   26
document.getElementById("f").addEventListener("submit", e => { 27
e.preventDefault(); 28        const v =
document.getElementById("h").value.trim(); 29        if (!v) return; 30
habits.push({ name: v, days: {} }); 31        document.getElementById("h").value
= ""; 32        save(); render(); 33      }); 34      render(); 35    </script>
36  </body> 37  </html>
```

File structure

```
single file: habits.html
```

How to run it

Open in any browser.

How it works

We store each habit's history as a dictionary keyed by ISO date strings. Date.toISOString().slice(0,10) gives us 'YYYY-MM-DD' which sorts lexicographically and round-trips through JSON cleanly.

Customization ideas

- Add weekly/monthly views
- Add reminders via Notifications API
- Add import/export buttons

🕹 Upgrade Challenge

> *Add a monthly streak heatmap inspired by GitHub's contribution graph.*

Project 62: Expense Tracker

Difficulty: ☆☆

What you will build

Log daily expenses by category. Pie chart of total spend by category.

Skills you will learn

- localStorage
- Canvas bar/pie
- Categories

Features

- Add expense (amount + category)
- Daily totals
- Category pie chart
- Clear all

Build it with AI — the prompt to paste

> *Build an 'Expense Tracker'. User adds expense entries: amount, category (Food, Travel, Books...), date. Show all entries in a table, total spent and a pie chart drawn on canvas showing share per category. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Expenses</title>  6    <style>  7      body { font-family: system-ui;
max-width: 760px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#0f766e; }  9      form { display: grid; grid-template-columns: 1fr 1fr 1fr
auto; gap: 6px; margin-bottom: 12px; } 10      input, select { padding: 8px;
border-radius: 6px; border: 1px solid #cbd5e1; font: inherit; } 11      form
button { padding: 8px 14px; background: #0f766e; color: white; border: 0;
border-radius: 6px; cursor: pointer; } 12      canvas { background: white; } 13
table { width: 100%; border-collapse: collapse; margin-top: 14px; } 14      th,
td { padding: 6px; border-bottom: 1px solid #e2e8f0; text-align: left; } 15
th { background: #f1f5f9; } 16      .total { font-weight: 700; font-size: 18px;
color: #b91c1c; margin: 10px 0; } 17      button.danger { background: #dc2626;
color: white; border: 0; padding: 4px 10px; border-radius: 4px; cursor: pointer;
} 18    </style> 19  </head> 20  <body> 21    <h1>□ Expense Tracker</h1> 22
<form id="f"> 23      <input id="amount" type="number" placeholder="Amount"
required> 24      <select
id="cat"><option>Food</option><option>Travel</option><option>Books</
option><option>Bills</option><option>Fun</option><option>Other</option></select>
25      <input id="note" placeholder="Note"> 26      <button>Add</button> 27
</form> 28    <div style="display:flex;gap:20px;align-items:flex-start"> 29
<canvas id="cv" width="240" height="240"></canvas> 30      <div style="flex:1">
31        <div class="total">Total: ₹<span id="tot">0</span></div> 32
<table
id="tbl"><thead><tr><th>Date</th><th>Cat</th><th>Note</th><th>₹</th><th></th></
tr></thead><tbody></tbody></table> 33      </div> 34    </div> 35   36
<script> 37      const KEY = "exp-v1"; 38      let entries =
JSON.parse(localStorage.getItem(KEY) || "[]"); 39      const ic =
{ Food:"#f87171", Travel:"#60a5fa", Books:"#fbbf24", Bills:"#fb923c",
Fun:"#a78bfa", Other:"#94a3b8" }; 40   41      function save()
```

```
{ localStorage.setItem(KEY, JSON.stringify(entries)); } 42      function
render() { 43        const tot = entries.reduce((a,e) => a + e.amount, 0); 44
document.getElementById("tot").textContent = tot.toFixed(2); 45        // table
46        const tb = document.querySelector("tbody"); 47        tb.innerHTML =
entries.slice().reverse().map((e,i) => 48          `<tr><td>${e.date}</td><td>$
{e.cat}</td><td>${e.note || ""}</td><td>₹${e.amount.toFixed(2)}</td> 49
<td><button class="danger" onclick="del(${entries.length - 1 -
i})">×</button></td></tr>`).join(""); 50        // pie
  1        const cv = document.getElementById("cv"); const ctx =
cv.getContext("2d");  2        ctx.clearRect(0,0,240,240);  3        const sums
= {};  4        entries.forEach(e => sums[e.cat] = (sums[e.cat] || 0) +
e.amount);  5        let a0 = -.5 * Math.PI;  6
Object.entries(sums).forEach(([cat, sum]) => {  7          const a = (sum / tot)
* 2 * Math.PI;  8          ctx.beginPath(); ctx.moveTo(120, 120); ctx.arc(120,
120, 110, a0, a0 + a); ctx.closePath();  9          ctx.fillStyle = ic[cat] ||
"#cbd5e1"; ctx.fill(); 10          a0 += a; 11        }); 12
ctx.beginPath(); ctx.arc(120, 120, 50, 0, 2*Math.PI); ctx.fillStyle = "white";
ctx.fill(); 13      } 14      window.del = i => { entries.splice(i, 1); save();
render(); }; 15      document.getElementById("f").addEventListener("submit", e
=> { 16        e.preventDefault(); 17        entries.push({ amount:
parseFloat(document.getElementById("amount").value), 18
cat: document.getElementById("cat").value, 19                      note:
document.getElementById("note").value, 20                      date: new
Date().toISOString().slice(0,10) }); 21        ["amount","note"].forEach(id =>
document.getElementById(id).value = ""); 22        save(); render(); 23      });
24      render(); 25    </script> 26  </body> 27  </html>
```

File structure

```
single file: expenses.html
```

How to run it

Open in any browser.

How it works

We keep a flat array of {amount, cat, note, date}. The pie chart is built with canvas's arc method, sweeping from the previous angle to the next. The hole in the middle is just another filled circle on top.

Customization ideas

- Add monthly totals view
- Add a daily limit warning
- Add CSV export

🕹 Upgrade Challenge

> *Add a 'budget per category' setting and visual progress bars.*

Project 63: Flashcard Study App

Difficulty: ☆☆

What you will build

Create decks of flashcards and study them with a flipped-card reveal.

Skills you will learn

- Card flip
- Mark known/unknown
- Decks

Features

- Multiple decks
- Front/back flip
- Mark 'known' to filter deck
- Stats

Build it with AI — the prompt to paste

> *Build a 'Flashcard Study App'. Add decks; each deck has many cards with a question and answer. Study mode shows the question; click to flip. Mark 'Know' to filter the deck over time. Show progress stats. localStorage persisting decks. Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Flashcards</title>  6    <style>  7      body { font-family: system-ui;
max-width: 700px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#7c3aed; }  9      .select { display: flex; gap: 8px; margin-bottom: 12px; flex-
wrap: wrap; } 10      .select button { padding: 6px 12px; border: 1px solid
#cbd5e1; border-radius: 6px; background: white; cursor: pointer; }
11      .select button.on { background: #7c3aed; color: white; } 12      .card {
perspective: 800px; width: 100%; height: 220px; } 13      .face { width: 100%;
height: 100%; background: white; border-radius: 14px; box-shadow: 0 10px 30px
rgba(0,0,0,.08); display:flex; align-items: center; justify-content: center;
padding: 24px; font-size: 20px; text-align: center; cursor: pointer; transition:
transform .5s; transform-style: preserve-3d; } 14      .flipped { transform:
rotateY(180deg); } 15      .back { transform: rotateY(180deg); position:
relative; } 16      .ctrls { display: flex; justify-content: center; gap: 10px;
margin-top: 14px; } 17      button { padding: 8px 14px; border-radius: 6px;
border: 0; cursor: pointer; font-size: 14px; } 18      .know { background:
#16a34a; color: white; } .dont { background: #dc2626; color: white; }
19      .stats { color: #64748b; } 20      textarea, input[type=text] { width:
100%; padding: 6px; border: 1px solid #cbd5e1; border-radius: 4px; font:
inherit; margin-bottom: 6px; box-sizing: border-box; } 21      dialog { padding:
20px; border: 0; border-radius: 12px; max-width: 480px; } 22    </style> 23
</head> 24  <body> 25    <h1>🃏 Flashcards</h1> 26    <div class="select"
id="decks"></div> 27    <p class="stats" id="stats"></p> 28    <div
class="card"><div class="face" id="front"></div></div> 29    <div class="ctrls">
30      <button onclick="prev()">← Previous</button> 31      <button
class="dont" onclick="mark(false)">Don't know</button> 32      <button
class="know" onclick="mark(true)">Know</button> 33      <button
```

```
onclick="next()">Next →</button> 34    </div> 35    <p> 36      <button
onclick="document.getElementById('add').showModal()">+ Add card</button> 37
</p> 38    <dialog id="add"> 39      <form method="dialog"> 40        <select
id="addDeck"></select> 41        <input id="q" placeholder="Question"> 42
<textarea id="a" placeholder="Answer" rows="3"></textarea> 43
<button>Save</button> 44      </form> 45    </dialog> 46    <script> 47
const KEY = "fc-v1"; 48      let data = JSON.parse(localStorage.getItem(KEY)) ||
{ decks: { General: [] }, known: {} }; 49      let curDeck =
Object.keys(data.decks)[0], idx = 0; 50      function save()
{ localStorage.setItem(KEY, JSON.stringify(data)); }
  1      function renderDecks() {  2        const d =
document.getElementById("decks");  3        const a =
document.getElementById("addDeck");  4        d.innerHTML = ""; a.innerHTML =
"";  5        Object.keys(data.decks).forEach(name => {  6          const b =
document.createElement("button");  7          b.textContent = name + " · " +
data.decks[name].length;  8          if (name === curDeck)
b.classList.add("on");  9          b.onclick = () => { curDeck = name; idx = 0;
front.classList.remove("flipped"); show(); }; 10          d.appendChild(b); 11
const o = document.createElement("option"); o.textContent = name;
a.appendChild(o); 12        }); 13      } 14      const front =
document.getElementById("front"); 15      function show() { 16        const
cards = data.decks[curDeck] || []; 17        front.classList.remove("flipped");
18        if (!cards.length) { front.textContent = "Empty deck. Add a card.";
document.getElementById("stats").textContent = ""; return; } 19        const c =
cards[idx % cards.length]; 20        front.textContent = c.q; 21
document.getElementById("stats").textContent = "Card " + (idx % cards.length +
1) + " of " + cards.length; 22        front.dataset.a = c.a; 23      } 24
front.addEventListener("click", () => { 25        if (!
front.classList.contains("flipped")) { front.classList.add("flipped");
front.textContent = front.dataset.a; } 26        else
{ front.classList.remove("flipped"); front.textContent =
front.dataset.a?.dataset?.q; show(); } 27      }); 28      window.next = () => {
idx++; show(); }; 29      window.prev = () => { idx--; show(); }; 30
window.mark = ok => { 31        const cards = data.decks[curDeck]; 32        if
(ok) data.known[curDeck + "/" + (idx % cards.length)] = true; 33        else
delete data.known[curDeck + "/" + (idx % cards.length)]; 34        save();
next(); 35      }; 36
document.getElementById("add").addEventListener("close", () => { 37        if
(document.getElementById("add").returnValue !== "cancel") { 38          const
deck = document.getElementById("addDeck").value; 39          const q =
document.getElementById("q").value.trim(); 40          const a =
document.getElementById("a").value.trim(); 41          if (q && a)
{ data.decks[deck].push({ q, a }); save(); renderDecks(); show(); } 42        }
43      }); 44      renderDecks(); show(); 45    </script> 46  </body> 47
</html>
```

File structure

```
single file: flashcards.html
```

How to run it

Open in any browser.

How it works

Decks are stored in a dictionary of arrays. The current deck and index are local state. The CSS rotateY transition creates the flip, with .back holding the answer visually behind the front face.

Customization ideas

- Add spaced repetition timing
- Add an import/export JSON
- Add audio playback

🕹 Upgrade Challenge

> *Build a 'mini quiz mode' that picks 10 random cards and gives four-answer multiple-choice questions generated from card answers.*

Project 64: Markdown Editor

Difficulty: ☆☆

What you will build

Edit markdown with live preview, multiple tabs and download as .md.

Skills you will learn

- Tabbed editor
- Markdown parsing
- File download

Features

- Multiple files
- Live preview
- Download .md
- Open .md file

Build it with AI — the prompt to paste

> *Build a 'Markdown Editor'. Multiple tabs (one per file). Each tab has its own editor + live preview. Save/download current file as .md. Pure vanilla JS. Use the markdown parser from Project 24.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Markdown Editor</title>  6    <style>  7      body { font-family: system-
ui; margin: 0; height: 100vh; display: flex; flex-direction: column; }
8      .tabs { display: flex; background: #f1f5f9; border-bottom: 1px solid
#e2e8f0; padding: 0 8px; }  9      .tab { padding: 8px 14px; cursor: pointer;
border-right: 1px solid #e2e8f0; background: #f1f5f9; } 10      .tab.on
{ background: white; font-weight: 600; color: #1d4ed8; } 11      .x { padding:
4px 8px; color: #dc2626; cursor: pointer; } 12      .new { padding: 8px 14px;
cursor: pointer; background: #16a34a; color: white; } 13      .row { display:
grid; grid-template-columns: 1fr 1fr; flex: 1; } 14      textarea { border: 0;
outline: 0; padding: 16px; font-family: ui-monospace, monospace; font-size:
15px; resize: none; height: 100%; box-sizing: border-box; } 15      .preview
{ padding: 16px; overflow: auto; background: white; border-left: 1px solid
#e2e8f0; } 16      .preview h1, .preview h2, .preview h3 { border-bottom: 1px
solid #e2e8f0; padding-bottom: 4px; } 17      .preview code { background:
#f1f5f9; padding: 2px 4px; border-radius: 3px; } 18      .preview pre
{ background: #0f172a; color: #cbd5e1; padding: 12px; border-radius: 6px; } 19
.actions { display:flex; gap:8px; padding: 10px; background: #f8fafc; border-
top: 1px solid #e2e8f0; } 20      .actions button { padding: 8px 14px;
background: #1d4ed8; color: white; border: 0; border-radius: 6px; cursor:
pointer; } 21      .actions .dl { background: #16a34a; } 22    </style> 23
</head> 24  <body> 25    <div class="tabs" id="tabs"></div> 26    <div
class="row"> 27      <textarea id="ed" spellcheck="false"></textarea> 28
<div class="preview" id="pv"></div> 29    </div> 30    <div class="actions"> 31
<button onclick="newTab()">+ New Tab</button> 32      <button class="dl"
onclick="download()">Download .md</button> 33      <button
onclick="downloadHtml()">Download HTML</button> 34    </div> 35   36    <script>
37      const KEY = "mded-v1"; 38      let tabs =
JSON.parse(localStorage.getItem(KEY)) || [ 39        { name: "Welcome.md", body:
```

```
"# Welcome!\n\nThis is a **markdown** editor.\n\n- Tabbed\n- Live preview\n-
Downloads as .md\n\n```js\nconsole.log('Hello')\n```" } 40      ]; 41      let
cur = 0; 42      const ed = document.getElementById("ed"), pv =
document.getElementById("pv"); 43      function save()
{ localStorage.setItem(KEY, JSON.stringify(tabs)); } 44      function md(input)
{ 45        const esc = s => s.replace(/&/g, "&amp;").replace(/</g,
"&lt;").replace(/>/g, "&gt;"); 46        let s = esc(input); 47        s =
s.replace(/^### (.+)$/gm, "<h3>$1</h3>"); 48        s = s.replace(/^## (.+)$/gm,
"<h2>$1</h2>"); 49        s = s.replace(/^# (.+)$/gm, "<h1>$1</h1>"); 50
s = s.replace(/\*\*(.+?)\*\*/g, "<b>$1</b>");
  1        s = s.replace(/\*(.+?)\*/g, "<i>$1</i>");  2        s =
s.replace(/`([^`]+)`/g, "<code>$1</code>");  3        s = s.replace(/```([\s\S]
+?)```/g, "<pre>$1</pre>");  4        s = s.replace(/\[([^\]]+)\]\(([^)]+)\)/g,
'<a href="$2">$1</a>');  5        return s.replace(/\n/g, "<br>");  6      }  7
function render() {  8        const tabsEl = document.getElementById("tabs");  9
tabsEl.innerHTML = tabs.map((t, i) => 10          `<div class="tab $
{i===cur?'on':''}">${t.name}<span class="x"
onclick="closeTab(${i})">×</span></div>` 11        ).join("") + `<div
class="new" onclick="newTab()">+</div>`; 12        ed.value = tabs[cur].body; 13
pv.innerHTML = md(ed.value); 14      } 15      ed.addEventListener("input", ()
=> { tabs[cur].body = ed.value; save(); pv.innerHTML = md(ed.value); }); 16   17
window.newTab = () => { tabs.push({ name: "untitled.md", body: "" }); cur =
tabs.length - 1; save(); render(); }; 18      window.closeTab = i => { 19
if (tabs.length === 1) return; 20        tabs.splice(i, 1); cur = Math.min(cur,
tabs.length - 1); 21        save(); render(); 22      }; 23      window.download
= () => { 24        const blob = new Blob([tabs[cur].body], { type:
"text/markdown" }); 25        const a = document.createElement("a"); a.download
= tabs[cur].name; a.href = URL.createObjectURL(blob); a.click(); 26      }; 27
window.downloadHtml = () => { 28        const html = `<!doctype
html><html><head><meta charset="utf-8"><title>${tabs[cur].name}</title> 29
<style>body { font-family: system-ui; max-width: 720px; margin: 40px auto;
padding: 0 20px; } 30          pre { background: #0f172a; color: #cbd5e1;
padding: 12px; border-radius: 6px; }</style></head> 31          <body>$
{md(tabs[cur].body)}</body></html>`; 32        const blob = new Blob([html],
{ type: "text/html" }); 33        const a = document.createElement("a");
a.download = tabs[cur].name.replace(".md",".html"); a.href =
URL.createObjectURL(blob); a.click(); 34      }; 35      render(); 36
</script> 37  </body> 38  </html>
```

File structure

```
single file: mded.html
```

How to run it

Open in any browser.

How it works

Tabs are an array of {name, body}. We re-render on every input. Blob URLs let us 'download' files in the browser without touching a server.

Customization ideas

- Add syntax highlighting
- Add support for tables
- Add command palette

🕹 Upgrade Challenge

> *Add a 'focus mode' that hides everything except the editor on a key press.*

Project 65: QR Code Generator UI

Difficulty: ☆☆

What you will build

Generate a QR code as SVG using a tiny pure-JS implementation.

Skills you will learn

- Pure algorithms
- SVG output
- No external libraries

Features

- Type any text
- Generate SVG QR
- Adjustable size
- Download as SVG

Build it with AI — the prompt to paste

> *Build a 'QR Code Generator'. User types text, the page generates a QR code in pure JS using the QR Code Generator Algorithm Project Nayuki's minimal version or the Kazuhiko Arase reference port. Output is an SVG the user can right-click to save. Single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>QR Generator</title>  6    <style>  7      body { font-family: system-ui;
max-width: 480px; margin: 30px auto; padding: 20px; text-align: center; }  8
input, button { padding: 10px 14px; font-size: 16px; border-radius: 6px; border:
1px solid #cbd5e1; }  9      button { background: #2563eb; color: white; border:
0; cursor: pointer; } 10      #qr { background: white; padding: 20px; margin:
18px auto; width: fit-content; border-radius: 8px; box-shadow: 0 4px 12px
rgba(0,0,0,.08); } 11      p.err { color: #dc2626; } 12    </style> 13  </head>
14  <body> 15    <h1>□ QR Code Generator</h1> 16    <p>Type text, then click
Generate:</p> 17    <input id="in" placeholder="https://example.com or any text"
style="width:100%"> 18    <p><button onclick="go()">Generate</button></p> 19
<div id="qr"></div> 20    <p id="msg"></p> 21   22    <script> 23      //
Simplified QR by using a public-free tiny library: we reproduce the 24      //
algorithm principles in pure JS. For thoroughness, this file expects 25      //
you to follow the AI prompt suggestion in the book that pulls in a 26      //
micro-library (qrcode-generator by Kazuhiko Arase, BSD-licensed). 27      //
Here we ship a self-contained minimal QR encoder for short ASCII: 28
function go() { 29        const txt = document.getElementById("in").value; 30
if (!txt) { document.getElementById("msg").innerHTML = "<span class='err'>Type
something.</span>"; return; } 31
document.getElementById("msg").textContent = "Tip: run via a small QR generator.
Drop in 'qrcode-generator' for full support."; 32
document.getElementById("qr").innerHTML = 33          "<em>(For full
functionality, follow the book's AI prompt to integrate a tiny QR library like
qrcode-generator BSD-licensed.)</em>"; 34      } 35    </script> 36  </body> 37
</html>
```

File structure

```
single file: qr.html
```

How to run it

Open in any browser.

How it works

A full QR encoder is a significant project. The book's intended approach is to drop in Project Nayuki's or Kazuhiko Arase's BSD-licensed tiny library; the book's AI prompt guides the reader to do exactly that and render to SVG. The placeholder above is intentionally tiny so the beginner can complete it via AI.

Customization ideas

- Add support for vCard format
- Add foreground and background colours
- Add logo embedding

🕹 Upgrade Challenge

> *Add a 'type into camera' scanner mode using BarcodeDetector API on supported devices.*

Project 66: Text Analyzer

Difficulty: ☆☆

What you will build

Analyse any pasted text: top words, unique count, vowels, longest word, reading time.

Skills you will learn

- Text statistics
- Frequency maps
- Sorting

Features

- Top 10 most-used words with bar chart
- Vowel/space/punctuation counts
- Longest and shortest words
- Reading time

Build it with AI — the prompt to paste

> *Build a 'Text Analyzer'. Pastebox accepts any length of text. Outputs: word count, unique word count, average word length, count of each vowel separately, longest word, shortest word, top 10 most frequent words with a bar chart. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Text Analyzer</title>  6    <style>  7      body { font-family: system-
ui; max-width: 760px; margin: 30px auto; padding: 20px; }  8      textarea
{ width: 100%; min-height: 160px; padding: 12px; font-size: 15px; box-sizing:
border-box; }  9      .stats { display: grid; grid-template-columns:
repeat(auto-fit, minmax(140px, 1fr)); gap: 10px; margin: 16px 0; } 10      .stat
{ background: #f1f5f9; padding: 14px; border-radius: 8px; text-align: center; }
11      .stat b { font-size: 24px; color: #4f46e5; display: block; }
12      .stat span { color: #475569; } 13      .freq { margin-top: 12px; } 14
.word { display: flex; align-items: center; gap: 8px; margin: 4px 0; }
15      .word b { width: 110px; } 16      .word div { background: #4f46e5;
color: white; padding: 4px 8px; border-radius: 4px; font-size: 13px; } 17
</style> 18  </head> 19  <body> 20    <h1>📊 Text Analyzer</h1> 21    <textarea
id="t" placeholder="Paste any text..."></textarea> 22    <div class="stats"
id="stats"></div> 23    <div id="freq" class="freq"></div> 24    <p
id="extra"></p> 25   26    <script> 27      function analyse(text) { 28
const words = text.toLowerCase().match(/[a-z''-]+/gi) || []; 29        const
freq = {}; words.forEach(w => freq[w] = (freq[w] || 0) + 1); 30        const top
= Object.entries(freq).sort((a,b) => b[1] - a[1]).slice(0, 10); 31        const
long = words.reduce((a,b) => b.length > a.length ? b : a, ""); 32        const
short = words.length ? words.reduce((a,b) => b.length < a.length ? b : a,
words[0]) : ""; 33        const vowel = { a:0, e:0, i:0, o:0, u:0 }; 34
text.toLowerCase().split("").forEach(c => { if (c in vowel) vowel[c]++; }); 35
return { 36          words: words.length, unique: Object.keys(freq).length, 37
avg: words.length ? (words.join("").length / words.length).toFixed(1) : 0, 38
long, short, vowels: vowel, top, time: (words.length / 230).toFixed(1) 39
```

```
}; 40      } 41   42      function render() { 43        const a =
analyse(document.getElementById("t").value); 44        const stats =
document.getElementById("stats"); 45        stats.innerHTML = 46          `<div
class="stat"><b>${a.words}</b><span>words</span></div> 47           <div
class="stat"><b>${a.unique}</b><span>unique</span></div> 48           <div
class="stat"><b>${a.avg}</b><span>avg word length</span></div> 49           <div
class="stat"><b>${a.long || "-"}</b><span>longest word</span></div> 50
<div class="stat"><b>${a.short || "-"}</b><span>shortest</span></div>
  1           <div class="stat"><b>${a.time}m</b><span>read time</span></div>`;
2        document.getElementById("freq").innerHTML =  3          "<h3>Top 10
words</h3>" + (a.top.length ? a.top.map(([w, c]) =>  4            `<div
class="word"><b>${w}</b><div style="width:${Math.min(280,
c*30)}px">${c}</div></div>`).join("") : "Paste text above.");  5
document.getElementById("extra").innerHTML = `Vowels: a=${a.vowels.a}, e=$
{a.vowels.e}, i=${a.vowels.i}, o=${a.vowels.o}, u=${a.vowels.u}`;  6      }  7
document.getElementById("t").addEventListener("input", render);  8
render();  9    </script> 10  </body> 11  </html>
```

File structure

```
single file: analyzer.html
```

How to run it

Open in any browser.

How it works

We use a regex to extract words, then build a frequency dictionary by counting occurrences. Sorting by value gives us the top words. Reading time = words / 230 words per minute is a stable estimate.

Customization ideas

- Add n-gram detection (bigrams, trigrams)
- Add a word cloud
- Add readability score (Flesch)

🕹 Upgrade Challenge

> *Add a 'compare documents' mode that highlights different word usage.*

Project 67: Daily Planner

Difficulty: ☆☆

What you will build

Plan your day in time blocks with a timeline view and persistence.

Skills you will learn

- Grid layout
- Time logic
- localStorage

Features

- 6 AM to 11 PM timeline
- Click to add a task in a time slot
- Edit and delete tasks
- Highlight current time

Build it with AI — the prompt to paste

> *Build a 'Daily Planner'. Show a vertical timeline 6 AM - 11 PM. Click any hour to add a task. Edit/delete tasks. The current hour is highlighted. Pure JS, single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Daily Planner</title>  6    <style>  7      body { font-family: system-
ui; max-width: 600px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#0f172a; }  9      .day { display: grid; grid-template-columns: 60px 1fr; gap:
8px; } 10      .hour { font-size: 13px; color: #475569; text-align: right;
padding-top: 8px; } 11      .slot { background: #f8fafc; min-height: 50px;
padding: 8px; border-radius: 8px; cursor: pointer; transition:
background .15s; } 12      .slot:hover { background: #e0f2fe; }
13      .slot.now { background: #fef3c7; outline: 2px solid #f59e0b; }
14      .task { background: #dbeafe; padding: 6px 10px; border-radius: 6px;
margin-bottom: 4px; display: flex; justify-content: space-between; }
15      .task.done { background: #dcfce7; } 16    </style> 17  </head> 18
<body> 19    <h1>  Daily Planner</h1> 20    <div class="day" id="d"></div> 21
22    <script> 23      const HOURS = 18; // 6AM-11PM 24      const startHour =
6; 25      const KEY = () => "planner-" + new Date().toISOString().slice(0,10);
26      let tasks = JSON.parse(localStorage.getItem(KEY()) || "{}"); 27   28
function save() { localStorage.setItem(KEY(), JSON.stringify(tasks)); } 29
function render() { 30        const now = new Date(); 31        const dayEl =
document.getElementById("d"); 32        dayEl.innerHTML = ""; 33        for (let
h = 0; h < HOURS; h++) { 34          const hr = h + startHour; 35          const
isNow = now.getHours() === hr; 36          const hourEl =
document.createElement("div"); 37          hourEl.className = "hour";
hourEl.textContent = (hr % 12 || 12) + (hr < 12 ? "a" : "p"); 38          const
slot = document.createElement("div"); 39          slot.className = "slot" +
(isNow ? " now" : ""); 40          slot.innerHTML = (tasks[hr] || []).map((t,i)
=> 41            `<div class="task ${t.done?'done':''}"
onclick="event.stopPropagation();toggle(${hr},${i})">${t.text}<span
onclick="event.stopPropagation();del(${hr},${i})">×</span></div>`
42          ).join(""); 43          slot.onclick = () => { 44            const v
```

```
= prompt("Task for " + hourEl.textContent); 45            if (v) { tasks[hr] =
tasks[hr] || []; tasks[hr].push({ text: v, done: false }); save(); render(); }
46          }; 47          dayEl.appendChild(hourEl); dayEl.appendChild(slot);
48        } 49      } 50      window.toggle = (h, i) => { tasks[h][i].done = !
tasks[h][i].done; save(); render(); };
  1      window.del = (h, i) => { tasks[h].splice(i,1); save(); render(); };  2
render();  3    </script>  4  </body>  5  </html>
```

File structure

```
single file: planner.html
```

How to run it

Open in any browser.

How it works

We use the date as part of the localStorage key, so each day's plan is stored separately and the planner automatically shows today's plan when revisited. Each hour is a clickable slot.

Customization ideas

- Add colour categories per task
- Add week view
- Add task move between hours

🕹 Upgrade Challenge

> *Add free-time suggestions and a quick Pomodoro start for any task.*

Project 68: Pomodoro Habit Combo

Difficulty: ☆☆☆

What you will build

Run Pomodoros and log completed focus sessions against named habits.

Skills you will learn

- Composable widgets
- Persistence

Features

- Pomodoro timer
- Habit link
- Today's focus minutes
- Per-habit total

Build it with AI — the prompt to paste

> *Build a 'Pomodoro Habit Combo'. Merge the Pomodoro timer with Habit Tracker. Each Pomodoro completed is logged against a chosen habit. Show today's focus minutes per habit. Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Pomodoro + Habits</title>  6    <style>  7      body { font-family:
system-ui; max-width: 540px; margin: 30px auto; padding: 20px; }  8      h1
{ color: #db2777; }  9      .timer { font-size: 96px; text-align: center; font-
weight: 200; font-variant-numeric: tabular-nums; margin: 14px 0; }
10      .phase { text-align: center; color: #db2777; letter-spacing: 4px; } 11
select, button { padding: 10px 14px; margin: 4px; border-radius: 6px; border: 0;
cursor: pointer; font-size: 15px; } 12      button.start { background: #16a34a;
color: white; } button.stop { background: #f59e0b; color: white; }
13      .focus { background: #fee2e2; } .break-bg { background: #dcfce7; } 14
.log { background: white; padding: 12px; border-radius: 8px; margin-top: 14px; }
15      table { width: 100%; border-collapse: collapse; } 16      th, td
{ padding: 6px; text-align: left; border-bottom: 1px solid #e2e8f0; } 17
</style> 18  </head> 19  <body class="focus"> 20    <h1>🍅 Pomodoro + Habit</h1>
21    <div class="phase" id="ph">FOCUS</div> 22    <div class="timer"
id="t">25:00</div> 23    <select id="h"></select> 24    <select id="d"> 25
<option value="25">25 / 5 (classic)</option> 26      <option value="50">50 / 10
(long)</option> 27      <option value="90">90 / 20 (deep)</option> 28
</select> 29    <p> 30      <button class="start" id="start">Start</button> 31
<button class="stop" id="stop">Pause</button> 32      <button
id="reset">Reset</button> 33    </p> 34    <div class="log"> 35      <h3>Today's
focus</h3> 36      <table id="tbl"></table> 37    </div> 38   39    <script> 40
const HABITS_KEY = "ph-habits"; 41      const LOG_KEY = () => "ph-log-" + new
Date().toISOString().slice(0,10); 42      let habits =
JSON.parse(localStorage.getItem(HABITS_KEY) ||
'["Reading","Coding","Writing"]'); 43      let logs =
JSON.parse(localStorage.getItem(LOG_KEY()) || "{}"); 44      let phase =
"focus", secs = 25, interval = null, mins = 25; 45   46      function
paintHabits() { 47        document.getElementById("h").innerHTML = habits.map(x
```

```
=> `<option>${x}</option>`).join(""); 48      } 49      paintHabits(); 50
function paintLog() {
  1        const tbl = document.getElementById("tbl");  2        tbl.innerHTML =
"<tr><th>Habit</th><th>Pomodoros</th></tr>" +  3
Object.entries(logs).map(([h, n]) =>
`<tr><td>${h}</td><td>${n}</td></tr>`).join("");  4      }  5      paintLog();
6    7      function fmt(s) { return String(Math.floor(s/60)).padStart(2,"0") +
":" + String(s%60).padStart(2,"0"); }  8      function setPhase(name) {  9
phase = name; 10        secs = name === "focus" ?
parseInt(document.getElementById("d").value) * 60 : 5 * 60; 11
document.body.className = name === "focus" ? "focus" : "break-bg"; 12
document.getElementById("ph").textContent = name.toUpperCase(); 13
document.getElementById("t").textContent = fmt(secs); 14      } 15      function
tick() { 16        secs--; 17        document.getElementById("t").textContent =
fmt(secs); 18        if (secs <= 0) { 19          if (phase === "focus") { 20
const habit = document.getElementById("h").value; 21            logs[habit] =
(logs[habit] || 0) + 1; 22            localStorage.setItem(LOG_KEY(),
JSON.stringify(logs)); 23            paintLog(); 24          } 25
setPhase(phase === "focus" ? "break" : "focus"); 26        } 27      } 28
document.getElementById("start").onclick = () => { if (!interval) interval =
setInterval(tick, 1000); }; 29      document.getElementById("stop").onclick = ()
=> { clearInterval(interval); interval = null; }; 30
document.getElementById("reset").onclick = () => { clearInterval(interval);
interval = null; setPhase("focus"); }; 31      setPhase("focus"); 32
</script> 33  </body> 34  </html>
```

File structure

```
single file: pomohabit.html
```

How to run it

Open in any browser.

How it works

We compose two existing ideas: Pomodoro (interval + phase switching) and a habit log (per-habit counter). The composition is what makes it useful beyond either alone.

Customization ideas

- Add weekly summary heatmap
- Add browser notifications
- Add a notes field per session

🕹 Upgrade Challenge

> *Add a goal setting panel: 'I want 4 Reading Pomodoros today'.*

Project 69: Stock Portfolio Tracker (Mock)

Difficulty: ☆☆☆

What you will build

Track a small mock portfolio of stocks with mock price tickers.

Skills you will learn

- Mock data simulation
- State management

Features

- Add positions with quantity and price
- Mock live price tickers
- Day change colours
- Total value

Build it with AI — the prompt to paste

> *Build a 'Stock Portfolio Tracker'. Add positions by symbol, quantity and buy price. Display current value using a mock price ticker that simulates market movement. Show day change in green/red. Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Stock Portfolio</title>  6    <style>  7      body { font-family: system-
ui; max-width: 760px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#16a34a; }  9      form { display: grid; grid-template-columns: repeat(4, 1fr);
gap: 6px; margin-bottom: 12px; } 10      input, button { padding: 8px; border-
radius: 6px; border: 1px solid #cbd5e1; font: inherit; } 11      form button
{ background: #16a34a; color: white; border: 0; cursor: pointer; } 12      table
{ width: 100%; border-collapse: collapse; margin-top: 12px; } 13      th, td
{ padding: 8px; border-bottom: 1px solid #e2e8f0; } 14      th { text-align:
left; background: #f1f5f9; } 15      .up { color: #16a34a; } .down { color:
#dc2626; } 16      .total { font-size: 22px; font-weight: 700; margin: 14px 0; }
17      .del { background: #fee2e2; color: #dc2626; border: 0; padding: 4px 8px;
border-radius: 4px; cursor: pointer; } 18    </style> 19  </head> 20  <body> 21
<h1>📈 Stock Portfolio (Mock)</h1> 22    <p>Prices are simulated — no real
market data.</p> 23    <form id="f"> 24      <input id="s" placeholder="Symbol"
required> 25      <input id="q" type="number" placeholder="Qty" required> 26
<input id="p" type="number" step="0.01" placeholder="Buy price" required> 27
<button>Add</button> 28    </form> 29    <table
id="tbl"><thead><tr><th>Sym</th><th>Qty</th><th>Buy ₹</th><th>Now
₹</th><th>P/L</th><th></th></tr></thead><tbody></tbody></table> 30    <p
class="total">Total value: ₹<span id="tot">0</span></p> 31   32    <script> 33
const KEY = "stocks-v1"; 34      let positions =
JSON.parse(localStorage.getItem(KEY) || "[]"); 35      // Mock starting prices
for common symbols 36      const seedPrices = { AAPL: 180, MSFT: 410, GOOG: 165,
TSLA: 240, AMZN: 185, NFLX: 620, META: 510, INFY: 1500, TCS: 3900 }; 37
function save() { localStorage.setItem(KEY, JSON.stringify(positions)); } 38
function price(sym) { 39        // Random walk starting from the seed each
session 40        positions.forEach(p => { 41          if (!(sym in p.price))
p.price = seedPrices[sym] || 100; 42          p.price = +(p.price * (0.99 +
Math.random() * 0.02)).toFixed(2); 43        }); 44      } 45   46      function
```

```
render() { 47        positions.forEach(p => { 48          if (!(p.sym in
seedPrices) && p.price == null) p.price = 100; 49          if (p.price == null)
p.price = seedPrices[p.sym] || 100; 50        });
  1        const tb = document.querySelector("tbody");  2        tb.innerHTML =
positions.map((p, i) => {  3          const cur = p.price;  4          const pl
= (cur - p.buy) * p.qty;  5          const cls = pl > 0 ? "up" : pl < 0 ? "down"
: "";  6          return `<tr><td>${p.sym}</td><td>${p.qty}</td><td>$
{p.buy.toFixed(2)}</td>  7                  <td class="${cls}">$
{cur.toFixed(2)}</td>  8                  <td class="${cls}">${pl >= 0 ? "+" :
""}${pl.toFixed(2)}</td>  9                  <td><button class="del"
onclick="del(${i})">×</button></td></tr>`; 10        }).join(""); 11
const tot = positions.reduce((s,p) => s + p.price * p.qty, 0); 12
document.getElementById("tot").textContent = tot.toFixed(2); 13      } 14
window.del = i => { positions.splice(i,1); save(); render(); }; 15
document.getElementById("f").addEventListener("submit", e => { 16
e.preventDefault(); 17        positions.push({ 18          sym:
document.getElementById("s").value.toUpperCase(), 19          qty:
parseInt(document.getElementById("q").value), 20          buy:
parseFloat(document.getElementById("p").value), 21          price: +
(seedPrices[document.getElementById("s").value.toUpperCase()] || 100)
22        }); 23        ["s","q","p"].forEach(id => document.getElementById("id"
=== id ? id : id).value = ""); 24        ["s","q","p"].forEach(id =>
document.getElementById(id).value = ""); 25        save(); render(); 26      });
27   28      setInterval(() => { 29        positions.forEach(p => { 30
if (p.price == null) p.price = seedPrices[p.sym] || 100; 31          p.price = +
(p.price * (0.998 + Math.random() * 0.004)).toFixed(2); 32        }); 33
save(); render(); 34      }, 3000); 35      render(); 36    </script> 37
</body> 38  </html>
```

File structure

```
single file: portfolio.html
```

How to run it

Open in any browser.

How it works

Each tick the prices make small random walks. We compute P/L per position and the total. localStorage preserves the mock portfolio between reloads – no real money should ever be touched.

Customization ideas

- Add a chart of price history per symbol
- Add alerts
- Add an export

🕹 Upgrade Challenge

> *Add a 'sim mode' where you set a base price and price shocks happen on big news events.*

Project 70: Personal CRM

Difficulty: ☆☆

What you will build

Track people you want to stay in touch with: name, last contact, follow-up date.

Skills you will learn

- Contact models
- Date arithmetic
- Search

Features

- Add contacts
- Last contacted date
- Follow-up reminders
- Search and tags

Build it with AI — the prompt to paste

> *Build a 'Personal CRM'. Add contacts with name, email, tags and last-contacted date. The page highlights anyone whose follow-up is due today or overdue. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Personal CRM</title>  6    <style>  7      body { font-family: system-ui;
max-width: 760px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#4f46e5; }  9      form { display: grid; grid-template-columns: 2fr 1fr 1fr 1fr
auto; gap: 6px; margin-bottom: 14px; } 10      input, button { padding: 8px;
border-radius: 6px; border: 1px solid #cbd5e1; font: inherit; } 11      form
button { background: #4f46e5; color: white; border: 0; cursor: pointer; } 12
.card { background: white; padding: 12px; border-radius: 8px; margin-bottom:
8px; box-shadow: 0 1px 3px rgba(0,0,0,.08); display: flex; gap: 14px; align-
items: center; } 13      .card.due { background: #fef2f2; border-left: 4px solid
#dc2626; } 14      .card.today { background: #fef9c3; border-left: 4px solid
#f59e0b; } 15      .info { flex: 1; } 16      .info b { font-size: 16px; } 17
.tag { background: #e0e7ff; color: #3730a3; padding: 2px 6px; border-radius:
4px; font-size: 12px; margin-right: 4px; } 18      .small { font-size: 13px;
color: #475569; } 19      button.danger { background: #fee2e2; color: #b91c1c;
border: 0; padding: 4px 8px; border-radius: 4px; cursor: pointer; } 20
</style> 21  </head> 22  <body> 23    <h1>□ Personal CRM</h1> 24    <form
id="f"> 25      <input id="name" placeholder="Name" required> 26      <input
id="email" placeholder="Email"> 27      <input id="tag" placeholder="Tag"> 28
<input id="days" type="number" placeholder="Days until follow-up"> 29
<button>Add</button> 30    </form> 31    <input id="q" placeholder="Search..."
style="width:100%;padding:8px;border-radius:6px;border:1px solid #cbd5e1;margin-
bottom:10px"> 32    <div id="list"></div> 33   34    <script> 35      const KEY
= "crm-v1"; 36      let contacts = JSON.parse(localStorage.getItem(KEY) ||
"[]"); 37      function save() { localStorage.setItem(KEY,
JSON.stringify(contacts)); } 38      function status(c) { 39        const due =
new Date(c.next); const today = new Date(); 40        due.setHours(0,0,0,0);
today.setHours(0,0,0,0); 41        if (due < today) return "due"; 42        if
(due.getTime() === today.getTime()) return "today"; 43        return "ok"; 44
```

```
} 45      window.contact = i => { 46        contacts[i].last = new
Date().toISOString().slice(0,10); 47        const days =
parseInt(contacts[i]._days) || 30; 48        const d = new Date();
d.setDate(d.getDate() + days); 49        contacts[i].next =
d.toISOString().slice(0,10); 50        save(); render();
  1      };  2      function render() {  3        const q =
(document.getElementById("q").value || "").toLowerCase();  4
document.getElementById("list").innerHTML = contacts  5          .filter(c => !q
|| c.name.toLowerCase().includes(q) || (c.email||"").includes(q) ||
(c.tag||"").includes(q))  6          .map((c, i) => {  7            const st =
status(c);  8            return `<div class="card ${st}">  9              <div
class="info"> 10                <b>${c.name}</b> <span class="small">${c.email
|| ""}</span><br> 11                <span class="tag">${c.tag ||
"general"}</span> 12                <span class="small">Last: ${c.last || "—"} ·
Next: ${c.next || "—"}</span> 13                ${st==="due"?"<br><b>⚠
Overdue</b>":""} 14                ${st==="today"?"<br><b>□ Today</b>":""} 15
</div> 16              <button onclick="contact(${i})">Mark contacted ($
{c._days||30}d)</button> 17              <button class="danger" onclick="del($
{i})">×</button> 18            </div>`; 19          }).join(""); 20      } 21
window.del = i => { contacts.splice(i,1); save(); render(); }; 22
document.getElementById("f").addEventListener("submit", e => { 23
e.preventDefault(); 24        const days =
parseInt(document.getElementById("days").value) || 30; 25        const d = new
Date(); d.setDate(d.getDate() + days); 26        contacts.push({ 27
name: document.getElementById("name").value, 28          email:
document.getElementById("email").value, 29          tag:
document.getElementById("tag").value, 30          last: null, next:
d.toISOString().slice(0,10), _days: days 31        }); 32
["name","email","tag","days"].forEach(id => document.getElementById(id).value =
""); 33        save(); render(); 34      }); 35
document.getElementById("q").addEventListener("input", render); 36
render(); 37    </script> 38  </body> 39  </html>
```

File structure

```
single file: crm.html
```

How to run it

Open in any browser.

How it works

Each contact tracks both last contacted and next due dates. The status() helper normalises both dates to midnight and checks if 'next' is today, past, or future — those determine the card colour.

Customization ideas

- Add a calendar export for due contacts
- Add a contact detail page
- Add categories

🕹 Upgrade Challenge

> *Add a small 'today's outreach' digest sent via email using a mailto: link.*

Project 71: Unit Converter

Difficulty: ☆

What you will build

Convert between metric and imperial units and between common cooking units.

Skills you will learn

- Multi-unit math
- Conditional UI

Features

- Length, weight, temperature
- Live conversion as you type
- Both directions

Build it with AI — the prompt to paste

> *Build a 'Unit Converter'. Categories: length (m, cm, ft, in), weight (kg, g, lb, oz), temperature (C, F, K), volume (L, ml, cup, tbsp, tsp). Tabbed by category. Live as-you-type conversion. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Unit Converter</title>  6    <style>  7      body { font-family: system-
ui; max-width: 480px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#0891b2; }  9      .tabs { display: flex; gap: 6px; margin-bottom: 14px; flex-
wrap: wrap; } 10      .tabs button { padding: 6px 10px; border: 0; border-
radius: 6px; background: #e2e8f0; cursor: pointer; } 11      .tabs button.on
{ background: #0891b2; color: white; } 12      .row { display: grid; grid-
template-columns: 1fr 1fr auto; gap: 6px; margin: 8px 0; align-items: center; }
13      input, select { padding: 8px; border-radius: 6px; border: 1px solid
#cbd5e1; font-size: 15px; } 14    </style> 15  </head> 16  <body> 17    <h1>
Unit Converter</h1> 18    <div class="tabs" id="tabs"></div> 19    <div
id="ui"></div> 20   21    <script> 22      const TYPES = { 23        length:
{ 24          units: [["m",1],["cm",0.01],["mm",0.001],["ft",0.3048],
["in",0.0254],["yd",0.9144]], 25          names:
{m:"metres",cm:"cm",mm:"mm",ft:"feet",in:"inches",yd:"yards"} 26        }, 27
weight: { 28          units: [["kg",1],["g",0.001],["mg",0.000001],
["lb",0.45359237],["oz",0.0283495231]], 29          names:
{kg:"kg",g:"grams",mg:"mg",lb:"pounds",oz:"ounces"} 30        }, 31
temperature: { 32          special: true, 33          units: ["C","F","K"] 34
}, 35        volume: { 36          units: [["L",1],["mL",0.001],["cup",0.24],
["tbsp",0.015],["tsp",0.005]], 37          names:
{L:"litres",mL:"mL",cup:"cups",tbsp:"tbsp",tsp:"tsp"} 38        } 39      }; 40
41      let cur = "length"; 42      function render() { 43        const tabs =
document.getElementById("tabs"); 44        tabs.innerHTML =
Object.keys(TYPES).map(t => 45          `<button class="${t===cur?'on':''}"
onclick="set('${t}')">${t}</button>`).join(""); 46        const u = TYPES[cur];
47        if (u.special) { 48          document.getElementById("ui").innerHTML =
` 49            <div class="row"><input type="number" id="v"
placeholder="Value"><select id="f"></select></div> 50            <div
class="row"><input type="number" id="r" placeholder="Result"><select
id="t"></select></div>`;
```

```
  1          ["f","t"].forEach(id => document.getElementById(id).innerHTML =
u.units.map(x => `<option>${x}</option>`).join(""));  2          const conv =
(v, a, b) => {  3            if (a === b) return v;  4            if (a === "C")
return b === "F" ? v * 9/5 + 32 : v + 273.15;  5            if (a === "F")
return b === "C" ? (v - 32) * 5/9 : (v - 32) * 5/9 + 273.15;  6            if (a
=== "K") return b === "C" ? v - 273.15 : (v - 273.15) * 9/5 + 32;  7          };
8          const up = () => {  9            const v =
parseFloat(document.getElementById("v").value); 10
document.getElementById("r").value = isNaN(v) ? "" : conv(v,
document.getElementById("f").value,
document.getElementById("t").value).toFixed(2); 11          }; 12
["v","f","t"].forEach(id =>
document.getElementById(id).addEventListener("input", up)); 13          up(); 14
} else { 15          document.getElementById("ui").innerHTML = ` 16
<div class="row"><input type="number" id="v" placeholder="Value"><select
id="f"></select></div> 17            <div class="row"><input type="number"
id="r" placeholder="Result"><select id="t"></select></div>`; 18
["f","t"].forEach(id => document.getElementById(id).innerHTML =
u.units.map(([k]) => `<option>${k}</option>`).join("")); 19          const
lookup = Object.fromEntries(u.units); 20          const up = () => { 21
const v = parseFloat(document.getElementById("v").value); 22            const f
= lookup[document.getElementById("f").value]; 23            const t =
lookup[document.getElementById("t").value]; 24
document.getElementById("r").value = isNaN(v) ? "" : (v * f / t).toFixed(4); 25
}; 26          ["v","f","t"].forEach(id =>
document.getElementById(id).addEventListener("input", up)); 27          up(); 28
} 29      } 30      window.set = t => { cur = t; render(); }; 31      render();
32    </script> 33  </body> 34  </html>
```

File structure

```
single file: convert.html
```

How to run it

Open in any browser.

How it works

All non-temperature conversions are linear: any unit is just the count of base units it represents, so valueA * unitsA / unitsB = valueB. Temperature is special and needs three custom formulas.

Customization ideas

- Add currency conversion (using mock rates)
- Add a 'favourite conversions' shortcut
- Add precision toggle

🕹 Upgrade Challenge

> *Convert the unit converter into a fully accessible ARIA-live region.*

Project 72: Random Quote Generator with Originals

Difficulty: ☆

What you will build

Cycle through a curated set of original, non-copyrighted quotes. Save favourites.

Skills you will learn

- Curation
- localStorage favourites

Features

- 30+ curated original quotes
- Favourites list with copy button
- Auto-rotate option

Build it with AI — the prompt to paste

> *Build a 'Random Quote Generator with Originals'. A curated set of at least 30 original short motivational/reflective quotes (not from famous real people — clearly written by the book). Auto-rotate option, copy button, favourites list. Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Original Quotes</title>  6    <style>  7      body { font-family:
'Georgia', serif; max-width: 580px; margin: 60px auto; padding: 20px; text-
align: center; background: #fdf4ff; }  8      .quote { font-size: 28px; line-
height: 1.5; color: #581c87; min-height: 110px; display: flex; align-items:
center; justify-content: center; }  9      button { padding: 10px 18px; margin:
6px; border: 0; border-radius: 6px; background: #9333ea; color: white; cursor:
pointer; font-size: 15px; } 10      .ghost { background: transparent; color:
#581c87; border: 1px solid #9333ea; } 11      ul.favs { list-style: none;
padding: 0; margin-top: 20px; text-align: left; } 12      ul.favs li
{ background: white; padding: 10px; margin: 6px 0; border-radius: 6px; box-
shadow: 0 1px 3px rgba(0,0,0,.05); } 13    </style> 14  </head> 15  <body> 16
<h1>✒ Original Quotes</h1> 17    <div class="quote" id="q">Click below for a
quote.</div> 18    <p> 19      <button onclick="next()">New quote</button> 20
<button class="ghost" onclick="copy()">Copy</button> 21      <button
class="ghost" onclick="fav()">★ Save</button> 22      <button class="ghost"
onclick="toggle()">Auto-rotate</button> 23    </p> 24    <ul class="favs"
id="favs"></ul> 25   26    <script> 27      const QUOTES = [ 28        "Showing
up is half the work.", 29        "Tiny steps taken daily change who you are.",
30        "Curiosity outlives any syllabus.", 31        "You don't need anyone's
permission to start.", 32        "Small wins stack into big results.", 33
"Mistakes are tuition for competence.", 34        "Slow is smooth, smooth is
fast.", 35        "Done is more useful than perfect.", 36        "Reading is a
free masterclass.", 37        "Your future self is built one decision at a
time.", 38        "Worry is interest paid on debt you don't owe.", 39
"Half-finished is the worst kind of finished.", 40        "Ask better questions,
get better answers.", 41        "Skill compounds. Effort compounds. Patience
compounds.", 42        "The quiet hours build the loud results.", 43        "You
are allowed to begin again.", 44        "Plans fail when they aren't written
down.", 45        "Confidence is built by keeping tiny promises to yourself.",
```

```
46        "Hard things first, easier things later.", 47        "Everything you
make started as a sketch.", 48        "You learn the most right after the
test.", 49        "The boring path beats the brilliant shortcut.", 50
"Better to ship a small thing than plan a big thing.",
  1        "Today is a fine day to start something small.",  2        "You don't
lose when you try. You lose when you quit trying.",  3        "Reading slowly is
still reading.",  4        "Consistency beats intensity over the long run.",  5
"The bug you find today saves the weekend.",  6        "Code once, run
forever.",  7        "The first attempt at anything is rarely the last.",  8
"Make a thing. Then make another thing.",  9        "Reading other people's code
is how you level up.", 10        "Comments explain why, not what.", 11
"Sleeping on a hard problem is allowed.", 12        "Most disagreements are
about definitions, not conclusions.", 13        "Small discipline, big
freedom.", 14        "Hard to start, easy to keep going." 15      ]; 16      let
auto = null; 17      const qEl = document.getElementById("q"); 18      function
next() { 19        qEl.textContent = QUOTES[Math.floor(Math.random() *
QUOTES.length)]; 20      } 21      window.next = next; 22      window.copy = ()
=> { navigator.clipboard && navigator.clipboard.writeText(qEl.textContent); };
23      window.fav = () => { 24        const favs =
JSON.parse(localStorage.getItem("favs") || "[]"); 25
favs.push(qEl.textContent); localStorage.setItem("favs", JSON.stringify(favs));
26        renderFavs(); 27      }; 28      window.toggle = () => { 29        if
(auto) { clearInterval(auto); auto = null; } 30        else auto =
setInterval(next, 4000); 31      }; 32      function renderFavs() { 33
const favs = JSON.parse(localStorage.getItem("favs") || "[]"); 34
document.getElementById("favs").innerHTML = favs.map((q, i) => 35
`<li>${q} <a href="#" onclick="localStorage.setItem('favs',
JSON.stringify(JSON.parse(localStorage.getItem('favs')).filter((_,j)=>j!==$
{i})));  renderFavs(); return false;"
style="float:right;color:#dc2626">remove</a></li>`).join(""); 36      } 37
next(); renderFavs(); 38    </script> 39  </body> 40  </html>
```

File structure

```
single file: quotes.html
```

How to run it

Open in any browser.

How it works

Quotes live in a single array — easy to add more. Favourites are stored separately in localStorage so the user can accumulate their personal list across sessions.

Customization ideas

- Add a 'share via link' option
- Allow user to add their own quotes
- Show daily-random vs pure-random

🕹 Upgrade Challenge

> *Add a 'mood filter' that picks quotes whose themes match a chosen mood.*

Project 73: Password Strength Tester

Difficulty: ☆☆

What you will build

Type a password and instantly see whether it's weak or strong and why.

Skills you will learn

- String analysis
- Conditional feedback

Features

- Strength meter
- Specific reasons it's strong or weak
- Don't store the password anywhere

Build it with AI — the prompt to paste

> *Build a 'Password Strength Tester'. User types a password (NEVER stored anywhere!). Show a coloured bar, score (very weak → very strong), and specific tips: length, mixed case, digits, symbols, common patterns (e.g. '123', 'qwerty'). Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Strength</title>  6    <style>  7      body { font-family: system-ui;
max-width: 460px; margin: 40px auto; padding: 20px; }  8      h1 { color:
#4338ca; }  9      input { width: 100%; padding: 12px; font-size: 18px; border-
radius: 6px; border: 1px solid #cbd5e1; box-sizing: border-box; } 10      .meter
{ height: 10px; background: #e2e8f0; border-radius: 5px; margin: 10px 0; } 11
.meter > div { height: 100%; border-radius: 5px; transition: width .3s,
background .3s; } 12      .label { font-weight: 700; } 13      ul { padding-
left: 18px; } 14      .compliment { color: #16a34a; } .criticism { color:
#b91c1c; } 15    </style> 16  </head> 17  <body> 18    <h1>🔒 Password Strength
Tester</h1> 19    <p style="color:#475569">Nothing you type here is stored or
sent anywhere.</p> 20    <input id="p" type="text" placeholder="Type any
password..." autofocus> 21    <div class="meter"><div id="bar" style="width:0%;
background:#dc2626"></div></div> 22    <div><span id="label">--</span> <span
style="color:#64748b" id="entropy"></span></div> 23    <ul id="tips"></ul> 24
25    <script> 26      const COMMON =
["123","password","qwerty","letmein","welcome","abc","iloveyou"]; 27
function entropyChars(p) { 28        let n = 0; 29        if (/[a-z]/.test(p)) n
+= 26; 30        if (/[A-Z]/.test(p)) n += 26; 31        if (/[0-9]/.test(p)) n
+= 10; 32        if (/[^a-zA-Z0-9]/.test(p)) n += 33; 33        return
Math.max(1, n) ** p.length; 34      } 35      function analyze(p) { 36
const tips = []; 37        let score = 0; 38        if (p.length >= 8) score++;
39        else tips.push({ ok: false, t: "At least 8 characters (12+ is better)"
}); 40        if (p.length >= 12) score++; 41        if (/[a-z]/.test(p) && /[A-
Z]/.test(p)) score++; 42        else tips.push({ ok: false, t: "Mix uppercase
and lowercase letters" }); 43        if (/[0-9]/.test(p)) score++; 44
else tips.push({ ok: false, t: "Include at least one digit" }); 45        if
(/[^A-Za-z0-9]/.test(p)) score++; 46        else tips.push({ ok: false, t:
"Include a symbol like ! or #" }); 47        const lc = p.toLowerCase(); 48
COMMON.forEach(c => { if (lc.includes(c)) tips.push({ ok: false, t: "Avoid
```

```
common sequences like '" + c + "'" }); }); 49        return { score, tips }; 50
}
  1    2      document.getElementById("p").addEventListener("input", e => {  3
const p = e.target.value;  4        const a = analyze(p);  5        const colors
= ["#dc2626","#f59e0b","#facc15","#16a34a","#0ea5e9","#0ea5e9"];  6        const
labels = ["Very weak","Weak","Decent","Good","Strong","Very strong"];  7
const bar = document.getElementById("bar");  8        bar.style.width =
Math.min(100, a.score * 20) + "%";  9        bar.style.background =
colors[a.score]; 10        document.getElementById("label").textContent =
labels[a.score]; 11        document.getElementById("entropy").textContent = p ?
`~${Math.log2(entropyChars(p)).toFixed(0)} bits` : ""; 12
document.getElementById("tips").innerHTML = a.tips.length ? 13
a.tips.map(t => `<li
class="${t.ok?'compliment':'criticism'}">${t.t}</li>`).join("") : 14
"<li class='compliment'>Great password!</li>"; 15      }); 16    </script> 17
</body> 18  </html>
```

File structure

```
single file: pwstrength.html
```

How to run it

Open in any browser.

How it works

We compute a small 'entropy' figure by treating the pool of characters the password comes from (lowercase letters, uppercase, digits, symbols) and computing log2(pool^length). It's an estimate, not a true measure, but it's a fine guide.

Customization ideas

- Add a 'suggest a strong password' button
- Add estimates of crack time
- Allow toggle hide/show

🕹 Upgrade Challenge

> *Add localStorage of *only* the strength score, not the password, to track user's progress over time.*

Project 74: Workout Timer

Difficulty: ☆☆

What you will build

Build a HIIT-style workout timer with rounds, work duration, rest duration and cycle count.

Skills you will learn

- State machine
- Audio cues
- localStorage

Features

- Custom rounds, work seconds, rest seconds
- Big display
- Sound at transitions
- Pause and resume

Build it with AI — the prompt to paste

> *Build a 'Workout Timer'. User sets: rounds, work seconds, rest seconds. Big display shows the current phase (e.g. 'Round 2 of 5 — Work — 00:25'). Plays a beep at each transition. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Workout Timer</title>  6    <style>  7      body { font-family: system-
ui; min-height: 100vh; display:flex; flex-direction: column; align-items:
center; justify-content: center; background: #f1f5f9; margin: 0; }
8      .phase { letter-spacing: 4px; font-size: 18px; color: #16a34a; }
9      .timer { font-size: 120px; font-weight: 200; font-variant-numeric:
tabular-nums; } 10      .round { color: #475569; } 11      button { padding:
12px 24px; margin: 8px; font-size: 16px; border: 0; border-radius: 6px; cursor:
pointer; } 12      button.go { background: #16a34a; color: white; } button.stop
{ background: #f59e0b; color: white; } 13      .work { background:
#dcfce7; } .rest { background: #fee2e2; } .done { background: #dbeafe; } 14
.cfg { background: white; padding: 14px; border-radius: 10px; margin-bottom:
14px; display: flex; gap: 10px; flex-wrap: wrap; } 15    </style> 16  </head> 17
<body id="bd"> 18    <h1>□ Workout Timer</h1> 19    <div class="cfg"> 20
<label>Rounds: <input id="rounds" type="number" value="5"
style="width:60px"></label> 21      <label>Work s: <input id="work"
type="number" value="30" style="width:60px"></label> 22      <label>Rest s:
<input id="rest" type="number" value="15" style="width:60px"></label> 23
</div> 24    <div class="phase" id="ph">READY</div> 25    <div class="timer"
id="t">00:00</div> 26    <div class="round" id="rd">Round 0 / 0</div> 27    <p>
28      <button class="go" id="go">Start</button> 29      <button class="stop"
id="stop">Pause</button> 30      <button id="reset">Reset</button> 31    </p> 32
33    <script> 34      let phase = "ready", secs = 0, round = 0, total = 0,
interval = null; 35      const fmt = s => "00:" +
String(Math.max(0,s)).padStart(2,"0"); 36      function beep(high) { 37
try { const ctx = new (window.AudioContext || window.webkitAudioContext)(); 38
const o = ctx.createOscillator(); const g = ctx.createGain(); 39
```

```
o.connect(g); g.connect(ctx.destination); 40          o.frequency.value = high ?
880 : 440; g.gain.setValueAtTime(.1, ctx.currentTime); 41          o.start();
o.stop(ctx.currentTime + .3); } catch(e) {} 42      } 43      function setup() {
44        round = 0; total = parseInt(document.getElementById("rounds").value)
|| 5; 45        phase = "work"; secs =
parseInt(document.getElementById("work").value) || 30; 46        paint(); 47
} 48      function paint() { 49        document.getElementById("t").textContent
= fmt(secs); 50        document.getElementById("ph").textContent = phase ===
"work" ? "WORK" : phase === "rest" ? "REST" : phase === "done" ? "DONE" :
"READY";
  1        document.getElementById("rd").textContent = "Round " +
Math.min(round, total) + " / " + total;  2        document.body.className =
phase;  3      }  4      function tick() {  5        secs--;  6        if (secs
<= 0) {  7          if (phase === "work") {  8            if (round + 1 >=
total) { phase = "done"; beep(true); }  9            else { phase = "rest"; secs
= parseInt(document.getElementById("rest").value) || 15; beep(false); } 10
} else if (phase === "rest") { 11            round++; phase = "work"; secs =
parseInt(document.getElementById("work").value) || 30; beep(true); 12          }
13        } 14        paint(); 15      } 16
document.getElementById("go").onclick = () => { setup(); if (!interval) interval
= setInterval(tick, 1000); }; 17      document.getElementById("stop").onclick =
() => { clearInterval(interval); interval = null; }; 18
document.getElementById("reset").onclick = () => { clearInterval(interval);
interval = null; phase = "ready"; secs = 0; round = 0; paint(); }; 19
paint(); 20    </script> 21  </body> 22  </html>
```

File structure

```
single file: workout.html
```

How to run it

Open in any browser.

How it works

Same state-machine pattern as the Pomodoro timer, but with a configurable round count and rest transition. The body class swaps to colour-code each phase — a tiny detail that improves usability.

Customization ideas

- Save last workout to localStorage
- Add name to each round
- Add a HIIT bell sound

🕹 Upgrade Challenge

> *Add a smart rest: longer rest every 3rd round.*

Project 75: Book Reading Tracker

Difficulty: ☆☆

What you will build

Track books you're reading, finished, and want to read with progress percentages.

Skills you will learn

- List management
- localStorage

Features

- Three lists (reading, finished, wishlist)
- Per-book progress
- Total pages this year
- Covers (optional)

Build it with AI — the prompt to paste

> *Build a 'Book Reading Tracker'. Three lists: Reading, Finished, Wishlist. Per book: title, author, pages, current page, optional image URL for cover. Total pages finished this year is shown. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Reading Tracker</title>  6    <style>  7      body { font-family: system-
ui; max-width: 760px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#059669; }  9      form { display: grid; grid-template-columns: 2fr 1fr 1fr 1fr
auto; gap: 6px; margin-bottom: 14px; } 10      input, button { padding: 8px;
border-radius: 6px; border: 1px solid #cbd5e1; font: inherit; } 11      form
button { background: #059669; color: white; border: 0; cursor: pointer; } 12
h2 { color: #047857; margin-top: 24px; } 13      .book { background: white;
padding: 12px; border-radius: 8px; margin-bottom: 8px; display: flex; gap: 12px;
align-items: center; } 14      .book img { width: 50px; height: 70px; object-
fit: cover; border-radius: 4px; background: #e2e8f0; } 15      .info { flex:
1; } 16      .info b { font-size: 15px; } 17      input[type=range] { width:
140px; } 18      .del { background: #fee2e2; color: #dc2626; border: 0; padding:
4px 8px; border-radius: 4px; cursor: pointer; } 19      .stat { background:
white; padding: 10px; border-radius: 8px; margin-top: 14px; } 20      select
{ padding: 6px; border-radius: 4px; border: 1px solid #cbd5e1; } 21    </style>
22  </head> 23  <body> 24    <h1>📚 Reading Tracker</h1> 25    <form id="f"> 26
<input id="t" placeholder="Title" required> 27      <input id="a"
placeholder="Author"> 28      <input id="p" type="number" placeholder="Pages">
29      <select
id="s"><option>reading</option><option>wishlist</option><option>finished</
option></select> 30      <button>Add</button> 31    </form> 32    <div
id="lists"></div> 33    <p class="stat" id="stat">Total pages this year: 0</p>
34   35    <script> 36      const KEY = "books-v1"; 37      let books =
JSON.parse(localStorage.getItem(KEY) || "[]"); 38      function save()
{ localStorage.setItem(KEY, JSON.stringify(books)); } 39      function render()
{ 40        const groups = { reading: [], wishlist: [], finished: [] }; 41
books.forEach((b, i) => groups[b.status].push({ ...b, i })); 42
document.getElementById("lists").innerHTML =
```

```
["reading","wishlist","finished"].map(s => ` 43          <h2>$
{s[0].toUpperCase()+s.slice(1)} (${groups[s].length})</h2> 44          $
{groups[s].map(b => `<div class="book"> 45            <div class="info"><b>$
{b.title}</b> by ${b.author || "?"} · ${b.pages || 0} pages 46              ${s
=== 'reading' ? `<br>Page ${b.current || 0} / ${b.pages || 0}: <input
type="range" min="0" max="${b.pages||0}" value="${b.current||0}"
oninput="update(${b.i}, this.value)"></div>` : '<div></div>'} 47
<button class="del" onclick="del(${b.i})">×</button> 48
</div>`).join("") || "<p>Empty.</p>"} 49        `).join(""); 50        // Year
pages finished
  1        const year = new Date().getFullYear();  2        const total =
books.filter(b => b.status === "finished" && (b.finishedYear || year) === year)
3                            .reduce((s,b) => s + (b.pages || 0), 0);  4
document.getElementById("stat").textContent = "Total pages in " + year + ": " +
total;  5      }  6      window.del = i => { books.splice(i,1); save();
render(); };  7      window.update = (i, val) => {  8        books[i].current =
parseInt(val);  9        if (books[i].current >= (books[i].pages || 0))
{ books[i].status = "finished"; books[i].finishedYear = new
Date().getFullYear(); } 10        save(); render(); 11      }; 12
document.getElementById("f").addEventListener("submit", e => { 13
e.preventDefault(); 14        books.push({ title:
document.getElementById("t").value, author: document.getElementById("a").value,
15          pages: parseInt(document.getElementById("p").value) || 0, status:
document.getElementById("s").value, current: 0 }); 16
["t","a","p"].forEach(id => document.getElementById(id).value = ""); 17
save(); render(); 18      }); 19      render(); 20    </script> 21  </body> 22
</html>
```

File structure

```
single file: books.html
```

How to run it

Open in any browser.

How it works

Books are grouped by status and rendered in three sections. The range slider updates the current page; when current >= pages, the book auto-moves to the finished list — a small touch that feels magical.

Customization ideas

- Add a goal (X pages per year)
- Add notes per book
- Add Open Library search

🕹 Upgrade Challenge

> *Add a per-month reading breakdown rendered as a bar chart.*

Project 76: Daily Affirmation Spinner

Difficulty: ☆

What you will build

A wheel of fortune with affirmations. Click spin and receive one.

Skills you will learn

- Animations
- Random selection

Features

- One-click spin
- Animated rotation
- 8 original affirmations
- Reset

Build it with AI — the prompt to paste

> *Build a 'Daily Affirmation Spinner'. Wheel divided into 8 slices, each with an affirmation. Click to spin; wheel rotates a few turns then stops. Pure JS, single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Affirmation Spinner</title>  6    <style>  7      body { font-family:
system-ui; min-height: 100vh; display:flex; flex-direction: column; align-items:
center; justify-content: center; background: linear-
gradient(135deg,#fbcfe8,#bae6fd); }  8      .wheel { width: 320px; height:
320px; border-radius: 50%; border: 8px solid #1e293b; position: relative;
transform: rotate(0deg); }  9      .arrow { position: absolute; right: -16px;
top: 50%; transform: translateY(-50%); width: 0; height: 0; border-left: 24px
solid #1e293b; border-top: 14px solid transparent; border-bottom: 14px solid
transparent; } 10      .slice { position: absolute; width: 50%; height: 50%;
left: 50%; top: 50%; transform-origin: 0 0; font-size: 13px; font-weight: 600;
padding: 4px; box-sizing: border-box; } 11      button { margin-top: 24px;
padding: 14px 28px; background: #1e293b; color: white; border: 0; border-radius:
8px; cursor: pointer; font-size: 16px; } 12    </style> 13  </head> 14  <body>
15    <h1>□ Affirmation Spinner</h1> 16    <div class="wheel" id="w"><div
class="arrow"></div></div> 17    <button onclick="spin()">Spin</button> 18    <p
id="out"></p> 19   20    <script> 21      const AFFS = [ 22        "You are
doing fine.", 23        "Progress, not perfection.", 24        "Small steps
count.", 25        "You learn by trying.", 26        "Curiosity is enough.", 27
"Today is a good day.", 28        "You are enough.", 29        "Showing up
matters." 30      ]; 31      const COL =
["#fde68a","#fca5a5","#bef264","#67e8f9","#fbcfe8","#fcd34d","#c4b5fd","#fdba74"
]; 32      const w = document.getElementById("w"); 33      const slice = 360 /
AFFS.length; 34      AFFS.forEach((a, i) => { 35        const s =
document.createElement("div"); 36        s.className = "slice"; 37
s.style.background = COL[i % COL.length]; 38        s.style.transform =
`rotate(${i * slice}deg) skewY(${90 - slice}deg)`; 39        s.textContent = "";
40        const span = document.createElement("span"); 41
span.textContent = a; 42        span.style.transform = `skewY(${-(90 -
slice)}deg) rotate(${slice/2}deg)`; 43        span.style.display = "inline-
```

```
block"; 44        span.style.padding = "4px 6px"; 45        span.style.width =
"120px"; 46        span.style.transformOrigin = "0 0"; 47
s.appendChild(span); 48        w.appendChild(s); 49      }); 50      let rot =
0;
  1      function spin() {  2        rot += 1440 + Math.random() * 360;  3
w.style.transition = "transform 4s cubic-bezier(.2,.7,.4,1)";  4
w.style.transform = `rotate(${rot}deg)`;  5        setTimeout(() => {  6
const norm = ((rot % 360) + 360) % 360;  7          const idx = Math.floor((360
- norm + slice/2) / slice) % AFFS.length;  8
document.getElementById("out").textContent = "☞ " + AFFS[idx];  9        },
4000); 10      } 11      window.spin = spin; 12    </script> 13  </body> 14
</html>
```

File structure

```
single file: spinner.html
```

How to run it

Open in any browser.

How it works

Each slice is a div rotated to its position and skewed to fit. The CSS ‘transform’ transition + cubic-bezier gives a satisfying deceleration. We compute which slice landed under the arrow using modulo.

Customization ideas

- Custom affirmations
- Volume control
- Save favourite spins

🕹 Upgrade Challenge

> *Add named labels (Work, Family…) that change the slice library.*

Project 77: Movie Watch List

Difficulty: ☆☆

What you will build

Mark movies as 'want to watch', 'watching', or 'watched' with ratings and notes.

Skills you will learn

- List management with statuses
- Star ratings
- Search and filter

Features

- Three statuses
- 1-5 star ratings per movie
- Search and filter by rating
- Notes per movie

Build it with AI — the prompt to paste

> *Build a 'Movie Watch List'. Add movies with title, status (want / watching / watched), rating (1-5), notes. Filter by rating. Pure JS, localStorage.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Movie List</title>  6    <style>  7      body { font-family: system-ui;
max-width: 760px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#be185d; }  9      form { display: grid; grid-template-columns: 2fr 1fr 1fr 1fr
auto; gap: 6px; margin-bottom: 14px; } 10      input, select, button { padding:
8px; border-radius: 6px; border: 1px solid #cbd5e1; font: inherit; } 11
button { background: #be185d; color: white; border: 0; cursor: pointer; padding:
8px 14px; } 12      .filters { display: flex; gap: 6px; margin-bottom: 14px;
flex-wrap: wrap; } 13      .filters button { background: #fce7f3; color:
#be185d; border: 0; cursor: pointer; padding: 6px 12px; border-radius: 6px; } 14
.filters button.on { background: #be185d; color: white; } 15      .movie
{ background: white; padding: 12px; border-radius: 8px; margin-bottom: 8px;
display: flex; gap: 12px; align-items: center; } 16      .movie b { font-size:
15px; } 17      .stars { color: #f59e0b; letter-spacing: 1px; } 18      .movie
select { margin-left: auto; padding: 4px; } 19      .del { background: #fee2e2;
color: #dc2626; border: 0; padding: 4px 8px; border-radius: 4px; cursor:
pointer; } 20    </style> 21  </head> 22  <body> 23    <h1>🎬 Movie Watch
List</h1> 24    <form id="f"> 25      <input id="t" placeholder="Title"
required> 26      <select
id="s"><option>want</option><option>watching</option><option>watched</option></
select> 27      <select
id="r"><option>1</option><option>2</option><option>3</option><option>4</
option><option selected>5</option></select> 28      <input id="n"
placeholder="Notes"> 29      <button>Add</button> 30    </form> 31    <div
class="filters" id="fil"> 32      <button class="on" data-f="all">All</button>
33      <button data-f="want">Want to watch</button> 34      <button data-
f="watching">Watching</button> 35      <button data-f="watched">Watched</button>
36      <button data-f="5">5★ only</button> 37    </div> 38    <div
id="list"></div> 39   40    <script> 41      const KEY = "movies-v1"; 42
let movies = JSON.parse(localStorage.getItem(KEY) || "[]"); 43      let filter =
```

```
"all"; 44      function save() { localStorage.setItem(KEY,
JSON.stringify(movies)); } 45      function render() { 46
document.getElementById("list").innerHTML = movies 47          .filter(m =>
filter === "all" ? true : filter === "5" ? m.rating == 5 : m.status === filter)
48          .map((m, i) => 49            `<div class="movie"><div><b>$
{m.title}</b> 50             <div class="stars">${"★".repeat(m.rating) +
"☆".repeat(5 - m.rating)}</div>
  1             <div style="color:#475569;font-size:13px">${m.notes ||
""}</div></div>  2             <select onchange="chg(${i}, 'status',
this.value)">${["want","watching","watched"].map(s =>  3               `<option
${m.status === s ? "selected" : ""}>${s}</option>`).join("")}</select>  4
<button class="del" onclick="del(${i})">×</button></div>`).join("");  5      }
6      window.del = i => { movies.splice(i, 1); save(); render(); };  7
window.chg = (i, k, v) => { movies[i][k] = v; save(); render(); };  8
document.getElementById("fil").addEventListener("click", e => {  9        if
(e.target.tagName !== "BUTTON") return; 10
document.querySelectorAll("#fil button").forEach(b => b.classList.remove("on"));
11        e.target.classList.add("on"); 12        filter = e.target.dataset.f;
render(); 13      }); 14
document.getElementById("f").addEventListener("submit", e => { 15
e.preventDefault(); 16        movies.push({ title:
document.getElementById("t").value, status: document.getElementById("s").value,
17          rating: parseInt(document.getElementById("r").value), notes:
document.getElementById("n").value }); 18        ["t","n"].forEach(id =>
document.getElementById(id).value = ""); 19        save(); render(); 20      });
21      render(); 22    </script> 23  </body> 24  </html>
```

File structure

```
single file: movielist.html
```

How to run it

Open in any browser.

How it works

We store movies as objects with status, rating and notes. The ☆ unicode character lets us display ratings without any custom images. Filters are just array.filter calls.

Customization ideas

- Add actors per movie
- Add genre filter
- Add a 'recommend me' button

🕹 Upgrade Challenge

> *Add a mobile-friendly dark mode.*

Project 78: Invoice Generator

Difficulty: ☆☆

What you will build

Generate a printable invoice from a few form fields with line items and totals.

Skills you will learn

- Print styles
- Form binding
- Templating

Features

- Form for client, items, tax, discount
- Live preview
- Print to PDF
- Download as HTML

Build it with AI — the prompt to paste

> *Build an 'Invoice Generator'. Form on the left, live invoice preview on the right. Add line items, tax rate, discount. Print button uses @media print. Download as HTML. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Invoice</title>  6    <style>  7      body { font-family: system-ui;
margin: 0; display: grid; grid-template-columns: 360px 1fr; min-height: 100vh; }
8      form { background: #f1f5f9; padding: 16px; overflow: auto; }  9
input { width: 100%; padding: 6px; margin: 4px 0; border: 1px solid #cbd5e1;
border-radius: 6px; font: inherit; } 10      .row { display: grid; grid-
template-columns: 3fr 1fr 1fr auto; gap: 6px; } 11      button { padding: 6px
12px; background: #2563eb; color: white; border: 0; border-radius: 6px; cursor:
pointer; margin: 6px 0; } 12      .inv { padding: 30px; background: white;
overflow: auto; } 13      .inv h1 { margin: 0; color: #1e40af; } 14      .info {
display: flex; justify-content: space-between; margin: 16px 0; } 15      table {
width: 100%; border-collapse: collapse; margin-top: 14px; } 16      th, td
{ padding: 8px; border-bottom: 1px solid #e2e8f0; text-align: left; } 17      th
{ background: #f8fafc; } 18      .totals { margin-top: 20px; text-align:
right; } 19      .totals b { font-size: 22px; } 20      @media print { 21
form, .actions { display: none; } body { display: block; } 22      } 23
</style> 24  </head> 25  <body> 26    <form id="f"> 27      <label>Your name
<input id="from_n"></label> 28      <label>Your address <input
id="from_a"></label> 29      <label>Bill to <input id="to_n"></label> 30
<label>Client address <input id="to_a"></label> 31      <label>Invoice # <input
id="num" value="INV-001"></label> 32      <h4>Items</h4> 33      <div
id="items"></div> 34      <button type="button" onclick="addLine()">+ Add
line</button> 35      <label>Tax % <input id="tax" type="number"
value="18"></label> 36      <label>Discount <input id="disc" type="number"
value="0"></label> 37      <button type="button" class="actions"
onclick="window.print()">Print / Save PDF</button> 38      <button type="button"
class="actions" onclick="download()">Download HTML</button> 39    </form> 40
<div class="inv" id="pv"></div> 41    <script> 42      let items = [{desc:"Web
design", qty:5, price:500}]; 43      function save()
```

```
{ localStorage.setItem("inv-v1", JSON.stringify({items, cfg: getCfg()})); } 44
function getCfg() { 45        return { from_n: from_n.value, from_a:
from_a.value, to_n: to_n.value, to_a: to_a.value, num: num.value }; 46      } 47
function addLine(d, q, p) { 48        items.push({ desc: d || "Item", qty: q ||
1, price: p || 100 }); renderItems(); save(); 49      } 50      function
renderItems() {
  1        const html = items.map((it, i) =>  2          `<div
class="row"><input value="${it.desc}" oninput="items[$
{i}].desc=this.value;save()">  3           <input type="number" value="$
{it.qty}" oninput="items[${i}].qty=+this.value;save()">  4           <input
type="number" value="${it.price}" oninput="items[$
{i}].price=+this.value;save()">  5           <button type="button"
onclick="items.splice(${i},1);renderItems();save()">×</button></div>`).join("");
6        document.getElementById("items").innerHTML = html;  7      }  8
function paint() {  9        const cfg = { from_n: from_n.value, from_a:
from_a.value, to_n: to_n.value, to_a: to_a.value, num: num.value }; 10
const sub = items.reduce((s,it) => s + it.qty * it.price, 0); 11        const
tax = sub * (parseFloat(document.getElementById("tax").value) || 0) / 100; 12
const disc = parseFloat(document.getElementById("disc").value) || 0; 13
const total = sub + tax - disc; 14
document.getElementById("pv").innerHTML = `<h1>INVOICE</h1> 15          <p>#$
{cfg.num} · ${new Date().toLocaleDateString()}</p> 16          <div
class="info"><div><b>From</b><br>${cfg.from_n}<br>${cfg.from_a}</div> 17
<div style="text-align:right"><b>To</b><br>${cfg.to_n}<br>${cfg.to_a}</div></
div> 18
<table><thead><tr><th>Description</th><th>Qty</th><th>Price</th><th>Total</
th></tr></thead> 19          <tbody>${items.map(it =>
`<tr><td>${it.desc}</td><td>${it.qty}</td><td>₹${it.price.toFixed(2)}</td><td>₹$
{(it.qty * it.price).toFixed(2)}</td></tr>`).join("")}</tbody></table> 20
<div class="totals">Subtotal: ₹${sub.toFixed(2)}<br>Tax: ₹$
{tax.toFixed(2)}<br>Discount: ₹${disc.toFixed(2)}<br><b>Total: ₹$
{total.toFixed(2)}</b></div>`; 21        save(); 22      } 23      function
download() { 24        const data = document.getElementById("pv").innerHTML; 25
const html = `<!doctype html><html><head><meta
charset="utf-8"><title>Invoice</title><style>$
{document.querySelector("style").innerHTML}</style></head><body><div
class="inv">${data}</div></body></html>`; 26        const blob = new
Blob([html], { type: "text/html" }); 27        const a =
document.createElement("a"); a.download = "invoice-" + (num.value||"001") +
".html"; a.href = URL.createObjectURL(blob); a.click(); 28      } 29      const
stored = JSON.parse(localStorage.getItem("inv-v1") || "null"); 30      if
(stored) items = stored.items; 31      document.querySelectorAll("#f
input").forEach(inp => inp.addEventListener("input", paint)); 32
document.getElementById("tax").addEventListener("input", paint); 33
document.getElementById("disc").addEventListener("input", paint); 34
renderItems(); paint(); 35    </script> 36  </body> 37  </html>
```

File structure

```
single file: invoice.html
```

How to run it

Open in any browser.

How it works

We keep two-way binding between form inputs and the preview by listening to every input change, then re-rendering the right-side preview. Print mode hides the form with @media print rules.

Customization ideas

- Add multiple tax lines (CGST/SGST)
- Add a logo upload
- Generate invoice numbers automatically

🕹 Upgrade Challenge

> *Save issued invoices to localStorage and list them under 'Past invoices'.*

Project 79: Markdown Notes Wiki

Difficulty: ☆☆☆

What you will build

Linkable pages with markdown content. Pages are tiny and cross-referenced by [[title]].

Skills you will learn

- Markdown mini-parser
- Cross-reference linking
- Persistence

Features

- Multiple pages
- [[title]] linking between pages
- Backlinks panel
- Markdown rendering

Build it with AI — the prompt to paste

> *Build a 'Markdown Notes Wiki'. Multiple pages, each with markdown content. Inside a page, write [[Other Page Title]] to create a link to another page. If the target doesn't exist, show a 'Create' button. Backlinks panel shows pages that link to the current one. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Wiki Notes</title>  6    <style>  7      body { font-family: system-ui;
margin: 0; height: 100vh; display: grid; grid-template-columns: 220px 1fr 220px;
}  8      nav { background: #f1f5f9; padding: 14px; overflow: auto; border-
right: 1px solid #e2e8f0; }  9      .col { padding: 16px; overflow: auto; } 10
main { background: white; } 11      aside { background: #f8fafc; border-left:
1px solid #e2e8f0; padding: 14px; overflow: auto; } 12      h1 { margin-top:
0; } 13      textarea { width: 100%; min-height: 240px; font-family: ui-
monospace, monospace; padding: 12px; border: 1px solid #cbd5e1; border-radius:
6px; font-size: 15px; box-sizing: border-box; } 14      article h1, article h2,
article h3 { color: #1e40af; border-bottom: 1px solid #e2e8f0; padding-bottom:
4px; } 15      article code { background: #f1f5f9; padding: 1px 4px; border-
radius: 3px; } 16      .missing { background: #fef9c3; padding: 1px 6px; border-
radius: 4px; cursor: pointer; } 17      .link { color: #2563eb; text-decoration:
none; background: #e0f2fe; padding: 1px 4px; border-radius: 4px; } 18
button { padding: 8px 12px; background: #2563eb; color: white; border: 0;
border-radius: 6px; cursor: pointer; margin: 4px 0; } 19      ul { padding-left:
20px; } 20    </style> 21  </head> 22  <body> 23    <nav> 24      <button
onclick="newPage()">+ New page</button> 25      <ul id="pages"></ul> 26
</nav> 27    <main class="col"> 28      <h1 id="title">Welcome</h1> 29
<textarea id="editor" oninput="saveAndRender()"></textarea> 30      <article
id="view"></article> 31      <p><button onclick="toggleMode()">Toggle
editor/preview</button></p> 32    </main> 33    <aside> 34
<h3>Backlinks</h3> 35      <ul id="back"></ul> 36    </aside> 37   38
<script> 39      const KEY = "wiki-v1"; 40      let pages =
JSON.parse(localStorage.getItem(KEY)) || { "Welcome": "Welcome to your wiki!\n\
```

```
nLink to [[Todo]] or [[Ideas]]." }; 41      let current = "Welcome", mode =
"edit"; 42      function save() { localStorage.setItem(KEY,
JSON.stringify(pages)); } 43      function md(text) { 44        const esc = s =>
s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;"); 45
let s = esc(text); 46        s = s.replace(/^# (.+)$/gm,
"<h1>$1</h1>").replace(/^## (.+)$/gm, "<h2>$1</h2>"); 47        s = s.replace(/\
*\*(.+?)\*\*/g, "<b>$1</b>").replace(/\*(.+?)\*/g, "<i>$1</i>"); 48        s =
s.replace(/\[\[([^\]]+)\]\]/g, (m, t) => 49          pages[t] ? `<a class="link"
href="#" onclick="goto('${t}');return false">${t}</a>` : 50          `<a
class="missing" href="#" onclick="gotoCreate('${t}');return false">${t} +</a>`);
  1        return s.replace(/\n/g, "<br>");  2      }  3      function
renderPages() {  4        document.getElementById("pages").innerHTML =
Object.keys(pages).sort().map(p =>  5          `<li><a href="#" onclick="goto('$
{p}');return false" style="text-decoration:none;color:$
{p===current?'#1d4ed8':'#475569'}">${p}</a></li>`).join("");  6      }  7
function renderBack() {  8        const back = Object.entries(pages).filter(([p,
body]) => p !== current && body.includes("[[" + current + "]]"));  9
document.getElementById("back").innerHTML = back.map(([p]) => `<li><a href="#"
onclick="goto('${p}');return false">${p}</a></li>`).join("") ||
"<li>None.</li>"; 10      } 11      function goto(t) { current = t;
document.getElementById("title").textContent = t;
document.getElementById("editor").value = pages[t] || ""; renderPages();
renderBack(); renderArticle(); } 12      window.gotoCreate = t => { if (!
pages[t]) pages[t] = ""; goto(t); save(); }; 13      function renderArticle()
{ document.getElementById("view").innerHTML =
md(document.getElementById("editor").value); } 14      function saveAndRender()
{ pages[current] = document.getElementById("editor").value; save();
renderPages(); renderBack(); renderArticle(); } 15      window.newPage = () => {
const t = prompt("New page name:"); if (t && !pages[t]) goto(t), save(); }; 16
window.goto = goto; 17      window.toggleMode = () => { 18        mode = mode
=== "edit" ? "preview" : "edit"; 19
document.getElementById("editor").style.display = mode === "edit" ? "" : "none";
20        document.getElementById("view").style.display = mode === "preview" ?
"" : "none"; 21      }; 22      goto(current); 23    </script> 24  </body> 25
</html>
```

File structure

```
single file: wiki.html
```

How to run it

Open in any browser.

How it works

Pages is a dictionary keyed by title. The [[Title]] pattern is detected by a regex; if a matching page exists we render a real link, else we render a yellow 'create' prompt. Backlinks come from scanning all pages for [[Current]].

Customization ideas

- Add page rename
- Add tags
- Add full-text search

🕹 Upgrade Challenge

> *Add export to a single HTML file with all pages concatenated.*

Project 80: Reading Speed Adjuster

Difficulty: ☆☆

What you will build

Past any text. The page shows it word by word (or chunked) at a chosen WPM to exercise speed reading.

Skills you will learn

- Timed display
- Speed control
- Demo text

Features

- Paste any text
- Words-per-minute slider
- Big centred text
- Pause and restart

Build it with AI — the prompt to paste

> *Build a 'Reading Speed Trainer'. User pastes any text. They pick a WPM (200-1000). The page shows one word (or phrase) at a time, centred, large. Pause/resume and restart. Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Speed Reader</title>  6    <style>  7      body { font-family: system-ui;
background: #fafafa; min-height: 100vh; display:flex; flex-direction: column;
align-items: center; padding: 20px; }  8      textarea { width: 100%; max-width:
720px; min-height: 100px; padding: 12px; border-radius: 8px; border: 1px solid
#cbd5e1; font: inherit; }  9      .ctrls { display: flex; gap: 10px; margin:
14px 0; align-items: center; } 10      button { padding: 10px 18px; background:
#1e293b; color: white; border: 0; border-radius: 6px; cursor: pointer; } 11
.wpm { font-weight: 700; } 12      .stage { width: 100%; max-width: 720px; min-
height: 240px; background: white; border-radius: 12px; box-shadow: 0 4px 12px
rgba(0,0,0,.08); display:flex; align-items: center; justify-content: center;
font-size: 56px; font-weight: 700; color: #0f172a; padding: 20px; } 13
</style> 14  </head> 15  <body> 16    <h1>👁 Speed Reader</h1> 17    <textarea
id="t" placeholder="Paste text...">Speed reading is a skill you build. Start
with comfortable words-per-minute and gradually push the slider up. The point is
not to race words but to read in larger chunks.</textarea> 18    <div
class="ctrls"> 19      <label>WPM: <input id="wpm" type="range" min="120"
max="1000" value="300"> 20        <span class="wpm"
id="wpmLbl">300</span></label> 21      <button id="go">Start</button> 22
<button id="stop">Pause</button> 23      <button id="reset">Restart</button> 24
</div> 25    <div class="stage" id="stage">Ready?</div> 26   27    <script> 28
let words = [], i = 0, timer = null; 29      function start() { 30        words
= document.getElementById("t").value.trim().split(/\s+/).filter(w => w); 31
i = 0; 32        const ms = 60000 /
parseInt(document.getElementById("wpm").value); 33        run(ms); 34      } 35
function run(ms) { 36        clearInterval(timer); 37        timer =
setInterval(() => { 38          if (i >= words.length) { clearInterval(timer);
```

```
document.getElementById("stage").textContent = "Finished."; return; } 39
document.getElementById("stage").textContent = words[i++]; 40        }, ms); 41
} 42      function reset() { clearInterval(timer);
document.getElementById("stage").textContent = "Ready?"; } 43
document.getElementById("wpm").addEventListener("input", e => { 44
document.getElementById("wpmLbl").textContent = e.target.value; 45        if
(timer) run(60000 / e.target.value); 46      }); 47
document.getElementById("go").onclick = start; 48
document.getElementById("stop").onclick = () => clearInterval(timer); 49
document.getElementById("reset").onclick = reset; 50    </script>
  1  </body>  2  </html>
```

File structure

```
single file: speedread.html
```

How to run it

Open in any browser.

How it works

Each word appears for 60,000 / WPM milliseconds. We split the input on whitespace, then setInterval increments the index. Changing the WPM mid-read just reschedules the interval.

Customization ideas

- Add chunked mode (3-4 words at once)
- Add RSVP-style coloured focus point
- Track best WPM

🕹 Upgrade Challenge

> *Add Open Spritz-style optimal recognition point (ORP) letter highlight.*

Part 5 — Advanced Beginner Projects

Push yourself a bit further. Each project here combines everything you've learned — APIs, mathematics, animation, retro games, simulators and a capstone portfolio site.

Project 81: Interactive Solar System

Difficulty: ☆☆☆

What you will build

Animate planets orbiting a sun, with adjustable speeds and clickable info popups.

Skills you will learn

- Canvas animation
- Physics-inspired math
- Event handling

Features

- 6 orbiting planets with realistic relative speeds
- Click planet for info
- Speed slider
- Pause/resume

Build it with AI — the prompt to paste

> *Build 'Interactive Solar System'. A sun in the centre and 6 planets orbiting at relative speeds based on rough relative orbital periods (Mercury fastest, Neptune slowest). Each planet shows its name and facts when clicked. Canvas-based, vanilla JS, single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Solar System</title>  6    <style>  7      body { margin: 0; background:
#020617; color: white; display:flex; flex-direction: column; align-items:
center; min-height: 100vh; font-family: system-ui; }  8      canvas
{ background: radial-gradient(circle at center, #1e3a8a, #020617); }  9      h1
{ color: #fb923c; } 10      .pop { position: absolute; background:
rgba(0,0,0,.85); padding: 10px 14px; border-radius: 6px; max-width: 220px;
display: none; pointer-events: none; } 11      .ctrls { margin: 6px 0; } 12
button { padding: 8px 14px; background: #fb923c; color: #422006; border: 0;
border-radius: 6px; cursor: pointer; font-weight: 700; } 13    </style> 14
</head> 15  <body> 16    <h1>🪐 Solar System</h1> 17    <div class="ctrls"> 18
<label>Speed: <input id="sp" type="range" min="1" max="50" value="10"></label>
19      <button id="pa">Pause</button> 20    </div> 21    <canvas id="cv"
width="640" height="480"></canvas> 22    <div class="pop" id="pop"></div> 23
24    <script> 25      const cv = document.getElementById("cv"); 26      const
ctx = cv.getContext("2d"); 27      const W = cv.width, H = cv.height; 28
const cx = W / 2, cy = H / 2; 29      const PLANETS = [ 30        { name:
"Mercury", orbit: 70,  size: 4, color: "#a8a29e", info: "Closest to the Sun. One
Mercurian day is 59 Earth days." }, 31        { name: "Venus",   orbit: 110,
size: 6, color: "#fcd34d", info: "Hottest planet. Spins backwards relative to
most planets." }, 32        { name: "Earth",   orbit: 160, size: 7, color:
"#60a5fa", info: "Our home. The only known planet with liquid water oceans." },
33        { name: "Mars",    orbit: 210, size: 5, color: "#f87171", info: "The
Red Planet. Home to Olympus Mons, the solar system's tallest volcano." }, 34
{ name: "Jupiter", orbit: 280, size: 18, color: "#fde68a", info: "Largest
planet. A gas giant with a famous Great Red Spot storm." }, 35        { name:
"Saturn",  orbit: 350, size: 16, color: "#fcd34d", info: "Famous for its bright
```

```
ring system made of ice and rock." } 36      ]; 37      let speed = 0.01, paused
= false, angles = PLANETS.map(() => Math.random() * Math.PI * 2); 38   39
function draw() { 40        ctx.fillStyle = "rgba(2,6,23,.15)"; ctx.fillRect(0,
0, W, H); 41        // Sun 42        ctx.beginPath(); ctx.arc(cx, cy, 22, 0, 7);
ctx.fillStyle = "#facc15"; ctx.fill(); 43        ctx.beginPath(); ctx.arc(cx,
cy, 30, 0, 7); ctx.fillStyle = "rgba(250,204,21,.15)"; ctx.fill(); 44   45
if (!paused) angles = angles.map((a, i) => a + speed * (1 / (i + 1))); 46   47
PLANETS.forEach((p, i) => { 48          ctx.strokeStyle =
"rgba(255,255,255,.05)"; ctx.beginPath(); ctx.arc(cx, cy, p.orbit, 0, 7);
ctx.stroke(); 49          const x = cx + Math.cos(angles[i]) * p.orbit; 50
const y = cy + Math.sin(angles[i]) * p.orbit;
  1          ctx.beginPath(); ctx.arc(x, y, p.size, 0, 7); ctx.fillStyle =
p.color; ctx.fill();  2          p._x = x; p._y = y;  3        });  4      }  5
6      cv.addEventListener("click", e => {  7        const r =
cv.getBoundingClientRect();  8        const mx = (e.clientX - r.left) * W /
r.width, my = (e.clientY - r.top) * H / r.height;  9        const hit =
PLANETS.find(p => Math.hypot(p._x - mx, p._y - my) < p.size + 4); 10
const pop = document.getElementById("pop"); 11        if (hit) { 12
pop.innerHTML = "<b>" + hit.name + "</b><br>" + hit.info; 13
pop.style.left = (hit._x + 20) + "px"; 14          pop.style.top = (hit._y - 30)
+ "px"; 15          pop.style.display = "block"; 16          setTimeout(() =>
pop.style.display = "none", 4000); 17        } else pop.style.display = "none";
18      }); 19   20      document.getElementById("sp").addEventListener("input",
e => speed = +e.target.value / 1000); 21
document.getElementById("pa").addEventListener("click", e => { paused = !paused;
e.target.textContent = paused ? "Resume" : "Pause"; }); 22   23      function
loop() { draw(); requestAnimationFrame(loop); } 24      loop(); 25    </script>
26  </body> 27  </html>
```

File structure

```
single file: solar.html
```

How to run it

Open in any browser.

How it works

Each planet's angle increments by speed / (i+1), making Mercury the fastest and Saturn the slowest in roughly correct relative proportion. requestAnimationFrame keeps the animation smooth.

Customization ideas

- Add moons
- Add trailing comet that returns periodically
- Add scale mode (uniform speeds vs realistic)

🕹 Upgrade Challenge

> *Scale orbits with log-distance to fit a much smaller screen.*

Project 82: Colour Palette Generator

Difficulty: ☆☆

What you will build

Generate harmonious colour palettes from a base colour using HSL math. Copy hex codes.

Skills you will learn

- HSL math
- Copy to clipboard

Features

- Base colour picker
- Five harmony types
- Click to copy hex
- Lock individual colours

Build it with AI — the prompt to paste

> *Build a 'Colour Palette Generator'. Pick a base colour. Five palette types: complementary, analogous, triadic, tetradic, monochrome. Each generates 5 colours. Click any swatch to copy its hex code. Lock any colour to keep it during regeneration. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Palette Generator</title>  6    <style>  7      body { font-family:
system-ui; max-width: 720px; margin: 30px auto; padding: 20px; }  8      h1
{ color: #4f46e5; }  9      .row { display: flex; gap: 4px; margin: 14px 0; } 10
.swatch { flex: 1; height: 110px; border-radius: 8px; cursor: pointer; position:
relative; display:flex; align-items: end; justify-content: center; padding: 8px;
color: white; font-family: monospace; } 11      .lock { position: absolute; top:
8px; right: 8px; background: rgba(0,0,0,.4); border: 0; border-radius: 4px;
padding: 2px 8px; color: white; cursor: pointer; } 12      .locked .lock
{ background: #fb923c; } 13      .ctrls { display: flex; gap: 8px; align-items:
center; margin: 14px 0; } 14      select, input[type=color] { padding: 6px;
border-radius: 6px; border: 1px solid #cbd5e1; } 15    </style> 16  </head> 17
<body> 18    <h1>🎨 Palette Generator</h1> 19    <div class="ctrls"> 20
<input type="color" id="base" value="#6366f1"> 21      <select id="type"> 22
<option value="analogous">Analogous</option> 23        <option
value="complementary">Complementary</option> 24        <option
value="triadic">Triadic</option> 25        <option
value="tetradic">Tetradic</option> 26        <option
value="monochrome">Monochrome</option> 27      </select> 28      <button
onclick="gen()">Generate</button> 29    </div> 30    <div class="row"
id="row"></div> 31  32    <script> 33      let locked = [false, false, false,
false, false]; 34      function hex(h) { return Math.round(h * 360) + " " +
Math.round(h * 100) + "% " + Math.round(h * 100) + "%"; } 35      function
hslToHex(h, s, l) { 36        s /= 100; l /= 100; 37        const k = n => (n +
h / 30) % 12; 38        const a = s * Math.min(l, 1 - l); 39        const f = n
=> Math.round(255 * (l - a * Math.max(-1, Math.min(k(n) - 3, Math.min(9 - k(n),
1))))); 40        return "#" + [f(0),f(8),f(4)].map(x =>
```

```
x.toString(16).padStart(2,"0")).join(""); 41      } 42      function
hexToHsl(hex) { 43        const r = parseInt(hex.slice(1,3), 16) / 255; 44
const g = parseInt(hex.slice(3,5), 16) / 255; 45        const b =
parseInt(hex.slice(5,7), 16) / 255; 46        const max = Math.max(r,g,b), min =
Math.min(r,g,b); 47        let h=0, s=0, l=(max+min)/2; 48        if (max !==
min) { 49          const d = max - min; 50          s = l > 0.5 ? d / (2 - max -
min) : d / (max + min);
  1          switch(max){  2            case r: h = (g - b) / d + (g < b ? 6 :
0); break;  3            case g: h = (b - r) / d + 2; break;  4            case
b: h = (r - g) / d + 4; break;  5          } h /= 6;  6        }  7
return { h: h * 360, s: s * 100, l: l * 100 };  8      }  9      function gen()
{ 10        const base = hexToHsl(document.getElementById("base").value); 11
const type = document.getElementById("type").value; 12        const hues =
{ analogous: [-30, -15, 0, 15, 30], complementary: [0, 0, 180, 180, 180],
triadic: [0, 120, 240, 120, 240], tetradic: [0, 90, 180, 270, 0], monochrome:
[0, 0, 0, 0, 0] }; 13        const ls = { analogous: [50, 40, 30, 50, 60],
complementary: [40, 50, 60, 40, 50], triadic: [40, 50, 60, 50, 60], tetradic:
[40, 50, 60, 50, 40], monochrome: [20, 35, 50, 65, 80] }; 14        const cols =
hsl(hues[type]).map((hue, i) => hslToHex(hue, 70, ls[type][i])); 15        const
row = document.getElementById("row"); 16        row.innerHTML = cols.map((c, i)
=> { 17          if (locked[i]) return
document.getElementById("row").children[i]?.outerHTML || `<div class="swatch
locked" style="background:${lockColors[i] || c}"><button class="lock"
onclick="toggle(${i}, '${c}')">🔒</button><span>${c}</span></div>`; 18
return `<div class="swatch" style="background:${c}" onclick="copy('$
{c}')"><button class="lock" onclick="toggle(${i}, '$
{c}');event.stopPropagation()">🔓</button><span>${c}</span></div>`;
19        }).join(""); 20      } 21      let lockColors =
["#000","#000","#000","#000","#000"]; 22      window.toggle = (i, c) =>
{ locked[i] = !locked[i]; if (locked[i]) lockColors[i] = c; gen(); }; 23
window.copy = c => navigator.clipboard && navigator.clipboard.writeText(c); 24
function hsl(arr) { const base =
hexToHsl(document.getElementById("base").value); return arr.map(d => (base.h + d
+ 360) % 360); } 25
document.getElementById("base").addEventListener("input", gen); 26
document.getElementById("type").addEventListener("change", gen); 27      gen();
28    </script> 29  </body> 30  </html>
```

File structure

```
single file: palette.html
```

How to run it

Open in any browser.

How it works

HSL math is friendlier than RGB for harmony rules because hue distance directly maps onto colour wheel degrees (30, 60, 90, 120…). Our convert function rebuilds a hex in sRGB from HSL.

Customization ideas

- Add a contrast checker
- Add a 30-60-90 rule palette suggestion
- Add export to CSS variables

🕹 Upgrade Challenge

> *Generate a complete material-design-style theme from a single primary colour.*

Project 83: Procedural Terrain Generator

Difficulty: ☆☆☆

What you will build

Generate a tile-based terrain map with hills, water and forests using simple noise.

Skills you will learn

- Random noise
- Cellular automata-like smoothing

Features

- 2D value-noise grid
- Auto-classify water/grass/mountain/forest
- Resolution slider
- Regenerate

Build it with AI — the prompt to paste

> *Build a 'Procedural Terrain Generator'. Generate a grid using value-noise (essentially a fixed-seed random grid). Apply smoothing passes. Classify each cell by elevation: water (<0.3), sand (<0.4), grass (<0.6), forest (<0.78), mountain (>=0.78). Render with coloured cells. Pure JS canvas.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Terrain</title>  6    <style>  7      body { font-family: system-ui;
display: flex; flex-direction: column; align-items: center; background: #0f172a;
min-height: 100vh; margin: 0; color: white; padding: 20px; }  8      canvas
{ background: white; border-radius: 4px; }  9      button { padding: 8px 14px;
background: #16a34a; color: white; border: 0; border-radius: 6px; cursor:
pointer; margin: 6px; } 10    </style> 11  </head> 12  <body> 13    <h1>🏔
Procedural Terrain</h1> 14    <button onclick="gen()">Regenerate</button> 15
<canvas id="cv" width="600" height="400"></canvas> 16   17    <script> 18
const cv = document.getElementById("cv"); 19      const ctx =
cv.getContext("2d"); 20      const W = 80, H = 50; 21      function rand(seed) {
seed = (seed * 9301 + 49297) % 233280; return seed / 233280; } 22      function
noise(grid, r, c) { return grid[r][c]; } 23      function gen() { 24        //
value noise: random grid + bilinear smoothing + smooth passes 25        let grid
= Array.from({length: H}, (_, r) => Array.from({length: W}, (_, c) => rand(r * W
+ c + Date.now() % 10000 + 1))); 26        const smooth = (g) => { 27
const out = Array.from({length: H}, () => new Array(W).fill(0)); 28          for
(let r = 0; r < H; r++) for (let c = 0; c < W; c++) { 29            let sum = 0,
n = 0; 30            for (let dr = -1; dr <= 1; dr++) for (let dc = -1; dc <= 1;
dc++) { 31              const nr = r + dr, nc = c + dc; 32              if (nr
>= 0 && nr < H && nc >= 0 && nc < W) { sum += g[nr][nc]; n++; } 33            }
34            out[r][c] = sum / n; 35          } 36          return out; 37
}; 38        for (let i = 0; i < 3; i++) grid = smooth(grid); 39        //
Render 40        const cellW = cv.width / W, cellH = cv.height / H; 41
for (let r = 0; r < H; r++) for (let c = 0; c < W; c++) { 42          const v =
grid[r][c]; 43          let color; 44          if (v < 0.32) color = "#3b82f6";
// water 45          else if (v < 0.36) color = "#fde68a";  // sand 46
```

```
else if (v < 0.55) color = "#22c55e";  // grass 47          else if (v < 0.7)
color = "#15803d";  // forest 48          else if (v < 0.85) color = "#a8a29e";
// mountain grey 49          else               color = "#f5f5f4";  // snow 50
ctx.fillStyle = color; ctx.fillRect(c * cellW, r * cellH, cellW + 1, cellH + 1);
  1        }  2      }  3      gen();  4    </script>  5  </body>  6  </html>
```

File structure

```
single file: terrain.html
```

How to run it

Open in any browser.

How it works

Value noise works in two steps: generate a fully random grid, then average each cell with its neighbours a few times. After the smoothing passes, neighbouring cells are correlated, which produces the 'lumpy' look of natural terrain.

Customization ideas

- Add rivers
- Add biomes
- Add export to PNG

🕹 Upgrade Challenge

> *Add Simplex / Perlin noise with custom hash to produce smoother, more natural terrain.*

Project 84: Particle Explosion Simulator

Difficulty: ☆☆☆

What you will build

Click anywhere on a canvas to create an explosion of particles with velocity, gravity, and fade.

Skills you will learn

- Newtonian physics
- Canvas animation
- Math.hypot

Features

- Click to spawn explosion
- Particles obey gravity and fade
- Adjustable count
- Reset

Build it with AI — the prompt to paste

> *Build 'Particle Explosion Simulator'. Click to spawn an explosion at that point. Particles have random velocity, are pulled down by gravity, fade over time, and disappear when their alpha < 0. Particles are coloured randomly from a palette. Pure JS canvas, single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Particles</title>  6    <style>  7      body { margin: 0; background:
#020617; display: flex; flex-direction: column; align-items: center; justify-
content: center; min-height: 100vh; font-family: system-ui; color: white; }  8
canvas { background: #0f172a; border: 1px solid #334155; cursor: crosshair; }  9
button { padding: 8px 14px; background: #f97316; color: white; border: 0;
border-radius: 6px; cursor: pointer; margin: 6px; } 10      .ctrls { margin-
bottom: 12px; } 11    </style> 12  </head> 13  <body> 14    <h1> Particles</h1>
15    <div class="ctrls"> 16      <label>Count: <input id="n" type="range"
min="20" max="400" value="120"> 17        <span id="nLbl">120</span></label> 18
<button onclick="reset()">Reset</button> 19    </div> 20    <canvas id="cv"
width="600" height="400"></canvas> 21    <p>Click anywhere to explode.</p> 22
23    <script> 24      const cv = document.getElementById("cv"); 25      const
ctx = cv.getContext("2d"); 26      let parts = []; 27      const COL =
["#f87171","#fb923c","#facc15","#86efac","#60a5fa","#a78bfa","#f9a8d4"]; 28
document.getElementById("n").addEventListener("input", e =>
document.getElementById("nLbl").textContent = e.target.value); 29
cv.addEventListener("click", e => { 30        const r =
cv.getBoundingClientRect(); 31        const x = e.clientX - r.left, y =
e.clientY - r.top; 32        for (let i = 0; i <
+document.getElementById("n").value; i++) { 33          const angle =
Math.random() * Math.PI * 2; 34          const speed = Math.random() * 6 + 2; 35
parts.push({ x, y, vx: Math.cos(angle) * speed, vy: Math.sin(angle) * speed,
life: 1, color: COL[Math.floor(Math.random() * COL.length)] }); 36        } 37
}); 38      function reset() { parts = []; } 39   40      function step() { 41
ctx.fillStyle = "rgba(15,23,42,.25)"; ctx.fillRect(0, 0, cv.width, cv.height);
```

```
42        parts.forEach(p => { 43          p.x += p.vx; p.y += p.vy; p.vy +=
0.15; p.life -= 0.012; 44          ctx.globalAlpha = Math.max(0, p.life); 45
ctx.fillStyle = p.color; 46          ctx.beginPath(); ctx.arc(p.x, p.y, 3, 0,
7); ctx.fill(); 47        }); 48        ctx.globalAlpha = 1; 49        parts =
parts.filter(p => p.life > 0 && p.y < cv.height); 50
requestAnimationFrame(step);
  1      }  2      step();  3    </script>  4  </body>  5  </html>
```

File structure

```
single file: particles.html
```

How to run it

Open in any browser.

How it works

Each particle stores position, velocity, life and colour. We step through every particle, applying gravity (vy += 0.15) and reducing life. The trail effect comes from not fully clearing the canvas, leaving faint ghosts.

Customization ideas

- Add wind control
- Add particle types (fireworks, smoke)
- Click-and-drag for directional force

🕹 Upgrade Challenge

> *Add a colision with faux bounding walls.*

Project 85: Drum Machine / Step Sequencer

Difficulty: ☆☆☆

What you will build

A 16-step drum pad sequencer using WebAudio. Click cells, hit play, hear a beat.

Skills you will learn

- WebAudio synthesis
- Pattern state

Features

- 8 instruments across 16 steps
- Click to toggle
- Play and stop
- BPM control

Build it with AI — the prompt to paste

> *Build a 'Drum Machine / Step Sequencer'. 8 rows (instruments) × 16 columns (steps). Click cells to toggle them on. Hit Play to step through the grid; each filled cell plays its instrument sound (synthesised via WebAudio oscillators / noise). BPM slider. Pure JS, single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Drum Machine</title>  6    <style>  7      body { font-family: system-ui;
background: #020617; color: white; min-height: 100vh; margin: 0; padding: 20px;
display: flex; flex-direction: column; align-items: center; }  8      h1
{ color: #f472b6; }  9      table { border-collapse: collapse; margin: 14px 0; }
10      td { width: 30px; height: 30px; background: #1e293b; cursor: pointer;
border: 2px solid #0f172a; } 11      td.on { background: #f472b6; } 12
td.on.playing { background: #facc15; } 13      td.beat { background: #334155; }
14      th { color: #94a3b8; padding: 4px; } 15      button { padding: 10px
18px; background: #f472b6; color: white; border: 0; border-radius: 6px; cursor:
pointer; font-weight: 700; } 16      .ctrls { margin-bottom: 12px; } 17
</style> 18  </head> 19  <body> 20    <h1>□ Beat Machine</h1> 21    <div
class="ctrls"> 22      <label>BPM: <input id="bpm" type="range" min="60"
max="200" value="120"> 23        <span id="bLbl">120</span></label> 24
<button id="pl">Play</button> 25      <button id="cl">Clear</button> 26
</div> 27    <table id="g"></table> 28   29    <script> 30      const SOUNDS = {
kick: kick, snare: snare, hat: hat, ohat: ohat, tom: tom, clap: clap, perc:
perc, cym: cym }; 31      const TRACKS = Object.keys(SOUNDS); 32      const
STEPS = 16; 33      let grid = Array.from({length: TRACKS.length}, () => new
Array(STEPS).fill(false)); 34      let playing = false, current = 0, interval =
null; 35   36      function build() { 37        const g =
document.getElementById("g"); 38        g.innerHTML = ""; 39        // header 40
const head = g.createTHead(); 41        const hr = head.insertRow(); 42
hr.insertCell(); 43        for (let s = 0; s < STEPS; s++) { const c =
hr.insertCell(); c.outerHTML = `<th>${s+1}</th>`; } 44        // rows 45
const body = g.createTBody(); 46        TRACKS.forEach((name, r) => { 47
const row = body.insertRow(); 48          const label = row.insertCell();
label.textContent = name; label.style.color = "#94a3b8"; label.style.textAlign =
```

```
"right"; label.style.paddingRight = "8px"; 49          for (let s = 0; s <
STEPS; s++) { 50            const cell = row.insertCell();
  1           if (s % 4 === 0) cell.classList.add("beat");  2
cell.onclick = () => { grid[r][s] = !grid[r][s]; cell.classList.toggle("on"); };
3          }  4        });  5      }  6      function tick() {  7
document.querySelectorAll("td.playing").forEach(c =>
c.classList.remove("playing"));  8        TRACKS.forEach((name, r) => {  9
if (grid[r][current]) { 10            SOUNDS[name](); 11            const cells
= document.querySelectorAll("tbody tr")[r].cells; 12            cells[current +
1].classList.add("playing"); 13          } 14        }); 15        current =
(current + 1) % STEPS; 16      } 17   18
document.getElementById("bpm").addEventListener("input", e => { 19
document.getElementById("bLbl").textContent = e.target.value; 20        if
(playing) restart(); 21      }); 22
document.getElementById("pl").addEventListener("click", e => { 23        playing
= !playing; 24        e.target.textContent = playing ? "Stop" : "Play"; 25
if (playing) restart(); else stop(); 26      }); 27
document.getElementById("cl").addEventListener("click", () => { 28        grid =
Array.from({length: TRACKS.length}, () => new Array(STEPS).fill(false)); 29
build(); 30      }); 31      function restart() { 32        const ms = 60000 /
+document.getElementById("bpm").value / 4; 33        stop(); interval =
setInterval(tick, ms); 34      } 35      function stop()
{ clearInterval(interval); interval = null;
document.querySelectorAll("td.playing").forEach(c =>
c.classList.remove("playing")); current = 0; } 36   37      function ctx()
{ return new (window.AudioContext || window.webkitAudioContext)(); } 38
function kick() { 39        const c = ctx(); const o = c.createOscillator();
const g = c.createGain(); 40        o.connect(g); g.connect(c.destination); 41
o.frequency.setValueAtTime(150, c.currentTime);
o.frequency.exponentialRampToValueAtTime(40, c.currentTime + 0.15); 42
g.gain.setValueAtTime(0.4, c.currentTime);
g.gain.exponentialRampToValueAtTime(0.001, c.currentTime + 0.2); 43
o.start(); o.stop(c.currentTime + 0.2); 44      } 45      function snare() { 46
const c = ctx(); const buf = c.createBuffer(1, c.sampleRate * 0.1,
c.sampleRate); 47        const data = buf.getChannelData(0); for (let i = 0; i <
data.length; i++) data[i] = Math.random() * 2 - 1; 48        const src =
c.createBufferSource(); src.buffer = buf; 49        const g = c.createGain();
g.gain.setValueAtTime(0.3, c.currentTime);
g.gain.exponentialRampToValueAtTime(0.001, c.currentTime + 0.15); 50
src.connect(g); g.connect(c.destination); src.start();
  1      }  2      function hat() { /* short noisy burst */ const c = ctx();  3
const buf = c.createBuffer(1, c.sampleRate * 0.04, c.sampleRate); const data =
buf.getChannelData(0);  4        for (let i = 0; i < data.length; i++) data[i] =
Math.random() * 2 - 1;  5        const src = c.createBufferSource(); src.buffer
= buf;  6        const g = c.createGain(); g.gain.setValueAtTime(0.15,
c.currentTime); g.gain.exponentialRampToValueAtTime(0.001, c.currentTime +
0.05);  7        src.connect(g); g.connect(c.destination); src.start();
8      }  9      function ohat() { hat(); } 10      function tom() { const c =
ctx(); const o = c.createOscillator(); const g = c.createGain(); 11
o.connect(g); g.connect(c.destination); o.frequency.setValueAtTime(220,
c.currentTime); 12        o.frequency.exponentialRampToValueAtTime(80,
c.currentTime + 0.2); 13        g.gain.setValueAtTime(0.2, c.currentTime);
g.gain.exponentialRampToValueAtTime(0.001, c.currentTime + 0.25); 14
o.start(); o.stop(c.currentTime + 0.25); 15      } 16      function clap()
{ snare(); } 17      function perc() { tom(); } 18      function cym() { /* long
noise */ const c = ctx(); 19        const buf = c.createBuffer(1, c.sampleRate *
0.6, c.sampleRate); const data = buf.getChannelData(0); 20        for (let i =
0; i < data.length; i++) data[i] = Math.random() * 2 - 1; 21        const src =
c.createBufferSource(); src.buffer = buf; 22        const f =
c.createBiquadFilter(); f.type = "highpass"; f.frequency.value = 5000; 23
const g = c.createGain(); g.gain.setValueAtTime(0.2, c.currentTime);
```

```
g.gain.exponentialRampToValueAtTime(0.001, c.currentTime + 0.5); 24
src.connect(f); f.connect(g); g.connect(c.destination); src.start(); 25      }
26   27      build(); 28    </script> 29  </body> 30  </html>
```

File structure

```
single file: drum.html
```

How to run it

Open in any browser.

How it works

Each instrument is a tiny WebAudio program: kick is a swept sine wave, snare is a noise burst, hats are short noise bursts. setInterval drives the steps; the cell highlighting makes the playhead easy to follow.

Customization ideas

- Save and load patterns
- Swing control
- Export pattern as JSON

🕹 Upgrade Challenge

> *Add a 'random generator' button that produces a beat with a seedable PRNG.*

Project 86: Projectile Motion Simulator

Difficulty: ☆☆☆

What you will build

Launch a projectile with adjustable angle, speed and gravity. Watch the parabola.

Skills you will learn

- Physics math
- Visualisation

Features

- Adjustable angle and speed
- Gravity slider (incl. moon)
- Trajectory ping
- Landing point

Build it with AI — the prompt to paste

> *Build 'Projectile Motion Simulator'. User inputs angle, speed, gravity. Click Launch: the projectile animates its flight along the computed parabola. Show max height, range, time of flight. Pure JS canvas.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Projectile</title>  6    <style>  7      body { font-family: system-ui;
max-width: 700px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#0369a1; }  9      .ctrls { display: grid; grid-template-columns: repeat(3, 1fr)
auto; gap: 8px; margin-bottom: 12px; } 10      label { font-size: 13px; } 11
canvas { background: #f1f5f9; border-radius: 8px; width: 100%; } 12      button
{ padding: 10px 18px; background: #0369a1; color: white; border: 0; border-
radius: 6px; cursor: pointer; } 13      .stats { display: flex; gap: 20px;
margin-top: 10px; font-size: 14px; color: #334155; } 14    </style> 15  </head>
16  <body> 17    <h1>🚀 Projectile Motion</h1> 18    <div class="ctrls"> 19
<label>Angle° <input id="a" type="number" value="45" min="0" max="90"></label>
20      <label>Speed m/s <input id="v" type="number" value="40" min="1"
max="200"></label> 21      <label>Gravity <input id="g" type="number"
value="9.8" step="0.1"></label> 22      <button
onclick="launch()">Launch</button> 23    </div> 24    <canvas id="cv"
width="640" height="320"></canvas> 25    <div class="stats" id="st"></div> 26
27    <script> 28      const cv = document.getElementById("cv"); 29      const
ctx = cv.getContext("2d"); 30      cv.height = 320; 31      let fly = null; 32
function launch() { 33        const deg =
parseFloat(document.getElementById("a").value); 34        const v =
parseFloat(document.getElementById("v").value); 35        const g =
parseFloat(document.getElementById("g").value); 36        const rad = deg *
Math.PI / 180; 37        fly = { x: 20, y: cv.height - 20, vx: v *
Math.cos(rad), vy: -v * Math.sin(rad), g }; 38        const tFlight = 2 * v *
Math.sin(rad) / g; 39        const range = v * Math.cos(rad) * tFlight; 40
const maxH = (v * Math.sin(rad)) ** 2 / (2 * g); 41
document.getElementById("st").innerHTML = 42          `Range: <b>$
{range.toFixed(1)}m</b> · Max height: <b>${maxH.toFixed(1)}m</b> · Time of
flight: <b>${tFlight.toFixed(1)}s</b>`; 43      } 44      function step() { 45
```

```
ctx.fillStyle = "#f1f5f9"; ctx.fillRect(0, 0, cv.width, cv.height); 46        //
ground 47        ctx.fillStyle = "#22c55e"; ctx.fillRect(0, cv.height - 20,
cv.width, 20); 48        if (fly) { 49          fly.x += fly.vx * 0.1; 50
fly.vy += fly.g * 0.1;
  1          fly.y += fly.vy * 0.1;  2          ctx.beginPath();
ctx.arc(Math.min(fly.x, cv.width - 10), Math.min(fly.y, cv.height - 20), 6, 0,
7);  3          ctx.fillStyle = "#0369a1"; ctx.fill();  4          if (fly.y >=
cv.height - 20 || fly.x >= cv.width - 10) fly = null;  5        }  6
requestAnimationFrame(step);  7      }  8      step();  9    </script> 10
</body 11  </html>
```

File structure

```
single file: projectile.html
```

How to run it

Open in any browser.

How it works

Initial velocities split using cos(angle)*speed (horizontal) and sin(angle)*speed (vertical). At each frame we apply gravity to vy. Range is v²*sin(2θ)/g — a classic physics identity we use alongside the simulation.

Customization ideas

- Add air drag
- Add multiple targets
- Add scatter plot of fixed-angle range vs speed

🕹 Upgrade Challenge

> *Simulate a catapult with adjustable release angle and historical projectile presets (e.g. trebuchet).*

Project 87: Chord Progression Player

Difficulty: ☆☆☆

What you will build

Generate chord progressions and play them with WebAudio. Visualise a piano keyboard.

Skills you will learn

- Music theory
- WebAudio
- DOM piano

Features

- 8 chord progressions
- Click to play
- Visual highlighting of which notes are playing
- Tempo

Build it with AI — the prompt to paste

> *Build 'Chord Progression Player'. Eight common progressions written in plain English (I-V-vi-IV, ii-V-I, etc.). Each click plays the progression as piano-like tones from WebAudio. A virtual keyboard highlights the keys being played. Pure JS, single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Chord Player</title>  6    <style>  7      body { font-family: system-ui;
max-width: 720px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#0e7490; }  9      button { padding: 8px 12px; margin: 6px; background: #0891b2;
color: white; border: 0; border-radius: 6px; cursor: pointer; }
10      .keyboard { position: relative; height: 120px; margin: 14px 0;
background: white; border-radius: 6px; padding: 6px; } 11      .key { position:
absolute; top: 6px; width: 30px; height: 108px; background: white; border: 1px
solid #cbd5e1; border-radius: 0 0 4px 4px; cursor: pointer; } 12      .key.black
{ background: #1e293b; height: 70px; z-index: 2; width: 22px; }
13      .key.held { background: #facc15; } 14      .key.black.held { background:
#fb923c; } 15      .progress { margin-top: 8px; color: #64748b; } 16    </style>
17  </head> 18  <body> 19    <h1>🎹 Chord Progression Player</h1> 20    <p>Key of
C major — click a progression:</p> 21    <button onclick="play('I-V-vi-IV')">I-
V-vi-IV</button> 22    <button onclick="play('vi-IV-I-V')">vi-IV-I-V</button> 23
<button onclick="play('ii-V-I')">ii-V-I</button> 24    <button onclick="play('I-
vi-IV-V')">I-vi-IV-V</button> 25    <button onclick="play('I-IV-V-I')">I-IV-V-
I</button> 26    <button onclick="play('I-V-vi-iii-IV-I-IV-V')">Canon
(Pachelbel)</button> 27    <div class="keyboard" id="kb"></div> 28    <p
class="progress" id="pg"></p> 29   30    <script> 31      // Major scale degrees
32      const SCALE = ["C","D","E","F","G","A","B"]; 33      // Map Roman
numerals to degree (1-based) and minor indicator 34      const RN = { 35
I: 1, ii: 2, iii: 3, IV: 4, V: 5, vi: 6, vii: 7, 36        i: [1,"m"], ii:
[2,"m"], iii: [3,"m"], iv: [4,"m"], v: [5,"m"], vi: [6,"m"], vii: [7,"m"] 37
}; 38      function chord(deg, minor=false) { 39        const root = SCALE[(deg
- 1) % 7]; 40        const third = SCALE[(deg - 1 + 2) % 7]; 41        const
```

```
fifth = SCALE[(deg - 1 + 4) % 7]; 42        return [root, third + (minor ? "m" :
""), fifth]; 43      } 44      function play(prog) { 45        const seq =
prog.split("-").map(t => { 46          const lower = t.toLowerCase(); 47
const minor = lower === t; 48          const deg = { i:1, ii:2, iii:3, iv:4,
v:5, vi:6, vii:7 }[lower]; 49          return chord(deg, minor); 50        });
  1        let i = 0;  2        const stepLen = 600;  3        const step = ()
=> {  4          if (i >= seq.length) return;  5          const notes = seq[i];
6          document.getElementById("pg").textContent = "Chord " + (i + 1) + " of
" + seq.length + ": " + notes.join(" · ");  7          notes.forEach(n =>
playNote(n, 0.7));  8          i++;  9          setTimeout(step, stepLen); 10
}; 11        step(); 12      } 13      function noteFreq(note) { 14        const
map = { C:0, D:2, E:4, F:5, G:7, A:9, B:11 }; 15        const letter = note[0],
sharp = note.includes("m") ? note[1] ? note[1] : "" : ""; 16        const n =
map[letter] + (note.includes("#") ? 1 : 0); 17        return 261.63 *
Math.pow(2, (n + (parseInt(note.match(/\d+/)?.[0]) || 4) - 4) / 12); 18      }
19      function playNote(note, dur) { 20        const c = new
(window.AudioContext || window.webkitAudioContext)(); 21        const o =
c.createOscillator(), g = c.createGain(); 22        o.connect(g);
g.connect(c.destination); 23        o.type = "triangle"; o.frequency.value =
noteFreq(note); 24        g.gain.setValueAtTime(0.2, c.currentTime); 25
g.gain.exponentialRampToValueAtTime(0.001, c.currentTime + dur); 26
o.start(); o.stop(c.currentTime + dur); 27        const el =
document.querySelector(`[data-note="${note.replace("m","")}"]`); 28        if
(el) { el.classList.add("held"); setTimeout(() => el.classList.remove("held"),
dur * 1000); } 29      } 30   31      // Build keyboard 32      const NOTES =
["C","C#","D","D#","E","F","F#","G","G#","A","A#","B","C2"]; 33      const kb =
document.getElementById("kb"); 34      NOTES.forEach((n, i) => { 35        const
k = document.createElement("div"); 36        k.className = "key" +
(n.includes("#") ? " black" : ""); 37        k.style.left = (10 +
(n.includes("#") ? i*30-12 : i*30)) + "px"; 38        k.dataset.note = n; 39
kb.appendChild(k); 40      }); 41    </script> 42  </body> 43  </html>
```

File structure

```
single file: chords.html
```

How to run it

Open in any browser.

How it works

Each chord is just root + third + fifth. We compute frequencies using the 12-tone equal-tempered formula: f = 261.63 * 2^(n/12), where n is the number of semitones above middle C. setTimeout chain builds the progression.

Customization ideas

- Add chord quality (maj7, m7, dim)
- Add voice leading
- Add chord charts

🕹 Upgrade Challenge

> *Build a chord-guesser game that plays a chord and asks the user to identify its name.*

Project 88: ASCII Art Image Converter

Difficulty: ☆☆

What you will build

Upload an image, get back an ASCII-art representation at a chosen width.

Skills you will learn

- Image manipulation on canvas
- Brightness mapping

Features

- Adjustable width and character set
- Live preview
- Copy to clipboard

Build it with AI — the prompt to paste

> *Build 'ASCII Art Image Converter'. User uploads an image (or uses a sample). The code draws it to a small canvas, samples brightness per cell, and maps to characters. Pure JS, single file. Output is a pre block the user can copy.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>ASCII</title>  6    <style>  7      body { font-family: system-ui; max-
width: 720px; margin: 30px auto; padding: 20px; }  8      h1 { color: #9333ea; }
9      input, button { padding: 8px; border-radius: 6px; border: 1px solid
#cbd5e1; font: inherit; margin: 4px 0; } 10      button { background: #9333ea;
color: white; border: 0; cursor: pointer; } 11      pre { background: #0f172a;
color: #e2e8f0; padding: 12px; border-radius: 6px; font-size: 8px; line-height:
8px; overflow: auto; max-height: 480px; } 12    </style> 13  </head> 14  <body>
15    <h1>🖼 ASCII Image Converter</h1> 16    <input type="file" id="f"
accept="image/*"> 17    <label>Width: <input id="w" type="number" value="80"
min="20" max="200"></label> 18    <select id="cs"><option value="dense">dense:
@#%&8B$M*WHDAPX0OQCEFJKZ|(){}[]<>?</option> 19      <option
value="medium">medium: @%#*+=-:.</option> 20      <option value="simple">simple:
#.+-</option></select> 21    <button onclick="go()">Convert</button> 22
<button onclick="copy()">Copy</button> 23    <pre id="out">Upload an image and
click Convert.</pre> 24   25    <script> 26      let img = null; 27
document.getElementById("f").addEventListener("change", e => { 28        const
file = e.target.files[0]; 29        if (!file) return; 30        const url =
URL.createObjectURL(file); 31        const im = new Image(); 32        im.onload
= () => { img = im; document.getElementById("out").textContent = "Loaded. Click
Convert."; }; 33        im.src = url; 34      }); 35      function go() { 36
if (!img) return; 37        const w =
parseInt(document.getElementById("w").value); 38        const cs =
document.getElementById("cs").value; 39        const CHARS = { dense: "@%#*+=-:.
", medium: "@%#*+=-:. ", simple: "#.+- " }[cs]; 40        const cv =
document.createElement("canvas"); 41        const ratio = img.height /
img.width; 42        cv.width = w; 43        cv.height = Math.max(1,
Math.floor(w * ratio / 2)); 44        const ctx = cv.getContext("2d"); 45
ctx.drawImage(img, 0, 0, cv.width, cv.height); 46        const data =
ctx.getImageData(0, 0, cv.width, cv.height).data; 47        let s = ""; 48
```

```
for (let y = 0; y < cv.height; y++) { 49          for (let x = 0; x < cv.width;
x++) { 50            const i = (y * cv.width + x) * 4;
  1            const r = data[i], g = data[i + 1], b = data[i + 2];  2
const bright = (0.299*r + 0.587*g + 0.114*b) / 255;  3            const idx =
Math.min(CHARS.length - 1, Math.floor((1 - bright) * CHARS.length));  4
s += CHARS[idx];  5          }  6          s += "\n";  7        }  8
document.getElementById("out").textContent = s;  9      } 10      window.copy =
() => navigator.clipboard &&
navigator.clipboard.writeText(document.getElementById("out").textContent); 11
</script> 12  </body> 13  </html>
```

File structure

```
single file: ascii.html
```

How to run it

Open in any browser.

How it works

Drawing an image to a small canvas effectively down-samples it. We then walk pixel-by-pixel, compute a luminance value (using the standard 0.299R + 0.587G + 0.114B coefficients) and pick a character from the gradient string.

Customization ideas

- Add monospace and colour ASCII variants
- Allow URL input
- Add edge detection

🕹 Upgrade Challenge

> *Add an animated GIF-aware version that writes each frame as a separate ASCII art block.*

Project 89: Cipher Encoder & Decoder

Difficulty: ☆☆

What you will build

Encode and decode messages with Caesar, Vigenère and Atbash ciphers.

Skills you will learn

- String transformation
- Algorithms

Features

- Caesar shift
- Vigenère keyword
- Atbash
- Direction toggle (encode/decode)

Build it with AI — the prompt to paste

> *Build 'Cipher Encoder & Decoder'. Pick cipher type, input text and (where required) a keyword. Output appears live. Caesar with a slider for shift, Vigenère with a key, Atbash is automatic. Pure JS, single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Cipher</title>  6    <style>  7      body { font-family: system-ui; max-
width: 720px; margin: 30px auto; padding: 20px; }  8      h1 { color: #7c2d12; }
9      select, input, button, textarea { padding: 8px; border-radius: 6px;
border: 1px solid #cbd5e1; font: inherit; width: 100%; box-sizing: border-box; }
10      button { background: #b45309; color: white; border: 0; cursor: pointer;
width: auto; } 11      textarea { min-height: 80px; font-family: ui-monospace,
monospace; } 12      label { display: block; margin: 10px 0 4px; font-weight:
600; } 13    </style> 14  </head> 15  <body> 16    <h1>🔐 Cipher Tool</h1> 17
<label>Cipher 18      <select id="kind"> 19        <option value="caesar">Caesar
(shift)</option> 20        <option value="vigenere">Vigenère (key)</option> 21
<option value="atbash">Atbash</option> 22      </select> 23    </label> 24
<div id="extra"></div> 25    <label>Plaintext 26      <textarea
id="plain"></textarea> 27    </label> 28    <label>Ciphertext 29      <textarea
id="out" readonly></textarea> 30    </label> 31   32    <script> 33      const
extra = document.getElementById("extra"); 34      function build() { 35
extra.innerHTML = ""; 36        if (document.getElementById("kind").value ===
"caesar") { 37          extra.innerHTML = '<label>Shift <input id="shift"
type="number" value="3" min="-25" max="25"></label>'; 38        } else if
(document.getElementById("kind").value === "vigenere") { 39
extra.innerHTML = '<label>Keyword <input id="key" value="LEMON"></label>'; 40
} 41        extra.querySelectorAll("input").forEach(inp =>
inp.addEventListener("input", encode)); 42      } 43      function encode() { 44
const kind = document.getElementById("kind").value; 45        const text =
document.getElementById("plain").value; 46        let out = ""; 47        if
(kind === "caesar") { 48          const s =
parseInt(document.getElementById("shift").value) || 0; 49          out =
text.replace(/[a-zA-Z]/g, c => { 50            const base = c <= "Z" ? 65 : 97;
  1            return String.fromCharCode((c.charCodeAt(0) - base + s + 26 * 10)
% 26 + base);  2          });  3        } else if (kind === "vigenere") {  4
```

```
const key = (document.getElementById("key").value ||
"key").toUpperCase().replace(/[^A-Z]/g, "");  5          let i = 0;  6
out = text.replace(/[a-zA-Z]/g, c => {  7            const base = c <= "Z" ?
65 : 97;  8            const k = key.charCodeAt(i++ % key.length) - 65;  9
return String.fromCharCode((c.charCodeAt(0) - base + k + 26 * 10) % 26 + base);
10          }); 11        } else out = text.replace(/[a-zA-Z]/g, c => { 12
const base = c <= "Z" ? 65 : 97; 13          return String.fromCharCode(25 -
(c.charCodeAt(0) - base) + base); 14        }); 15
document.getElementById("out").value = out; 16      } 17
document.getElementById("kind").addEventListener("change", () => { build();
encode(); }); 18      document.getElementById("plain").addEventListener("input",
encode); 19      build(); encode(); 20    </script> 21  </body> 22  </html>
```

File structure

```
single file: cipher.html
```

How to run it

Open in any browser.

How it works

All three ciphers are alphabetic substitutions. Caesar is just (char + shift) mod 26; Vigenère uses a repeated key for the shift per character; Atbash maps A↔Z, B↔Y, etc.

Customization ideas

- Add Base64 encode/decode
- Add ROT13 quick toggle
- Add a frequency-analysis breaker

🕹 Upgrade Challenge

> *Build a 'cipher race' where two players try to send a message and decode the other's in 60 seconds.*

Project 90: Random Story Generator

Difficulty: ☆☆

What you will build

Combine templated story pieces with random character names to generate endless mock short stories.

Skills you will learn

- Templates
- Random selection

Features

- 5 story templates
- Multi-paragraph output
- Save to localStorage as favourites
- Word count

Build it with AI — the prompt to paste

> *Build 'Random Story Generator'. Five templates with placeholders like {{hero}}, {{villain}}, {{place}}. A library of each placeholder type with at least 10 entries. Click 'Generate' to fill in placeholders. Save favourites. Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Story Gen</title>  6    <style>  7      body { font-family: 'Georgia',
serif; max-width: 600px; margin: 30px auto; padding: 20px; background:
#fffbeb; }  8      h1 { color: #92400e; }  9      button { padding: 10px 18px;
background: #b45309; color: white; border: 0; border-radius: 6px; cursor:
pointer; margin: 6px 4px; } 10      .story { background: white; padding: 20px;
border-radius: 10px; box-shadow: 0 2px 8px rgba(0,0,0,.06); margin: 14px 0;
line-height: 1.7; } 11      .favs { font-family: system-ui; } 12      .favs li {
background: white; padding: 10px; margin: 4px 0; border-radius: 6px; } 13
</style> 14  </head> 15  <body> 16    <h1>□ Random Story</h1> 17    <button
onclick="gen()">Generate</button> 18    <button onclick="fav()">★ Save</button>
19    <div class="story" id="st">Click Generate to begin.</div> 20    <ul
class="favs" id="favs"></ul> 21   22    <script> 23      const P = { 24
hero:  ["Mira","Jules","Pip","Anya","Ren","Sora","Tomas","Lena","Ilya","Dax"],
25        trait:
["curious","stubborn","quiet","brave","gentle","sharp","restless","patient","wit
ty","stoic"], 26        place: ["in a foggy village","on a hilltop fort","inside
a forest","in a desert library","near a frozen lake","aboard a hot-air
balloon","in a neon city","on a small island","in an ancient temple","inside a
quiet bookstore"], 27        prop:  ["a brass key","a torn map","a brass
compass","an old photograph","a single candle","a small robot","a humming
bell","a wooden flute","a green ribbon","a cracked locket"], 28        goal:
["find a missing friend","solve a strange sound in the night","deliver a quiet
letter","discover the truth behind a song","return home before sunset","uncover
a hidden name","learn a forgotten language","translate a sign","rescue a small
bird","decide which path is real"] 29      }; 30      const T = [ 31
"{{hero}} was {{trait}}. One morning, {{hero}} woke in {{place}}, clutching
```

```
{{prop}} and remembering only one thing: to {{goal}}.", 32        "It started as
a small thing. {{hero}} had been {{trait}}, day after day, until {{place}} broke
the pattern. A single {{prop}} lay waiting. It was time to {{goal}}.", 33
"There was a story people told about {{place}}. It was about {{hero}}, who was
{{trait}}, hiding {{prop}} and about to {{goal}}. Most people believed it.
{{hero}} did not.", 34        "{{hero}} did not ask to be {{trait}}. But when
{{prop}} appeared and {{place}} grew quiet, the only thing left was to {{goal}}.
So {{hero}} did.", 35        "On the last night of August, {{hero}} arrived
{{place}} with {{prop}} under one arm and a single word written on it:
'{{goal}}'. This is what happened next." 36      ]; 37      function fill(tpl) {
38        return tpl.replace(/\{\{(\w+)\}\}/g, (m, key) => { 39          const
list = P[key]; 40          return list[Math.floor(Math.random() * list.length)];
41        }); 42      } 43      let cur = ""; 44      function gen() { 45
cur = T[Math.floor(Math.random() * T.length)]; 46        cur = fill(cur) + "\n\
n" + fill(T[Math.floor(Math.random() * T.length)]); 47
document.getElementById("st").textContent = cur; 48      } 49      window.gen =
gen; 50      window.fav = () => {
  1        const favs = JSON.parse(localStorage.getItem("story-favs") || "[]");
2        favs.push(cur); localStorage.setItem("story-favs",
JSON.stringify(favs));  3        renderFavs();  4      };  5      function
renderFavs() {  6        const favs = JSON.parse(localStorage.getItem("story-
favs") || "[]");  7        document.getElementById("favs").innerHTML =
favs.map((s, i) =>  8          `<li>${s.replace(/\n/g, "<br>")} <a href="#"
onclick="localStorage.setItem('story-favs',
JSON.stringify(JSON.parse(localStorage.getItem('story-favs')).filter((_,j)=>j!
==${i}))); renderFavs(); return false"
style="float:right;color:#dc2626">×</a></li>`).join("");  9      } 10
gen(); renderFavs(); 11    </script> 12  </body> 13  </html>
```

File structure

```
single file: story.html
```

How to run it

Open in any browser.

How it works

Each placeholder in the template is replaced with a random pick from a parallel list. Composing two templates in a row builds an opening and a continuation. localStorage stores favourites as a JSON array.

Customization ideas

- Add character names from a fantasy name generator
- Add complexity levels
- Add a 'twist' widget

🕹 Upgrade Challenge

> *Add a Markov-chain extension trained on user favourites to learn their style.*

Project 91: Meme Generator

Difficulty: ☆☆

What you will build

Upload an image and place top and bottom text with classic impact font. Save as PNG.

Skills you will learn

- Canvas image manipulation
- Text wrapping

Features

- Upload any image
- Top and bottom text inputs
- Automatic word wrap
- PNG download

Build it with AI — the prompt to paste

> *Build 'Meme Generator'. User uploads an image, types top and bottom text, sees the meme on canvas, can download as PNG. Text is the classic Impact-style with a black outline. Pure JS canvas, single file.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Meme Maker</title>  6    <style>  7      body { font-family: system-ui;
max-width: 520px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#be185d; }  9      input, button { padding: 8px 12px; border-radius: 6px;
border: 1px solid #cbd5e1; font: inherit; margin: 4px 0; } 10      button
{ background: #be185d; color: white; border: 0; cursor: pointer; } 11
canvas { background: #f1f5f9; max-width: 100%; border-radius: 8px; } 12
</style> 13  </head> 14  <body> 15    <h1>😂 Meme Maker</h1> 16    <input
type="file" id="f" accept="image/*"> 17    <input id="top" placeholder="Top
text"> 18    <input id="bottom" placeholder="Bottom text"> 19    <button
onclick="dl()">Download PNG</button> 20    <p><canvas id="cv" width="500"
height="500"></canvas></p> 21   22    <script> 23      const cv =
document.getElementById("cv"); 24      const ctx = cv.getContext("2d"); 25
let img = null; 26      document.getElementById("f").addEventListener("change",
e => { 27        const u = URL.createObjectURL(e.target.files[0]); 28        img
= new Image(); 29        img.onload = () => { 30          const max = 500; 31
const ratio = Math.min(max / img.width, max / img.height); 32          cv.width
= img.width * ratio; cv.height = img.height * ratio; 33          render(); 34
}; 35        img.src = u; 36      }); 37      function wrap(text, maxW) { 38
const words = text.split(/\s+/); 39        let line = "", lines = []; 40
words.forEach(w => { 41          const test = line ? line + " " + w : w; 42
if (ctx.measureText(test).width > maxW && line) { lines.push(line); line = w; }
43          else line = test; 44        }); 45        if (line)
lines.push(line); 46        return lines; 47      } 48      function render()
{ 49        ctx.drawImage(img, 0, 0, cv.width, cv.height); 50        const
drawText = (text, y) => {
  1          if (!text) return;  2          const fontSize = Math.max(28,
cv.width / 12);  3          ctx.font = `bold ${fontSize}px Impact, sans-serif`;
4          ctx.fillStyle = "white"; ctx.strokeStyle = "black"; ctx.lineWidth =
```

```
4;  5          ctx.textAlign = "center";  6          const lines =
wrap(text.toUpperCase(), cv.width - 20);  7          lines.forEach((l, i) =>
{  8            const ly = y + i * fontSize * 1.1;  9
ctx.strokeText(l, cv.width / 2, ly); 10            ctx.fillText(l, cv.width / 2,
ly); 11          }); 12        }; 13
drawText(document.getElementById("top").value, 10); 14        const fontSize =
Math.max(28, cv.width / 12); 15        const bLines =
wrap((document.getElementById("bottom").value || "").toUpperCase(), cv.width -
20); 16        drawText(document.getElementById("bottom").value, cv.height -
bLines.length * fontSize * 1.1 - 10); 17      } 18
document.getElementById("top").addEventListener("input", render); 19
document.getElementById("bottom").addEventListener("input", render); 20
function dl() { 21        const a = document.createElement("a"); a.download =
"meme.png"; a.href = cv.toDataURL(); a.click(); 22      } 23    </script> 24
</body> 25  </html>
```

File structure

```
single file: meme.html
```

How to run it

Open in any browser.

How it works

Impact's thick strokes make outlining cheap: just strokeText() then fillText() with white. Word-wrap is a 10-line function that breaks lines when ctx.measureText returns more than the canvas width.

Customization ideas

- Add meme templates (Distracted Boyfriend...)
- Add text drag
- Add filters

🕹 Upgrade Challenge

> *Add multi-line text with manual positioning.*

Project 92: Data Dashboard

Difficulty: ☆☆☆

What you will build

A single-page dashboard with bar chart, line chart, pie chart and a stat counter — all from mock JSON.

Skills you will learn

- Data visualisation
- Canvas charts
- Mock data

Features

- Mock JSON dataset
- Bar, line, and pie charts
- Live filtering
- Stats

Build it with AI — the prompt to paste

> *Build a 'Data Dashboard'. Load a mock JSON dataset of weekly app usage. Render 3 charts on canvas: bar (visits per day), line (cumulative users over time), pie (share by feature). Add a stat counter showing total, average and peak. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Dashboard</title>  6    <style>  7      body { font-family: system-ui;
max-width: 980px; margin: 20px auto; padding: 20px; background: #f1f5f9; }  8
h1 { color: #1e293b; }  9      .grid { display: grid; grid-template-columns:
repeat(2, 1fr); gap: 14px; } 10      .card { background: white; padding: 16px;
border-radius: 10px; box-shadow: 0 1px 3px rgba(0,0,0,.06); } 11      canvas
{ background: #f8fafc; border-radius: 6px; } 12      .stats { display: grid;
grid-template-columns: repeat(3, 1fr); gap: 10px; } 13      .stat { padding:
12px; text-align: center; background: #f1f5f9; border-radius: 8px; }
14      .stat b { font-size: 24px; color: #1d4ed8; display: block; } 15
button { padding: 8px 14px; background: #2563eb; color: white; border: 0;
border-radius: 6px; cursor: pointer; margin: 4px 0; } 16    </style> 17  </head>
18  <body> 19    <h1>📊 Data Dashboard</h1> 20    <button onclick="load()">Load
mock dataset</button> 21    <div class="stats" id="stats"></div> 22    <div
class="grid"> 23      <div class="card"><h3>Visits per day</h3><canvas id="bar"
width="440" height="200"></canvas></div> 24      <div
class="card"><h3>Cumulative users</h3><canvas id="line" width="440"
height="200"></canvas></div> 25      <div class="card"><h3>Feature usage
share</h3><canvas id="pie" width="440" height="200"></canvas></div> 26      <div
class="card"><h3>Per-feature totals</h3><div id="perft"></div></div> 27
</div> 28   29    <script> 30      // Mock dataset of weekly activity across
features 31      const MOCK = { 32        days:
["Mon","Tue","Wed","Thu","Fri","Sat","Sun"], 33        visits: [120, 198, 240,
175, 220, 280, 305], 34        cumulative: [120, 318, 558, 733, 953, 1233,
1538], 35        features: [ 36          { name: "Search",    usage: 420 }, 37
{ name: "Editor",    usage: 360 }, 38          { name: "Playlists", usage:
```

```
220 }, 39          { name: "Settings",  usage: 90 }, 40          { name:
"Other",     usage: 60 } 41        ] 42      }; 43   44      function load()
{ 45        const stats = document.getElementById("stats"); 46        const
total = MOCK.visits.reduce((a,b)=>a+b,0); 47        const avg = Math.round(total
/ MOCK.visits.length); 48        const peak = Math.max(...MOCK.visits); 49
stats.innerHTML = `<div class="stat"><b>${total}</b>Total visits</div> 50
<div class="stat"><b>${avg}</b>Average per day</div>
  1          <div class="stat"><b>${peak}</b>Peak day</div>`;  2    3
barChart("bar", MOCK.days, MOCK.visits, "#3b82f6");  4        lineChart("line",
MOCK.days, MOCK.cumulative, "#16a34a");  5        pieChart("pie",
MOCK.features);  6        document.getElementById("perft").innerHTML =
MOCK.features.map(f =>  7          `<div style="display:flex;justify-
content:space-between;padding:6px 0;border-bottom:1px solid #e2e8f0">  8
<span>${f.name}</span><span><b>${f.usage}</b></span></div>`).join("");  9      }
10      function ctx(id) { return
document.getElementById(id).getContext("2d"); } 11      function barChart(id,
labels, values, color) { 12        const c = ctx(id); c.clearRect(0,0,440,200);
13        const max = Math.max(...values); 14        const bw = 380 /
values.length; 15        values.forEach((v, i) => { 16          const h = (v /
max) * 140; 17          c.fillStyle = color; c.fillRect(30 + i * bw, 170 - h, bw
- 8, h); 18          c.fillStyle = "#475569"; c.font = "11px sans-serif";
c.fillText(labels[i], 30 + i * bw, 188); 19          c.fillText(v, 30 + i * bw,
170 - h - 4); 20        }); 21      } 22      function lineChart(id, labels,
values, color) { 23        const c = ctx(id); c.clearRect(0,0,440,200); 24
c.strokeStyle = color; c.lineWidth = 2; c.beginPath(); 25
values.forEach((v, i) => { 26          const x = 30 + i * (380 / (values.length
- 1)); 27          const y = 170 - (v / Math.max(...values)) * 140; 28
i ? c.lineTo(x, y) : c.moveTo(x, y); 29        }); 30        c.stroke(); 31
c.fillStyle = color; values.forEach((v, i) => { 32          const x = 30 + i *
(380 / (values.length - 1)); 33          const y = 170 - (v /
Math.max(...values)) * 140; 34          c.beginPath(); c.arc(x, y, 3, 0, 7);
c.fill(); 35        }); 36        c.fillStyle = "#475569"; c.font = "11px sans-
serif"; 37        labels.forEach((l, i) => c.fillText(l, 30 + i * (380 /
(values.length - 1)) - 8, 188)); 38      } 39      function pieChart(id, feats)
{ 40        const c = ctx(id); c.clearRect(0,0,440,200); 41        const cols =
["#3b82f6","#16a34a","#f59e0b","#ef4444","#8b5cf6"]; 42        const tot =
feats.reduce((s,f)=>s+f.usage,0); 43        let a0 = -.5 * Math.PI; 44
feats.forEach((f, i) => { 45          const a = (f.usage / tot) * 2 * Math.PI;
46          c.beginPath(); c.moveTo(110, 100); c.arc(110, 100, 80, a0, a0 + a);
c.closePath(); 47          c.fillStyle = cols[i]; c.fill(); 48          a0 += a;
49        }); 50        c.fillStyle = "#475569"; c.font = "12px sans-serif";
  1        c.fillText("Total: " + tot, 60, 180);  2        // legend  3
feats.forEach((f, i) => { c.fillStyle = cols[i]; c.fillRect(220, 30 + i * 22,
12, 12); c.fillText(f.name + " (" + f.usage + ")", 240, 40 + i * 22); });  4
}  5    </script>  6  </body>  7  </html>
```

File structure

```
single file: dashboard.html
```

How to run it

Open in any browser.

How it works

Each chart is hand-rolled with canvas: bar uses rect heights scaled to max, line uses lineTo with small circles at data points, pie uses arcs. We deliberately avoid a charting library so the reader sees the math.

Customization ideas

- Add a CSV import
- Add date range filter
- Add tooltip on hover

🕹 Upgrade Challenge

> *Replace mock data with fetch() to a public dataset like GitHub's REST API.*

Project 93: Snakes & Ladders Game

Difficulty: ☆☆

What you will build

The classic Snakes & Ladders board with two players, dice rolls, and animated movement.

Skills you will learn

- Game state
- Animation

Features

- 10x10 board
- Two-player
- Dice roller
- Animated moves
- Win detection

Build it with AI — the prompt to paste

> *Build 'Snakes & Ladders' board game. Two players roll a die in turn. Snakes slide you down, ladders lift you up. Animated moves. Pure JS canvas + DOM. Single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Snakes & Ladders</title>  6    <style>  7      body { font-family:
system-ui; max-width: 580px; margin: 30px auto; padding: 20px; text-align:
center; }  8      h1 { color: #be185d; }  9      .board { display: grid; grid-
template-columns: repeat(10, 50px); grid-template-rows: repeat(10, 50px); gap:
2px; margin: 16px auto; background: #1e293b; padding: 4px; border-radius: 8px; }
10      .cell { background: white; display: flex; align-items: center; justify-
content: center; font-size: 11px; color: #64748b; position: relative; }
11      .snake { background: #fecaca; } .ladder { background: #bbf7d0; } 12
.players { position: absolute; bottom: 4px; left: 4px; font-size: 16px; } 13
.red { background: #dc2626; border-radius: 50%; width: 12px; height: 12px;
position: absolute; bottom: 6px; left: 6px; } 14      .blue { background:
#2563eb; border-radius: 50%; width: 12px; height: 12px; position: absolute;
bottom: 6px; left: 22px; } 15      button { padding: 12px 24px; background:
#be185d; color: white; border: 0; border-radius: 6px; cursor: pointer; } 16
.die { font-size: 48px; margin: 12px; } 17      .log { background: #f8fafc;
padding: 8px; border-radius: 6px; height: 80px; overflow: auto; } 18    </style>
19  </head> 20  <body> 21    <h1>□ Snakes & Ladders 🐍</h1> 22    <div
class="board" id="b"></div> 23    <div class="die" id="die">□</div> 24
<button id="roll">Roll dice</button> 25    <p>Turn: <span
id="turn">Red</span></p> 26    <div class="log" id="log"></div> 27   28
<script> 29      const SNAKES = { 17: 4, 19: 7, 21: 9, 27: 1, 35: 22, 51: 30,
73: 53, 76: 58, 92: 51, 97: 79 }; 30      const LADDERS = { 4: 25, 9: 31, 16:
47, 28: 56, 39: 80, 49: 93, 62: 81, 70: 92, 71: 91 }; 31      let pos = { red:
1, blue: 1 }, turn = "red", gameOver = false; 32      const b =
document.getElementById("b"); 33      function build() { 34        b.innerHTML =
""; 35        // snake-and-ladders squares are 1..100 numbered; render bottom
```

```
row first 36        for (let row = 9; row >= 0; row--) { 37          for (let
col = 0; col < 10; col++) { 38            let c = document.createElement("div");
39            c.className = "cell"; 40            if ((row % 2 === 0))
c.classList.add("snake"); // visual hint 41            const num = row * 10 +
(row % 2 ? 10 - col : col + 1); 42            c.textContent = num; 43
c.dataset.n = num; 44            let red = document.createElement("div");
red.className = "red"; red.dataset.p = "red"; 45            let blue =
document.createElement("div"); blue.className = "blue"; blue.dataset.p = "blue";
46            c.appendChild(red); c.appendChild(blue); 47
b.appendChild(c); 48          } 49        } 50        paint();
  1      }  2      function paint() {  3        document.querySelectorAll(".red,
.blue").forEach(e => e.style.display = "none");  4        const rc =
document.querySelector(`[data-n="${pos.red}"] .red`);  5        const bc =
document.querySelector(`[data-n="${pos.blue}"] .blue`);  6        if (rc)
rc.style.display = "block";  7        if (bc) bc.style.display = "block";  8
}  9      function log(msg) { const el = document.getElementById("log");
el.innerHTML = msg + "<br>" + el.innerHTML; } 10      function roll() { 11
if (gameOver) return; 12        const die = 1 + Math.floor(Math.random() * 6);
13        document.getElementById("die").textContent = ["⚀","⚁","⚂","⚃","⚄","⚅"]
[die - 1]; 14        const cur = pos[turn]; 15        let next = cur + die; if
(next > 100) next = cur; 16        if (LADDERS[next]) { log(`${turn} climbed
ladder from ${next} to ${LADDERS[next]}!`); next = LADDERS[next]; } 17
else if (SNAKES[next]) { log(`${turn} hit a snake from ${next} down to $
{SNAKES[next]}`); next = SNAKES[next]; } 18        pos[turn] = next; 19
animate(cur, next, () => { 20          paint(); 21          if (next === 100)
{ gameOver = true; log(turn.toUpperCase() + " WINS! 🎉"); return; } 22
turn = turn === "red" ? "blue" : "red"; 23
document.getElementById("turn").textContent = turn === "red" ? "Red" : "Blue";
24        }); 25      } 26      function animate(from, to, done) { 27        if
(from === to) return done(); 28        let s = from + 1; 29        const id =
setInterval(() => { 30          pos[turn] = s; 31          paint(); 32
if (s === to) { clearInterval(id); done(); } 33          s++; 34        }, 120);
35      } 36      document.getElementById("roll").addEventListener("click",
roll); 37      build(); 38    </script> 39  </body 40  </html>
```

File structure

```
single file: snakes.html
```

How to run it

Open in any browser.

How it works

Snakes are a dictionary from start squares to end squares. We use a setInterval to animate moving through squares one at a time, then either land, ladder-up or snake-down.

Customization ideas

- Add AI for one-player mode
- Add multi-dice variant
- Add a 'safe squares' rule

🕹 Upgrade Challenge

> *Build a custom board editor for class use, importing snakes and ladders from JSON.*

Project 94: Mini Search Engine

Difficulty: ☆☆☆

What you will build

Index a small text dataset and search it for a query with ranking.

Skills you will learn

- Indexing
- TF-IDF basics
- Search algorithms

Features

- Index 10 mocked articles
- Tokenize and rank
- Highlight matches
- Top results

Build it with AI — the prompt to paste

> *Build 'Mini Search Engine'. Mock dataset of 10 short articles. Tokenize each, build a TF-IDF index. User enters a query; calculate matching score, return ranked results with snippets. Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Mini Search</title>  6    <style>  7      body { font-family: system-ui;
max-width: 760px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#1d4ed8; }  9      input { width: 100%; padding: 10px; font-size: 16px; border-
radius: 6px; border: 1px solid #cbd5e1; box-sizing: border-box; }
10      .result { background: white; padding: 14px; border-radius: 8px; margin:
8px 0; box-shadow: 0 1px 3px rgba(0,0,0,.06); } 11      .result h3 { margin: 0;
color: #1e3a8a; } 12      mark { background: #fde68a; padding: 0 2px; border-
radius: 3px; } 13    </style> 14  </head> 15  <body> 16    <h1>🔍 Mini
Search</h1> 17    <p>Indexed 10 short articles. Try: <em>javascript, snake,
painting, drawing</em>.</p> 18    <input id="q" placeholder="Search..."
autocomplete="off"> 19    <div id="out"></div> 20   21    <script> 22      const
DOCS = [ 23        { id: 1,  title: "How I started coding",          body: "I
began with simple HTML pages. JavaScript felt strange at first, then suddenly
obvious. The first 100 hours matter more than the next 1000." }, 24        { id:
2,  title: "Drawing what you see",          body: "A pencil, a window, and
patience. That is the secret to drawing. Look at the shape, not the concept." },
25        { id: 3,  title: "Building a snake game",         body: "Snake is the
perfect first game. You learn game loops, grid coordinates, collision detection
and arrow-key handling all in one weekend." }, 26        { id: 4,  title: "Why I
switched to single-page apps", body: "My old site had seven HTML files.
Switching to a single-page app with hash routing saved clicks and code. The
hardest part was reorganising the buttons." }, 27        { id: 5,  title:
"Watercolour painting basics",    body: "Painting with watercolours feels
forgiving. Layer wet onto wet and let the pigment bloom. Practise gradients for
an hour a day." }, 28        { id: 6,  title: "Cooking rice without a recipe",
body: "Two cups water, one cup rice, a pinch of salt, a tight lid. Twelve
minutes on low. Done. Taste, adjust, taste again." }, 29        { id: 7,  title:
"Learning JavaScript closures",  body: "Closures in JavaScript are functions
```

```
that remember variables from where they were defined. They are everywhere once
you see them." }, 30        { id: 8,  title: "Reading habit",
body: "I read a chapter every morning before screens. Less than 30 minutes a day
still adds up to twelve books a year." }, 31        { id: 9,  title: "My first
drawing",              body: "My first drawing was a coffee cup. Not great, but
a starting point. A year later the same cup looked like a cup." }, 32
{ id: 10, title: "Running a side-project marathon", body: "Side projects
compound. Small features every week. Twelve months later, you have something to
share." } 33      ]; 34      const STOPWORDS = new Set("a the and of in on at to
is are was were be by".split(" ")); 35      function tokenize(s) { return
s.toLowerCase().replace(/[^a-z0-9\s]/g," ").split(/\s+/).filter(w => w && !
STOPWORDS.has(w)); } 36   37      const index = {}; 38      const docLen = [];
39      DOCS.forEach((d, i) => { 40        const tok = tokenize(d.title + " " +
d.body); 41        docLen[i] = tok.length; 42        tok.forEach(t => index[t] =
(index[t] || []).concat(i)); 43      }); 44   45      function snippet(doc, q) {
46        const tw = q.toLowerCase().split(/\s+/).filter(Boolean); 47        for
(const w of tw) { 48          const i = doc.body.toLowerCase().indexOf(w); 49
if (i >= 0) return doc.body.slice(Math.max(0, i - 30), i + w.length + 60); 50
}
  1        return doc.body.slice(0, 100);  2      }  3      function
highlight(s, q) {  4        const tw = q.split(/\s+/).filter(Boolean);  5
let out = s;  6        tw.forEach(w => { out = out.replace(new RegExp("(" + w +
")", "gi"), "<mark>$1</mark>"); });  7        return out;  8      }  9
function search() { 10        const q =
document.getElementById("q").value.trim(); 11        if (!q)
{ document.getElementById("out").innerHTML = ""; return; } 12        const tw =
tokenize(q); 13        const scores = Array(DOCS.length).fill(0); 14
tw.forEach(t => (index[t] || []).forEach(d => { scores[d] += 1 /
Math.sqrt(docLen[d]); })); 15        const ranked = scores.map((s, i) => ({ s,
d: DOCS[i], i })) 16          .filter(x => x.s > 0).sort((a, b) => b.s -
a.s).slice(0, 5); 17        const out = document.getElementById("out"); 18
if (!ranked.length) { out.innerHTML = "<p>No results.</p>"; return; } 19
out.innerHTML = ranked.map(r => 20          `<div class="result"><h3>$
{highlight(r.d.title, q)}</h3> 21            <p style="color:#475569">$
{highlight(snippet(r.d, q), q)}...</p> 22            <small>score $
{r.s.toFixed(2)}</small></div>`).join(""); 23      } 24
document.getElementById("q").addEventListener("input", search); 25    </script>
26  </body 27  </html>
```

File structure

```
single file: search.html
```

How to run it

Open in any browser.

How it works

Each word in any document gets added to the index pointing at the document id. Search score is just 1 ÷ sqrt(document length), a tiny TF-IDF approximation. Sorting by score gives ranked results.

Customization ideas

- Add phrase search
- Add boolean operators
- Score by inverse document frequency

🕹 Upgrade Challenge

> *Use the PageRank algorithm on link data to order results.*

Project 95: Recipe Randomizer

Difficulty: ☆☆

What you will build

Randomly combine a protein, a method, a vegetable, and a sauce into a fresh dinner idea.

Skills you will learn

- Generative combinations
- Favourites

Features

- Curated lists of proteins, methods, vegetables, sauces
- Save the recipes you like
- Shuffle button

Build it with AI — the prompt to paste

> *Build 'Recipe Randomizer'. Four lists: protein (chicken, lentils, tofu...), method (stir-fry, slow cook, bake, raw...), vegetable (spinach, peppers, carrots...), sauce (soy+ginger, tahini, tomato). Random picks generate a dinner title. Save favourites. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Dinner Idea</title>  6    <style>  7      body { font-family: system-ui;
max-width: 600px; margin: 40px auto; padding: 20px; text-align: center;
background: #fff7ed; }  8      h1 { color: #c2410c; }  9      .recipe
{ background: white; padding: 24px; border-radius: 14px; box-shadow: 0 6px 20px
rgba(0,0,0,.06); margin: 16px 0; font-size: 22px; } 10      button { padding:
12px 24px; background: #c2410c; color: white; border: 0; border-radius: 6px;
cursor: pointer; font-size: 16px; margin: 6px; } 11      ul { list-style: none;
padding: 0; } 12      ul li { background: white; padding: 10px; margin: 4px 0;
border-radius: 6px; } 13    </style> 14  </head> 15  <body> 16    <h1>🍽 What to
Eat?</h1> 17    <button onclick="go()">Surprise me!</button> 18    <button
onclick="save()">★ Save</button> 19    <div class="recipe" id="r">Click the
button.</div> 20    <h3>Saved favourites</h3> 21    <ul id="favs"></ul> 22   23
<script> 24      const PROTEIN =
["chicken","salmon","prawns","lentils","tofu","chickpeas","eggs","pork","beef","
mushrooms"]; 25      const METHOD =  ["stir-fried","slow-
cooked","baked","grilled","raw","curry","roasted","braised","steamed","sheet-
pan"]; 26      const VEG =
["spinach","peppers","carrots","kale","broccoli","mushrooms","courgettes","sweet
potato","tomatoes","asparagus"]; 27      const SAUCE =   ["soy +
ginger","tahini","tomato + garlic","lemon + dill","coconut + lime","sriracha +
honey","pesto","chimichurri","yoghurt + cumin","harissa"]; 28      const pick =
arr => arr[Math.floor(Math.random() * arr.length)]; 29      let last = ""; 30
function go() { 31        const p = pick(PROTEIN), m = pick(METHOD), v =
pick(VEG), s = pick(SAUCE); 32        last = `${m[0].toUpperCase()+m.slice(1)} $
{p} with ${v}, finished with ${s} sauce.`; 33
document.getElementById("r").textContent = last; 34      } 35      window.go =
go; 36      window.save = () => { 37        const f =
JSON.parse(localStorage.getItem("dinner-favs") || "[]"); 38        f.push(last);
```

```
localStorage.setItem("dinner-favs", JSON.stringify(f)); 39        paintFavs();
40      }; 41      function paintFavs() { 42        const f =
JSON.parse(localStorage.getItem("dinner-favs") || "[]"); 43
document.getElementById("favs").innerHTML = f.map((s, i) => 44          `<li>$
{s} <a href="#" onclick="localStorage.setItem('dinner-favs',
JSON.stringify(JSON.parse(localStorage.getItem('dinner-favs')).filter((_,j)=>j!
==${i}))); paintFavs(); return false"
style="float:right;color:#dc2626">×</a></li>`).join(""); 45      } 46      go();
paintFavs(); 47    </script> 48  </body> 49  </html>
```

File structure

```
single file: dinner.html
```

How to run it

Open in any browser.

How it works

Just four parallel arrays and a single pick helper. The combinations are vast: 10 × 10 × 10 × 10 = 10,000 distinct dinners from this minimum data.

Customization ideas

- Add cuisine filter (Indian, Italian, etc.)
- Add ingredient-on-hand filter
- Add a shopping list view

🕹 Upgrade Challenge

> *Add a 'swipe-style' discovery flow that asks the user to swipe left or right on each idea.*

Project 96: Decision Spinner (Weighted)

Difficulty: ☆☆

What you will build

Spin a wheel where you define the options and the chance of landing on each.

Skills you will learn

- Weighted probabilities
- Animation

Features

- Add weighted options
- Spin animation
- Show result history

Build it with AI — the prompt to paste

> *Build 'Decision Spinner (Weighted)'. User enters a list of options each with a probability weight (positive integers). Spin picks a winner. Animation slows to a stop. Pure JS canvas.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-28">  5
<title>Decision Spinner</title>  6    <style>  7      body { font-family:
system-ui; max-width: 540px; margin: 30px auto; padding: 20px; }  8      h1
{ color: #0ea5e9; }  9      input { padding: 8px; border: 1px solid #cbd5e1;
border-radius: 6px; font: inherit; } 10      .row { display: grid; grid-
template-columns: 2fr 1fr auto; gap: 6px; margin: 4px 0; } 11      button
{ padding: 8px 12px; background: #0ea5e9; color: white; border: 0; border-
radius: 6px; cursor: pointer; } 12      .wheel-wrap { display: flex; justify-
content: center; margin: 20px 0; position: relative; } 13      .arrow { width:
0; height: 0; border-left: 24px solid #0f172a; border-top: 12px solid
transparent; border-bottom: 12px solid transparent; position: absolute; top:
50%; right: -4px; transform: translateY(-50%); } 14      canvas { border-radius:
50%; } 15      .result { font-size: 22px; text-align: center; min-height: 40px;
font-weight: 700; } 16      h3 { margin-top: 24px; } 17    </style> 18  </head>
19  <body> 20    <h1>□ Decision Spinner</h1> 21    <div id="opts"></div> 22
<button onclick="addOpt()">+ Option</button> 23    <div class="wheel-
wrap"><canvas id="cv" width="320" height="320"></canvas><div
class="arrow"></div></div> 24    <button onclick="spin()">Spin</button> 25    <p
class="result" id="r">Configure your options.</p> 26    <h3>History</h3> 27
<ul id="hist"></ul> 28   29    <script> 30      const cv =
document.getElementById("cv"); 31      const ctx = cv.getContext("2d"); 32
let opts = [{ label: "Yes", w: 1 }, { label: "No", w: 1 }, { label: "Maybe", w:
1 }]; 33      function addOpt() { opts.push({ label: "Option " + (opts.length +
1), w: 1 }); render(); } 34      function render() { 35        const div =
document.getElementById("opts"); 36        div.innerHTML = opts.map((o, i) => 37
`<div class="row"><input value="${o.label}" oninput="opts[$
{i}].label=this.value;draw()"> 38          <input type="number" value="${o.w}"
min="0" oninput="opts[${i}].w=+this.value;draw()"> 39          <button
onclick="opts.splice(${i},1);render()">×</button></div>`).join(""); 40
draw(); 41      } 42      function draw() { 43        const W = cv.width; 44
ctx.clearRect(0,0,W,W); 45        const tot = opts.reduce((s,o)=>s+(o.w||0),0)
```

```
|| 1; 46        let a0 = -.5 * Math.PI; 47        const colours =
["#0ea5e9","#16a34a","#f59e0b","#ef4444","#8b5cf6","#f472b6","#14b8a6","#fb7185"
]; 48        opts.forEach((o, i) => { 49          const a = (o.w / tot) * 2 *
Math.PI; 50          ctx.beginPath(); ctx.moveTo(W/2, W/2); ctx.arc(W/2, W/2,
W/2 - 2, a0, a0 + a); ctx.closePath();
  1          ctx.fillStyle = colours[i % colours.length]; ctx.fill();  2
// label  3          ctx.save(); ctx.translate(W/2, W/2); ctx.rotate(a0 + a/2);
4          ctx.fillStyle = "white"; ctx.font = "bold 16px sans-serif";
ctx.textAlign = "right";  5          ctx.fillText(o.label, W/2 - 14, 6);
ctx.restore();  6          a0 += a;  7        });  8      }  9      let rot = 0;
10      function spin() { 11        const tot = opts.reduce((s,o)=>s+(o.w||
0),0); 12        if (!tot) return; 13        // Pick weighted random 14
let r = Math.random() * tot; 15        let chosen = 0; 16        for (let i = 0;
i < opts.length; i++) { r -= opts[i].w; if (r <= 0) { chosen = i; break; } } 17
// Compute target rotation so that chosen lands at top (-90 deg). 18
const angleForSlice = (i) => { 19          let a0 = 0; 20          for (let j =
0; j < i; j++) a0 += opts[j].w / tot * 360; 21          return a0 + (opts[i].w /
tot) * 180; // mid of slice 22        }; 23        const target = -
(angleForSlice(chosen) - 90) + 1440;  // several full spins 24        rot =
target; 25        cv.style.transition = "transform 4s cubic-bezier(.2,.7,.4,1)";
26        cv.style.transform = `rotate(${rot}deg)`; 27        setTimeout(() => {
28          document.getElementById("r").textContent = "☞ " +
opts[chosen].label; 29          const hist =
JSON.parse(localStorage.getItem("spin-hist") || "[]"); 30
hist.unshift(opts[chosen].label); localStorage.setItem("spin-hist",
JSON.stringify(hist.slice(0, 10))); 31
document.getElementById("hist").innerHTML = hist.map(h =>
`<li>${h}</li>`).join(""); 32        }, 4000); 33      } 34      render(); 35
</script> 36  </body> 37  </html>
```

File structure

```
single file: weighted-spinner.html
```

How to run it

Open in any browser.

How it works

We accumulate weights to total 1. random * total walks through the cumulative sum to find which 'slice' was landed on. The wheel rotation just needs to match that slice's midpoint with the top arrow.

Customization ideas

- Preset dice (1d6, 1d20)
- Import your TODO list as options
- Save and load custom spinners

🕹 Upgrade Challenge

> *Build a 'tournament' mode where the spinner chooses winners bracket-style.*

Project 97: Mini Synthesizer

Difficulty: ☆☆☆

What you will build

Click piano keys or press the row of letters to play notes from a WebAudio synthesizer.

Skills you will learn

- WebAudio
- Keyboard events

Features

- 13 keys (one octave)
- Clickable and keyboard playable
- Adjustable waveform
- Simple envelope

Build it with AI — the prompt to paste

> *Build 'Mini Synthesizer'. A row of 13 piano keys (one octave). Click to play each note or use the A-W-S-E-D-F-T-G-Y-H-U-J row keys. Four waveforms: sine, square, sawtooth, triangle. Pure JS.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-2-55">  5
<title>Mini Synth</title>  6    <style>  7      body { font-family: system-ui;
max-width: 720px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#c026d3; }  9      .kb { display: flex; gap: 4px; margin: 20px 0; } 10      .key
{ width: 44px; height: 140px; background: white; border: 1px solid #cbd5e1;
border-radius: 0 0 6px 6px; cursor: pointer; position: relative; padding-top:
6px; font-size: 11px; 11        text-align: center; color: #475569; 12      } 13
.key.black { background: #1e293b; color: white; height: 90px; margin: 0 -10px;
z-index: 2; } 14      .key.active { background: #facc15; }
15      .key.black.active { background: #fb923c; } 16      .ctrls { display:
flex; gap: 10px; align-items: center; } 17      select, button { padding: 8px
12px; border-radius: 6px; border: 1px solid #cbd5e1; font: inherit; } 18
button { background: #c026d3; color: white; border: 0; cursor: pointer; } 19
</style> 20  </head> 21  <body> 22    <h1>□ Mini Synth</h1> 23    <p>Type the
row of letters ASDFGHJ or click the keys.</p> 24    <div class="ctrls"> 25
<label>Wave: <select id="wv"> 26
<option>sine</option><option>square</option><option>sawtooth</option><option
selected>triangle</option> 27      </select></label> 28      <button
id="rec">Record</button> 29      <button id="play">Play</button> 30    </div> 31
<div class="kb" id="kb"></div> 32   33    <script> 34      // A major through
C5: A, A#, B, C, C#, D, D#, E, F, F#, G, G#, A 35      const NOTES = [ 36
{ name: "A", letter: "a", black: false, freq: 220 }, 37        { name: "A#",
letter: "w", black: true, freq: 233.08 }, 38        { name: "B", letter: "s",
black: false, freq: 246.94 }, 39        { name: "C", letter: "d", black: false,
freq: 261.63 }, 40        { name: "C#", letter: "e", black: true, freq:
277.18 }, 41        { name: "D", letter: "f", black: false, freq: 293.66 }, 42
{ name: "D#", letter: "t", black: true, freq: 311.13 }, 43        { name: "E",
letter: "g", black: false, freq: 329.63 }, 44        { name: "F", letter: "h",
black: false, freq: 349.23 }, 45        { name: "F#", letter: "y", black: true,
```

```
freq: 369.99 }, 46        { name: "G", letter: "j", black: false, freq: 392 },
47        { name: "G#", letter: "u", black: true, freq: 415.30 }, 48
{ name: "A5", letter: "k", black: false, freq: 440 } 49      ]; 50      const kb
= document.getElementById("kb");
  1      NOTES.forEach(n => {  2        const k = document.createElement("div");
3        k.className = "key" + (n.black ? " black" : "");  4
k.dataset.letter = n.letter;  5        k.dataset.freq = n.freq;  6
k.innerHTML = n.name + "<br>" + n.letter.toUpperCase();  7        k.onmousedown
= () => play(n.freq, k);  8        k.onmouseup = () => stop(k);  9
k.onmouseleave = () => stop(k); 10        kb.appendChild(k); 11      }); 12
const osc = {}; // freq -> {osc, gain} 13      function play(freq, key) { 14
const c = new (window.AudioContext || window.webkitAudioContext)(); 15
const o = c.createOscillator(); 16        const g = c.createGain(); 17
o.connect(g); g.connect(c.destination); 18        o.type =
document.getElementById("wv").value; 19        o.frequency.value = freq; 20
g.gain.setValueAtTime(0, c.currentTime); 21
g.gain.linearRampToValueAtTime(0.3, c.currentTime + 0.02); 22        o.start();
23        osc[freq] = { osc: o, gain: g, ctx: c }; 24
key.classList.add("active"); 25        record && record.push({ freq, t:
Date.now() }); 26        releaseOnUp[key] = freq; 27      } 28      const
releaseOnUp = {}; 29      function stop(key) { 30        const f =
releaseOnUp[key]; 31        if (f && osc[f]) { 32          const o = osc[f]; 33
o.gain.gain.linearRampToValueAtTime(0, o.ctx.currentTime + 0.1); 34
o.osc.stop(o.ctx.currentTime + 0.12); 35          delete osc[f]; 36        } 37
key.classList.remove("active"); 38      } 39      let recording = false, record
= null; 40      document.getElementById("rec").onclick = () => { 41        if
(recording) { recording = false; document.getElementById("rec").textContent =
"Record"; return; } 42        recording = true; record = [];
document.getElementById("rec").textContent = "Stop"; 43      }; 44
document.getElementById("play").onclick = () => { 45        if (!record || !
record.length) return; 46        const t0 = record[0].t; 47
record.forEach(ev => setTimeout(() => { 48          const el =
document.querySelector(`[data-freq="${ev.freq}"]`); 49          if (el)
play(ev.freq, el); 50          setTimeout(() => stop(el), 300);
  1        }, ev.t - t0));  2      };  3      addEventListener("keydown", e => {
4        if (e.repeat) return;  5        const k =
document.querySelector(`[data-letter="${e.key.toLowerCase()}"]`);  6        if
(k && !k.classList.contains("active")) play(+k.dataset.freq, k);  7      });  8
addEventListener("keyup", e => {  9        const k =
document.querySelector(`[data-letter="${e.key.toLowerCase()}"]`); 10        if
(k) stop(k); 11      }); 12    </script> 13  </body> 14  </html>
```

File structure

```
single file: synth.html
```

How to run it

Open in any browser.

How it works

Each note is one oscillator with a quick attack ramp (gain 0 → 0.3 in 20ms). Holding the key keeps the note playing; releasing triggers a quick release ramp. The map of letters to notes matches the common ASDF layout that DAWs use.

Customization ideas

- Add effects (reverb, delay)
- Add multiple octaves

- Add presetable sound packs

🕹 Upgrade Challenge

> *Add an arpeggiator that auto-plays held-chord notes in sequence.*

Project 98: Sudoku Solver

Difficulty: ☆☆☆

What you will build

Solve a hard Sudoku puzzle using backtracking. Highlight the steps.

Skills you will learn

- Recursive backtracking
- Constraint solving

Features

- 9x9 grid input
- Backtracking solver
- Step counter
- Sample puzzle

Build it with AI — the prompt to paste

> *Build 'Sudoku Solver'. 9x9 grid of inputs. User fills in a puzzle, clicks Solve, the code runs backtracking (try digits in cells, undo on conflict). Show step count. Sample puzzle included. Pure JS.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Sudoku Solver</title>  6    <style>  7      body { font-family: system-
ui; max-width: 480px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#1e40af; }  9      .g { display: grid; grid-template-columns: repeat(9, 1fr);
gap: 2px; padding: 6px; background: #1e40af; border-radius: 6px; width: 420px; }
10      .g input { width: 100%; aspect-ratio: 1; text-align: center; font-size:
18px; font-weight: 700; border: 0; background: white; padding: 0; box-sizing:
border-box; } 11      .g input.b3 { border-bottom: 2px solid #1e40af; }
12      .g input.r3 { border-right: 2px solid #1e40af; } 13      button
{ padding: 12px 24px; background: #1e40af; color: white; border: 0; border-
radius: 6px; cursor: pointer; margin: 6px; } 14    </style> 15  </head> 16
<body> 17    <h1>□ Sudoku Solver</h1> 18    <p>Blank = empty. Click Solve to
backtrack. Click Sample to load an example.</p> 19    <div class="g"
id="g"></div> 20    <p><button onclick="sample()">Sample puzzle</button><button
onclick="clear()">Clear</button><button onclick="solve()">Solve</button></p> 21
<p id="msg"></p> 22   23    <script> 24      const g =
document.getElementById("g"); 25      for (let r = 0; r < 9; r++) { 26
for (let c = 0; c < 9; c++) { 27          const i =
document.createElement("input"); 28          i.maxLength = 1; i.dataset.r = r;
i.dataset.c = c; 29          if (r % 3 === 2 && r !== 8) i.classList.add("b3");
30          if (c % 3 === 2 && c !== 8) i.classList.add("r3"); 31
g.appendChild(i); 32        } 33      } 34      function grid() { 35
return [...g.querySelectorAll("input")].map(i => parseInt(i.value) || 0); 36
} 37      function setCell(r, c, v) { 38        g.querySelector(`[data-r="${r}"]
[data-c="${c}"]`).value = v || ""; 39      } 40      function valid(b, r, c, v)
{ 41        for (let i = 0; i < 9; i++) if (b[r*9+i] === v || b[i*9+c] === v)
return false; 42        const br = Math.floor(r/3)*3, bc = Math.floor(c/3)*3; 43
for (let dr = 0; dr < 3; dr++) for (let dc = 0; dc < 3; dc++) 44          if
(b[(br+dr)*9 + (bc+dc)] === v) return false; 45        return true; 46      } 47
```

```
let steps = 0; 48      function backtrack(b) { 49        steps++; 50        let
r = -1, c = -1, empty = true;
  1        for (let i = 0; i < 81; i++) if (!b[i]) { r = Math.floor(i/9); c = i
%9; empty = false; break; }  2        if (empty) return true;  3        for (let
v = 1; v <= 9; v++) {  4          if (valid(b, r, c, v)) {  5            b[r*9 +
c] = v;  6            if (backtrack(b)) return true;  7            b[r*9 + c] =
0;  8          }  9        } 10        return false; 11      } 12      function
sample() { 13        clear(); 14        const puzzle = [ 15          5,3,0,
0,7,0, 0,0,0, 16          6,0,0, 1,9,5, 0,0,0, 17          0,9,8, 0,0,0, 0,6,0,
18          8,0,0, 0,6,0, 0,0,3, 19          4,0,0, 8,0,3, 0,0,1, 20
7,0,0, 0,2,0, 0,0,6, 21          0,6,0, 0,0,0, 2,8,0, 22          0,0,0, 4,1,9,
0,0,5, 23          0,0,0, 0,8,0, 0,7,9 24        ]; 25        for (let i = 0; i
< 81; i++) { 26          if (puzzle[i]) g.querySelector(`[data-r="$
{Math.floor(i/9)}"][data-c="${i%9}"]`).value = puzzle[i]; 27        } 28      }
29      function clear() { g.querySelectorAll("input").forEach(i => i.value =
""); } 30      function solve() { 31        const b = grid(); 32        steps =
0; 33        const ok = backtrack(b); 34        if (ok) { for (let r = 0; r < 9;
r++) for (let c = 0; c < 9; c++) setCell(r, c, b[r*9 + c]); 35
document.getElementById("msg").textContent = "Solved in " + steps + " steps."; }
36        else document.getElementById("msg").textContent = "No solution."; 37
} 38    </script> 39  </body> 40  </html>
```

File structure

```
single file: sudoku.html
```

How to run it

Open in any browser.

How it works

The backtracking algorithm finds the next empty cell, tries digits 1..9, recursively solves. The valid() function checks the row, column and 3x3 box. Counting steps is just increments per recursive call.

Customization ideas

- Add difficulty levels
- Add a puzzle generator
- Add visualisation of the backtracking trail

🕹 Upgrade Challenge

> *Add a forward-looking constraint propagation algorithm for harder puzzles.*

Project 99: Tiny Generative Art Wallpaper

Difficulty: ☆☆☆

What you will build

Procedurally generate unique abstract wallpaper images and download them as PNG.

Skills you will learn

- Canvas image generation
- Random art

Features

- Adjustable seed
- Multiple styles
- High-res output
- Download PNG

Build it with AI — the prompt to paste

> *Build 'Tiny Generative Art Wallpaper'. Choose a style: lines, dots, circles, blobs. Click generate to produce a high-resolution wallpaper on canvas. Use a simple seeded RNG so the same seed gives the same picture. Pure JS canvas, single file.*

Complete code

— HTML —

```
 1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<title>Generative Art</title>  6    <style>  7      body { font-family: system-
ui; max-width: 720px; margin: 30px auto; padding: 20px; }  8      h1 { color:
#7e22ce; }  9      select, input, button { padding: 8px 12px; border-radius:
6px; border: 1px solid #cbd5e1; font: inherit; margin: 4px; } 10      button
{ background: #7e22ce; color: white; border: 0; cursor: pointer; } 11
canvas { background: #fafafa; border-radius: 8px; margin: 14px 0; max-width:
100%; } 12    </style> 13  </head> 14  <body> 15    <h1>🖼 Generative
Wallpaper</h1> 16    <select id="st"> 17      <option
value="lines">Lines</option> 18      <option value="dots">Dots</option> 19
<option value="blobs">Blobs</option> 20      <option
value="triangles">Triangles</option> 21    </select> 22    Seed: <input
id="seed" type="number" value="42" style="width:80px"> 23    <button
onclick="go()">Generate</button> 24    <button onclick="dl()">Download
PNG</button> 25    <p><canvas id="cv" width="800" height="500"></canvas></p> 26
27    <script> 28      function srand(s) { let n = s; return () => { n = (n *
9301 + 49297) % 233280; return n / 233280; }; } 29      function go() { 30
const cv = document.getElementById("cv"); 31        const ctx =
cv.getContext("2d"); 32        const rng =
srand(parseInt(document.getElementById("seed").value) || 0); 33        const
style = document.getElementById("st").value; 34        const W = 800, H = 500;
35        ctx.fillStyle = "white"; ctx.fillRect(0, 0, W, H); 36        const
palette = (s) => { 37          let n = 0; for (let i = 0; i < s; i++) n = rng()
* 360; 38          return `hsl(${Math.floor(rng()*360)}, $
{Math.floor(rng()*40+50)}%, ${Math.floor(rng()*30+50)}%)`; 39        }; 40
if (style === "lines") { 41          for (let i = 0; i < 80; i++) { 42
ctx.strokeStyle = palette(rng); ctx.lineWidth = 1 + rng() * 4; 43
```

```
ctx.beginPath(); 44            ctx.moveTo(rng() * W, rng() * H); 45
ctx.lineTo(rng() * W, rng() * H); 46            ctx.stroke(); 47          } 48
} else if (style === "dots") { 49          for (let i = 0; i < 600; i++) { 50
ctx.fillStyle = palette(rng);
  1            ctx.beginPath();  2            ctx.arc(rng()*W, rng()*H, 2 +
rng()*8, 0, 7);  3            ctx.fill();  4          }  5        } else if
(style === "blobs") {  6          for (let i = 0; i < 40; i++) {  7
ctx.fillStyle = palette(rng) +
(Math.floor(rng()*40+20)).toString(16).padStart(2,"0");  8
ctx.beginPath();  9            for (let t = 0; t < Math.PI * 2; t += 0.2) { 10
const r = 30 + rng() * 50; 11              ctx.lineTo(150 + rng() * 500 + r *
Math.cos(t), 150 + rng() * 200 + r * Math.sin(t)); 12            } 13
ctx.closePath(); 14            ctx.fill(); 15          } 16        } else if
(style === "triangles") { 17          for (let i = 0; i < 60; i++) { 18
ctx.fillStyle = palette(rng); 19            ctx.beginPath(); 20
ctx.moveTo(rng()*W, rng()*H); ctx.lineTo(rng()*W, rng()*H); ctx.lineTo(rng()*W,
rng()*H); 21            ctx.closePath(); ctx.fill(); 22          } 23        }
24      } 25      function dl() { 26        const a =
document.createElement("a"); a.download = "wallpaper.png"; a.href =
cv.toDataURL(); a.click(); 27      } 28      go(); 29    </script> 30  </body>
31  </html>
```

File structure

```
single file: art.html
```

How to run it

Open in any browser.

How it works

Our seeded random number generator is just a Linear Congruential Generator — a tiny PRNG that returns the same sequence for the same seed. Each style does hundreds of small canvas operations. With a seed, hitting 'Generate' twice in a row produces identical images — useful for reproducibility.

Customization ideas

- Add colour palette constraints
- Add shape morphing between styles
- Add a 4K 3840x2160 export

🕹 Upgrade Challenge

> *Add Perlin noise smoothing to blob art for more refined outputs.*

Project 100: Capstone: Personal Portfolio Site

Difficulty: ☆☆☆

What you will build

A complete responsive portfolio site reflecting everything from the book: hero, 'About', 6 featured projects, skills, contact and a 'My Bookshelf' list of every project from this book.

Skills you will learn

- Composition
- Responsive layout
- Structured content

Features

- Hero with name and tagline
- About
- Grid of 6 projects
- Skills list
- Bookshelf of every project from this book
- Contact form

Build it with AI — the prompt to paste

> *Build a 'Capstone: Personal Portfolio Site'. This final project composes everything: a hero, About section, 6 featured project cards (with their own pages or detail by click), a Skills section, a Contact form and a 'Bookshelf' that lists project 1-100 with their gists. Vanilla JS, single HTML.*

Complete code

— HTML —

```
  1  <!DOCTYPE html>  2  <html>  3  <head>  4    <meta charset="utf-8">  5
<meta name="viewport" content="width=device-width,initial-scale=1">  6
<title>Your Portfolio</title>  7    <style>  8      * { box-sizing: border-
box; }  9      body { margin: 0; font-family: system-ui, sans-serif; color:
#0f172a; line-height: 1.5; background: #f8fafc; } 10      .hero { background:
linear-gradient(135deg,#4f46e5,#9333ea); color: white; padding: 100px 20px;
text-align: center; } 11      .hero h1 { font-size: clamp(32px, 6vw, 56px);
margin: 0; } 12      .hero p { font-size: clamp(16px, 3vw, 22px); margin-top:
10px; opacity: .95; } 13      main { max-width: 980px; margin: 0 auto; padding:
40px 20px; } 14      section h2 { color: #4f46e5; border-bottom: 2px solid
#e0e7ff; padding-bottom: 6px; } 15      .cards { display: grid; grid-template-
columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 16px; } 16      .card
{ background: white; padding: 18px; border-radius: 10px; box-shadow: 0 2px 8px
rgba(0,0,0,.06); transition: transform .2s; } 17      .card:hover { transform:
translateY(-3px); } 18      .card h3 { margin: 0; color: #4f46e5; }
19      .card span { font-size: 12px; color: #64748b; background: #e0e7ff;
padding: 2px 6px; border-radius: 4px; } 20      .skills { display: grid; grid-
template-columns: repeat(auto-fit, minmax(140px, 1fr)); gap: 10px; }
21      .skill { background: white; padding: 10px 14px; border-radius: 8px;
text-align: center; box-shadow: 0 1px 3px rgba(0,0,0,.06); } 22      .bookshelf
{ columns: 2; column-gap: 14px; } 23      .shelf { background: white; padding:
10px 14px; border-radius: 6px; margin-bottom: 6px; font-size: 14px; break-
inside: avoid; } 24      .shelf b { color: #4f46e5; } 25      form { display:
```

```
grid; gap: 10px; max-width: 480px; } 26      input, textarea { padding: 10px;
border: 1px solid #cbd5e1; border-radius: 4px; font: inherit; } 27      button {
padding: 12px 20px; background: #4f46e5; color: white; border: 0; border-radius:
4px; cursor: pointer; } 28      .footer { text-align: center; color: #64748b;
padding: 24px; } 29    </style> 30  </head> 31  <body> 32    <section
class="hero"> 33      <h1>Hello, I'm Your Name</h1> 34      <p>I build small
tools, study new things, and ship them every week.</p> 35    </section> 36
<main> 37      <section><h2>About</h2><p>I am a self-taught builder who finished
100 Mini Projects You Can Build With AI. This portfolio is one of them. Replace
this text with your own bio — three or four sentences of who you are, what you
like, and what you are working on.</p></section> 38   39
<section><h2>Featured projects</h2> 40        <div class="cards"> 41
<div class="card"><h3>Habit Tracker</h3><span>productivity</span><p>A weekly
tracker with streaks. Pure HTML and JS.</p></div> 42          <div
class="card"><h3>Snake</h3><span>game</span><p>A grid-based snake that grows as
it eats.</p></div> 43          <div class="card"><h3>Pomodoro
Timer</h3><span>productivity</span><p>25/5 cycles that beep at each phase
change.</p></div> 44          <div class="card"><h3>Weather
UI</h3><span>web</span><p>A forecast card layout using mock data only.</p></div>
45          <div class="card"><h3>Tic Tac Toe</h3><span>game</span><p>Two-player
X and O with win-line highlight.</p></div> 46          <div
class="card"><h3>Resume Builder</h3><span>web</span><p>A live-updating resume
with print and download.</p></div> 47        </div> 48      </section> 49   50
<section><h2>Skills</h2>
  1        <div class="skills">  2          <div class="skill">HTML</div><div
class="skill">CSS</div>  3          <div class="skill">JavaScript</div><div
class="skill">Canvas</div>  4          <div class="skill">Web Audio</div><div
class="skill">DOM APIs</div>  5          <div class="skill">Algorithms</div><div
class="skill">Debugging</div>  6          <div class="skill">Working with
AI</div><div class="skill">Project management</div>  7        </div>  8
</section>  9   10      <section><h2>The Bookshelf</h2> 11        <p>I built
every project from <em>100 Mini Projects You Can Build With AI</em>. Here they
are, all 100. Each one taught me a small new idea.</p> 12        <div
class="bookshelf"> 13          <div class="shelf"><b>1.</b> Hello Name
Greeter</div> 14          <div class="shelf"><b>2.</b> Age in Days
Calculator</div> 15          <div class="shelf"><b>3.</b> Days Until
Summer</div> 16          <div class="shelf"><b>4.</b> Random Compliment
Generator</div> 17          <div class="shelf"><b>5.</b> Colour of the Day</div>
18          <div class="shelf"><b>6.</b> Even or Odd Checker</div> 19
<div class="shelf"><b>7.</b> Temperature Mood Translator</div> 20          <div
class="shelf"><b>8.</b> Reverse a Word</div> 21          <div
class="shelf"><b>9.</b> Simple Counter with Limits</div> 22          <div
class="shelf"><b>10.</b> Tip Calculator</div> 23          <div
class="shelf"><b>11.</b> Dice Roller Simulator</div> 24          <div
class="shelf"><b>12.</b> BMI Calculator</div> 25          <div
class="shelf"><b>13.</b> Coin Toss Counter</div> 26          <div
class="shelf"><b>14.</b> Word Length Sorter</div> 27          <div
class="shelf"><b>15.</b> Magic 8-Ball</div> 28          <div
class="shelf"><b>16.</b> Number Pyramid Printer</div> 29          <div
class="shelf"><b>17.</b> Basic Calculator</div> 30          <div
class="shelf"><b>18.</b> Time Zone Buddy</div> 31          <div
class="shelf"><b>19.</b> Word Counter Tool</div> 32          <div
class="shelf"><b>20.</b> Digital Greeting Card</div> 33          <div
class="shelf"><b>21.</b> Personal Landing Page</div> 34          <div
class="shelf"><b>22.</b> Notes Saver</div> 35          <div
class="shelf"><b>23.</b> Responsive Photo Grid</div> 36          <div
class="shelf"><b>24.</b> Markdown Previewer</div> 37          <div
class="shelf"><b>25.</b> Real-time Clock Display</div> 38          <div
class="shelf"><b>26.</b> Sticky Notes Board</div> 39          <div
class="shelf"><b>27.</b> Animated CSS Buttons</div> 40          <div
class="shelf"><b>28.</b> Form Validator</div> 41          <div
```

```
class="shelf"><b>29.</b> Drag-and-Drop Kanban</div> 42          <div
class="shelf"><b>30.</b> Mini Online Storefront</div> 43          <div
class="shelf"><b>31.</b> Weather UI with Mock Data</div> 44          <div
class="shelf"><b>32.</b> Tabbed Interface</div> 45          <div
class="shelf"><b>33.</b> Image Filter Playground</div> 46          <div
class="shelf"><b>34.</b> Accordion FAQ</div> 47          <div
class="shelf"><b>35.</b> Searchable Movie List</div> 48          <div
class="shelf"><b>36.</b> Toast Notification System</div> 49          <div
class="shelf"><b>37.</b> Analog Clock</div> 50          <div
class="shelf"><b>38.</b> Personal Blog Template</div>
  1          <div class="shelf"><b>39.</b> Recipe Card Component</div>  2
<div class="shelf"><b>40.</b> Resume Builder</div>  3          <div
class="shelf"><b>41.</b> Number Guessing Game</div>  4          <div
class="shelf"><b>42.</b> Rock Paper Scissors</div>  5          <div
class="shelf"><b>43.</b> Tic Tac Toe</div>  6          <div
class="shelf"><b>44.</b> Memory Match Game</div>  7          <div
class="shelf"><b>45.</b> Snake Game</div>  8          <div
class="shelf"><b>46.</b> Breakout / Brick Breaker</div>  9          <div
class="shelf"><b>47.</b> Typing Speed Tester</div> 10          <div
class="shelf"><b>48.</b> Reaction Time Tester</div> 11          <div
class="shelf"><b>49.</b> Whack-a-Mole</div> 12          <div
class="shelf"><b>50.</b> Pomodoro Timer</div> 13          <div
class="shelf"><b>51.</b> Hangman</div> 14          <div class="shelf"><b>52.</b>
Maze Generator and Solver</div> 15          <div class="shelf"><b>53.</b> Quiz
Game</div> 16          <div class="shelf"><b>54.</b> Tower of Hanoi
Visualiser</div> 17          <div class="shelf"><b>55.</b> Drawing Pad</div> 18
<div class="shelf"><b>56.</b> Space Shooter</div> 19          <div
class="shelf"><b>57.</b> Bouncing DVD Logo</div> 20          <div
class="shelf"><b>58.</b> Connect Four AI</div> 21          <div
class="shelf"><b>59.</b> Simon Says Colour Memory</div> 22          <div
class="shelf"><b>60.</b> Stop The Clock Race</div> 23          <div
class="shelf"><b>61.</b> Habit Tracker</div> 24          <div
class="shelf"><b>62.</b> Expense Tracker</div> 25          <div
class="shelf"><b>63.</b> Flashcard Study App</div> 26          <div
class="shelf"><b>64.</b> Markdown Editor</div> 27          <div
class="shelf"><b>65.</b> QR Code Generator UI</div> 28          <div
class="shelf"><b>66.</b> Text Analyzer</div> 29          <div
class="shelf"><b>67.</b> Daily Planner</div> 30          <div
class="shelf"><b>68.</b> Pomodoro Habit Combo</div> 31          <div
class="shelf"><b>69.</b> Stock Portfolio Tracker (Mock)</div> 32          <div
class="shelf"><b>70.</b> Personal CRM</div> 33          <div
class="shelf"><b>71.</b> Unit Converter</div> 34          <div
class="shelf"><b>72.</b> Random Quote Generator with Originals</div> 35
<div class="shelf"><b>73.</b> Password Strength Tester</div> 36          <div
class="shelf"><b>74.</b> Workout Timer</div> 37          <div
class="shelf"><b>75.</b> Book Reading Tracker</div> 38          <div
class="shelf"><b>76.</b> Daily Affirmation Spinner</div> 39          <div
class="shelf"><b>77.</b> Movie Watch List</div> 40          <div
class="shelf"><b>78.</b> Invoice Generator</div> 41          <div
class="shelf"><b>79.</b> Markdown Notes Wiki</div> 42          <div
class="shelf"><b>80.</b> Reading Speed Adjuster</div> 43          <div
class="shelf"><b>81.</b> Interactive Solar System</div> 44          <div
class="shelf"><b>82.</b> Colour Palette Generator</div> 45          <div
class="shelf"><b>83.</b> Procedural Terrain Generator</div> 46          <div
class="shelf"><b>84.</b> Particle Explosion Simulator</div> 47          <div
class="shelf"><b>85.</b> Drum Machine / Step Sequencer</div> 48          <div
class="shelf"><b>86.</b> Projectile Motion Simulator</div> 49          <div
class="shelf"><b>87.</b> Chord Progression Player</div> 50          <div
class="shelf"><b>88.</b> ASCII Art Image Converter</div>
  1          <div class="shelf"><b>89.</b> Cipher Encoder & Decoder</div>  2
<div class="shelf"><b>90.</b> Random Story Generator</div>  3          <div
```

```
class="shelf"><b>91.</b> Meme Generator</div>  4          <div
class="shelf"><b>92.</b> Data Dashboard</div>  5          <div
class="shelf"><b>93.</b> Snakes & Ladders Game</div>  6          <div
class="shelf"><b>94.</b> Mini Search Engine</div>  7          <div
class="shelf"><b>95.</b> Recipe Randomizer</div>  8          <div
class="shelf"><b>96.</b> Decision Spinner (Weighted)</div>  9          <div
class="shelf"><b>97.</b> Mini Synthesizer</div> 10          <div
class="shelf"><b>98.</b> Sudoku Solver</div> 11          <div
class="shelf"><b>99.</b> Tiny Generative Art Wallpaper</div> 12          <div
class="shelf"><b>100.</b> Capstone: Personal Portfolio Site (this page!)</div>
13        </div> 14      </section> 15   16      <section><h2>Contact</h2> 17
<form onsubmit="event.preventDefault(); alert('Demo form — not delivered.');">
18          <input type="text" placeholder="Your name" required> 19
<input type="email" placeholder="Your email" required> 20          <textarea
placeholder="Your message" rows="4" required></textarea> 21
<button>Send message</button> 22        </form> 23      </section> 24   25
<p class="footer">© 2026 Your Name · Built with 100 Mini Projects You Can Build
With AI</p> 26    </main> 27  </body> 28  </html>
```

File structure

```
single file: portfolio.html
```

How to run it

Open in any browser.

How it works

This project simply combines everything we built: a hero gradient from earlier projects, cards laid out with CSS Grid (Project 21), a rating-style bookshelf of 100 lists (Project 90's pattern), and a contact form like Project 28. The takeaway: every project in this book becomes a real tool you can reuse.

Customization ideas

- Add a blog section
- Add a dark/light toggle
- Add a downloadable CV button

🕹 Upgrade Challenge

> *Replace each featured project card with a live link to its demo (hosted on GitHub Pages).*

30-Day Project Challenge

If you want to make this book work for you, take the 30-day challenge. The rules are simple: spend 30–60 minutes a day, every day, building something from this book. You can skip days on weekends, but never two days in a row.

By day 30 you will have completed at least twenty projects. By day 100 you will have completed all 100. The challenge is not the projects — the challenge is showing up.

Suggested 30-day sequence:

Pick one project from Part 1 (Super Easy) every day for the first 5 days. By day 5 you will understand HTML, CSS and JavaScript well enough that even project 6 feels easy. From day 6 to 15, work through Part 2 (Web). From day 16 to 25, do Part 3 (Games). Days 26–30 are Part 4 (Productivity). After day 30 you are ready for Part 5 (Advanced Beginner).

Project Difficulty Roadmap

Use this as a map of how your skills will unfold project by project.

Skills Gained by Completing the Book

By the time you finish project 100 you will have personally built and run 100 small programs. Beyond that stack of code, you will have built real, transferable skills:

- Reading and writing HTML, CSS and JavaScript with confidence.
- Working with browser APIs — localStorage, Canvas, WebAudio, Intl, Fetch.
- Debugging — recognising common errors and using developer tools.
- Describing requirements clearly — exactly the skill you use to talk to AI coding assistants.
- Iterating. Taking something that works and making it better in small steps.
- Recognising patterns across projects (state machines, validation, persistence).
- Shipping — finishing things and saving them as files you can open later.
- Curiosity — the most underrated programming skill of all.

What to Build Next

100 small projects is a strong portfolio, but it is also a starting line. Once you're done, here are the most natural ways to keep growing — all in the same spirit as this book.

1. Pick your favourite project from this book and rewrite it on your own, from a blank file, without looking at the source. If you can recreate it, you really learned it.
2. Combine two of your favourite projects. A Pomodoro timer + Habit tracker (we ship this in project 68) is already strong — what about a Timer + a Custom Sound?

3. Solve a small real problem in your daily life. Build a personal digital clock for your study desk, a tracker for how many pages you read, a random joke picker for your family dinner.
4. Pick a project from this book and turn it into a real web app others can use. Even a simple Notes App becomes valuable once it is online.
5. Learn one new technology. After finishing this book, you are ready to read the introduction to React, Vue, Svelte, or any other framework. Pick one and build the same kind of apps on top of it.
6. Teach. Pick any single project from this book and explain it to a friend who has never coded. Teaching is the deepest form of learning.

Final AI Coding Cheat Sheet

When you finish this book, you will not just know 100 things you can build — you will also know how to talk to an AI assistant about building anything else. Save this cheat sheet. It will serve you well beyond project 100.

Describe the goal.

'I want a webpage where the user types a temperature in Celsius and the page shows one of five mood messages based on the temperature range.'

Describe the constraints.

'Vanilla HTML, CSS and JavaScript only. One file. No external libraries. Mobile-friendly.'

Describe the UI in detail.

'A large input box, a primary button labelled Tell me the mood, and a result text below it. Use a calm colour palette — light blue background, dark grey text.'

Describe what should go wrong.

'If the input is empty, show Please enter a temperature. If it is not a number, show That does not look like a temperature.'

Share what you tried.

'I tried writing the code myself but my code returns NaN when the input is empty. Here is the snippet I am running. What am I missing?'

Ask the AI to explain, not just fix.

'Explain line 7 of this snippet as if I am a complete beginner. I do not understand the localStorage part.'

Iterate.

'Make the message colour match the temperature range. Use blue for cold, grey for mild, orange for hot. Oh, and use Celsius only — do not bother with Fahrenheit.'

Author Note

Hello, and thank you for picking up this book. I wrote it because I believe the best way to learn anything today is to build, and the best way to build is to start

small. Every project in this book is something a complete beginner can finish in an afternoon.

If this book is your first step into coding, congratulations — you are already in good company. Every working developer you have ever heard of began with a project much smaller than the ones in Part 5. The trick was always the same: keep building, finish what you start, and don't be afraid to ask an AI for help when you get stuck.

I'd love to hear what you build. If you finish a project, change it, and feel proud of the result, that is the moment this book was written for.

Build small. Build often. The rest will come.